AI 未成之路

> **一个走了三年 AI 探索的项目负责人，写下的「没做成」记录**

> **副标题**：为什么大多数企业的 AI 都没做成，以及怎么认出自己正在哪个坑里 【待核实：待拍板】 **定价**：【待核实：待拍板】 **署名**：【待核实：待拍板，候选署名「米悦说说」】 **版本**：定稿候选版 v1.21 · 2026-11-06 **体量**：约 15 万中文字，十章 + 附录 A–H

脱敏口径：全文不出现真实企业名，涉及处统一用「某大型制造企业」「某制造业集团」等泛化称呼。文中出现的开源系统与商业平台名称，用于说明技术路线，不指向任何具体企业的内部部署。

免责说明：书中案例基于作者所在项目的真实经历，金额、人数、效率数据均已做相对量级处理，仅用于说明方法，不构成对任何具体企业的评价。

脱敏口径：全文不出现真实企业名，涉及处统一用「某大型制造企业」「某制造业集团」等泛化称呼。文中出现的开源系统与商业平台名称，用于说明技术路线，不指向任何具体企业的内部部署。

免责说明：书中案例基于作者所在项目的真实经历，金额、人数、效率数据均已做相对量级处理，仅用于说明方法，不构成对任何具体企业的评价。

> **一个走了三年 AI 探索的项目负责人，写下的「没做成」记录**

副标题：为什么大多数企业的 AI 都没做成，以及怎么认出自己正在哪个坑里

作者：某大型制造企业 智能与数字化中心 项目负责人

版本：定稿候选稿 v1.13 （ 2026-11-03 ）｜全书约 15 万中文字

全书目录

章	标题	这章要回答什么
0	引言	AI 的量级该怎么估，这本书写给谁
1	企业 AI 落地的 5 个坑	输的都不是模型不够强
2	场景选择：先选事，再选技术	场景选错的代价远大于技术选错
3	数据准备：把数据治理翻译成 AI 能听懂的话	数据没准备好就想上 AI
4	技术方案：三棵决策树	「哪个模型更强」是伪问题
5	企业 AI 落地五步法	从「方案通过」到「真的有人每天用」
6	算账：老板视角的三问	AI 项目不赚钱就等于没做
7	合规与风险：哪条线不能碰	AI 的位置必须按出错代价来定
8	真实案例复盘	三个成的，一个没成的，一个半失败的
9	数字员工建设	数字员工不是系统，是岗位
附录	工具箱 A–H	能直接抄走的表

阅读提示

- 只想看真相与结论 → 直接读**第八章**
- 正负责一个在跑的 AI 项目 → 读**第五章 5.4 止损条件**
- 在纠结要不要做某个场景 → 读**第二章、第三章**的判断表
- 想知道制造业六个环节各自的 AI 适合度 → 读**第八章 8.0.1**
- 想要能直接抄走的表格 → 读**附录**

- 想要能直接抄走的表格 → 读**附录**
- 手上有个新场景，先问一句「这事该不该做」 → 读**附录 H 场景筛选表**（ 12 问， 5 分钟）

本书纪律

不编造未确认的案例、金额与效果数字；不确定处一律标注【待核实】；企业名、人名、产品名、金额与内部精确数字已做泛化处理。

本书结论不是任何一家企业的专利——素材来自一家大型制造企业三年三段的实践（技术探索 → 创新专项 → 持续探索），但书里讲的是制造业六个环节的共性规律。

> **本章说明**：本书素材来自作者亲历的企业内部实践与行业通行做法。**凡涉及具体企业、金额、效果数字之处，不确定的均标** `【待核实】`。 脱敏口径：全文不出现真实企业名，涉及处统一用「某大型制造企业」「某制造业集团」等泛化称呼。

---

0. 开篇：三年里， AI 大多没做成

> **这一节是这本书为什么存在的答案。** 下面三条「真相」是它的内容，这一节是它的立场。

★ 先把这三年说清楚：三段不同的性质

很多读者读到后面会问一个问题：**这些数字，到底是攒了三年攒出来的，还是一年之内做出来的？**

这个问题很关键，因为它决定了后面的每一组数字该怎么读。



★ 请把这张图和下一张图放在一起看。

下一张图里的「推出来的」那一组数字——培训场次、组队数量、需求清单——**全部发生在第二年这一年里**。而「扎出来的」那一组——可核算的年度效益——出现在第三年。

这就是本书全部的主题：**同样是三年，第一年的试错和第三年的模式转变，中间夹着第二年那场声势浩大、却没有兑现的「推」**。

先看两组数字

同一家企业，**第二年（AI 创新专项正式展开那年）**的时间轴上，躺着两组完全不同的数字。

第一组（推出来的）	
★ 以上全部发生在同一个年度（第二年）	
培训累计	数百人次（含 6 场专题培训）
内部AI应用创新赛	三十余支队伍、一百五十余人
最终胜出	十余支队伍
AI 创新专项参与总人数	两百余人
数字员工平台登录 / 调用	四千余次 / 两千余次
成熟度自评	从近乎零起步，一年内进入第二档
收集到的业务需求	一百五十余项（多轮创研累计）
★ 最终真正落地的场景	数个（★ 均为全年）

第二组（扎出来的）	
★ 出现于第三年（持续探索期）	
合同智审	四大合同类型全部上线 ★ 全公司覆盖
印章识别	合同审阅部门落地 ★ 可复制
产出	千余小时/年 50%~96% 效率提升 漏检风险降 30%+

★ 这两组数字之间的落差，就是这本书的全部主题：

> **培训拉来了人，人变多了，场景没变多。**
>
> **而两个被「扎」出来的场景，拿出了可核算的年度效益。**

先说清楚一件事：这不是我们一家公司的故事

有读者会问：**这些坑，是你一家公司没做好，还是行业里都这样？**

答案是行业里都这样。 因为制造业的业务链条几乎不随规模和技术路线变化：

```
研发 → 采购 → 生产 → 库存 → 质量 → 销售
          ↓
★ 不管是精密生产、流程生产还是离散生产，这六个环节都在
   而 AI 在每个环节遇到的问题结构，也是共通的
```

所以这本书写的不是一家公司的私有经验，而是六个环节各自的「AI 适合度」：

环节	AI 适合度	典型结局
研发	差（验证周期太长）	试点悬空
采购	**最好（规则明确、文档密集）**	**最容易先成功**
生产	差（实时与完整互斥）	卡在数据侧
库存	中	卡在口径统一
质量	中（责任不可转移）	必须配套留痕与签字设计
销售	**差（口径在人脑、入口在别人地盘）**	**典型失败场景**

★ 而由此得出的一条普适结论，是这本书最想留下的东西之一：

> **AI 落地的顺序，不该从「最痛的地方」开始，而该从「规则最明确的地方」开始。**
>
> 因为「痛」不等于「适合」——最痛的地方，往往同时是口径说不清、入口改不动、效果验不了的。

那为什么没做成？

如果你在企业里负责过 AI，你大概率经历过这样的一年：

```
年初：定了目标，要推 AI
春天：办训练营，做大赛，收集需求，报规划
夏天：立项了几个场景，平台上线了，助手发下去了
秋天：开会问进度——"用得怎么样？"
        "挺好的，大家都在用。"
年末：算账，算不出 ROI
        → 明年：目标再定一遍，人再办一遍
```

这个循环里，最贵的不是钱，是一整年的注意力。

而这个循环之所以会重复，通常不是因为这一年没努力，而是因为**没有人记录失败**：

```
成功的部分有汇报材料
试错的部分只有几句"还在推进"
半失败的部分没人愿意讲——因为讲出来像甩锅
彻底失败的部分干脆从记录里消失
          ↓
★ 于是每一年都是崭新的开始，每一年都在重新踩同一批坑
```

这本书想说的是：失败的部分才是资产

市面上的 AI 落地书几乎都在讲怎么做成。 讲怎么做成的书有一个共同的特点：

```
它们的成功案例，往往来自同一批人——
  有人懂技术，有人有数据，有人有决策权，还有人愿意试错
          ↓
★ 于是读者读完的问题不是"我该怎么做"
   而是"他们有我没有的条件"
```

而这本书讲的是没做成的那些，包括：

- 一个功能做出来了，用户没有来的营销助手
- 一个试过了、推了、但没继续用的智能问数
- 一批「计划做」至今躺在清单上的 46 个场景
- 一堆上线了但一个流程都没改的助手
- 以及——**两个真的做成的，和它们做成的那个共同点**

★ 全书的核心命题只有一句：

> **AI 落地失败，不是努力不够，是「推」与「扎」的资源配比错了。**
>
> **推的产出是声量，扎的产出是数字。而绝大多数企业，把绝大部分资源花在了推上。**

这本书写给谁：一份立场声明

> **这一节是全书最重要的一节。** 因为它决定了这本书写什么、不写什么， 也决定了你在读每一章的时候，应该把它当成「参考」还是「镜子」。

★ 先把话说清楚：这本书是站在甲方这边写的。

```
市面上讲 AI 落地的书，绝大多数是乙方视角写的——
    卖方案、卖工具、卖培训、吹成功案例
          ↓
★ 而这本书的作者是甲方：
    是一家制造企业的 AI 创新专项负责人
    拿着自己的预算、自己的考核、自己团队的 KPI
    走完了三年
          ↓
★ 所以这本书里有些东西，是乙方视角的书不会写的：
    · 哪些场景其实是"看着能做"，其实做不了
    · 哪些钱花了但换不回任何东西
    · ★ 以及——一个项目组在自己的考核压力下，是怎么做出错误判断的
```

★ 具体的读者，是这三类人：

读者	你会在书里找到什么	建议从哪读起
甲方 IT / 技术团队	场景怎么筛、数据怎么备、技术怎么选、模型怎么管、接口怎么接、留痕怎么做	第三章 → 第四章 → 附录 A/C/D
甲方管理者 / 业务负责人	该投什么、怎么判断一个场景值不值得投、怎么算账、什么时候该叫停	第一章 → 第二章 → 第六章 → 5.4 止损条件
乙方（服务商 / 创业团队）	**★ 一份反向提醒**：客户真正需要什么，以及什么样的推销是有害的	8.5 营销助手失败 → 8.6 智能问数 → 第六章

★ 而给乙方的那一部分，是这本书最想说的话之一：

> **不要为了自己的业绩，去推动甲方投入。**
>
> **★ 要为甲方设身处地地考虑解决方案—— 而不是为乙方自己的业绩表考虑方案。**

★ 乙方最容易走的三条路：

□ 1. "这个场景很有前景" → 甲方投了，三个月后发现不该做
↓
而这时，已经投进去的人力、算力、时间，
早就沉没了 —— ★ 沉没成本让决策人更难承认失败
↓
★ 所以"劝甲方投一个不合适的场景"，
代价从来不在乙方那边，在甲方那边

□ 2. "AI 能解决一切" → 甲方照做了，
效果达不到预期 → 甲方对整个 AI 的判断被一起否掉
↓
★ 一家企业的 AI 信任，是很脆弱的资产。
★ 乙方每做一次过度承诺，就在消耗它

□ 3. 只讲能做的事，不讲做不到的事
↓
★ 甲方付了钱、投了人，最后发现自己买的是一台"没有说明书的能力"

★ 而这三件事，本书作者在甲方这边全都见过：

★ 所以这本书写给乙方的那部分，说的其实只有一句话：

↓

★ ★ ★ 如果你的方案让甲方投了钱却没做成，
那笔账最后是要甲方来付的。 ★ ★ ★

↓

★ 真正为甲方着想的乙方，
会在签合同之前就告诉甲方"这个场景现在不该做"，
并且说清楚为什么。

★ 而这恰恰是甲方视角写出来的书才敢说的话：

如果这本书是乙方写的，它不会告诉你
"有些场景现在不该做"

↓

★ 因为那等于在劝客户别买

↓

★ 而甲方作者可以写这句话，
因为他花的是自己的钱、担的是自己的责

> **金句**
>
> **「这本书站在甲方这边——因为钱是甲方出的，责任也是甲方担的。」**
>
> **「不要为了自己的业绩推动甲方投入。真正为甲方着想的乙方，会在签合同前就劝客户别买。」**
>
> **「一家企业的 AI 信任是脆弱资产，乙方每做一次过度承诺就在消耗它。」**
>
> **「劝甲方投一个不合适的场景，代价从来不在乙方那边。」**

★★ 先把 AI 这件事的量级说对：★ 它更像财务软件，不像工业革命

★ 这一节是全书的定调，写在前面，是为了让你用对尺子。 ★ 作者 2026-11-03 补充。

```
★★ ★★ 一个类比：★★ AI 之于今天的信息化，★ ★★ 很像当年的财务软件之于计算器和算盘
        ↓
★★ ★★ 而财务软件这件事，★★ 我们都知道它后来怎么样了：
   ★★ 没有一夜之间取代一切，★★ ★★ 它是 ★★ 逐步、不断周期性地淘汰迭代
   ★★ 一个部门一个部门、一个岗位一个岗位地深入下去
   ★★ ★★ 而今天回头看，★ 它当然赢了——★★ ★★ 但它赢的方式，★★ 是"渗透"，不是"爆破"
```

★★ ★★ 而这个类比，最要紧的地方在于它纠正了两种普遍的误判：

```
★★ 误判一：★★ "★ AI 会像宣传的那样，★ 直接替代掉什么"
        ↓
★★ ★★ 而历史经验是：★★ 重大替代从来不是一次完成的
★★ ★★ 它是 ★★ 每一个周期替换掉一小块，★★ ★★ 连续几轮之后才显出全貌
        ↓
★★ ★★ 而正因为它慢，★★ ★★ 中途的每一次局部替换，★★ ★★ 都会被误读为失败
```

```
★★ 误判二：★★ "★ 那些真正高端的岗位，★★ AI 一来就该被替代"
        ↓
★★ ★★ 而事实是：★★ 在数学、医学研究这类高端领域，
★★ ★★ 就算不用 AI，★★ ★★ 也有其他先进技术手段在替它
★★ ★★ ★★ 而那些手段，★★ ★★ 本身也在被 AI 改写着
        ↓
★★★ ★★★ ★★ 也就是说：★★ ★★ 高端领域的替代，★★ ★★ 并不是"人 vs AI"
★★★ ★★★ ★★ 而是 "★★ 人 + 一堆技术手段 vs 人 + 一堆技术手段 + AI ★★"
```

★★★ ★★★ 而这两条，★ 合起来就画出了传统制造企业 AI 机会的真实形状：

```
★★★ ★★★ 传统制造企业里，★★★ ★★ 大部分岗位没有"高端到能被 AI 直接替代"的东西
★★★ ★★★ ★★ 而更多的是这一类：
        ↓
   · ★★ 提高效率
   · ★★ 降低成本
   · ★★ 提高决策能力
   · ★★ ★★ ★★ 减少对外咨询的费用
        ↓
★★★ ★★★ ★★ 而最后这一条，★★★ ★★ 是 AI 落在传统制造企业上最实在、也最少被算的一笔账
```

★ ★★ 而由此，本书对「AI 这件事」的基本判断也就定下来了：

```
★★★ ★★★ ★★ 不指望 AI 带来一场革命，★★★ ★★ 只指望它成为一个新的底座
★★★ ★★★ ★★ 而新的底座不会一夜建成，★★ ★★ 它是一个周期一个周期地被建起来
          ↓
★★★ ★★★★ ★★ 于是这本书要回答的问题，★★★ ★★ 就不是"★ 怎么抓住这个风口"
★★★★    ★★★★ ★★ 而是：★★ "★ 在这个漫长的渗透过程里，★★★ ★★ 哪些事现在就该做，哪些事现在做了是浪费
```

> **★ 金句**
>
> **「AI 更像当年的财务软件——它不是一夜之间取代一切，是逐步、不断周期性地淘汰迭代，然后不断深入。」**
>
> **「在传统制造企业里，AI 最实在的一笔账，不是替代谁，而是减少对外咨询的费用。」**

这本书不写什么

```
❌ 不写模型科普              —— 你可以在任何地方查到
❌ 不写厂商对比              —— 明年就不准了
❌ 不写"如何做好 AI 转型"这类宏大叙事
❌ 不把失败包装成"虽然没做成但意义重大"
          ↓
★ 尤其最后一条是本书的纪律：
   失败要讲失败，但要讲清楚代价和结论
   —— 否则这本书就成了一本自我安慰读物
```

那这本书写什么

第一章　五个坑：为什么大部分项目在开头就埋雷
第二章　场景选择：别先问能做什么，先问值不值得做
第三章　数据准备：把数据治理翻译成 AI 能听懂的话
第四章　技术方案：三个决策树，不追最新
第五章　五步法：从决定做，到真的被用起来
第六章　算账：成本五项、收益四类，以及"失败的代价"
第七章　合规与风险：哪些事绝对不能干
第八章　★ 真实复盘：三年三阶段、一个失败、一个半失败、两个真成功
第九章　数字员工：把 AI 变成岗位，而不是工具
附录　七个可以直接抄走的模板

★ 怎么读这本书：按你的角色选一条路。

【甲方 · 技术团队】
第三章 数据准备 → 第四章 技术方案 → 附录 A/C/D
（你想知道的是"怎么把它建起来"）

【甲方 · 管理者 / 业务负责人】
第一章 五个坑 → 第二章 场景选择 → 第六章 算账 → 5.4 止损条件
（你想知道的是"该不该投、投多少、什么时候叫停"）

【乙方 · 服务商 / 创业团队】
8.5 营销助手失败 → 8.6 智能问数 → 第六章 算账 → 2.2.1 六环节速查表
（你想知道的是"客户真正需要什么，以及什么推销是有害的"）

【所有人 · 只有 30 分钟】
直接读第八章

★ 读全书的话，建议顺序是：第一章 → 第二章 → 第三章 → 第四章 → 第五章 → 第六章 → 第七章 → 第八章 → 第九章。

★ 而如果你是带着一个具体问题来的：

你的问题	去哪
「这个场景到底该不该做？」	第二章 2.2 三问 + 2.2.1 六环节速查表
「数据要准备到什么程度？」	第三章 + 附录 D 三张清单
「模型怎么选、微调还是 RAG？」	第四章三棵决策树
「项目什么时候该叫停？」	第五章 5.4 止损条件
「老板问 ROI 怎么答？」	第六章 + 附录 B
「这算不算合规风险？」	第七章 + 附录 E
「有哪些真实案例？」	第八章
「怎么把 AI 变成岗位？」	第九章

三个可以带走的自查问题

这本书很长，但如果你只记住三件事，就记住这三个问题：

① 我这个场景，如果做砸了，代价是多少？
—— 代价可控，就试；代价不可控，先别动

② 我做这件事，是为了让更多人知道，还是为了让某个数变好？
—— 后者才是产出数字，前者只是产出声量

③ 这个场景做成了，能复制到下一个部门吗？
—— 不能复制就不是成果，是表演

最后一件事：关于时间

```
第一年 · 技术探索期
    试模型、试工具、试数据能不能出。
    没规模，没声量，但没这一年的试错，后面连坑都踩不上。
          ↓
第二年 · AI 创新专项正式展开  ★ 声势最大的一年
    一年之内：多轮创研、6 场专题培训、需求清单收到一百五十余项、
    三十余支队伍参赛，最后落地数个场景。
    —— 投入是实打实的，产出与投入不成比例。
          ↓
第三年 · 持续探索期  ★ 真正跑出数字的一年
    合同智审全公司覆盖，印章识别进了合同审阅部门，
    落地应用模式从"让人学用"转向"嵌进流程"。
          ↓
★ 从"试"到"扎"，中间隔的不是时间，是
    —— 你有没有把中间那年没成的事，认真记下来
    —— 以及你有没有在第三年敢推翻第二年的做法
```

而这正是这本书存在的理由：把那三年里没记下来的东西，现在记下来。

0.5 这本书想解决什么问题

这几年，几乎每家企业都在谈 AI 。

有的在买模型，有的在建算力中心，有的在开「 AI 专项行动」会议，有的在给员工发提示词模板。热闹是真热闹。但如果你问一个 CIO 或者数字化总监：「你们公司去年上的那个 AI 项目，现在还在用吗？」——十有八九会得到一个含糊的回答。

不是他们不想说清楚。是真说不清。

因为从一开始，就没有人算过这笔账。

我做过二十年企业信息化，做过数据治理体系，也参与过 AI 相关规划的落地与评估。看了太多企业上 AI 的路径——**技术上没有一家是输在模型不够强的，输的都是别的地方**。输在选错了场景、输在数据根本不能用、输在组织不配合、输在做完了没人说得清省了多少。

这四个「输」里面，没有一个跟「模型不够强」有关。

所以这本书的立场很明确：

> **别人教你怎么把 AI 用起来，我告诉你为什么你的 AI 用不起来，以及在出事之前先做什么。**

这本书不教你写提示词，不给你工具测评，不做模型对比。它写给的是这样一类人： 35 到 55 岁，企业的 IT 负责人、数字化总监、 CIO 助手，来自国企、央企、制造业、制药这类数据敏感、流程沉重、层级复杂的组织。他们不缺技术科普，缺的是**在有限预算和有限权威下做决策的依据**。

先说清楚一件事：这不是一本让人兴奋的书。企业 AI 落地的真实过程，坦白说，大部分时候是憋屈的、憋屈的、慢的、经常令人沮丧的。但只有把这层滤镜去掉，你才能看见真正决定成败的那几件事。

---

1. 真相一：技术不是瓶颈，数据是入场券

现象

一个常见的场景是这样的：企业上了大模型，买了算力，搭了平台，做了个知识问答助手，上线仪式很风光。然后业务部门用了两次，发现答非所问，就再也不用了。

而技术团队的第一反应是：模型不够强，换个更强的模型试试。

为什么会这样

因为在 2023 到 2026 这几年，「模型能力」是整个市场最响亮的叙事。发布会、跑分、榜单、多模态、推理能力——每一次迭代都有明确的、可量化、可截图传播的进步。相比之下，「数据」这件事既不性感也不能立刻演示。

但只要把 AI 落地拆开看，结论是绕不开的：

```
当 AI 落地企业时，真正有价值的是四样东西：

  ① 数据      ← 一切的基础
  ② 流程      ← 最终也是数据
  ③ 知识      ← 最终也是数据
  ④ 规则      ← 最终也是数据

关键洞察：不管怎么分类，流程、知识、规则
最终在数字化系统里，都是「数据」的形式呈现
```

AI 转型要回答的三个问题：

```
问题 1：数据是什么？
问题 2：关联关系是什么？
问题 3：规则在哪里？

结论：
如果没有数据治理
连数据都说不清
AI 转型就是空谈
```

请注意这三个问题的顺序。它们不是三个并列的技术问题，是一条**因果链**：你没法在不知道「数据是什么」的情况下回答「关联关系是什么」；也没法在不知道「关联关系是什么」的情况下指望模型替你找到「规则」。

这不是 AI 时代的新问题。这是企业信息化二十年来的老问题——只不过以前叫「数据标准不统一」「主数据不一致」「报表对不上账」，现在换了个名字叫「AI 效果不好」。

怎么识别自己在坑里

三个自测问题，任何一个答不上来，你大概率就站在坑边上了：

① 你能不能用一句话说清，这个 AI 项目要吃的是哪几份数据？

如果答案是「各个系统的数据」——你不知道。你需要精确到：哪个系统、哪张表、哪个字段、多少数据量、多久更新一次。

② 这些数据有没有主人？

不是「谁在维护」这种笼统的说法，而是：当这份数据的口径被质疑时，**谁有权拍板说「以我为准」**。一份没有主人的数据，喂给模型就是在赌。

③ 业务方说「我们要 AI 帮我们做预测」，你能问出「预测什么、依据什么、错了谁负责」吗？

如果对方只想要一个「智能」的结果，不想要这三个答案，那这不是一个能落地的需求。

正确做法

不是「先做数据治理再上 AI」——这句话太绝对，而且会让很多企业直接放弃。正确的说法是：

只治理 AI 马上要吃的那一口。

不要为 AI 治理全公司的数据，只治理 AI 马上要吃的那一口。选定了场景，就沿着这个场景的数据链路往前追：需要什么 → 从哪来 → 现在什么状态 → 谁负责 → 多久能改好。把这口「数据」治理干净，比治理一万条无关数据更有价值，也更快能拿出成果。

这一点我在《数据治理实战》里已经完整讲过（详见该书引言与第二章）——治理方法论的细节那里写得更全，本书只讲 **AI 语境下的差别**：AI 不需要「干净的数据」，它需要「说得清的数据」。

> **AI 吃进去的是数据，吐出来的是责任的具象化。**

---

2. 真相二：阻力不是技术，是政治

现象

技术方案评审通过了，预算批了，项目启动了。然后推进到一半就推不动了。

数据要不出来。业务部门说「这个数据涉密」。要人配合，部门负责人说「我们人手不够」。要改流程，流程归另一个部门管，改动要先过他们的会。半年过去，试点没跑起来。

会开了一堆， PPT 做了几十页，就是没有进展。

为什么会这样

因为大家把这件事当成了技术问题，而它其实是**权责问题**。

先看一组对比，这是我在数据治理里反复验证过的：

```
主数据治理为什么阻力不大？

因为主数据是"别人的数据"
   各部门用别人的数据
   用好了，大家都受益

比如：
   组织架构主数据 → 各部门都要用
   客户主数据      → 销售、市场、客服都要用
   供应商主数据    → 采购、财务、仓库都要用

结论：主数据治理，各部门都能受益，阻力自然不大
```

反过来，真正的阻力在哪里？

```
┌──────────────────────────────────────┐
│          一线业务数据和实际业务数据          │
│       （这些数据掌握在业务部门手里）          │
└──────────────────────────────────────┘

这些数据的特点：
  代表部门的业绩
  代表部门的责权
  代表部门的价值
  动了这些数据 = 动了部门的"蛋糕"
```

AI 项目要吃的，恰恰多是第二类数据。为什么？因为有价值的 AI 场景，几乎都长在业务的痛点上——那是业务自己的数据。而业务自己的数据，动了就是动了它的蛋糕。

所以阻力有明确的四个来源：

阻力一：部门不愿公开数据。 典型心态是「这些数据是我们部门的核心竞争力，一旦公开，其他部门都能看到我们的业绩，那我们还有什么话语权」。深层是：数据透明 = 业绩透明 = 压力透明。某些部门宁愿数据不准，也不愿透明。

阻力二：数据/IT 部门不愿放权。 「如果数据都归业务部门管，我们数据管理部门的价值在哪里？」担心职能被架空、价值被边缘化。这种心态不只是业务部门有， IT 部门自己也有。

阻力三：工作方式被迫改变。 很多部门习惯用 Excel 管数据、习惯手工台账、习惯「我说了算」的数据。改用系统的代价是：要学新东西、要守新标准、要放弃旧习惯。这是最不政治、也最真实的阻力。

阻力四：责权博弈与再分配。 AI 的本质是**重分配**。谁原来做这件事，现在 AI 做了，那个人干什么？谁原来签字担责，现在 AI 出了错谁担？——**AI 不是解放工具，是权责重分配工具。** 所有绕不开的责任问题，都会变成阻力。

怎么识别自己在坑里

① 你的项目要的数据，是不是「某个部门的核心业绩数据」？

是的话，阻力已经写在数据属性里了。不要以为是沟通问题。

② 你的立项方案里，有没有「业务部门在项目里承担什么」这一节？

如果只有 IT 部门在干活，业务只在最后验收——那不是协同，那是让业务替你的项目背锅。业务一旦意识到这一点，就会用最省力的方式让它黄掉。

③ 你有没有算过：这个 AI 上了以后，谁的岗位/考核会变？

这个问题如果连你都不敢在方案里写，那阻力一定会在中途冒出来，而且会以「技术问题」的面貌出现。

正确做法

先做「能让人少干活的」，不做「能让人被考核的」。

第一个 AI 场景，务必选业务部门**自己都嫌烦**的事——那种没人愿意干、干了也没人夸、但确实消耗人力的活。这类场景里，业务的收益是即时的、个人的、不涉及谁被替代，所以阻力最小。

同时，把使用者变成标准共建者，而不是被规范的对象。培训会开完没人提问，往往不是因为满意，而是因为提问等于承认自己干不好。

> **员工不抵触 AI，抵触的是用 AI 之后自己没价值了。**

一个可以立刻用的判据

如果你要在几个候选场景里挑，可以用一个非常朴素的判断：

这个场景要吃的是「自己的数据」还是「别人的数据」？

- 用**自己的数据**补自己的短板 → 阻力大（动了自己的蛋糕）
- 用**别人的数据**补自己的短板 → 阻力小（大家都受益）

这就是为什么同一个企业内部，做「跨部门信息汇总」类助手通常比做「本部门数据深度分析」更容易推起来——虽然直觉上前者听起来更复杂。

---

3. 真相三：没人算过账，所以项目没法活过第一年

现象

项目做完了，验收的时候很顺利。演示很漂亮，指标很好看。

然后就没有然后了。

没有进入常态化运营，没有算过这笔投入值不值，第二年预算收紧的时候，第一个被砍掉的就是它。半年后有人问「我们不是有 AI 吗」，谁也说不清那个东西现在还在不在用。

为什么会这样

因为 AI 项目的成本结构，和一般的信息化项目不一样。

一般的信息化项目，成本是**可见的、一次性的、可预测的**：买软件、招外包、三个月上线。 AI 项目的成本有一大块是**隐性且持续的**：

```
AI 项目的成本（不完整版）：

  ① 算力 / 部署          ← 私有化 or API
  ② 模型调用与微调       ← 用得越多越贵
  ③ 数据治理前置投入     ← 最常被漏算，也最贵
  ④ 人员与培训           ← 贵，且不体现在预算表里
  ⑤ 错误处理与兜底人工   ← 模型必然会错，这笔账没人算
```

第③和第⑤两项，是绝大多数 AI 项目在立项时完全没有的。

第③项容易被理解：数据本来就脏，把数据治理做干净，这是一笔前置投入。这笔钱花得值不值，要看没有它 AI 的效果会不会归零。

第⑤项几乎总是被漏掉：**模型会出错，而出错的处理成本是持续的**。 一个准确率 85% 的助手，剩下 15% 的错误谁来兜？这些错误会进入流程、进入文档、进入客户的感知里。你不去算这笔账，它就以「用户不信任这个工具」的形式收场。

怎么识别自己在坑里

① 你的立项文档里，有没有「什么都不做的现状基线」？

这是最要命的一条。没有基线，就没有对照，就没有 ROI 。所有人都在说「引入了 AI 之后效率提升明显」，但「明显」是相对什么？没有基线，提升就无从谈起。

② 你的收益侧写的是「人时节省」还是「人的感受」？

「体验更好」「效率提升」「智能化水平提高」——这些都不是指标。指标必须是：某个流程的周期从 X 缩到 Y ，某类错误从 N 件降到 M 件。

③ 你的项目有「运营负责人」吗？上线之后谁维护提示词、修数据、盯效果、处理投诉？

如果没有，说明这个项目在你心里是个一次性交付物。而它实际上是一个**需要长期供养的系统**。

正确做法

立项的时候就把账算出来，哪怕算得粗。

收益分三类，三类都得算：

- **提效类**：人时节省。多少人、每天省多少分钟、折成年化人力。
- **决策类**：风险降低、决策提速。避免了一次什么事故、少了几轮评审。
- **质量类**：差错减少、合规事件减少。受监管制造行业里，这一类的价值经常被低估——但它往往最大。

成本侧则必须把第③项「数据治理前置投入」写进去，并且明说它是**沉没但必要**的。

> **AI 项目最贵的那一项成本，从不出现在预算表里——它叫「数据原本就不干净」。**

> **算不出账的 AI 项目，本质上是一笔由未来偿还的贷款。**

---

4. 顺序搞反了，比做错了更贵

上面三个真相合起来，指向一个更根本的问题：**大多数企业把顺序搞反了**。

常见路径（问题路径）：

```
想上 AI → 选模型 → 搭平台 → 找场景 → 补数据 → 发现效果差
                                                  ↑
                                          成本已经沉下去了
```

推荐路径：

```
找场景 → 补那一口数据 → 选技术 → 小步试点 → 算账 → 推广
```

看起来只是顺序调了一下，后果差很远。问题路径里，你在还不知道数据能不能用的时候，就已经把算力、平台、人力投进去了。等发现效果不行，损失已经发生，而且大部分是沉没成本。

一个可借用的成熟度分档

判断一家企业现在该不该上 AI、先上多少，可以套用数据治理的成熟度分档逻辑：

```
L0  数据没有主人，口径靠人脑
    → 这个阶段谈 AI 是危险的。先治数据。

L1  数据有标准有主人，但没进平台，靠人搬运
    → 仍然先治数据。可以做"内部试验"，但别承诺产出。

L2  数据基本就绪，平台可用，有运营机制
    → 可以试点，选一个业务部门自己都烦的场景。

L3  治理常态化，数据资产可发现可复用
    → 才适合规模化推广和多场景铺开。
```

这个分档的用处不在于给企业贴标签，而在于**避免在 L0/L1 阶段做 L3 的承诺**。很多项目的失败，是立项时就许了一个 L3 的收益，而组织实际还在 L1。

关于「AI 转型三问」的一个补充

前面提到的三问——数据是什么、关联关系是什么、规则在哪里——还有一个前置问题经常被跳过：

> **你不可能在没看过全量数据之前，就宣布哪些数据是重要的。**

这句话原本是讲主数据治理的：主数据不是选出来的，是摸出来的——你得先盘清家底，才知道哪几个域是真正的核心。同样的道理在 AI 场景选择上成立：**你不能先决定「我们要做智能质检」，再去想数据从哪来**。很多时候盘一遍数据域，会发现真正值钱、且技术已经够用的场景，跟你一开始想做的完全不是一回事。

> **抄别人的顺序，等于抄了别人的前提条件。**

---

5. 本书路线图

前面讲的是为什么，下面是怎么办。全书八章，结构如下：

```
第一章  企业 AI 落地的 5 个坑
        坑一 老板以为 AI 能解决一切
        坑二 技术选型盲目追新
        坑三 数据没准备好就想上 AI   ← 全书重点
        坑四 忽视员工抵触情绪
        坑五 没有 ROI 评估就上马

第二章  场景选择——别先问能做什么，先问值不值得做
        12 个场景全景 + 价值×可行性双维筛选

第三章  数据准备——把数据治理翻译成 AI 能听懂的话
        数据可用性清单 / 文档治理清单 / 反馈闭环清单

第四章  技术方案——私有化、API、RAG、微调、Agent 怎么选
        三棵决策树，不做科普

第五章  落地五步法
        场景 → 数据 → 试点 → 上线 → 运营

第六章  算账——AI 项目的成本结构与 KPI 设计
        成本五项，收益四类

第七章  合规与风险——制造企业特有的红线
        GxP/CSV、数据不出厂、审计留痕

第八章  真实案例复盘
        2-3 个案例，含至少一个失败案例

附录    7 套模板：场景评估表 / ROI 计算表 / 模型管理规范模板
        数据准备三张清单 / 风险自查清单 / 技术名词速查
```

如果只读一章，读第一章。那一章的作用是让你在坑里的时候，能自己认出来。

如果只读一章，那也读第一章。后面六章是方法，第一章是诊断。诊断错了，后面全白做。

如果你正在做一个 AI 项目的立项，先跳到第六章，把账算一遍。算不下去就先别立项。

6. 写在最后

我不打算在这本书里告诉你 AI 有多神。

模型会变强，会变得更让人惊艳，这几乎肯定会发生。但模型变强这件事，和你企业里那个 AI 项目能不能活过第一年，是两个几乎不相关的问题。

企业 AI 落地的真实战场不在模型能力上。它在三个更朴素的地方：

你选的场景对不对。 你吃的数据干不干净。 你算没算过那笔账。

这三件事， 20 年前叫信息化选型和数据治理， 20 年后叫 AI 落地。名字换了，规律没换——甚至变得更硬了，因为这次的赌注更大、期望更高、失望也更快到来。

而所有关于 AI 落地的智慧，最终都会落到一个很朴素的地方：**你必须真的懂你所在的组织，才能把它推下去。** 这一点，没有任何模型能替你做。

> **AI 项目死得最多的地方，不是模型不够强，是没人算过这笔账。**

> **本章要回答**：我是不是已经在坑里了？ **纪律声明**： ① 本稿**已写全五节**（坑一~坑五），每节统一按「现象 → 为什么会这样 → 怎么识别自己在坑里 → 正确做法 → 一句金句」五段式；坑五的定性已按作者意见由「没有 ROI 就上马」调整为「**算不清就上马**」（理由见 1.5 第二层）。 ② 企业名、集团名、子公司名、事业部代号、产品名**全部泛化**，统一用「某大型制造企业」「某制造业集团」「某集团研究院」。 ③ 涉及第三方公开报告的数字（ MIT 报告的 95%、 Gartner 的 3:1 等）均标明出处与年份，**与本书企业案例中的数字不可混算**，且部分来源为转述，需核对原文【待核实】。 ④ 涉及模型品牌、版本号与准确率的段落，**按「写机制不写品牌」的默认口径处理**，是否点名待作者拍板【待核实】。

1.0 本章要回答什么

前面引言里有一句话：技术上没有一家企业是输在模型不够强的，输的都是别的地方。

那「别的地方」具体是哪几个地方？

这一章只做一件事：把最常踩的五个坑摊开，每个坑按同一个结构写到底——

```
现象 → 为什么会这样 → 怎么识别自己在坑里 → 正确做法 → 一句金句
```

「现象」是现场画面，「为什么会这样」是机制，「怎么识别」是你能当场对照的信号，「正确做法」是能照做的动作，「一句金句」是能被你截图转发的结论。

为什么先讲坑，不先讲「AI 能做什么」？

因为**读者心里那个没说出口的问题，不是「AI 是什么」，而是「我是不是已经在坑里了」**。前者可以自己去查，后者查不到——因为坑的形状，恰恰是「看起来一切正常」的样子。

而这五个坑是有顺序的：

```
坑一  老板以为 AI 能解决一切    ←  起点错了，后面全错
坑二  技术选型盲目追新          ←  选型时把项目变成了技术债
坑三  数据没准备好就想上 AI     ←  技术选完了，发现喂不进去
坑四  忽视员工抵触情绪          ←  系统上线了，人不用
坑五  没有 ROI 评估就上马       ←  用起来了，说不清值不值
```

这五个坑不是并列的五个清单，是**一条链上的五个环节**——

```
坑一 起点错了（决策期） → 坑二 选型时变形（决策期） → 坑三 喂不进去（准备期）
     → 坑四 人不用（运营期） → 坑五 说不清（运营期）
```

坑一坑二在**决策期**，坑三在**准备期**，坑四坑五在**运营期**。前面坑错了，后面几坑会自动加固它——这条链会自我闭环。1.6 节把这条链合起来讲。

1.1 坑一：老板以为 AI 能解决一切

现象

有一类领导，对 AI 特别感兴趣、特别在乎。

他相信 AI 能解决很多问题——公司那些经营上的难题、流程上的难题、部门之间扯皮的问题，他都想试一试。按他的说法，他会到处查资料了解，希望各个部门都去推进。

于是就有了这样一幕：

> 各部门领到任务，做出来一堆演示型项目。有的部门做了个「智能问答」，有的做了个「文档生成」，有的做了个看起来很炫的看板。年底汇报，每个部门都有成果。**但没有一个项目，能说清楚自己到底替公司省下了什么**。

到了下一轮，领导的判断是： AI 没用。

注意这个结局——它不是偶然，它是必然的。

而这类领导的画像，往往还带着另一层：

> 立项文件的第一页写着「**重塑公司整体运作模式**」，到了第三章才开始写」我们要做一个合同审核助手「。

从上到下，中间隔着两个数量级的期望。

★ 而这个「数量级」的差距，往往被量化成一个具体的数字，于是变成一场无法调和的争论：

```
老板心里的那个数字：★ AI 能替代 40%~50% 的人
        ↓
而在传统企业的环境里，★ AI 实际能替代的大约是 20%~30%
        ↓
★★ 而且更关键的是：★★ ★★ 它替代的不是"岗位"，★★ ★★ 是"岗位里的一些点上"
```

★ 这两个数字之间差的那一截，会在项目第一个季度就变成冲突：

- 老板按 40%~50% 配资源、设目标；
- 团队按 20%~30% 做方案、报工作量；
- **★★ 中间的差额没人负责，★★ 最后由项目失败来买单**。

> ★ **来源**： 20%~30% 与「替代的是点上」为作者在传统企业环境下的经验判断【口径：行业通行说法，非研究结论】。

为什么会这样

第一层： AI 在直觉上像解放工具，但它在组织里其实是重分配工具

这是坑一的根。

一把手把 AI 想成了解放工具——把人从重复劳动里解放出来，把效率提上去，把成本降下来。这个想象很自然，因为 AI 的演示确实是这样：输入一段话，输出一段话，**没有人被替代，一切看起来都是增益**。

但 AI 在组织里的真实作用是**权责重分配**。

为什么必然触及权责？

AI 要替人做事，就必须先回答三件事：

① 这件事原来归谁负责？
② 出了错谁认？
③ 干得好不好，功劳算谁的？

这三个问题都不是技术问题，是权责问题。
而权责问题，在任何组织里都有既得利益者。

所以当一件事交给 AI 去做，**受益的人不是「被解放的人」，而是「原本承担了这个责任风险的人」**——因为风险转移了，但责任没跟着转移。

而一把手最初设想的收益，往往恰恰是压在这个位置上：**效率提升的成本，由一线承担；红利，由上面拿走**。

于是这个项目的结局在第一天就注定了：**执行层会用最省力的方式交差——做演示，不做落地**。

第二层：没有路径的推动，会自我实现地生产出反例

这一层更狠。因为它解释了那个「AI 无用」的结局是怎么长出来的。

热情型领导的推动路径：

到处查资料、到处讲、要求各部门推进
↓
部门为应付而做 —— 资源有限、优先级排在业务之后
↓
产出一堆演示型项目（能演示的先做，能落地的往后排）
↓
项目上线，无一产生业务价值
↓
反过来印证了领导原本的怀疑："AI 嘛，就是没用"
↓
领导更坚决反对 AI，下一轮更难推动

没有路径的推动，会自我实现地生产出反例。

所以这件事**不能被简化成「老板糊涂」**。他不糊涂，他甚至比多数人都懂 AI 的重要性。他缺的不是热情，是**路径**——而路径这个东西，讲一次是讲不出来的。

第三层：就算全员都用上了，红利也不一定兑现

这里有一个几乎所有 AI 推广大会上都不会讲的悖论：

> 经过近三年的实施观察，很多人认为**员工能力提升了、效率提升了，就等于降本增效了。但这往往是一个悖论。**

因为如果公司收入没有增加，就不可能给员工加薪。

员工用 AI 提升了效率之后，**更多的反而是空余时间变多了**。

```
全员层的 AI 效率红利，默认形态是"时间盈余"，
不是"收入增量"。

要把时间盈余变成收入增量，
只有高层层面能做这件事——
因为只有高层能重新配置资源、重新设定考核、重新定价产出。
```

这句话把坑一从「老板的认知问题」升级成了「组织的兑现机制问题」：

> **如果高层不兑现，全员层的 AI 效率就只会变成员工的空闲时间。** 而员工的空闲时间多了，通常意味着焦虑，而不是从容。

怎么识别自己在坑里

以下四条，**每一条都可以当场对照**。对上两条以上，你大概率在坑里。

信号一：立项文件里没有「不做这个 AI 我们会怎样」的基线。

这句话是全章最有用的一个判据。基线不是「我们要提效 30%」，基线是：

- 现在这份合同审核，每个月人工花多少小时？
- 出错率是多少？谁在复核？
- 如果什么都不做，一年后会是什么样？

没有基线，就没有验收标准；没有验收标准，项目就只能用「看起来很牛」来验收。

信号二：目标里出现「重塑」「赋能」「全面重构」这类词，但正文第一个可交付物是「做一个助手」。

期望和交付物之间差了两个数量级。领导会按前者验收，团队会按后者交付，然后双方都觉得对方不配合。

信号三：项目数量在涨，能说清业务价值的项目数量没涨。

部门都在报成果，但成果都是「上线了」「发布了」「参加了路演」。**问一句「它现在还在用吗」，答不上来的就是它。**

信号四：你发现自己在准备「给领导解释 AI 能做什么」的材料，而不是在准备「这个项目怎么验收」的材料。

这一条最隐蔽，也最要命。

如果你在做的第一份文档是「 AI 能力介绍」，而不是「验收口径」，你已经在坑里了——因为你的工作是让领导相信 AI 有用，而不是让项目有用。

信号五（对照类，用来识别你面对的是哪一类领导）：

还有另一类领导，判断完全不同：

> 他们对 AI 的认识比较清楚，但觉得 **AI 对传统企业的主营业务支持提升没有太大实际价值和意义**；同时觉得 **AI 投入比较大**，所以不愿意，甚至抵触。

这一类和坑一是**镜像关系**：坑一是「以为 AI 能解决一切」，这一类是「认定 AI 解决不了什么」。两者都错，但错的方向相反。

> ★ **这一类领导的方法部分，作者尚未补充**【待核实】。本稿只能指出其存在，不能给方法。**这属于素材缺口，不是可以编造的地方。**

正确做法

坑一的解法，不是「劝领导冷静」——劝不动。是从那段「热情无路径型」的素材里最要命的动作开始改。

做法一：陪跑，不摊派。

热情型领导的问题不是热情，是没有路径。**而路径不是讲一次能讲出来的，是陪着他走出来的。**

所以第一件事不是立项，是**多与领导沟通**——把你对「AI 现在到底能做什么、不能做什么」的判断，反复讲到他心里去。不是一次汇报，是持续的对话。

为什么必须持续？ 因为「AI 能解决一切」这个念头不是一次沟通能拔掉的，它是在每一次部门反馈困难时被默认接受的。你要在每一个他犹豫的节点上出现。

做法二：双轨交付——既有落地方案，也有长远规划。

只给落地方案，领导三个月后会问「然后呢」——因为他不打算只交这一件事。

只给长远规划，他会说「先做个眼前的」——三年战略他接不住，他需要今天能讲的东西。

所以要**同时给**：

轨道	内容	给谁看
近期轨	一个 30 天内能看见结果的落地动作	领导本人（他要看到进展）
远期轨	一条 3–5 年的演进规划	领导的**上级**（他要能交代）

★ 关键在第二条：这份长期规划是「为领导做的」，不是「为项目做的」。

他真正的压力不在项目，在**他上面那个人**。你要给他一张能讲清楚的时间表——什么时候能做到什么、投多少、风险在哪。这样他才敢在内部推。

（规划里放什么？按作者给的思路：解释清楚 AI 以及此次大语言模型到底能做什么。这既是技术沟通，也是路线图的基础。）**

做法三：用第三方报告代替你的说服。

这是最省力、也最有效的一招。

不要在会议室里跟领导争「AI 到底能做什么」。**去找行业分析报告、去找国际知名的论文，把结论摆给他。**

> **给领导讲道理，是权力博弈；给他看报告，是提供证据。** 前者他可以驳回，后者他没法驳回——因为那是他自己转发的。

一套可以直接推过去的「三件套」（1 页纸）：

1. **一份泼冷水的研究报告。** 有一份 2025 年 7 月由 MIT Media Lab 发布的《The GenAI Divide: State of AI in Business 2025》，结论是：在 300–400 亿美元的企业生成式 AI 投入下，**95% 的试点没有产生可衡量的财务回报**，只有 5% 产生了可衡量价值；并指出自建方案的失败率约为采购方案的两倍，主要差距在于「不知道怎么设计能真正捕获收益的工作流」。【出处： MIT Media Lab ， 2025 ； 转述口径，需核对原文，见 1.4 待核实】

> ★ 这份报告适合推给领导，恰恰因为它**不是唱衰 AI**，而是泼完冷水之后给出了那 5% 的路径。领导看完不会得出「AI 没用」，会问「那我们怎么进那 5%」——**这正是你要的对话方向。**

1. **两段学术结论。** 一份关于 Agentic RAG 的国际综述里有两句极硬的结论：「Agent 的推理能力弥补不了稳定的检索质量」，以及 「Agent 的自主性必须配显式约束」。【arXiv:2501.09136 】

> 翻译给领导：**Agent 越强，越掩盖底层数据的烂账。** 项目里演示得越顺，数据问题暴露得越晚——而那时，它已经在生产上了。

1. **一个反直觉的行业判断。** Gartner 预测，到 2027 年，**任务专用小模型在企业部署中将以 3:1 超过通用前沿模型——「专业化，而不是更大」才是方向。**【Gartner ， 2025 ； 转述口径，需核对原文】

> 这一条对预算和自主可控都极有利：**你不需要追最大的模型。**

★ **顺带说一句这份报告里最容易被忽略、但对你这本书极有用的一条**：报告发现，员工正在用**未经雇主批准的消费级工具**干自己的工作——他们管这叫「影子 AI 经济」。

> 这条对第四章（员工抵触）和第七章（合规红线）都是一手外部证据：**员工不是抵触 AI，员工已经在用了，只是公司不知道。**

（这三份材料我已整理成一份可直接转发的清单，含出处、适用场合和三条引用纪律。）

做法四：把「能做什么」和「不能做什么」一起讲。

只讲能做什么，他的期望就只会往上走。

讲清楚边界——**会出错、需要有人复核、敏感数据能不能出厂、做不到什么**——他才会给出你能交付的范围。

> **边界不是限制，是保护项目。** 边界之内，他给你资源；边界之外，他会替你把项目送死。

做法五：第一个 AI 项目，选「业务部门自己都烦」的事。

这是全章最实操的一条。

不选「一把手最想要的」，选「业务部门自己也觉得烦」的。

为什么？两个理由。

其一，一把手最想要的事，通常是最难的事：它跨部门、动权责、数据最脏。这些坑你在第一个项目里一个都躲不掉。

其二，也是更重要的——**「业务部门自己都烦」这七个字，决定了项目的验收人是谁。**

```
第一个 AI 项目的选人三个维度：

  信任成本低    ×    决策权高    ×    业务痛感强
          （三者缺一不可）

  只有信任没有决策权 → 做出来，但推不动
  只有决策权没有信任 → 拿不到真实使用数据
  只有信任和决策权，没有痛感 → 变成"给高管的玩具"
```

★ 这一条在企业里的实际含义是：**第一个项目不要交给一把手自己，要交给一个「信任成本低、决策权高、天天被这件事烦」的业务负责人**。

他烦，所以他会用；他有决策权，所以他能推动；他信任你，所以他愿意说真话——**包括说「这东西不好用」**。

一句金句

> **AI 不是解放工具，是权责重分配工具。**
>
> **而所有没有路径的推动，都会自我实现地生产出反例。**

1.2 坑二：技术选型盲目追新

现象

发布会一场接一场。

新模型出来了，跑分又涨了，官方说「在某某评测上超过上一代」。群里一阵骚动，有人已经在问「我们那个项目是不是该换一下」。

于是项目换了一代模型。过两个月，又一代出来了，再换。

模型换了三代，项目还在原地。

不是没进展——每个季度的确实验证了一些新能力。但当初立项时承诺的那个业务问题，**一直没有被解决**。而且每次换代，都要付出一次代价：改配置、重新调提示词、重新跑评估、重新验证一遍。

为什么会这样

第一层：把「选型」当成了技术能力竞赛

这是坑二的根。

在一个技术团队内部，「我们用的模型是不是最好的」是一个**极其诱人的话题**。它可量化、可排名、可在技术会上拿来比较——比「我们这个项目到底有没有替公司省下钱」好用太多了。

于是选型从「解决一个业务问题」滑向「证明我们技术团队很先进」。

而企业 AI 的选型标准，恰恰和这个竞赛的评分标准是相反的：

```
企业 AI 的选型标准只有三条：

  运维得起吗？    —— 谁来训练、谁来盯着、谁来兜底
  查得到吗？      —— 审计留痕、模型台账、出处可追溯
  赔得起吗？      —— 换代成本、数据迁移成本、错误处理成本
```

三条里没有一条是「它是不是最强」。

第二层：AI 系统是企业里唯一一类生命周期极不稳定的东西

这是坑二最容易被低估的技术事实，出自某大型制造集团的智数化顶层规划原文：

> **常规系统：生命周期较稳定，版本迭代由业务功能驱动。AI 系统：生命周期依赖数据和算法，需要持续更新数据集和模型，否则性能会衰减。** AI 系统的核心是模型，需进行模型选型、训练、评估、上线等 MLOps 流程；上线后也需不断迭代优化模型性能。

把这句话翻译成选型语言，就是：

> **你换的不是一台机器的零件，你换的是整条依赖链。**

因为模型一换，跟着一起变的是：

```
换模型 = 换提示词 + 换评估集 + 换工作流 + 换知识库结构
       + 重新做安全评估 + 重新过一遍审批
       + 重新积累的踩坑经验（对新版本无效）
```

这才是「追新」的真实成本——它不是模型下载那点时间，是把你已经稳定下来的那一整套东西重新变成不确定的。

第三层：真实的代价长什么样

我们在某大型制造企业的一个合同审核项目里，看过一次完整的「换代代价」。

> 那个项目的基础模型选型，最终结论是：选一个**开源的 32B 参数量模型**作为合同审核的基础模型，另有一个推理模型作为差异定位的专项模块补充。选型理由是**开源性、算力适配性及长上下文能力——没有一条是「业界最先进」**。
>
> 然后事情开始发生：
>
> - 上线评估阶段，**该模型的旧版本出现了准确率评估流程无限挂起的问题**，最终是通过修改模型配置文件、集成第三方推理框架的最新提交才解决。
> - 在一个偏学术推理的评测场景上，**该 32B 模型的准确率为 59.5%**，而对比模型为 **71%**。
> - 方案里同时写着，计划在下一代 64B 模型中引入新的架构，目标是在同等参数量级上追平对比模型 90% 的推理性能。

> ★ **口径说明**：上述内容来自该项目的**实施方案文档**，属第三方方案设计口径，非本方实测结果；模型品牌与版本号按「写机制不写品牌」的默认口径处理，**是否写明品牌、准确率数字能否保留，等待确认**【待核实】。

你看清楚这里发生了什么：

一次「追新」，代价是**改配置、集第三方最新提交、重跑评估**；不追新的代价是**能力差距 11.5 个百分点**。

而这两个选择，**在选型那一刻根本看不出来**——因为你无法预知下一个版本会不会让你的评估流程挂起，也无法预知下一代会补上多少差距。

这就是坑二最本质的困境：追新的成本是确定的（你每次都要重做一遍），不追新的成本是不确定的（你不知道差距会拉大多少）。而人在不确定面前，天生偏向选择那个「看起来更先进」的选项。

第四层：选型的另一种死法——永远停在选型

选型不只死于「追」，也死于「散」和「拖」。

在某大型集团的一个统一知识库项目上， 2024 年 10 月立项，计划 2025 年 12 月交付。这个项目的选型状态是这样的：

> 一边是「现尝试云端（多家厂商的云知识管理平台）」，另一边是「**搭建至少 4 个开源知识管理平台还在选型阶段**」。

12 个月的计划，交付的时候还在选型。

而且注意它的形态：一边在试云，一边在试 4 个开源——**这不是选型，这是发散**。4 个候选同时评估，每个都投一点人力，每个都做不深，最后哪个都定不下来。

怎么识别自己在坑里

信号一：选型理由里出现「业界最先进」，但说不出「能撑几年不换」。

这是大纲里给坑二定的识别信号，也是最锋利的一条。

因为「业界最先进」这个理由有一个致命特征：**它没有有效期**。

- 如果理由是「开源的 32B 模型，显存够、长上下文够、评估流程跑得通」——这个理由**可以被检验**，也可以在未来某天被推翻，但推翻时你知道推翻的是什么。
- 如果理由是「业界最先进」——它**今天成立，明天就不成立**，而且你无法用它做任何决策。

> ★ **一个可操作的自查**：把选型文档里「为什么选它」的那段话读一遍，逐条问自己——**这条理由如果反过来（我们不选它），会损失什么？** 答不上来的那一条，就是「业界最先进」。

信号二：选型文档里没有「我们为什么不用另外三个」这一段。

这是我从上面那个「4 个平台还在选型」的真实事项里提炼的候选判据【待核实，待作者确认是否入书】。

一份真正的选型文档，必然包含**落选分析**——另外三个候选各自好在哪、为什么这次不选它。

没有落选分析，说明这份文档没有做过决策，它只是一份采购意向书。

信号三：文档里找不到「如果换模型，要重做哪几样」。

一份成熟的选型文档应该有一节叫「可替换性」，写清楚：换模型要重做提示词吗？要重做评估集吗？工作流要重配吗？知识库结构要改吗？

如果这一节是空的，那说明你们默认了「不会换」——而这恰恰是最危险的假设，因为它把一个必然会发生的动作，写成了不会发生的意外。

信号四：模型升级这件事，是在技术例会上决定的，不是在选型文档里决定的。

正常顺序是：先有业务需求 → 再有选型文档（讲清楚为什么这个模型适合这个需求）→ 再有升级机制（什么条件下升级、升级要做什么）。

如果变成「新模型出来了 → 技术团队决定升 → 顺带适配一下业务」，那你已经在坑里了。

正确做法

做法一：先定「不换的」，再选型。

这是大纲给坑二定的正确做法，也是全章最反直觉的一条：

> **先定「不换的」——数据在哪、审计怎么留、模型怎么替换，再选型。**

顺序反了，就是追新；顺序对了，追新的成本就被提前算清了。

具体要落三个东西：

（1）数据在哪。

模型可以换，数据不能轻易换。**数据在哪、谁能访问、出了厂怎么审计**——这几件事定了，模型就是个可替换件。数据没定，模型就永远是个「不能动的东西」，因为一动就要重做数据链路。

（2）审计怎么留。

每一次模型调用、每一次输出、每一个引用出处，都要有痕迹。这一条在受监管制造企业是硬要求，在别的行业是后来补的 painful lesson 。

审计留痕的真正意义不是「合规」，是「你敢换」——如果你连三个月前的输出都追不回来，你就永远不敢换任何东西，因为换不了就等于赌。

（3）模型怎么替换。

把它当成一个明确的技术动作来设计，而不是当成一个「应该不会发生的事」：

模型替换预案（要写在选型文档里，不是写在脑子里）

- □ 模型在系统中的边界在哪（哪些逻辑在模型里，哪些在模型外）
- □ 提示词存放在哪（能改还是写在代码里）
- □ 评估集存放在哪（能不能在换模型后重跑）
- □ 接口契约是什么（换一个模型，改动面有多大）
- □ 谁批准替换（流程上的一环，不是技术判断）

★ 这五个框填不满，**就不要开始选型**。因为它们填不满意味着：你换一次模型的代价，你算不出来。

做法二：用三条标准替代「哪个更强」。

回到那三条标准，实际操作时可以变成三个必须写进文档的问题：

问题	具体要回答	对应什么
运维得起吗?	谁训、谁盯、谁兜底、多久巡检一次	第四章（技术方案）、第五章（运营机制）
查得到吗?	审计留痕、模型台账、出处可追溯	第七章（合规与风险）
赔得起吗?	换代成本、迁移成本、错误处理成本	第六章（算账）

★ **一个来自行业侧的佐证**：有研究机构对 1200 个生产环境部署做过分析，结论是——**把一个系统从 80% 质量水平推到 95% 质量阈值，所做的几乎全是基础设施和数据的工作，不是模型的工作。**【ZenML 研究，经第三方转述，见 1.4 待核实】

翻译成选型语言就是：

> **决定你这个系统好不好用的，主要不是模型，是模型周围那些你懒得做的活。**

这条也顺带回答了「要不要追新」：**如果你连数据和检索都没做扎实，换模型带来的收益，会被数据问题吃掉。**

做法三：把「追新的冲动」改成「有节奏的替换」。

不是不换，是**换要有触发条件**。

建议在选型文档里写死升级的触发条件，例如：

> 允许升级的条件（满足任一）
>
> - □ 现有模型无法支撑上线场景，且已确认不是数据/检索问题
> - □ 现有模型出现无法通过配置修复的稳定性问题
> - □ 单位调用成本超过预算线，且有可验证的替代方案
> - □ 合规或安全要求发生变化
>
> 不构成升级理由的情况
>
> - ✗ 新模型跑分更高
> - ✗ 同行换了
> - ✗ 团队想试试

★ **把「跑分更高」明确写进「不构成理由」，是这一节最有用的一句话**。因为跑分是唯一一种**每次都在变**的理由；不把它排除掉，它就会反复赢。

★ **同时要写清「升级要做什么」**——这是把追新的成本提前摆上桌：

> 每次升级的固定动作（写进流程）
>
> 1. 重跑评估集（评估集必须与模型解耦、独立存放）
> 2. 重做安全与合规检查
> 3. 抽样人工复核输出质量
> 4. 更新模型台账与文档
> 5. 灰度发布，先小流量

把这五步变成流程之后，「换模型」从一个刺激的冲动，变成一个例行的、低情绪的运维动作。

做法四：选型要留落选分析。

回到「4 个平台还在选型」那个教训。

一份选型文档，必须包含落选分析。至少三个候选，每个写清：优势是什么、劣势是什么、这次为什么没选它。

好处有两个：

1. **当场就能筛掉「选型发散」**——如果四个候选你没有落选分析，那就是还没开始选。
2. **未来有人问「为什么不选那个更好的」时，你有答案**——这在评审会上是能救命的。

★ **顺带一个可自检的问题**：如果你的选型文档里，候选是「腾讯、阿里等众多厂家的云知识管理平台」这种写法，那不是选型，那是**在列可以打勾的清单**。

一句金句

> **模型可以换一年换一次，企业 IT 的债要还十年。**
>
> **你换的不是模型，是你三年后还在维护的那套提示词、评估集和工作流。**

1.3 坑三：数据没准备好就想上 AI

现象

有一类项目，节奏看起来特别顺：

```
立项会开了 → 方案写完了 → 模型选好了 → 团队搭起来了
                                                ↓
                                        开始接数据，停了
```

停下来的那一刻通常不是技术问题，而是一句很朴素的话：

> **「这个数在哪个系统里？」**

然后你会发现，要做一个销售预测场景，需要的数据散在三**个内部系统**里，各自有**两套不同的数据周期**，还缺一块**只能从外部拿**的第三方数据。

> 这是某大型制造企业内部真实做过的一份「场景所需数据盘点表」里的样子：一个场景，三个内部数据源（BIS 的发货明细表、 BI 的终端纯销表、 BIS 的经销商进销存报表），数据周期分别是 2023–2025 与 2024–2025 两段，此外还有外部数据源。**表格的表头列是：数据描述、数据来源、数据周期、核心字段、数据量、数据质量。**
>
> 换句话说，**那张表一半的篇幅是在问「数据够不够、能不能用」**。

数据没准备好，不是「数据还没清洗完」。

数据没准备好是：你要的那个数字，现在没有唯一算法。

为什么会这样

第一层：数据的问题，从来不是「字段有没有」，而是「这个数字怎么算」

同一个指标在不同地方叫不同的名字、算不同的口径，是所有数据项目的起点：

```
同一个"金额"：
  OA 表单里叫"金额"
  合同附件里叫"合计"
                ↓
      两个数都对，但它们不能相加、不能比较、不能做趋势

同一笔业务，三套口径：
  纯销口径  /  发货口径  /  达成率口径
                ↓
      领导问"这个月怎么样"，三个部门给三个答案
```

这就是为什么「数据没准备好」的第一层机制是：数据的定义权不在用数据的人手里，也不在管数据的人手里。

而在 AI 场景里，这件事的代价被放大了——

> 报表上口径不一致，最坏结果是两张表的数对不上。**模型上口径不一致，最坏结果是它把两套数一起学了，然后自信地给你一个第三个数。**

而且这里有个关键区别：

```
报表场景：人看到数不对，会去查、会去问、会发现
AI 场景  ：数是"合理"地错出来的
              没有人会去质疑一个看起来很专业的输出
```

口径问题的杀伤力，不在于它错，而在于它错得很合理。

第二层：数据的「时间轴」是断的

很多场景要的是趋势、是预测、是「和去年同期比」。但实际拿到的历史有多长？

> 某大型制造企业内部数据治理的盘点结论是：**可用数据不超过 3 年，最短的只有 1.5 年**。

```
一个需要看 3 年趋势的场景，
手里只有 1.5 年数据
        ↓
模型能做的，只是"把过去 1.5 年往前延"
而这 1.5 年里可能刚好包含一次口径调整、一次组织调整
        ↓
它学到的"规律"，有一半是那两次调整造成的
```

这就是「数据没准备好」的第二层机制：不是没有数据，是数据的这段历史不能支撑你要的那个结论。

第三层：数据不在一个地方，也不在一个责任人手里

> 某大型制造企业销项系统的数据库清单里，**数十张表分属多个系统**。而预算系统与财务共享系统的数据并不打通。

于是每一次建模，取数过程长这样：

```
需要"客户维度" → 客户主数据在 A 系统
需要"交易事实" → 在 B 系统
需要"组织口径" → 在 C 系统（且 C 的组织结构和 A 的不一致）
        ↓
中间要做关联、映射、去重、对齐
        ↓
每一步都是一个"这次的数怎么和上次不一样"的可能来源
```

第三层机制：数据没准备好，还包括「**没有一个地方能说清全部的数据**」。

第四层：真正要的不是「很多数据」，是「被治理过的关键指标」

这一层最容易被跳过。

> 某大型制造企业做高管 AI 助手时，前提被写得很清楚：**必须有统一的流程系统、统一的数据平台，以及数据治理——至少针对高层关心的指标进行了完整治理以后，这个智能体才能推出，才有作用。**
>
> 而同样的材料里还有一句更狠的对比：
>
> 「收邮件、爬网、做提醒」这三件事**不依赖公司数据**，所以**任何一家公司做出来都长一样，也都没什么价值**；而「高管要的」——经营指标、预算与达成、合同与合规、制度的适用范围、会议的结论与承诺——**全部依赖公司内部数据的完整与一致**。

这第四层机制，是整个坑三最值钱的一句：

```
很多企业做 AI 项目时，
把"做一个好用的 AI"理解成了"给它更多数据"。

正确的关系是反过来的：
  不是"数据越多，AI 越有用"
  而是"对的那几个指标治理好了，AI 才有价值"
```

一个不能忽略的背景数字

如果读者觉得「数据没准备好」只是个别项目的问题，那看看另一个数字：

> 某大型制造企业做过一次覆盖 **16 个部门的访谈、 12 个业务单元逐一体检**的 AI 能力成熟度自评，结果是：**全集团 AI 能力成熟度处于起步水平，属 L1–L2 级**（ L1 简单探索 → L2 AI 应用 → L3 AI 标准化 → L4 领域级 → L5 领先）。

起步水平（满分 5 分），意味着这个企业绝大多数部门停在「探索」和「应用」，还没有到「标准化」。

> ⚠◆ **【待核实】** 该得分为该集团**内部自评版**结果，不代表行业普遍水平；是否在书中保留具体分值，待作者确认。

怎么识别自己在坑里

以下五条，**每一条都可以当场对照**。对上三条以上，你大概率在坑里。

信号一：你的场景需要「历史」，但你能拿到的最短数据只有一两年。

问数据同事一句话就能验证。对方如果回答「再往前我这边没有」——这就是坑三。

信号二：立项文件里，「数据准备」这一行的时间是零。

```
项目排期表：
  模型部署      ████████ 2周
  接口开发      ██████ 1.5周
  效果调优      ████████ 2周
  数据准备      （空）
```

数据准备这一格空着，通常不是「数据已经准备好了」，是「还没人去问」。

信号三：你的方案里有一整节在讲「AI 能做什么」，却没有一节在讲「这个数字从哪来、怎么算、谁来维护」。

这是最好用的一条，因为它不需要问任何人，看文档就能发现。

信号四：你要的核心指标，在不同部门那里有不同的名字或不同的算法。

不用做全量盘点，**挑一个你最在意的指标，找三个部门问一遍**，问题就出来了。

信号五：你的项目里，有一部分能力是「任何公司做出来都长一样的」——收邮件、爬网、做提醒。

> 这一条不是否定它，而是要把它**从「AI 项目」里拿出来**。它可以是工具，但它不该承担「这个项目创造了什么价值」的举证责任。

正确做法

坑三的解法不是「把数据治理好」——那是十年期工程。这一节要的是**能在这个季度就开始做的四件事**。

做法一：先做指标级治理，不做全量治理。

> 前提是**统一的流程系统、统一的数据平台，以及数据治理——至少针对高层关心的指标进行了完整治理以后**，这个智能体才能推出，才有作用。

「至少」这两个字是全章最实用的一个词。

```
全量治理：全公司的数据、全部口径、三年以上历史
          → 做不完，永远做不完，AI 项目等不到那一天

指标级治理：先把高层/业务最关心的那几个指标治到位
            （口径 / 来源 / 计算 / 历史 / 权限 五个字段）
          → 三个月能出一批，这个季度就能用上
```

指标级治理的五个字段，抄下来就能用：口径（有没有唯一算法）、来源（是不是唯一权威系统）、计算（有没有文档能复算）、历史（够不够长）、权限（谁能看、谁能改）。

做法二：给每个场景做一张「数据体检表」。

不用发明新东西，**照着那份真实的盘点表结构做**：

体检项	要问出来的东西
数据描述	需要哪些数据、什么粒度、要哪些核心字段
数据来源	在哪个系统（写系统代号，不写厂商名）
数据周期	有多少历史、最短的一段有多长
数据量	行数级别、能不能跑
数据质量	有没有缺失、重复、口径断层
外部依赖	有没有必须外购或外接的数据

这张表的作用不是判断「能不能做」，是判断「这个场景现在值不值得做「。

很多场景在这一步就被诚实地否掉了——**而这是好事**，因为否掉它省下的是整个项目。

做法三：把「数据不具备」写进立项文件，而不是等评审时才发现。

```
写法一（错的）：
  "本项目将实现 XX 智能分析功能，覆盖 XX 场景"

写法二（对的）：
  "本项目需要 YYY 指标（口径待定，来源：Z 系统，历史 2 年）
   该指标目前在三处口径不一致，需先完成口径统一，
   预计 4 周；若口径统一不成立，本项目降级为原型验证。"
```

写法二看起来是在给项目设限，实际上是在给项目上保险。因为它把「项目失败」的原因从「团队不给力」改成了「数据前提不成立」——**这两者在组织里的后果完全不同**。

做法四：把「依赖公司数据」和「不依赖公司数据」的能力分开立项。

```
不依赖公司数据的：收邮件、爬网、做提醒、通用写作、通用问答
    → 定位：工具。可以快做，可以采购，不要指望它产生业务价值

依赖公司数据的：经营指标、预算达成、合同合规、制度适用、会议结论
    → 定位：能力。要先做指标级治理，慢做，但这是真价值
```

这个区分能帮企业省下一大笔钱：因为大部分被否决的「AI 项目」，其实是第一类被当成了第二类来汇报。

一句金句

> **AI 的价值曲线，不随模型能力上升，只随数据治理程度上升。**
>
> **数据准备不是项目的前置条件——数据准备就是项目本身。**

1.4 坑四：忽视员工抵触

现象

系统上线了，演示也做完了，然后：

> **没有人用。**

不是不会用，是**有别的更省事的办法**。

一线继续用 Excel 填那张表，业务方在群里问「这个能不能不用 AI 做」，管理员问「能不能把权限再放开一点」，而**真正该用的那部分场景，试点三个月后使用量只剩一个人——那个做演示的人**。

而项目复盘会上，最常说的一句是：

> **「大家积极性不高。」**

为什么会这样

第一层：抵制的不是 AI，是 AI 带来的四种变化

在数据治理里，有一个被验证过的「阻力四来源」。把它改写成 AI 语境，**四层全都要换主语**：

```
治理时代（阻力四来源）          →    AI 时代（抵触的四层）

① 部门不愿公开数据              →    部门不愿"喂"数据
   数据透明 = 业绩透明                 不愿把自己的业务数据交给模型去学
   宁愿数据不准，也不愿透明            抵触理由从"话语权"变成"被替代"

② 数据管理部门不愿放权          →    IT/数据部门不愿放权
   担心管理职能被架空                  担心职能被架空 + 担心业务绕过他们自己写工具

③ 工作方式的改变                →    工作方式被迫改变
   学新系统、遵守新标准                从"我说了算"变成"模型说了算"

④ 责权的博弈与再分配            →    谁为模型的错负责
   数据归谁管                          模型给错了，责任归谁
```

注意第①层的变化：治理时代大家守的是「数据」，AI 时代大家守的是「判断权」。

数据公开了，业绩被看穿，只是难看。**判断权交出去了，是位置没了。**

第二层：效率红利没有兑现，抵触就有收益

这一层是整个坑四里最现实、也最常被忽略的机制：

> **经过近三年的实施观察，很多人认为员工能力提升了、效率提升了，就等于降本增效了。但——这往往是一个悖论。**

```
员工用 AI 提效 30%
        ↓
公司收入没有增加 → 不会因此给员工加薪
        ↓
员工的实际获得：空闲时间多了 30%
        ↓
而空闲时间多了，通常意味着焦虑，而不是从容
        ↓
结论：对员工而言，抵触 AI 是有收益的
      —— 抵触最省事，配合最少，出错最少
```

所以「员工积极性不高」这个判断，几乎总是错的诊断。

它把一个**红利分配问题**，误诊成了**态度问题**。而态度问题是没有解法的——你不能靠动员让人更愿意干活。

第三层：工作方式的改变，是四层里最重的一层。

前三层还能谈，第四层（责权）还能设计，但第三层是躲不掉的：

> 过去一个分析师做完月度分析，产出是一张表，**表是他做的，逻辑是他想的，专业性体现在「他知道该看什么」**。
>
> 现在产出可能是一段模型生成的结论，**他要做的是判断这段结论对不对**。
>
> 而「判断模型对不对」，比「自己算一遍」更难，也更没有成就感、更没有话语权。

第三层机制：AI 拿走的不只是工作量，还有「我是内行」这件事本身。

第四层：如果没人回答「模型错了谁负责」，抵触就永远有土壤

这一层是本节最实操的部分：

```
同一个模型错误，两种业务里代价差一个量级：

场景 A：内部办公助手，给错了 → 返工，多花一小时
场景 B：质量预测，参与放行判断 → 一批产品的质量事故
```

如果一个场景里「模型错了谁负责」没有明确答案，那么：

> **一线员工会用脚投票——因为承担后果的是他们**。

这不是觉悟问题，是**理性反应**。所以「提高员工积极性」这种话，在责任边界没定之前，是没有着力点的。

1.4.1 ★★ 抵触不是均匀分布的：它集中在文书岗位，而不在核心岗位

★ 这一节要说的事情很反直觉，而且它有直接的操作后果。

现象

先看两个观察，这两条都来自作者的亲身经历：

```
★★ 观察一：★★ 抵触的两个来源很明确
   · 不理解 AI 大模型能做什么
   · 担心自己的工作被替代

★★ 观察二：★★ 但这两个来源在岗位上的分布，和直觉相反
   · 核心审核岗位（合规 / 法务 / 财务）★ 通常不会反对，
     ★ ★★ 也不会觉得"AI 代替了我的工作"
   · 抵触主要集中在文书类工作的岗位 ★ —— ★★ 因为这类岗位 AI 替代性较高
   · 其他岗位则没有那么多顾虑
```

为什么会这样

★ 核心结论只有一句：

> **★ 抵触强度，与「该岗位的 AI 替代性」成正比；★ 而与「该岗位的业务重要性」无关——★ 甚至常常相反。**

```
   岗位 AI 替代性高  ——→  ★ 员工担心被替换  ——→  ★ 抵触强
   岗位 AI 替代性低  ——→  ★ 员工不怕被替换  ——→  ★ 抵触弱
         ↓
   ★★ 而替代性低的岗位，★★ 往往正是业务的核心岗位（合规 / 法务 / 财务）
```

★★ 而这背后的机制其实很清楚：

```
★★ ★★ 核心岗位的价值不在"做"，★★ ★★ 在"判"和"担"

   AI 可以起草一份合同 →  但不能签字
   AI 可以提示风险点 →  但不能承担法律责任
   AI 可以把对账差异跑出来 →  但不能解释这笔差异该怎么处理
         ↓
★★ ★★ 而"签字"和"担责"，★★ ★★ 恰恰是 AI 目前最不擅长的两件事
         ↓
★★ ★★ ★★ 所以这些岗位的人，★★ ★★ 并不觉得 AI 会抢走自己的位置
```

★ 换个说法：人抗拒的不是「AI 很强」，是「我会不会被换掉」。 ★ 抗拒的对象是位置，不是技术。

★ 而这带来一个很有操作价值的结果：

```
★★ ★★ 抵触地图，★ ★★ 可以在立项前就画出来 —— ★★ 按岗位清单标一遍替代性就完了
★★ ★★ 它不需要调研，★★ ★★ 它需要的是一份岗位清单
```

★★★ 然后是最值钱的一条推论：试点挑得太好推，本身是一种风险

```
★★ ★★ 反直觉的推论：★★ ★★ 试点挑得太"好推"，★ ★★ 推广期才爆炸
```

★ 因为逻辑是连着的：

```
试点选在合规 / 法务 / 财务（低替代性岗位）
      ↓
★ 抵触最小 ★ → 数据最好看 → 汇报最漂亮
      ↓
★★ 但推广到文书密集的岗位时，★★
★★ 抵触最大的那一类人，★★ ★★ 一个都没经过测试
      ↓
★★★ ★★★ 而这正好解释了那个极常见的现象：★★★ ★★★
★★★ ★★★ "试点很成功，推广很失败。"
```

★ 所以正确的做法不是「挑最好推的试点」，而是反过来：

- **★ 试点必须至少覆盖一个高替代性岗位**（哪怕只有 3 个人）
- ★ 或者：**★ 把抵触当作一个被观测的指标**，而不是一个要被藏起来的阻力

怎么识别自己在坑里

1. **★ 你能说出「抵触最强的是哪个岗位」吗？**

   → 如果答案是「感觉大家都不太积极」，说明你在看情绪，没看结构。

1. **★ 你的试点团队里，有没有文书 / 事务性岗位的人？**

   → 如果一个都没有，**★ 你的试点结论不覆盖推广风险**。

1. **★ 你给员工讲 AI 的时候，是不是只讲了「AI 很强」？**

   → ★ 没讲「它替代不了你身上的哪一样」，那「担心被替代」这个情绪就根本没被处理过。

1. **★ 你有没有数过，抵触最强的那类岗位，实际有多少人？**

   → 如果只占 5%，**★ 那把「抵触」当成项目的主要风险来管理，本身就是风险。**

正确做法

做法一：立项前画一张岗位替代性地图。

```
按岗位清单，逐个标三档：
    高替代性  →  文书 / 事务 / 录入 / 汇总
    中        →  半结构化（部分靠规则、部分靠判断）
    低替代性  →  判断 / 责任 / 对外谈判 / 签字
    ↓
★ 抵触地图自动生成，★ 而且它还能直接当推广排期用
```

做法二：★ 对不同岗位，讲不同的话。

岗位类型	别说	要说
高替代性（文书、事务）	「AI 能帮你提高效率」	**★ 「你的哪些活会交出去、交出去之后你剩下的那部分为什么更值钱」**
低替代性（合规、法务、财务）	「AI 效率很高」	**★ 「AI 干不了的那部分，你更值钱了」**

★ 注意高替代性那一栏——★ 不要讲技术，要讲「活」的结构变化。 ★ 讲「效率提升 30%」，员工的解读是「我要被裁」；讲「你的活从三件变成两件，而第三件更值钱」，才是他听得进去的话。

做法三：★ 试点里放一个高替代性岗位。

哪怕只有 3 个人。**★ 否则推广期会替你还账，而且账比试点成功那点小得多。**

一句金句

> ★ **员工抗拒的不是 AI，是位置。**
>
> ★ **抵触强度与岗位的 AI 替代性成正比，与该岗位的业务重要性无关。**
>
> ★ **试点挑得太好推，推广期才爆炸。**

怎么识别自己在坑里

信号一：上线三个月，你的使用数据只有一个活跃用户——演示那次的操作者。

不需要精确统计，问一句「上周谁在上面做过一次真实业务」就够了。

信号二：业务方在群里问「这个能不能不用 AI 做」。

这句话翻译过来是：**「它比我自己做更麻烦，而且出了事是我的。」**

信号三：一线还在用 Excel 做同一张表。

不是因为他们不会用 AI，是因为**AI 的输出没有成为他们工作流的一部分**。他们要先「搬」数据进去、再「搬」结果出来——那当然不如自己拉个表。

信号四：你的项目从来没回答过「模型给错了，谁负责」。

这一条最好用，因为它是一个**你答不上来就说明没准备好**的问题。

信号五：你的推动方式里，有「要求」但没有「给」。

> 如果项目对业务方的全部承诺是「你们要用起来」，而没有任何一项是「我们为你们减少什么、给你们什么」，那抵触不是态度问题，是**这笔交易根本不对等**。

正确做法

做法一：把「业务部门自己都烦的事」放在第一批。

这是坑一讲过的做法，在坑四这里有了第二层含义：

> **员工抵触的本质是「这件事本来就不在我要解决的清单上」。** 挑那件业务部门自己也烦的事， AI 是**帮他解决问题**，不是**给他派活**。

同一件事，对「部门 KPI 」是负担，对「部门的痛」才是帮助。

做法二：试点与辐射，不要一上来就全员。

> 高管推进 AI 的可行路径是**试点与辐射**：先在小范围内跑通，用真实结果说话，再向周边扩散。

这不只是技术上的稳妥，更是**对抵触的正面处理**：

```
全员推行：
  抵触的人没有退出选项 → 表面配合，实际绕过 → 反例产生 → 结论"AI 没用"

小范围试点：
  愿意的人先跑通 → 拿出真实数据 → 周边主动来问 → 变成"我们部门也在用"
```

对抗抵触最有效的武器不是动员，是让身边人先受益。

做法三：把红利兑现写进方案，而不是等它自己出现。

回到第二层机制，答案是很直接的：**效率红利如果不兑现，抵触就会成为最优策略。**

那就在方案里明确回答：

这件事做完了，谁的时间被省下来了？
省下来的时间，去干什么了？
有没有人因此多承担了，但没有多拿？

第三个问题最要紧。**如果有，就要在这个方案阶段处理，而不是等它变成「员工积极性不高」。**

做法四：先定「谁为模型的错负责」，再让它参与决策。

顺序不能反：

❌ 先让模型参与决策，再想责任
✅ 先划清责任边界，再让模型进入那个环节

具体到场景，就是回到资料包里那条卡片模板的最后一栏：**出错谁担责。**

- 参与**预警** → 出错代价是「多看一条提示」，风险可控
- 参与**放行判定** → 出错代价是「一整批产品的质量事故」，必须有人类签字与责任主体

这两种场景在企业里的推进速度，会差十倍。而这个差别，在立项时就能算出来。

做法五（可选但有效）：让抵触的人参与设计。

> **抵触你的人，往往掌握着你没有的信息**：他知道哪个报表他每天要看三次、知道哪一步审批其实可以取消、知道哪张表从来没人信。
>
> 让他来挑第一个场景，比你自己挑十个都管用。

一句金句

> **治理时代大家不肯给的是数据， AI 时代大家不肯给的是判断权。**
>
> **「员工积极性不高」是一个误诊——抵触背后，通常是一笔没算平的账。**
>
> **★ 员工抗拒的不是 AI，是位置。**

1.5 坑五：算不清就上马

现象

有两种相反的现场，但它们是同一个病。

第一种：

```
立项：没有基线，没有目标数字
上线：没有验收口径
半年后：领导问"这个到底省了多少"
项目组：……
```

第二种，更常见，也更隐蔽：

```
立项：认真算了 ROI
      人力节省 → 算得清清楚楚
      其他收益 → 没算
半年后：算出来的结论是"不划算"
      于是项目被砍
      而那些没被算出来的收益，正在被使用中悄悄产生
```

第二种现场里，项目组其实是对的——数字没错，结论错。

为什么会这样

先看量级：这类项目的钱花在哪

在讨论“算不算得清”之前，先得知道这类项目的钱大致是什么量级。以下区间来自作者的采购与实施经验，已做模糊化处理，不代表任何具体项目：

支出项	量级
外部咨询	一百万到两百万
设备购买与硬件算力	几百万
应用实施	几十万到一百多万

【待核实】以上为作者口述的**泛化区间**，作者明确要求“不按真实情况写”；是否以“行业量级”的形式入书、是否保留三档并列，待确认。

量级本身不说明问题，但它决定了一件事：在“几百万”这个量级上，一个项目的成败，靠的不是省一点预算，而是**选对了事**。

第一层：ROI 是后补的，不是前置的。

这和坑一信号一是同一件事，但在这里它的后果更直接：

> **没有基线，就没有验收标准；没有验收标准，项目就只能用「看起来很牛」来验收。**

「看起来很牛」是没法算 ROI 的。所以：

```
顺序错了：
  立项 → 做事 → 补 ROI 测算 → 发现算不清 → 挨批

顺序对了：
  立项 → 先定基线（现在人工花多少、出错率多少、不做会怎样）
       → 再定目标（省多少、快多少、少错多少）
       → 再选场景 → 再定投入
```

坑五的第一层机制：ROI 不是项目做完后的总结，是项目开始前的输入。

第二层：收益有两种，只算一种的人一定算亏。

这一层是本章最值钱、也最容易被忽略的：

> 作者（某大型制造企业的数字化负责人）在讲怎么算账时明确区分了两类收益：
>
> **可计量的收益**——人时节省、决策提速、差错减少。 **不可计量但真实的收益**——员工与技术人员的能力提升、对新应用模式的认识、组织认知水位的变化。
>
> 并且他给了一个判断：**尽量采用最小化投入来实现最大化的收益价值**，而这个「最小化投入」的重点，**是通过这个过程让员工和技术人员的能力得到提升**。

于是：

```
第一版 ROI 测算通常只有：
  人力节省：X 人天
  采购与实施成本：Y 万元
  结论：回收期 = ...

而真实收益里还有：
  · 团队学会了新的工作方式（下一个项目的启动成本会低）
  · 一批人从"不敢用"变成"能验收 AI 的输出"
  · 组织对 AI 的认知水位上移（这个不会出现在任何表格里）
  → 全都没算
          ↓
结论必然是"不划算"
```

第二层机制：算不清，往往不是因为数字难算，是因为你只算了一半。

这一层同时把坑五从「财务纪律」抬升成「认知纪律」——**它不是一个财务问题，是一个完整性的问题**。

第三层：真正该算的不是「投入」，是「锁定成本」。

这是作者在采购决策上最锋利的一段判断：

> **AI 发展速度太快，从 23 年开始到现在，每年基本上年底年初都会有大的新应用模式出现。** 假如说企业刚投入，**花 3 个月到半年上线一套 AI 平台，花了大价钱，结果还没用就已经要被淘汰了**，然后又要赶快等着大平台去升级——这样的话牵动的东西很多。

把这段话翻译成一条可以当场用的判据：

```
这笔钱花出去，如果明年的范式变了，我损失多少？

  损失接近零              → 买（算力、存储、标准的模型服务接口、可导出的知识库工具）
  损失是重做一遍          → 先别买（把业务逻辑、数据格式、工作流全焊死在一个平台里的那种）
```

第三层机制：在变化极快的领域，「投入」是一次性成本，「锁定」才是持续成本。

而大多数企业算账时，只算了前者。

第四层：两个极端都是错的。

```
一个极端：什么都自建
  → 外部研究（2025）指出：自建方案的失败率约为采购方案的两倍，主因是工程基础薄弱

另一个极端：什么都买平台
  → 作者的判断：花 3-6 个月上线一套平台，还没用上就被新范式淘汰，还得跟着升级
```

把两者放在一起，答案其实很清楚：

> **买「能力」，别买「范式」；别全自建，也别全外包。**
>
> **采购是结果，不是起点——你要落实哪些应用场景，就采购哪些。**

> ⚠ ◆ **【待核实】** 「自建失败率约为采购两倍」引自第三方 2025 年研究，目前为转述口径，引用前需核原文。

第五层：把两类钱混在一个池子里，账永远算不清。

作者还有一条很实际的提醒：

> 对于研发，比如医药研发、化工产品研发这种场景，就比较特殊一些……**大多数都是合作模式，基本上就是投钱**，相对于传统研发来说投入更低效、速度更快。 所以，**这个方面的预算，由整个企业的研发战略来确定，而不是由技术战略决定**。

```
一般场景（质检、文档、客服、风控…）
  → 属于技术战略：按场景采购、最小化投入、先跑通
  → 判据：省了多少人力、提速多少

研发场景（药物研发、化工研发…）
  → 属于研发战略：要合作、要外部数据、要投钱
  → 判据：缩短了多少研发周期
          ↓
把两类钱放进同一个 ROI 表 → 两套判据互相打架 → 永远算不清
```

第五层机制：算不清，很多时候是因为尺子用错了。

怎么识别自己在坑里

信号一：立项文件里找不到「不做这个，我们会怎样」。

找不到基线，就说明 ROI 是打算事后补的——补不出来的。

信号二：你的 ROI 表里只有「人力节省」这一项收益。

一张只有一行收益的表，算出来的一定是负数。

信号三：你的投入清单里有「平台」，而你说不清它明年还在不在。

用那条判据测一下：**如果明年的范式变了，这一项我要重做多少？**

信号四：你把研发项目的钱和技术项目的钱放进了同一张表。

用同一把尺子量两种东西，结论一定没有说服力。

信号五（对照类）：你的项目在汇报时，从来没有出现过「基线」这个词。

汇报里只有「我们已经完成了什么」，没有「我们原来是怎么样」。**没有原来，就没有对比，就没有 ROI。**

正确做法

做法一：先立基线，再谈目标。

基线不用复杂，一页就够：

```
现在这份工作，每月人工花多少小时？
出错率是多少？谁在复核？
如果什么都不做，一年后会是什么样？
        ↓
把这三行写进立项文件的第一页
        ↓
后面的所有数字，才有地方可以放
```

基线这件事还有个隐藏收益：它是唯一能让「业务部门自己都烦的事」变得可论证的方式。 业务方对「AI 很好」无感，但对「这件事现在每月花 40 小时」有感。

做法二：收益分两类写，一张表给 CFO，一张表给老板。

	可计量的收益	不可计量但真实的收益
内容	人时节省、决策提速、差错减少	团队能力提升、组织认知水位上移
给谁看	财务口径、可进预算表	战略口径、进决策层汇报
怎么写	用工时、单据量、错误率做乘除	用「下一个项目的启动成本会低多少」「能独立验收 AI 输出的有几个人」这种可讨论的说法
⚠ ◆ **注意**	**不要把第二类塞进第一类算 ROI**	**不要因为第二类无法量化就不写**

做法二的真正要点不是「多写一类」，而是「分开放」。

> **把不可计量的收益硬折算成金额，是自欺；把它完全删掉，是自欺的另一种形式。正确做法是：算账归算账，论证归论证，分开呈现，让决策者自己权衡。**

做法三：用「损失接近零」这条判据筛采购项。

这条判据最好用的地方在于：**它不需要精确估算，只需要诚实回答「重做一遍要多久」。**

采购项逐条过一遍：

- 算力、存储 → 重做代价接近零 → 买
- 标准模型服务接口 → 重做代价接近零 → 买
- 可导出的知识库工具 → 重做代价接近零 → 买
- 某某 AI 平台 → 重做 = 业务逻辑+数据格式+工作流全重做 → 先别买

同时记住另一句：自建失败率约为采购两倍。 两句话一起用，采购清单才不会走极端。

做法四：把两类预算分开，各用各的判据。

技术战略预算（走技术口径）

- 按场景倒推采购：你要落实哪些场景，就采购哪些
- 判据：能不能在 3 个月内跑通一个场景

研发战略预算（走研发口径）

- 合作模式为主，该投钱就投钱
- 判据：研发周期缩短了多少

做法五：把「最小化投入」这句话说完整。

> **最小化的是「为不确定的未来预付的钱」，不是「为确定要做的场景付的钱」。**

这句话值得单独贴出来，因为很多人引用「最小化投入」的时候，砍掉的是**后者**——砍掉了确定要做的场景，只因为采购方报了个整包。

一句金句

> **你算不清，可能是因为你只算了一半。**
>
> **采购是结果，不是起点——你要落实哪些场景，就采购哪些。**

1.6 小结：五个坑其实是一条链

前面五个坑，看起来是五件事——一个讲人，一个讲技术，一个讲数据，一个讲组织，一个讲账。

但它们其实是**同一条链上的五个环节**：

```
坑一　起点错了
　　一把手以为 AI 能解决一切 → 项目被摊派 → 部门应付
　　　　　　　　　↓
坑二　选型时变形
　　把选型当能力竞赛 → 不断追新 → 项目变成技术演示
　　　　　　　　　↓
坑三　喂不进去
　　选完了模型，才发现关键指标没有唯一算法、历史不够长
　　　　　　　　　↓
坑四　人不用
　　系统上线了，但抵触在，而且效率红利还没兑现
　　　　　　　　　↓
坑五　说不清
　　回头算账，发现收益漏了一半、投入锁死在了一个平台上
　　　　　　　　　↓
　　　最后一句结论："AI 没用"
```

这条链有个残酷的特点：它会自我闭环。

坑一造出来的「AI 没用」，会被后面四个坑**加固**——因为它们本来就是失败的原因。所以老板的怀疑看上去很有根据，其实是他自己前面那几步的必然结果。

如果只能记住这一章的一件事，请记住这个顺序：

> **先把「不做什么」想清楚，再去想「用什么做」。**

坑一的正确做法是选业务部门自己都烦的事——**缩小范围**。坑二的正确做法是先定「不换的」再选型——**锁定长期成本**。坑三的正确做法是先做指标级治理——**把地基打在少数关键指标上**。坑四的正确做法是先定责任边界、再谈推广——**把抵触从态度问题变成边界问题**。坑五的正确做法是先立基线、收益分两类写——**让账算得完整，而不是算得漂亮**。

这五件事都在做同一件事：把不可控的冲动，约束成可交付的承诺。

而这恰恰也是这本书叫「避坑」的原因——**你避的不是失败，你避的是那些看起来像成功的东西**：一堆 Demo 、一次追新、一句「 AI 很有潜力」。

> **本章要回答**：我们该用 AI 做哪件事？ **纪律声明**：① 企业名、集团名、子公司名、事业部代号、产品名**全部泛化**，统一用「某大型制造企业」「某集团研究院」「某子公司 A/B/C」。② 场景卡片中**素材未覆盖处一律标** `【待核实】`，不编造系统、金额、效果数字。③ 场景清单的取舍**待作者确认**：本章按作者 10-29 口述保留 9 个场景，其中 4 个已划掉的场景（设备预测性维护、预测性决策、研发文献/数据挖掘、智能客服与知识库）不进入正文，但保留在 2.4 节末的「已划掉」清单里——**删掉的理由本身也是内容**。

2.0 本章要回答什么

在讲「怎么用」之前，先讲一件更靠前的事：**用在哪**。

企业做 AI，最先启动的通常是技术侧：选模型、算力、工具链、平台。场景讨论往往被放在第二轮，而且是**由已经选定的东西去找场景**——于是场景变成了「这套技术能做什么」的注脚。

本章要把这个顺序倒过来。

因为在企业里，**场景选错的代价远大于技术选错**：

```
技术选错了 → 换掉它，最多损失几个月和一个预算
场景选错了 → 半年投入、一群人半年时间、
                以及一句从此没人再信的"AI 没用"
```

本章要做三件事：

1. 说明为什么场景总是选不对（**用一份真实的集团级场景台账说话**）
2. 给出一个能当场用的筛选方法：**选场景三问**
3. 把 9 个真实场景摆出来，每一个都标明**它的边界在哪、出错谁担责**

最后这一项是本章与大部分同类书最不一样的地方。

大部分写 AI 场景的书，告诉你有哪些场景可以选。这一章告诉你的是：选中的那个场景，它会从谁手里拿走什么。

2.1 为什么场景总是选不对

先看一份真实的台账。

某大型制造企业做过一次集团级的 AI 业务场景盘点，把所有 AI 相关需求收敛成了一张表。链条是完整的：

```
上百项业务需求
   ↓ 78 项与 AI 相关
76 条需求台账
   ↓
46 个"计划做"的场景 + 23 个"已实现"的场景
```

这张表有四个字段是标准的：**场景可行性（满分 4 分）、效益重要度（满分 4 分）、场景紧迫度（满分 5 分）、价值维度**。

看起来很规范。

但这张表，恰好把场景选不对的三个原因全摆了出来。

洞察一：排了 46 个场景，却没有排优先级

> 46 个「计划做」是**并列的，没有先后顺序**。 一张排了 46 个场景却没排优先级的表，等于没有路线图。

这句话值得放在本书的开头不远处，因为它适用于无数企业：

你手上那张"AI 场景清单"，长这样吗？

□ 智能问答　　□ 质量预测　　□ 供应链优化
□ 合同审核　　□ 知识库　　□ 排产优化
□ 客服机器人　　□ 文档生成　　…… 共 46 项

↓

每一项都"重要"，每一项都"紧迫"

↓

于是先做哪个？—— 谁声音大谁先做

判断一张场景清单是不是路线图，只需要问一句：它排过序吗？

> ★ **可用的金句：「一张排了 46 个场景却没有排优先级的表，不是路线图，是愿望清单。」**

洞察二：越靠近业务价值，越做不动

这张台账里有一个非常干净的数据分布：

业务域	已实现	计划做	计划做占比
办公	19	6	13%
财经	3	3	7%
营销	1	9	20%
供给	0	19	41%
生产	0	5	11%
研发	0	4	9%

已实现的 23 个里，83% 是办公类。而营销、供给、生产、研发这四个业务域合计 37 个场景，**全部停在「计划做」，一个都没动**。

```
已经做成的：写材料、查知识、做问答、跑流程      ← 通用、好做、不碰业务
还没做的：预测需求、管理库存、优化排产、
          算清渠道、盯住质量                    ← 通用方案做不了
```

为什么办公类做得动？因为办公类的需求**不依赖业务口径、不依赖业务数据、错了也没人受伤**。它只需要「把文本处理好」。

而供给、生产、营销这些域，**每一个动作都压在一条已经建好的业务链上**——改一个口径、一个判断，就会牵动一批人和一批考核。

所以：

> **洞察二说的其实不是「业务部门推不动」，而是「越有价值的场景，越难被通用 AI 方案覆盖」。**
>
> **而企业通常是从最容易的那一端开始做的——所以做了一堆办公场景，然后得出结论「AI 只能做办公」。**

这一条对读者极有杀伤力，因为它把「我们推不动」从**态度问题**重新定义成了**难度分布问题**。

洞察三：需求来源 100% 是「业务需求」，没有一条来自「行业成熟场景」

这张台账在表头里规定，需求来源有四类：

```
(1) 行业成熟场景，对应公司基础业务
(2) 业务需求
(3) 专家建议
(4) 供应商产品/解决方案
```

而实际填进去的 **46 个「计划做」场景，来源全部是 (2) 业务需求**。

一条 (1) 都没有。

这一条的分量在于：

```
如果场景全来自业务部门自己提的需求，
那么这张清单反映的是——
    这个企业现在哪里最痛
而不是——
    这个行业已经有哪些跑通的场景、值不值得抄
```

没有一条「行业成熟场景」，意味着这家企业没有系统性地看过行业里已经跑通的东西。

它完全可能拥有一个在任何行业都能落地的场景清单——但那份清单是**从内部长出来的**，不是从外部比出来的。

> **一家企业选了 46 个场景，全部是自己想到的。这既可能是因为它特别了解自己，也可能是因为它没看过别人。**

一个背景数字：这张表是在什么水平上选出来的

> 该集团做过一次覆盖 **16 个部门访谈、12 个业务单元逐一体检**的 AI 能力成熟度自评，结果是 **起步水平（满分 5 分），属 L1–L2 级**（L1 简单探索 → L2 AI 应用 → L3 AI 标准化 → L4 领域级 → L5 领先）。

> ⚠◆ **【待核实】** 该得分为某集团**内部自评版**结果，不代表行业普遍水平；是否在书中保留具体分值待作者确认。

把它和上面的台账放在一起看，图景就完整了：

```
成熟度：L1-L2（多数部门还在"探索"与"应用"）
台账：    46 个场景、没排优先级、需求来源全是内部
            ↓
这张台账不是路线图不足，
是它对应的组织能力，本来就还停在 L1-L2
```

这也解释了为什么「选场景」这件事这么难——它不是一个分析问题，是一个能力问题。

2.2 选场景三问

场景清单难用，是因为大多数清单只有「场景名 + 一句话描述」。要能用来做决策，**每个场景至少要能回答三个问题**，而且**顺序不能换**。

```
第一问　这件事做错了，代价大不大？　　← 先答，答不过就不用往下看
第二问　现在人工是怎么做的？基线在哪？
第三问　做完之后，谁的工作方式会变？
```

第一问：这件事做错了，谁赔？

这是三个问题里**唯一一个具有否决权**的。

因为前两个问题答得再好，如果做错了的代价是企业承受不起的，那这个场景就不该由 AI 进入**决策环节**——它最多只配做个**提示工具**。

同一个模型，放在不同环节，代价差一个量级：

```
放在"提示"位置　　→ 错了，多看一条提示，多花一小时
放在"判定"位置　　→ 错了，一整批产品的质量事故
```

所以这一问的正确问法不是「这个场景重不重要」，而是：

> **如果这个场景里模型判断错了，损失由谁承担、多久能被发现、能不能挽回？**

三个追问，缺一不可：

追问	如果答不上来
错了谁承担	责任不明 → 一线会用脚投票（见第一章 1.4 第四层）
多久能发现	发现太晚 → 模型会持续输出错误结论，越错越远
能不能挽回	不可挽回 → 模型只能进「建议位」，不能进「决策位」

这一问还有一个副作用，值得单独说：它是最有效的范围裁剪器。

很多企业的场景清单之所以做不下来，根因是**把所有场景都当成同一种东西在排优先级**。而实际上：

```
决策类场景（错一次就是事故）  → 少而深，一个一个啃
提示类场景（错了只是浪费点时间）→ 多而浅，可以铺开做
```

分完之后你会发现，「要做 AI 决策」这件事的规模，比你以为的小得多。

第二问：现在人工是怎么做的？

这一问看起来最简单，其实最容易被跳过。

```
如果不先知道人工基线，你无法回答：

  AI 要替代的那段工作，现在到底花多少时间？
    → 不知道 → 你算不出 ROI（第一章 1.5 第一层）

  AI 给出的结果，好在哪？
    → 不知道 → 你无法设计验收标准

  AI 是不是在重复一件大家早就在用 Excel 做的事？
    → 不知道 → 你可能花大力气做了一个更笨的 Excel
```

一个具体到可以照抄的问法：

> **「这个场景现在由谁做、每周花多少时间、其中哪一步最花时间？」**

第三步往往就是 AI 的切入点——**通常不是整个流程，而是那个最花时间、又最机械的一步**。

第三问：做完之后，谁的工作方式会变？

这一问最容易被忽略，也最容易被低估。

```
如果一个场景做完了，谁的工作方式一点没变——
那它多半不是真需求，只是"技术上能做"。

反过来：
如果一个场景会改变某个岗位每天的工作方式，
那它就是真需求。
```

为什么这一问重要？因为它决定了推广阻力从哪里来。

第一章 1.4 讲过：员工抵触的本质是「这件事本来就不在我要解决的清单上」。第三问就是那个清单的探针。

第三问的答案	含义
没人改变	伪需求，或价值极低
某个岗位的日常变了	真需求，但要先解决该岗位的抵触
整个岗位的产出方式变了	**真需求，且是战略级的**——这类场景最值得做，也最难推

三问用完之后的动作：排序，而不是淘汰

```
不要用三问淘汰场景，要用三问给场景分层：

  第一问（代价）  → 决定能不能进"决策位"
  第二问（基线）  → 决定能不能算得清 ROI
  第三问（人）    → 决定推进难度

  三问都过得去 → 候选场景，进入排序
  答不上第二问 → 不是不能做，是"现在还做不了"，先补前置
  第一问答不过 → 降到"提示位"，不进决策
```

这一步做对之后，一个典型的项目清单会变成这样形状：

```
少数高代价场景（深度做，需要跨部门治理）
多数低代价场景（广度做，可以铺开）
一批"现在做不了"的场景（等数据、等口径）
```

它比「46 个场景」有用得多，因为它同时告诉了你做什么、做什么、和现在不做什么。

2.2.1 ★ 制造业六环节场景速查表（拿一张表就能筛）

> ★ **这一节是本章最实用的一张表**：拿一张表就能把自己的场景筛一遍。★ 它来自第八章 8.0.1 的推演（制造业六个环节的 AI 适合度），前置到这里是因为**它属于「选场景」，不属于「复盘」**。✅ **【已确认】六环节分级已由作者定稿**（2026-11-02），本表与第八章 8.0.1 的表述一致，可直接使用。

为什么要有这张表：

```
很多企业选场景，是从"哪里最痛"开始的
        ↓
★ 但"痛"不等于"适合"。最痛的地方往往同时是三个"最难"：
   ① 口径在人的脑子里          → 说不清
   ② 工作入口在别人地盘上      → 改不动
   ③ 效果要好几年才看得见      → 验不了
        ↓
★ 而 AI 最擅长的恰好是反面：
   规则明确 + 文档密集 + 错了当场能复核 + 今天做今天看对不对
```

★ 速查表（按规则密度，不按部门）：

环节	数据与知识形态	AI 适合度	典型结局	你该先问自己的一句话
研发	知识密集、数据私有、验证周期长	★ 差	效果无法快速验证，试点容易悬空	我做的是「研发算法」，还是「研发文档与知识的管理」？
采购	规则明确、文档密集、错一条能复核	**★ 最好**	**最容易先成功**	这件事的规则，能不能写在一张纸上？
生产	实时性要求高、数据在设备与工艺里、口径在老师傅脑子里	★ 差	实时与完整互斥，卡在数据侧	我要的是「实时」还是「完整」？两个都要就会卡死
库存	数据完整但滞后、周转口径各部门分歧	中	卡在口径统一，不卡在取数	这个「周转」，几个部门的算法一样吗？
质量	责任不可转移，AI 只能出草稿、人必须签字	中	能用，但必须配套责任与留痕设计	出了错，谁签字？谁担责？
销售·文档	合同、协议、政策：规则明确、文档密集	★ 好	**可做**	是不是一份「审核意见」就能定的活？
销售·数据	查业绩、要过程指标：口径在销售脑子里、入口在社交软件	★ 差	**典型失败场景**	一线愿意放弃微信，用你的工具吗？

★ 这张表最关键的一行，是「研发」和「采购」的对照：

```
同一个企业里：
    采购 → 规则写在纸上 → 好做
    研发 → 效果几年后才看得见 → 难做
          ↓
★ 但★ 研发中心的"制度与知识管理"又是好做的
    （真实案例：LLM-WIKI 知识助手，提效 20%~30% 以上，见 8.2.6）
          ↓
★ 于是这张表的用法就清楚了：
    ┌──────────────────────────────────────────────┐
    | ★ 判断的单位不是"部门"，                      |
    |    是"这个部门里具体是哪一件事"。             |
    |                                              |
    |    研发的算法探索        → 验证周期长 → 难    |
    |    研发的制度/知识管理 → 规则明确、文档密集 → 易 |
    └──────────────────────────────────────────────┘
```

★ 拿这张表筛场景的三步：

```
第一步：把你的场景挂到六个环节上
第二步：挂不上去的 → 它不属于"生产型企业的 AI 场景"，
          要么是职能型场景（财务/法务/人力），要么是纯办公场景
          —— 那两类按第 2.4 节的九大场景走
第三步：挂上去了 → 看"适合度"那一列
          ★ 高 → 优先做（这是你的快赢）
          ★ 中 → 先补前置（口径统一 / 责任设计），别急着投人
          ★ 差 → ★ 别用"这里最痛"说服自己
```

★ 最后一句必须写出来，因为它最容易被违反：

> AI 落地的顺序，不应该从「最痛的地方」开始，而应该从「规则最明确的地方」开始。
>
> 因为按部门粗分场景必然失败，按规则密度找场景不会。

> **金句**
>
> 「痛的地方不一定适合，适合的地方不一定痛。选场景要的是后者。」
>
> 「一个部门里同时住着适合 AI 的活和不适合 AI 的活。判断的单位是活，不是部门。」

2.3 AI 出错的代价，差一个量级

这一节是本章的高光，也是我们把「责任边界」摆到场景讨论前面的原因。

把两个真实场景并排放：

场景 A：面向患者的健康管理陪伴

> 某大型制造企业十五五战略里规划了一条面向老年慢病人群的**全流程健康管理陪伴**服务——从机构、社区到居家，含并发症预警、就医协助、子女远程关怀，后续延伸至中医养生指导、心理健康疏导、康复训练辅助。

它的业务价值不需要论证：**面向的是患者和家属**。

但它同时带来一个别处没有的问题：

```
模型误判了某个并发症风险等级
        ↓
患者家属没有收到预警，或收到了错误预警
        ↓
后果不是"返工一小时"
是——直接的、可追责的、可能涉及人身健康的后果
```

在这个场景里，「出错谁担责」不是一个可以推迟回答的问题。因为它决定了：

- 模型能不能进「决策位」（我的判断是：初期只能进「提示位」，且必须有人工复核与签字）
- 出了问题，企业对谁交代（患者与家属，不是内部部门）
- 数据边界怎么划（涉及健康信息，外部数据与模型调用的合规要求完全不同于内部办公场景）

场景 B：面向内部员工的办公助手

```
模型把一份分析报告的某个数字算错了
        ↓
员工按错数字做了一份 PPT
        ↓
后果：返工，纠正，多花一小时
```

同样是「AI 出错」，代价差一个量级。

把这个对照放回方法论

这正是 2.2 第一问的底层依据：

	场景 A（面向患者）	场景 B（面向内部）
错误后果	人身健康与责任	返工成本
可追责性	直接对外	内部消化
数据敏感度	极高	中
模型能进的位置	初期只能提示	可进流程
推进速度	慢	快

这张表就是本书的立场：

> **「这个场景能不能上」的第一问不是「能不能做」，是「做错了谁赔」。**
>
> **AI 出错的代价，差一个量级；所以 AI 的位置，必须按代价来定。**

一个必须一起讲的机制：AI 的价值并不随模型能力上升

把这两个场景放在一起，还会看到第二层结论。

> 同一批材料里有一段极重要的对照：「收邮件、爬网、做提醒」这三件事**不依赖公司数据**，所以**任何一家公司做出来都长一样，也都没什么价值**；而「高管要的」——经营指标、预算与达成、合同与合规、制度的适用范围、会议的结论与承诺——**全部依赖公司内部数据的完整与一致**。
>
> 由此得出：**AI 助手的价值曲线，不是随模型能力上升，而是随数据治理程度上升**。

这两句话放在一起，就是本章与第三章之间的桥：

```
场景 A 和场景 B 的差别，不在模型，在数据与责任
                ↓
所以选场景的时候，模型参数不是判据，
数据条件和责任边界才是
                ↓
这两件事分别指向第三章（数据准备）和第七章（合规与风险）
```

这也是为什么本书不把「技术选型」放在前面讲。

2.4 九个场景

下面是 9 个场景卡片。每张卡片只填**素材能支撑的部分**，其余标 【待核实】。

> **卡片五栏**：解决什么业务问题 / 涉及哪些系统 / 当前 AI 能做到什么程度 / 人工基线是什么 / **出错谁担责**
>
> 最后一栏是本章特有的——前四栏任何一份场景清单都有，第五栏决定了它能不能进决策位。

场景 1 ｜质量预测

卡片项	内容
解决什么业务问题	生产制造领域的**质量预测**（与生产工艺优化、设备预测性维护并列，见 AI 工程分中心章程的生产领域条目）
涉及哪些系统	【待核实】——是否接入 LIMS / MES / 检验记录，未确认
当前 AI 能做到什么程度	【待核实】
人工基线是什么	【待核实】——现在靠什么判断、靠多少经验
出错谁担责	【待核实】——**本场景最关键的一问**：是**预警辅助**（错了是多看一条提示），还是**参与放行决策**（错了是一整批产品的质量事故）？

> ★ 写这张卡时必须区分清楚这两种形态：**发预警**与**参与放行**的场景写法完全不同， 2.3 的那张表说的就是这个差别。

场景 2 ｜偏差调查辅助

> 作者口述中的「偏差的预测」，在本章按场景名称写作「偏差调查辅助」。

卡片项	内容
解决什么业务问题	出现偏差后的**调查辅助**——定位原因、缩小范围
涉及哪些系统	【待核实】
当前 AI 能做到什么程度	【待核实】
人工基线是什么	【待核实】——这一步通常靠资深工程师的经验和翻记录
出错谁担责	【待核实】——给出**排查方向**则责任轻，给出**结论**则重

> ★ **素材缺口提示**：这是 9 个场景里素材最空的一个（没有找到「偏差调查」相关的一手材料相关条目）。它要么是内部正在做、还没进材料的项目，要么在一手盘点表里但未被检索到关键词。**需要作者补一句它现在到哪一步了。**

场景 3 ｜个性化健康管理 ★ 与其他场景不同类

卡片项	内容
解决什么业务问题	面向老年慢病人群的**全流程健康管理陪伴**——从机构、社区到居家；含并发症预警、就医协助、子女远程关怀；后续延伸至中医养生指导、心理健康疏导、康复训练辅助
涉及哪些系统	【待核实】——是否接入可穿戴设备数据接口、区域医疗/零售机构系统
当前 AI 能做到什么程度	【待核实】——原战略材料中是**五年演进里程碑**（1.0→5.0），需作者说明**现在实际到哪一版**
人工基线是什么	【待核实】
出错谁担责	【待核实】——**面向患者的服务，责任口径与内部场景完全不同**，详见 2.3

> ★ 这张卡的价值不在业务本身，在于它和场景 1 构成了本章的**核心对照**：同样是「AI 出错」，代价差一个量级。
>
> ★ 同时它提醒一件事：**这个场景是面向外部消费者的服务战略，而其他场景多是「给员工用」或「给决策用」。** 读者的企业未必有这类场景，但读者一定会遇到「要不要做面向客户的 AI」这个选择——**这本身就是一张有价值的卡片**。

场景 4 | 文档自动化

卡片项	内容
解决什么业务问题	【待核实】——按作者确认入选，具体业务边界待补
涉及哪些系统	【待核实】
当前 AI 能做到什么程度	【待核实】
人工基线是什么	【待核实】
出错谁担责	【待核实】——**同类场景可参考 2.5 之前的判断**：文档类错误多为「内容错误」而非「判定错误」，代价通常是返工；但若用于合同/合规类文件，**责任口径会立刻变重**（见场景 5 与第三章）

场景 5 ｜精准追溯

卡片项	内容
解决什么业务问题	【待核实】——作者已确认入选；**注意大纲中「精准追溯 / 智能风控 / 预测性决策」三者的真实状态此前均待确认**，前两者已确认入选
涉及哪些系统	【待核实】
当前 AI 能做到什么程度	【待核实】
人工基线是什么	【待核实】
出错谁担责	【待核实】——追溯类场景的特殊性在于**它是「查」不是「判」**：查错了会误导决策，但通常不会直接造成事故；一旦被用于「判责」，性质立刻改变

场景 6 ｜智能风控

卡片项	内容
解决什么业务问题	【待核实】——作者已确认入选
涉及哪些系统	【待核实】
当前 AI 能做到什么程度	【待核实】
人工基线是什么	【待核实】
出错谁担责	【待核实】——**风控类场景的第一问最尖锐**：漏判与误判的代价不对称（漏判是事故，误判是成本与体验），**所以第一问要拆成两个方向分别问，不能只问「错不错」**

场景 7 ｜营销线索挖掘 ★ 一张「否定行业主流做法」的卡片

卡片项	内容
解决什么业务问题	营销侧的管理与提效：**经济下滑、销售乏力、库存积压**。作者判断：这三个问题的答案不在对话模型里，在数据与分析体系里
涉及哪些系统	【待核实】——销售/库存/客户数据源、指标体系（BI 或自建）、预测模型的运行环境
当前 AI 能做到什么程度	**AI 大模型用于快速建立预测模型的方法论与体系**（指标框架、归因框架、模型骨架、更新流程）；**但指标与参数数据必须自行构建与完善**。作者**明确不推荐**数字人带货、智能客服这类当前主流做法——**价值不高**
人工基线是什么	【待核实】——现在做需求预测/库存分析靠什么
出错谁担责	【待核实】——**这个场景的责权特别值得写**：预测错了是「备货错了还是预测错了」？前者担责在后者的结论上，**责任会一路回溯到指标口径是谁定的**

★ 这张卡是本章唯一一张**直接否定行业主流做法**的大场景卡，而它正是读者最常问的问题的答案：「那为什么不做数字人、不做智能客服？」

作者的三层框架（可直接进正文）：

```
第 1 层（工具层）：AI 帮人干一件具体的活
    写文案、查知识库、做数字人、做客服
    → 可见、易演示、易算工时
    → 但价值往往不高，因为可替代性也高

第 2 层（体系层）：AI 帮人把一套方法/机制建起来
    把需求预测的指标体系、归因框架、数据更新流程搭出来
    → 看不见、不好演示、不好算工时
    → 但这才是企业真正缺的东西

第 3 层（能力层）：AI 帮人的能力水位上移
```

★ **第 2 层是本书的差异化位置**：大部分同类书在讲第 1 层（好写、好举例），少量在讲第 3 层（好听）。**讲清楚「第 1 层价值不高、真正该建的是第 2 层」，是这本书与别的不一样的地方**。

场景 8 ｜供应链预测

卡片项	内容
解决什么业务问题	【待核实】——作者已确认入选
涉及哪些系统	【待核实】
当前 AI 能做到什么程度	【待核实】
人工基线是什么	【待核实】
出错谁担责	【待核实】——与场景 7 同族：**备货与预测的责权切分**问题（谁对「多备的货」负责，谁对「没备的货」负责）

场景 9 ｜ Vibe Coding 部门应用矩阵模式 ★ 与前 8 个不同类

> **它不是第 9 个业务场景，是第 1 种组织模式。** 详见 2.5 。

卡片项	内容
解决什么业务问题	让业务部门在**不排队等 IT** 的前提下自行产出可用的小型 AI 应用，把「业务最懂业务」变成实际产出
涉及哪些系统	【待核实】——AI 编程工具链（工具名待定）、代码托管、制品库、测试环境、发布流水线、权限与审计
当前 AI 能做到什么程度	业务部门可自行完成需求→可运行工具的全过程；**技术部门事后介入做四道把关**（安全 / 整体推广判断 / 编码完善性与产品化 / 正式管理授权），并提供「开放发布测试一条龙」的通道
人工基线是什么	【待核实】——过去这类需求要排 IT 的开发周期（周期长度待补）
出错谁担责	【待核实】——**本场景最核心、也最需要作者定的问题**：一个部门自建的小工具被推广到全集团后，**维护责任留在原部门，还是转到技术部门成为正式产品**？

作者已划掉、不进正文的 4 个场景

> **这一节的存在本身有价值**：读者会拿自己公司的场景清单来对，删掉的理由和留下的理由一样有信息量。

场景	作者的处理	本章给出的解读
设备预测性维护	划掉	属于「预测」类，与质量预测、供应链预测重叠
预测性决策	划掉	属于「预测」类
研发文献/数据挖掘	划掉	数据挖掘类，与营销场景的方法论建设同源
智能客服与知识库	划掉	**工具层，价值不高且可替代性高**（见场景 7 的三层框架）

> ★ 作者 10-29 原话：「所有的预测都划掉，数据挖掘取消掉，智能客服也被我删除了。」
>
> ⚠◆ **【待核实】** 上述「划掉」的理由解读为本章所加，非作者原话，待确认。

待圈定的备选池：8 个（作者尚未划定）

这 8 个来自同一份集团级场景台账，**它们来自台账，不是来自作者的记忆**——这一点本身就是个问题：台账上写着，不等于有人在推。

备选	场景	域	与已入选场景的关系
C1	企业级 AI 智能问答与知识助手	办公	面向**内部员工**的知识助手，与已划掉的“智能客服/知识库”不是一类
C2	文件智能审核与校对（QC/QA/法务）	办公	文档类的第二个样本，与合同审核同族
C3	智能文档与报告生成	办公	与场景 4 同族，可作对照
C4	药物警戒 AI 工作台（语音识别与术语编码）	办公	强合规场景，可与第三章“数据不能出厂”对照
C5	产线视觉 AI 检测平台	生产	图像**判定**，不是预测，与场景 1 不同类
C6	生产工艺 AI 智控（经验算法化）	生产	“把老师傅经验变成规则”的样本
C7	AI 视觉安全监控平台（EHS）	供给	—
C8	设备预测性维护与智能诊断平台	供给	作者已划掉“预测”类，此项**默认不用**，仅留作对照

【待核实】作者 10-29 口述确认了 9 个场景，但**未圈定是否需要增补**。这 8 个里哪些是真的在推、哪些是“台账上写了但没人管”，需要作者划线；圈定之前，本章正文只按已确认的 9 个写。

2.5 第九个场景不是第九个业务场景

前 8 个场景回答的是同一个问题：**用 AI 做哪件事**。

第九个场景回答的是另一个问题：**当业务自己动手做 AI 工具时，企业该怎么办**。

前提变了

> 过去「业务部门不会开发」是默认前提，所以所有 AI 需求都要排队等 IT 。**现在这个前提正在被 Vibe Coding 打破——业务部门自己就能做出一个能跑的工具。**

于是管理问题整个变了：

```
过去：业务部门不会开发
        → 所有需求排队等 IT
        → 核心问题：怎么让业务别乱做

现在：业务部门自己就能做
        → 核心问题：怎么让业务做的东西能被管理
```

分工必须写死

谁	负责什么	不负责什么
业务部门	需求定义、开发、**自己运维管理**	不负责全集团推广、不负责技术选型的合规性
技术部门	① **安全**（从前到后）② **整体推广判断**（这个工具值不值得推广到别的部门）③ **编码完善性与产品化**（把能跑的东西变成能用的东西）④ **正式管理授权** ⑤ **开放发布测试一条龙**	不替业务决定做什么、**不替业务日常运维**

> ★ **这套分工里最有价值的一句话**：
>
> **技术部门保留的是「治理权」，交出去的是「运维权」。**
>
> 业务自己运维，意味着技术部门不必为几百个小应用承担运维负担；而技术部门保留授权与发布通道，意味着企业不会变成一个失控的「影子软件园地」。**两难被这样切开了：既不扼杀业务的自发创造力，也不放弃集团的管控。**

为什么它该放在第二章

放法	结果
放法 A（本章采用）：作为第二章第 9 场景，单列一节并说明它换了轴	第二章「选场景」之后紧接着讲「开放自建通道」，形成**先选对事、再放开手**的递进
放法 B：移到第五章作为规模化推广机制	第二章变成 8 个场景，第五章里会突然插入一段新东西

本章采用的定位表述：

> **前八个场景回答「用 AI 做哪件事」，第九个回答「当业务自己动手做 AI 工具时，企业该怎么办」。它不是第 9 个业务场景，是第 1 种组织模式。**

它的前置条件，与成熟度的关系

这个模式不是想开就能开的。它有一个硬前提：

> **在 L1-L2 阶段，组织的核心瓶颈是「组织与协同能力」，不是模型能力。**
>
> → 业务部门自建工具越多，对**授权、验收、发布、归档**这套治理机制的需求就越迫切。
>
> → **反过来说：一个还停在「业务不会开发」阶段的企业，开放自建通道只会得到一堆没人维护的小工具。**

这句话也是本章对第一章的一个呼应：第一章说「先想清楚不做什么」，本章说「想清楚之后，才谈得上放开手」。

一个必须回答的开放问题

> ★ **本章的最后一个待决问题**（同时也是这一节最有用的一问）：
>
> 一个部门自建的小工具被推广到全集团后，**它的维护责任是留在原部门，还是转到技术部门（成为正式产品）？**
>
> 这一问直接决定「整体推广」这条职责能不能落地——**如果推广之后没人维护，推广本身就会变成新的坑。**

2.6 本章小结

```
选场景三问，按顺序：

  ① 做错了谁赔？          ← 有否决权
  ② 现在人工怎么做？      ← 决定 ROI 能不能算
  ③ 谁的工作方式会变？    ← 决定推广难度
```

这三个问题之所以能替代一份 46 项的场景清单，是因为它们管的是三件不同的事：能不能做、值不值做、推得动吗。

而这一章真正想让读者记住的，其实是三句话：

第一句：场景选错的代价远大于技术选错，所以顺序不能反——**先选事，再选技术**。

第二句：「**做错了谁赔**」**是一个否决性问题**，它同时决定了场景能不能做、和 AI 能站的位置。

第三句：真正值得做的，往往不是「AI 帮你干一件活」（工具层，可替代性高、价值不高），而是「**AI 帮你把这套方法建起来**」（体系层，看不见、不好演示、但这才是企业真正缺的）。

★ 而 2.2.1 那张速查表，把这三问压成了一张可以贴在墙上的纸：

```
制造业六环节 → 按规则密度分档 → 高=先做 / 中=先补前置 / 差=别用"最痛"说服自己
                    ↓
★ 落到实操层面的一句：
   判断的单位不是"部门"，是这个部门里具体是哪一件事
```

第四句：**上 AI 别从「最痛的地方」开始，从「规则最明确的地方」开始**。最痛的地方往往同时是「口径说不清、入口改不动、效果验不了」——恰好是 AI 最不擅长的。

2.7 金句墙

> 一张排了 46 个场景却没有排优先级的表，不是路线图，是愿望清单。

> 越有价值的场景，越难被通用 AI 方案覆盖——所以企业总从最容易的那一端开始做。

> 没有一条来自「行业成熟场景」的场景清单，反映的是这个企业现在哪里最痛，而不是这个行业已经跑通了什么。

> 「这个场景能不能上」的第一问不是「能不能做」，是「做错了谁赔」。

> AI 出错的代价，差一个量级；所以 AI 的位置，必须按代价来定。

> AI 助手的价值曲线，不随模型能力上升，只随数据治理程度上升。

> 能让企业持续变好的 AI，不是替你干活的那个，是帮你把方法建起来的那个。

> 模型的精度不来自模型，来自口径。

> 技术部门保留授权权，交出运维权——两难就这样被切开了。

> **当业务能自己动手，IT 的角色不是变少了，是从「排队接单」变成「设闸放行」。**

> **痛的地方不一定适合，适合的地方不一定痛。选场景要的是后者。**

> **一个部门里同时住着适合 AI 的活和不适合 AI 的活。判断的单位是活，不是部门。**

> **按部门粗分场景必然失败，按规则密度找场景不会。**

> **本章要回答**：AI 项目要数据，IT 部门怎么接？ **纪律声明**：① 企业名、子公司名、事业部代号、产品名**全部泛化**，统一用「某大型制造企业」「某子公司 A/B/C」。② 素材未覆盖处标 `【待核实】`，不编造系统名、金额与效果数字。③ 本章的清单结构是**可执行的工程模板**，不是某家企业的既有制度——首次落笔，供作者按实际做法增删。

3.0 本章要回答什么

第一章的坑三说了一句话：**数据没准备好就想上 AI**。第二章把场景选完了，第三章就要面对那件最不性感的事——

```
场景选对了，方案也定了。
然后 IT 部门问了一句：
      "数据从哪来？"
```

这一问，会让项目停两个月。

而这两个月里发生的所有事，几乎都源于一个误会：

> **很多人以为「数据准备」是数据部门的事。**
>
> **实际上，「数据准备」是把企业的业务口径，翻译成 AI 能听懂的形式——这是一次翻译工作，不是一次搬运工作。**

搬运会失败，因为 AI 不接受原始数据：它需要**口径统一、边界清晰、权限可控、来源可溯**的信息。

翻译不会失败，因为翻译的对象是**口径**——而口径是业务部门定的，不是数据部门定的。

这就是本章的立场：数据准备的第一责任人不是 IT，是业务。

3.1 先复习： AI 落地的四大基石

第二章选场景时用的那四个判据，这里再挂一次，因为**数据准备的每一步都能挂到其中一条上**：

```
基石一   场景要成立          —— 场景不对，数据再准也没用
基石二   数据要能算          —— 口径不清，AI 算的是"第三个数"
基石三   流程要能承接        —— AI 出了结论，没人接，等于没出
基石四   组织要能管住        —— 权责不清，出了事无人认领
```

数据准备主要压在基石二上，但它的成败却常常由基石一和基石四决定。

这句话很反直觉，但很重要：

```
数据准备的失败，九成不是"数据脏"，
是"这个场景本就不该做" 或 "做完了没人认领"。
```

一个不该做的场景，你为它治理数据，等于**为一个错误结论修路**。一个没人认领的场景，你为它治理数据，等于**修了一条没人走的路**。

所以在动手整理数据之前，先回头看第二章那三问：这件事做错了谁赔？现在人工怎么做？

3.2 三张清单

数据准备不需要一个庞大的治理体系，需要三张**能在一周内填完**的清单。

三张清单的分工是：

```
┌─ 数据可用性清单 — 机器要吃的：口径 / 时效 / 完整性 / 权限
│    → 回答：这份数据能不能用？
├─ 文档治理清单 —— 人要查的：所有者 / 版本 / 生效期 / 密级
│    → 回答：这份文件现在算不算数？
└─ 反馈闭环清单 —— 用起来之后：报错 / 纠错 / 回流
     → 回答：用错了怎么办、多久能改好？
```

为什么是三张而不是一张：因为它们回答的是三个不同的问题，而且**只有第一张是数据部门的事**。

3.2.1 数据可用性清单：机器要吃的那一口

这张清单的字段，可以直接抄：

字段	要写清什么	不写清会怎样
口径	这个数字怎么算出来的，唯一算法是什么	模型学进去两套口径，给你第三套
来源	在哪个系统，哪张表，谁是权威源	同一指标从三个地方取，各取各的
时效	数据更新到什么时候，延迟多少	拿上周的库存做今天的预测
完整性	覆盖多少业务、多少时间、多少维度	训练集只覆盖了三成场景
权限	谁能看到、谁能改、能不能出模型	越权访问，事故从数据泄露开始

用一张真实的盘点表来看这张清单长什么样。

某大型制造企业在做销售预测场景时，做过一份「场景所需数据盘点表」，它的表头列是：

```
数据描述 | 数据来源 | 数据周期 | 核心字段 | 数据量 | 数据质量
```

而这张表填出来的内容长这样（脱敏后）：

```
┌──────────────┬──────────┬──────────────┬──────────────┬──────────┐
| 内部发货明细  | BIS      | 2023-2025    | 规格/数量/日期 | 百万行级 |
├──────────────┼──────────┼──────────────┼──────────────┼──────────┤
| 终端纯销数据  | BI       | 2024-2025    | 终端/销量/日期 | 千万行级 |
├──────────────┼──────────┼──────────────┼──────────────┼──────────┤
| 经销商进销存  | BIS      | 2024-2025    | 经销商/进货/出货 | 百万行级 |
├──────────────┼──────────┼──────────────┼──────────────┼──────────┤
| 外部医疗场景数据| 外部     | 待落实       | 待定         | 待定     |
└──────────────┴──────────┴──────────────┴──────────────┴──────────┘
```

这张表就是「数据准备」的实物形态：

```
一个场景
  ├─ 三个内部系统（不是同一个）
  ├─ 两套数据周期（2 年 vs 3 年）
  ├─ 一块还得去外面谈的外部数据
  └─ 每个系统各有各的字段口径
```

三个洞察，比表本身更值钱：

洞察一：这张表最有价值的列是「数据周期」，不是「数据量」。

> 因为数据量决定跑不跑得动，**数据周期决定结论靠不靠谱**。一个场景需要 3 年趋势，而其中一块数据只有 1.5 年——**这才是要提前拍板的事**。

洞察二：表里最刺眼的是「外部数据待落实」。

一个内部场景，依赖一块**还没谈下来的外部数据**，那么项目排期里就一定藏着一个风险：外部一延，整个项目延。

> **凡是要外部数据才能成立的场景，都必须写进立项文件的风险栏，而不是藏在附件里。**

洞察三：这张表本身就是一个「要不要做这个场景」的判断器。

如果填完这张表，发现三块内部数据里有两块口径不明、一块要靠外部采购才能拿到——**那这个场景现在不该做**，先做别的。

省下的是整个项目，不是几个月。

★★ 3.2.1.1 一份「数据资产盘点」最终长什么样：四十几张表

★ 上面那份盘点表是一个场景的视角。而如果把镜头再往后拉一格，看到的是整张数据地图。

```
★★ 某大型制造企业做过一份销项系统的数据资产盘点
★★ 结果是：★★ 四十几张表
          ↓
★★ 而这四十几张表，★ 恰恰是"数据准备"这项工作的真实体量
```

★★ 而这份清单最值得看的，不是表的数量，是表之间的三种关系：

```
┌──────────────────────────────────────────────────────┐
│ ① 主表与明细表（★ 一对多）                            │
│    单据主表 ↔ 单据明细表                               │
│    ★ 一个"单据"概念，被拆成了两张表                    │
│    → 这就是第三章要说的第一件事：★ 业务概念和数据结构不一一对应│
├──────────────────────────────────────────────────────┤
│ ② 字典表（★ 给其他表提供枚举值）                       │
│    数据字典 ↔ 各类单据表                               │
│    ★ 某个字段的"合法取值"，不在字段里，在另一张表里     │
│    → ★★ 切分文档会把这种关系切断（见 3.3）             │
├──────────────────────────────────────────────────────┤
│ ③ 基础信息表（★ 给其他表提供参照）                     │
│    商品编码 / 客户信息 / 行政区划 → 各类单据表          │
│    ★ 单据上的"客户""商品"字段，本身是另一张表的引用     │
│    → ★★ 这就是"数据血缘"：★ 表与表之间有引用关系       │
└──────────────────────────────────────────────────────┘
          ↓
★★ ★★ 于是这份清单立刻回答了一个问题：
★★    ★★ "为什么数据治理不是 IT 部门一个人就能干完的？"
          ↓
    ★ 因为这张图上的每一个关系，★ 都要有人能说清业务含义：
    · 这个字典里的哪些值现在还生效？
    · 这张主表和那张明细表，★ 哪张才是权威？
    · 这个商品编码表，★ 是谁在维护、多久更新一次？
    · ★ 这张表里的字段，★ 出厂以后能不能带出去？
          ↓
★★★ ★★★ 而最后一条，★ 就是下一节 3.4 要处理的问题
```

★★ 而这份清单还有一层更深的意义——它把「数据治理」这件事的形态说清楚了：

```
★ 很多人以为数据治理是"把数据洗干净"
★★ 从这份清单看，数据治理的本体其实是：★★ 把关系理清楚
          ↓
★ 一张表脏了可以修
★★ 但"这张表和那张表是什么关系"没人说得清，★ 那是治理的真正难点
          ↓
★★★ ★★★ 而这也解释了为什么数据治理永远做不完：
★★★ 它不是一份可以一次性交付的清单，
★★★ ★★★ 它是每接一个新场景，就要重新确认一遍的关系网
```

★★ 而这里有一个特别值得单拎出来的点： ★★ 上面那三类关系——主表明细、字典、基础信息—— ★★ 正是第三章 3.3 要讲的「 RAG 切片四类死结」在数据侧的同构版本。 ★★ 文档被切碎，关系会断；表被切碎，关系一样会断。 ★★ 所以本章讲的治理方法，对文档和对表是同一套。

★★ 3.2.1.2 从盘点表到能开工，中间还差三步

★ 一份盘点表填出来，不等于能开工。中间还差三步，而这三步每一步都会掉人。

```
★ 盘点表填完（第 1 步）
   · 有哪些数据、在哪、多大、多久更新
        ↓
★ 第 2 步：★ 口径对齐
   · ★ 同一个指标，★ 三个系统里三个算法 → 谁是权威源？
   · ★ 这一步做不掉的场景，★ 就该在第三章开头被砍掉
        ↓
★ 第 3 步：★ 打通
   · 数据在一个系统，场景在另一个系统 → 要接口、要权限、要责任人
   · ★ 这一步最慢，而且它不在任何人的 KPI 里
        ↓
★ 第 4 步：★ 试跑一次最小样本
   · ★★ 拿一小块真实数据，★ 让模型答一个真问题
   · ★★ 这一步不做，★ 前三步做得再漂亮也是纸上谈兵
        ↓
★ 才能进入训练/微调/上线
```

★★ 而这四步里，最容易被跳过的恰好是第 3 步和第 4 步—— ★★ 因为第 3 步「没有产出物」，第 4 步「看起来像是在小打小闹」。

★★★ ★★★ 但全书第八章那个失败案例，卡死的就是这两步： ★★★ ★★★ 数据口径不全、数据不完整（卡在第 2 步）； ★★★ ★★★ 而「另开一个入口」这个决定，★ 让本来能打通的第 3 步彻底不存在了。 ★★★ ★★★ 所以第三章讲的方法，★ 在第八章会原形毕露地再出现一次。

3.2.2 文档治理清单：人要查的那一份

第二张清单管的是**非结构化材料**：制度、规程、合同范本、会议纪要、检验标准、 SOP 。

这类东西的特点是：**AI 要读它们，但它们自己会过期**。

```
一份三个月前的制度文件
  ├─ 内容没错
  ├─ 但已被新版取代
  └─ AI 照样把它当现行有效引用 → 结论错，且无法追溯
```

所以文档治理清单的字段是：

字段	含义	为什么必须有
所有者	这份东西归谁维护	没主人的文件会一直「有效」下去
版本	当前是第几版	没有版本号的文档无法判断新旧
生效期	什么时候开始有效、什么时候失效	**这是 AI 出错的高发点**
密级	内部 / 受控 / 秘密	决定它能不能被谁检索到

这张清单背后有一个致命细节：生效期。

> 制度、规程、标准类文件，**绝大多数不是「写错了」，而是「过期了还被引用」**。
>
> 人判断一份文件是否有效，靠的是「我记得这版后来改过」。 **AI 判断不了这件事——除非你把生效期喂给它。**

这就是为什么文档治理的第一件事不是上传，而是**打标签**。

这张清单的第二个致命细节：所有者。

一份没有所有者的文件，在 AI 场景下会变成：

- 没有人能回答「这份东西对不对」

- 没有人能修它
- 出错时没有人能负责

而这正是第一章 1.4 讲过的那个机制在文档场景的重演：责任无处落地，抵触与敷衍就有土壤。

3.2.3 反馈闭环清单：用起来之后的那三件事

第三张清单最容易被忽略，也最能决定一个 AI 场景的**实际寿命**。

```
场景上线
   ↓
用户用它 → 发现错了
   ↓
???
   ↓
（如果这里没有清单，90% 的场景就停在这一步）
```

反馈闭环清单要写清三件事：

事项	具体要定	常见的坑
错误上报	谁在什么入口报错、报什么（场景/输入/错误输出）	只留了个邮箱，没人看
纠错责任	谁来判断「这是错的」、多久内给结论	反馈回来没人认领，闭环断在第二跳
回流路径	纠错的结论怎么回到知识库/规则/数据，让下次不再错	改了单次记录，没改源头，**同一个错犯三次**

第三条最关键，也最常被漏。

一个只记录、不回流的反馈系统，本质上是一**个抱怨箱**：

```
用户报错 → 运营记录 → 用户收到"已收到" → 下次继续错
```

而回流机制要做的是：

```
用户报错 → 定位到具体是哪一段知识/哪一条规则/哪一个数据口径
         → 改源头（不是改这一次的答案）
         → 记录一条"这类错误"的通用规则
         → 下次同类问题不再犯
```

这就是「纠错」和「改进」的区别：前者修一次，后者修一类。

能不能做对，取决于反馈闭环有没有回流这一步。这也是本书后面讲「运营」时会再出现的一个机制。

3.2.4 三张清单怎么填：一周工作法

不要做全景盘点。**只填第一个场景要用的那一份。**

```
第 1-2 天   拿一张表跑通
            挑第一个场景，按"数据可用性清单"把这张场景要用的数据逐项列出来
            （照着那份试点数据盘点表的列抄）

第 3 天     补文档清单
            这个场景会引用哪些制度/规程/标准？
            逐份确认：所有者、版本、生效期、密级

第 4 天     定反馈闭环
            这个场景最可能出哪三类错？谁报？谁判断？改完回到哪里？

第 5 天     开会，做三件事
            ① 清单里标"待落实/待确认"的，当场定责人和时限
            ② 清单里标"拿不到"的，判断这个场景还做不做
            ③ 确认哪些字段需要 IT 侧提供，哪些需要业务侧补口径
```

第 5 天的第②件事才是重点：清单填完不是为了开始做，是为了允许你决定不做。

3.3 RAG 与数据治理的真实接口

这一节要讲一件很多人没意识到的事：

> **RAG （检索增强生成）不是绕过数据治理的捷径，它只是把数据治理的问题从「全量」缩小到「检索得准不准」。**

3.3.1 为什么 RAG 看起来像「不用治理也能用」

因为 RAG 的直觉 appealing：

```
把文档丢进向量库 → 用户提问 → 检索出几段 → 塞给模型 → 生成答案
```

看起来，脏数据、过期文档、口径混乱，全都被「检索」这一步吸收了。

但事实是：检索只能在你已经治理好的东西上做筛选，做不了治理本身。

具体地说， RAG 会把数据治理的四类问题原封不动地搬到检索环节：

治理问题	在 RAG 里的表现形式
口径不统一	检索出两段互相矛盾的内容，模型「综合」出一个第三种说法
版本过期	检索出旧版制度，答案看起来完全正确
权限不清	检索出用户本不该看到的机密段落
溯源缺失	答案对了，但说不出来自哪份文件——**在受监管场景里等于没答**

3.3.2 四个真实接口

接口一：切片（chunking）——治理问题被放大三倍

```
一份 80 页的规程，切成 100 个片段
    ↓
每个片段脱离上下文：
  "本条适用于……"    ← 指代谁？在别的片段里
  "例外情况见附录B" ← 附录 B 不在这 100 片里
  "（本条自2025年起废止）" ← 废止信息在第 3 页，在第 47 片里
    ↓
检索召回的是第 47 片
    ↓
模型拿着一段没有生效期、没有例外说明的条文回答问题
```

这就是为什么「文档治理清单」直接决定 RAG 的质量上限：你标没标生效期，切片里就没有。

切片切得再聪明，也救不了没打过标签的文档。

接口二：混合检索——为什么纯向量检索在制度类文档上会翻车

制度、规程、标准这类文本有个特点：**关键词比语义更可靠**。

```
问："偏差调查的处理时限是多少个工作日？"
文档："偏差调查应在 5 个工作日内完成（见 3.2.1）"

纯向量检索：靠语义相似度找 → 找到一堆提到"调查""时限"的段落
关键词检索：命中"5 个工作日" → 精确
```

混合检索（关键词 + 语义 + 结构化过滤）是制度类场景的必需项，不是可选项。

> 【待核实】作者所在项目的实际检索策略（是否用混合检索、是否接图谱）——素材中未见具体记载，本节为通用工程实践，需按实情修订。

接口三：权限继承——RAG 最大的合规缺口

这是本节最要紧的一条。

```
传统系统：权限在数据源那一层
    → 你查报表，SQL 带权限条件，返回的就是你能看的

RAG 系统：权限最容易"掉"在检索这一层
    → 全员共用一个向量库 → 检索按语义命中 → 命中了就能看到
```

如果不把权限继承到检索层，RAG 就会变成「全员都能看到所有文件」的知识库。

正确的做法是把权限作为**检索的前置过滤条件**，而不是生成后的兜底：

```
✅ 权限过滤在检索前（按用户身份过滤候选片段）
⚠️ 权限过滤在生成后（先检索再判断）—— 已经泄露了
❌ 不做权限（默认全员可见）
```

这不是技术偏好问题，是第七章要讲的红线问题。在这里先记住一句：**「检索到」就等于「看到了」，所以权限必须落在检索之前。**

接口四：引用溯源——在受监管场景里，答案不带出处等于没答

```
模型答："偏差调查应在 5 个工作日内完成。"
                    ↑
          错一个数字，整条答案作废
          而且没人知道它从哪来
```

所以 RAG 的输出必须带引用：

```
✅ 每句结论后面挂来源（文件名 + 版本 + 条款号 + 生效期）
✅ 检索不到 → 明确回答"没有依据"，而不是编一个
✅ 引用了旧版 → 治理清单里的生效期字段在这一刻兑现价值
```

「没有依据就说不知道」这条规则，是把幻觉从技术问题变成工程问题的唯一办法。

接口五：结构化数据——表被切碎，关系一样会断

★ 前面四个接口讲的都是文档。而企业的数据资产里，结构化的表占了大头。

```
★ 一份四十几张表的数据地图（见 3.2.1.1）
        ↓
★ 有人想："★ 把它导出来扔进向量库，让 AI 也能查"
        ↓
★★★ ★★★ 结果：★★ 关系全断了
```

★★ 而断在哪，是可以一条条说出来的：

```
① 主表与明细表被拆开
   → 单独看明细表，★ 不知道它属于哪张单据
   → 模型回答"这张单据的总金额"时，★ 得自己把明细加起来
   → ★★ 而"总金额"这个口径，★ 恰恰在主表的某个字段里

② 字典表被拆开
   → 单据表里存的是编码，★ 人能看懂的含义在字典表里
   → ★★ 编码进模型，★ 模型答出来的是一串数字，不是业务含义

③ 基础信息表被拆开
   → 单据上写"客户编号 A"，★ 客户名字在另一张表
   → ★★ 模型答"这个客户是谁"，★ 答不出来
```

> **★★ ★★ 而这三条，★ 和第三章开头讲的文档切片是同一个病：★★ ★★ 切片切碎的不是内容，★ 是内容之间的关系。★★ ★★ 而关系一旦断了，★ 模型就只能用「看起来很像答案」的东西糊弄过去。**
>
> **★★★ ★★★ 所以正确的做法是反过来的：★★★ ★★★ ★ 不是「把数据切碎扔进去」，★ 而是「把关系保留下来」—— ★★★ ★★★ ★ 要么在切分时把关联字段一起带上（主表 ID + 明细行），★★★ ★★★ ★ 要么干脆别用检索，直接查库。★★★ ★★★ ★ 而「直接查库」这条路，★ 就是第四章讲的 Agent 与工具调用形态。**

★★ 而这也顺带解释了第四章 4.2.4 那个结论的适用边界：

```
★★★ RAG/LLM Wiki 这套方法，★ 治的是"★ 文档"的问题
★★★ 而"★ 数据"的问题，★ 靠的是工具调用，不是检索
          ↓
★★ ★★ 所以第三章讲的治理方法，
★★    文档侧是"标注 + 不切片破坏关系"，
★★    数据侧是"口径 + 权威源 + 打通"
★★ ★★ 两条路，★ 都要走
```

3.3.3 一句话结论

> **RAG 把数据治理的难度从「全公司」缩小到「一个场景」，但没有把难度消掉。**
>
> **它把治理的粒度，从「数据表」变成了「可检索的片段」。**

而「可检索的片段」能不能被正确检索，取决于：口径清不清、版本标注没标注、生效期写没写、权限挂没挂。

这四件事，全在上一节那三张清单里。

3.4 数据分级：能不能进模型

RAG 之外， AI 还要面对一个更直接的问题：**哪些数据能进模型**。

这里最容易出事的地方，是把「能不能给 AI 看」当成一个是非题。其实它是三档：

```
┌──────────────────────────────────────────────────────┐
│ 第一档　可进                                          │
│　　· 已公开的资料、公开的行业数据                    │
│　　· 内部通用文档：制度、规程、模板、流程说明        │
│　　· 脱敏后的业务数据（去标识化、无个人标识）        │
│　　条件：可追溯来源、可标注密级                      │
├──────────────────────────────────────────────────────┤
│ 第二档　需脱敏后方进                                  │
│　　· 含个人信息的记录（患者、客户、员工、供应商联系人）  │
│　　· 含商业敏感的数据（价格、销量、毛利、渠道策略）  │
│　　做法：去标识化 / 聚合化 / 只给摘要不给原始        │
│　　★ 例：子女端只看健康摘要，不看原始心率数据        │
├──────────────────────────────────────────────────────┤
│ 第三档　绝对不进                                      │
│　　· 受控文件与商业秘密原文                          │
│　　· 依法不得外传的数据                              │
│　　· 客户/患者个人身份信息原文                       │
│　　做法：不进模型；必要时在企业内私有部署、且只做检索  │
└──────────────────────────────────────────────────────┘
```

3.4.1 分级的判断标准：不是「重要不重要」，是「出去之后能不能收回」

很多企业的分级做不下去，是因为用错了标准——按「重要性」分，结果一二三档都有人争。

用另一个标准会清楚得多：

这份数据一旦出了厂，我还能不能把它要回来？

能 → 第一档或第二档（脱敏后）
不能 → 第三档（受控数据 / 个人信息原文）

这个标准的好处是它不靠主观判断，靠后果判断。

3.4.2 第二档的三个常见脱敏手法

手法	做什么	适用
去标识化	去掉姓名、证件号、手机号等直接标识，保留分析价值	人员、客户、供应商数据
聚合化	明细变成区间/汇总（「某区域某品类销量区间」）	商业敏感数据
分层给摘要	不给原始，只给结论性摘要	面向不同角色的场景
★ 精度截断	★ 保留可判断的量级，砍掉精确到小数位的细节	★ 合同金额、成本单价这类「必须知道量级、但不必知道精确值」的数据

分层给摘要是最容易被忽略、也最有效的一种。

> 同一个数据，不同角色该看到的东西不一样。
>
> 一个面向患者的健康陪伴场景，和一个面向内部运营的报表场景，**需要的是同一份数据的不同颗粒度**。
>
> 而「只给摘要不给原始」这个设计，**同时解决了数据最小化问题和权限边界问题**——它在技术上要求你先把数据分层，而这正是治理想推动的方向。

★ 一个可以立刻用的判例（★ 来自行业实践中的通行做法，不指向任何一家企业的实测值）：

```
★ 合同金额这类数据，★ 分级时最常被争论"能不能给模型"
        ↓
★★ 一个很实用的处理方式：★ 保留量级，砍掉精度
   · 合同金额 → 保留到"个位/万元级区间"
   · 成本单价 → 保留到"档位"，不给精确到小数
   · 客户名称 → 给编号 + 客户等级，不给名称
        ↓
★★ ★★ 为什么这个处理有效？
   ★★ 因为它保留了"能判断"的部分：
   ★★ "这个合同金额在什么量级"—★ 绝大多数分析只需要这个
   ★★ 而"精确到小数点后四位"—★ ★ 几乎从不做决策，只增加泄露面
        ↓
★★★ ★★★ 所以脱敏的判据不是"给多少"，是"★ 给到够判断的那一档就停"
```

3.4.3 分级不是一次做完的，是逐场景做的

```
正确做法：
  场景 A → 只做 A 涉及的那几类数据的分级（通常 3–5 类）
  场景 B → 复用已有分级，只补新增的

错误做法：
  先做全公司的数据分级体系 → 一年后还没做完 → 场景全部停摆
```

这和第一章 1.5 第三层（只治理第一口）是同一个原则在数据侧的应用。

3.5 最小起步：只治理第一口

大纲给这一节的标题很直白——**不要一上来全量治理，先治理第一个场景要用到的那 10%**。

这一节要把它讲透，因为它是本章最容易被忽略、也最省钱的做法。

3.5.1 为什么「全量治理」必然做不完

```
全量治理的诉求：
  全公司的数据，全部口径，三年以上历史，全部权限梳理
        ↓
它的规模由公司决定，不由你决定
        ↓
而它交付的时间点，由 AI 项目的排期决定
        ↓
结果：AI 项目等它 → 等不到 → 妥协 → 带着脏数据硬上
        ↓
然后所有人得出结论："数据治理做不了，AI 做不了"
```

注意最后那一句——这是最坏的结果：治理的漫长被误读成「治理没用」。

实际上它只是**顺序错了**。

3.5.2 最小起步的四条做法

做法一：以场景为圆心，而不是以部门为圆心。

```
以部门为圆心：
  先治理财务部 → 再治理销售部 → 再治理生产部
  → AI 项目在第三年才开始受益

以场景为圆心：
  先治理"销售预测"这一个场景用到的数据（约 10%）
  → 这个季度就能跑通一个闭环
  → 闭环跑通之后，治理的方法本身也被验证了
```

做法二：先治「被问得最多的那几个指标」。

> 前提是**至少针对高层关心的指标进行了完整治理**——而不是全部数据。
>
> 高层关心的指标通常只有十几个。**把它们治透，比把三千张表治一半有用得多。**

做法三：把「数据不具备」写成可决策的结论。

```
写法一（错的）：
  "本项目需完成数据治理"

写法二（对的）：
  "本项目需要 3 个口径指标：
     ① 纯销口径 —— 现存 3 套算法，需统一，责任人：销售运营部，4 周
     ② 经销商进货 —— 数据仅到 2024 年，不足以支撑 3 年趋势，需外部数据
     ③ 终端动销 —— BI 与业务系统两处不一致，需确认权威源
   若 ①②③ 在 X 月 X 日前未达成，本项目降级为原型验证，不承诺业务结论。"
```

写法二把「数据不具备」从一句抱怨，变成了一张有责任人、有时限、有降级方案的计划。

做法四：把第一口的治理成果做成「可复制的模板」，而不是一次性成果。

```
好的第一口：
  三张清单填出来了 → 清单模板也定下来了 → 下一个场景直接套用
  → 第二个场景的治理成本是第一口的 1/4

坏的第一口：
  第一个场景靠某个能人手工整理了一堆 Excel → 下次还要重来
```

3.5.3 一个可以立刻用的判断表

在决定「这个场景现在做不做」时，用这张表：

问题	答「是」	答「否」
这个场景最核心的 1-3 个指标，**有唯一算法**吗？	继续	**先治理口径，或换场景**
需要的数据**能在 2 个系统内拿到**吗？	继续	**先做取数，或换场景**
关键数据的历史**够长**吗（趋势类场景 ≥ 3 年）？	继续	**降级为当期判断类场景**
数据**权限清晰**、知道谁能看吗？	继续	**先做权限梳理**
有明确的**文档来源**（制度/标准/规程）可引用吗？	继续	**没有引用源的场景，RAG 无解**
出错**有人认领**吗？	继续	**先定责任边界（回第一章 1.4）**

六个「是」才启动。一个「否」，就先解决那一个「否」，或者换场景。

3.5.4 最小起步的真实风险

必须说清楚它省不了什么：

```
最小起步 ≠ 降低标准
最小起步 = 把标准用在必要的地方，先形成闭环

它省掉的是：给用不上的数据做治理
它不省掉的是：第一个场景该做的治理，一样都不能少
```

最容易出现的翻车是：拿「最小起步」当借口，把第一个场景的数据治理也省了。

那不叫最小起步，那叫没做治理，只是没被人发现。

3.6 本章小结

数据准备这件事，可以压成四句话：

第一句：数据准备是翻译，不是搬运。

搬运的是数据文件，翻译的是业务口径——**而口径是业务部门定的。**

第二句：三张清单，够用一整年。

数据可用性（机器吃的）、文档治理（人查的）、反馈闭环（用错之后）。**三张表都只要一周填完，但只填第一个场景要用的那一份。**

第三句：RAG 不是绕过治理的捷径，它是把治理的粒度从「数据表」缩小到「可检索的片段」。

切片、混合检索、权限继承、引用溯源、表结构关系——**五个接口，五个治理断点。** 其中最危险的一条是权限：**「检索到」就等于「看到了」，所以权限必须落在检索之前。**

★★★ 而这一章最重要的判断，其实是这一句：

> **★★★ 切片切碎的不是内容，是内容之间的关系。 ★★★ 文档被切碎，关系会断；表被切碎，关系一样会断。 ★★★ 而数据的解法是工具调用，文档的解法是保留结构的检索——两条路都要走。**

第四句：数据分级不看重要性，看「出去之后能不能收回」。

可进 / 需脱敏 / 绝对不进——**能收回的可以谈，收不回的别碰。**

而这一切的起点，是第一章那句话：

> **不要为 AI 治理全公司的数据，只治理 AI 马上要吃的那一口。**

> **切片切碎的不是内容，是内容之间的关系。**

> 脱敏的判据不是「给多少」，是「给到够判断的那一档就停」。

> 数据治理的本体不是把数据洗干净，是把关系理清楚——所以它永远做不完。

3.7 金句墙

> 数据准备不是数据部门的事——口径是业务部门定的。

> 搬运不会失败，翻译才会失败。而翻译的对象是口径。

> 数据准备的失败，九成不是「数据脏」，是场景本就不该做，或做完了没人认领。

> 一份 80 页规程切成 100 片之后，每一片都失去了它的上下文——包括「本条自 2025 年起废止」那一行。

> RAG 不是绕过数据治理的捷径，它只是把治理的问题从「全量」缩小到「检索得准不准」。

> 「检索到」就等于「看到了」，所以权限必须落在检索之前。

> 答案不带出处，在受监管的场景里等于没答。

> 数据分级不看重要性，看「这份数据出去之后，我还能不能把它要回来」。

> 不要为 AI 治理全公司的数据，只治理 AI 马上要吃的那一口。

> **切片切碎的不是内容，是内容之间的关系。**

> **脱敏的判据不是「给多少」，是「给到够判断的那一档就停」。**

> **数据治理的本体不是把数据洗干净，是把关系理清楚——所以它永远做不完。**

> **全量治理让企业得出「治理做不了， AI 做不了」的结论——那不是治理没用，那是顺序错了。**

> **反馈闭环没有「回流」，就只是一个抱怨箱。**

> **本章要回答**：给一个技术总监能用的决策树，而不是科普。 **纪律声明**： ① 本章不点名任何模型厂商、产品与版本，一律用「通用大模型 API 」「开源权重模型」「通用大模型」等泛称；协议名（ MCP 、 RBAC 、 GxP 、 GMP 、 OCR 、 CSV ）可保留。 ② 所有金额只写量级区间并已模糊化——作者明确要求「不按自家真实情况写」。 ③ 涉及第三方公开报告的数字（ MIT 报告 95%、自建失败率约为采购两倍、 Gartner 「 3:1」、 ZenML 「 80%→95%」）均为转述口径，须标明出处与年份，引用前核原文【待核实】。 ④ 企业名、集团名、事业部代号、产品名一律泛化，统一用「某大型制造企业」「某集团研究院」。

4.0 为什么「哪个模型更强」是伪问题

技术总监的会议室里，最常出现的第一个问题是：

> **「我们到底用哪个模型？」**

这个问题看起来专业，实际是伪问题。原因是——**它把一个已经基本解决的技术问题，放在了一个还没解决的管理问题前面**。

先把量级摆清楚。一个中等体量的企业把 AI 这件事做起来，钱大致花在三处：

支出项	量级（泛化）
外部咨询	一百万到两百万
设备购买与硬件算力	几百万
应用实施	几十万到一百多万

【待核实】以上为作者口述的泛化区间，作者明确要求不写真实情况；是否以「行业量级」形式入书待确认。

在这个量级上，**模型选错与数据用错之间，根本不是一个数量级的损失**。

外部研究给的两条参照口径：一份 2025 年的第三方研究指出，把系统从 80% 提到 95% 质量阈值的工作**几乎全是基础设施与数据工作，不是模型工作**，且**自建方案的失败率约为采购方案的两倍**【待核实】；另一份 2025 年的公开研究显示，**95% 的生成式 AI 试点没有产生可衡量的财务回报**，并且到 2027 年，任务专用小模型在企业部署中将以 3:1 超过通用前沿模型【待核实】。

```
模型之间的差距（年度级，趋于收敛）
        vs
你的数据烂、检索烂、权限没管住（周级，天天疼）

前者一年改一次，后者每一条业务链路都在流血。
```

还有一个更隐蔽的机制。国际学界关于检索增强生成的综述里有一句结论，值得原样记住：

> **Agent 的推理能力，弥补不了不稳定的检索质量。失败往往源于检索覆盖不足、索引构建粗糙、结构化与非结构化知识融合不够。**

翻译成企业语言：**Agent 越强，越会掩盖检索的烂账**。演示越顺，底层数据问题暴露得越晚——而那时系统已经在生产上了。

所以本章不讲模型排名，只讲三棵决策树：**部署形态怎么定、知识与能力怎么搭、工具怎么接**。

> **本章第一性问题不是「哪个更强」，是「哪个你敢让它拿到手」。**

4.1 决策树一：私有化 vs API

4.1.1 为什么会这样：这是一个假二选一

常见的问法是「我们该私有化还是用 API」。这个问题之所以要改写，是因为它把**连续的风险面**压缩成了一个开关。

真实的分歧点只有两个：

```
问题一：数据能不能出厂？
问题二：模型和算力，谁来养？

"私有化 vs API"只是这两个问题的答案组合，
不是问题的本身。
```

作者在讲采购决策时，给过一条被低估的解法：

> 如果**知识库、数据、文件全部在本地**，大模型获取到的可能都是一些局部或部分的东西，也存在一些问题。 但**可以通过切换多模型（使用路由模型），把这个信息分拆到各个板块、各种模型当中来使用**。

这句话的技术含义，本章把它固定成一条设计原则：

> **路由模型在这里的价值不是「省成本」，是「把数据外发的范围切到最小」**。
>
> 一个模型看全部、一个模型只看一段，是两种完全不同的合规结论。

于是「私有化 vs API 」的真实解，是**按数据分层的外发边界**：

```
绝对不出厂（商业秘密、未公开研发数据、个人信息）  → 本地推理 / 专属实例
可脱敏后出厂（统计口径、已公开法规、通用问答）      → 通用大模型 API
可分片出厂（按板块、按任务切给不同模型）            → 模型网关 + 路由层
```

4.1.2 四因子判断表

因子	问什么	指向私有化/自建	指向 API/采购
数据能不能出厂	有多少比例的业务内容必须留在厂内？有没有哪一类是绝对红线？	核心业务数据、客户数据、未公开研发数据占多数	内容多为公开/可脱敏；或已按分片规则设计
成本规模	调用量与数据留存周期多大？峰值出现在什么时候？	高频、持续、批处理型调用	峰值集中、低频、波动大
合规审计	有没有需要监管穿透检查的环节？留痕要求到什么粒度？	需物理隔离、全链路可追溯、日志长期留存	可用云侧合规能力满足，但需确认数据驻留与删除条款
运维能力	有没有能长期维护推理集群、版本升级、容量规划的人？	**有稳定的运维班组与预算**	无专职团队，或人手要投到场景上

这张表最有用的不是它的结论，是**第四行**。

因为前三行都指向「应该自建」的场景里，只要运维能力那一格是空的，**自建这条路就基本走不通**——外部研究里「自建失败率约为采购两倍」的原因正是「工程基础薄弱」【待核实】。

4.1.3 常见错误组合

错误组合一：合规要求高 + 运维能力弱 → 硬上自建。

领导说「数据不能出厂」，团队就买了硬件、招了人、上了推理集群。半年后遇到的是：模型升级要停服、显存不够要扩容、推理框架和业务代码耦合到无法迭代、专门养运维的人成了新的成本中心。

机制：自建不是一个技术决定，是一个**运维承诺**。签下采购合同那一刻，你同时签下了一份未来三年的设备折旧与人力承诺。

错误组合二：调用量不大 + 想「一步到位」建私有平台。

机制：规模不匹配时，私有化的单位成本远高于 API；平台的沉没成本会锁死后续每一次技术路线调整。

错误组合三：把「不可外发」当成「全都不外发」。

一刀切禁止外发，把本可安全外发的通用问题（写通知、通用问答、格式转换）也拖进自建，成本翻倍。

机制：外发风险是**按数据分类**的，不是按公司划线的。

错误组合四：把 API 当成「零成本、随时可撤」。

调用量涨、上下文变长、多步 Agent 把一次任务变成十几次模型调用——**Agent 越自主，token 账单越像指数。**

机制：成本口径从「每千字」变成「每任务」，而任务长度由编排层决定，不由业务决定。

错误组合五：先买平台，后定场景。

作者的态度非常明确：

> **AI 发展速度太快**，从 23 年开始到现在，每年基本上年底年初都会有大的新应用模式出现。假如说企业刚投入，**花 3 个月到半年上线一套 AI 平台，花了大价钱，结果还没用就已经要被淘汰了**，然后又要赶快等着大平台去升级。

由此提炼成一条可当场使用的采购判据：

```
这笔钱花出去，如果明年的范式变了，我损失多少？

  损失接近零          → 买（算力、存储、标准的模型服务接口、可导出的知识库工具）
  损失是重做一遍 → 先别买（把业务逻辑、数据格式、工作流都焊死在一个平台里的那种）
```

采购是结果，不是起点——你要落实哪些应用场景，就采购哪些。

4.1.4 本节结论

```
先问：有没有哪类数据绝对不能出厂？
  有 → 这些数据走本地，其余按分类走 API + 路由分层
  没有 → API 起步，把省下的运维人力投到场景上

再问：调用规模与运维能力撑不撑得起自建？
  撑不起 → 采购，别自建
  撑得起 → 再问明年的范式变了，这套东西损失多少
```

> **私有化不是安全选项，它是一份运维合同。一个模型看全部、一个模型只看一段，是两种完全不同的合规结论。**

4.2 决策树二： RAG vs 微调 vs Agent

4.2.1 三选一其实是一问三答

这条决策树的入口不是「哪个技术更先进」，而是一个具体的问题：

你要解决的是什么	对应做法	判据
知识会变（法规更新、文件天天改、口径调整）	**RAG**	知识是否需要在**不重训模型**的前提下更新？
风格与格式（输出必须像我们的样子、术语必须统一、格式必须合规）	**微调**	问题是不是**「模型知道但不会按我们的方式说」**？
多步自主（要读多个系统、要判断、要循环、要决定下一步）	**Agent**	任务是否**需要模型自己决定步骤**？

三条判据都指向同一个意思：**先看变化发生在哪一层**。

```
变化发生在「知识」层 → 知识外挂（RAG）
变化发生在「行为」层 → 行为固化（微调）
变化发生在「流程」层 → 流程自主（Agent）
```

4.2.2 各自的失败模式

RAG 的失败模式：不是模型答错，是它根本没查到。

检索质量是主瓶颈。失败通常来自三处：检索覆盖不足（该用的文档没进索引）、索引构建粗糙（切片切坏了语义，一段话被切成三片）、结构化与非结构化融合不够（台账里的数字和制度里的文字放不到一起）。

工程上的补救手段（可抄）：

- **长文档切分策略**：合同、注册申报这类超长材料，要按语义边界切（条款、章节、条目），不要按固定字数硬切；切片要保留「所属文件 + 章节路径 + 页码」，否则引用回不到原文。
- **结构化与非结构化分开处理**：台账、指标走结构化查询，制度文本走语义检索，不要用一个索引硬吞两类。
- **检索策略分场景**：开放式生成推理用宽口径检索，窄口径精确抽取用细粒度检索。
- **引用可追溯**：答案必须能指回原文片段。这不是锦上添花，是 RAG 能不能进生产的门槛——**不能回溯的引用，在审计面前等于没有证据**。

微调的失败模式：把会变的东西烤进了不会变的东西里。

```
失败机制一：知识过期
  微调把知识写进权重 → 知识要改，只能再训一次
  → 制度类、法规类知识一进微调，就变成定期重训的债

失败机制二：样本不足与过拟合
  企业里典型的样本分布是"绝大多数正常 + 极少数异常"，
  而异常恰恰是最需要学的

失败机制三：不可审计
  微调后的行为来自权重，很难向审计解释
  "为什么它这次这么判" —— 权重给不出答案，RAG 给得出

失败机制四：版本管理失控
  每次微调产生一个新模型，评估、回归、审批、灰度全要重做一遍
```

所以微调只在该固化的地方固化：格式、语气、术语口径、分类边界、判断规则。**不该固化的是事实**。

Agent 的失败模式：自主性没有配约束。

同一份国际综述给出的另一条结论是：**Agent 的自主性必须配显式约束**。

```
没有约束的 Agent 典型死法：

① 步骤循环 —— 反复调用同样的工具直到超时，烧钱且无产出
② 权限越界 —— 为了完成任务，把不该读的读了、不该写的写了
③ 错误累积 —— 第一步的小错被当成事实往下推，最后一步已经离谱
④ 目标漂移 —— 没人告诉它"做到什么程度算完成"，它自己定义完成
```

对应的四条硬约束，建议写进架构基线：

约束	内容
动作白名单	允许调用哪些工具、读写哪些数据域，写死在配置里
步数与预算上限	单任务最大步数、最大 token 、最大墙钟时间，超限即停并转人工
置信度阈值	输出置信度低于阈值时，不给结论，转人工复核
不可逆动作人工确认	写回、发出去、提交审批这类动作，必须有人工确认环节

4.2.3 现实形态： RAG 打底 + Agent 编排 + 少量微调

上面三棵树是分开讲的，落地时几乎不会是三选一，而是三层叠加：

```
┌──────────────────────────────────────────────┐
│  微调层（薄）                                  │
│  固化：输出格式 / 术语口径 / 语气 / 判断规则    │
│  只做"怎么说"，不做"知道什么"                   │
├──────────────────────────────────────────────┤
│  Agent 层（编排）                              │
│  决定：读什么、调谁、循环几次、何时转人工       │
│  必须配：白名单 + 步数上限 + 人工确认          │
├──────────────────────────────────────────────┤
│  RAG 层（打底）                                │
│  提供：可追溯的事实 + 引用原文 + 实时更新       │
│  必须配：切片策略 + 检索评估 + 引用回链         │
└──────────────────────────────────────────────┘
```

为什么这个组合是常态，而不是折中？

因为它把三种「变化」分给了三种「可改的地方」：

```
知识天天变  →  改索引即可，不重训、不重测
行为偶尔变  →  改提示与配置，必要时才微调
流程经常变  →  改编排，改完就生效
```

只有当**行为**这一层需要固化到「每次都稳定复现」时，微调才有不可替代的价值。**微调是把一次性工程投入换成长期稳定性，不是把知识塞进模型。**

> **RAG 负责「知道什么」，微调负责「怎么说」， Agent 负责「先做什么」。 Agent 越强，越会掩盖检索的烂账。**

4.2.4 ★★ 企业知识库场景： RAG 的四个死结，与 LLM Wiki 为什么会赢

> **这一节讲的不是「 RAG 这个技术不好」。 它讲的是一个更具体的问题：当你要做的不是「检索一堆网页」， 而是「★ 让全公司多个系统共用一套知识」时， RAG 路线里「★ 自动分段」和「★ 向量库作知识底座」这两个选择会在哪里卡住。**

现象

企业想做知识库，第一反应几乎都是 RAG： 文档切段 → 向量化 → 检索 top-k → 拼提示词 → 生成。

流程跑通，演示效果也不错。**但进生产之后，四道坎始终过不去。**

★ 而这四道坎有一个共同特点：**它们都不是「模型不够强」造成的，也不是「检索参数没调好」造成的。** 它们出在流程的最前面——**文档变成向量的那一刻。**

★ 死结一：分段这件事，非人工不可自动化

```
★ 切得自动（按固定字数/按分隔符）：
    → "第三章 第2条 第3款....."被切成两半
    → ★ 条款的上下文断了，检索出来的片段读不懂
    → ★ 表格被按行切开，表头和数据分家

★ 切得人工：
    → 质量确实好
    → ★★ 但文档一更新就要重切
    → ★★ 而企业的制度、标准、模板，是天天更新的
```

★ 本质问题： RAG 把「理解文档结构」这件事推给了切分器，而切分器只会数长度。

★ 切分器不懂「这是第几款的例外条款」，也不懂「这张表的三列要一起看」。

★ 死结二：前处理同样非自动化

★ 企业的知识不是纯文本流，而是**有结构的文档**：

```
★ 编号条款   →  第一条 / 第一款 / （一）—— 切开后引用链断裂
★ 表格       →  表头与数据分离后，数字失去含义
★ 图表       →  切完只剩一段描述
★ 附件清单   →  变成一堆孤立字符串
```

★ 而 RAG 的输入端期待的偏偏是「一段自洽的纯文本」。**两者的形态不匹配，而且这个不匹配无法靠调参解决**。

★ 死结三：平台迁移难

```
★ 换嵌入模型 → 向量维度变了 → 全量重灌
★ 换向量数据库 → 数据迁移 + 索引重建 + 调参重做
★ 换检索策略（向量 → 混合 → 图） → 又是重灌
```

★ ★ 本质在于：你以为存的是「知识」，其实存的是「向量」。

★ 向量是**绑死的**：它同时绑定了嵌入模型、维度、切分策略、检索后端。**知识是活的，向量是死的**。

★ 死结四：多系统对接、共用同一套知识库——★ 这一条最要命

```
★ 每个系统各自建一套 RAG → 知识孤岛堆叠
★ 同一份制度，在 5 个系统里有 5 份拷贝
★ ★ 制度更新了，5 个系统要更新 5 次，漏一个就出事
★ 要真共用 → 必须统一切分口径 → ★ 迁就最差的那个系统
```

★ ★ 共用知识库的前提是：知识要有一个★ 不依赖任何系统的形态。 ★ 而向量库天然绑系统——**这条前提，从架构上就不成立**。

★ 所以很多企业的现状是：**知识库很多，知识却很少**。

四个死结的共同根因

> ★ RAG 路线把「知识」提前降维成了「片段 + 向量」，★ 而企业真正需要的是「★ 结构完整、平台无关、可被人读懂的知识」。
>
> ★★ 而降维是不可逆的——你没法从一堆向量里还原出那份制度。

★ 正确做法：LLM Wiki （把知识整理成 Wiki，而不是切成碎片）

★ 机制换了一下：

```
RAG:    文档 → 【切段】→ 向量化 → 检索碎片 → 拼提示词 → 生成
        ↑ 这一步在"数长度"

Wiki:   文档 → 【整篇交给 AI 做逻辑整理】→ 结构化 Wiki 页面 → 直接基于页面回答
        ↑ 这一步在"理解内容"
```

★ ★ **核心差别只有一处：分段这件事，从「数长度的人」交给了「能读懂的人」。**

★ 而且这个替换，**一条一条地把四个死结解掉了**：

★ 死结	★ LLM Wiki 怎么解
分段非自动化	★ 不切段——整篇交给 AI 按逻辑层级重组，**切分由理解内容的一方来做**
前处理非自动化	★ 表格、条款、编号被保留在结构化页面里，**上下文完整**
平台迁移难	★ Wiki 就是 Markdown/文档，**不绑向量库、不绑嵌入模型**，换平台就是拷文件
多系统共用难	★ 同一份 Wiki 被所有系统读；**共用所需的那个「独立于系统的形态」，Wiki 天然具备**

★★ 一句必须说清的话：LLM Wiki 不是要取代 RAG

★ 否则这一节会跟 4.2.3 「RAG 打底」自相矛盾。准确的说法是：

```
★ LLM Wiki 取代的是 RAG 路线里的两件事：
   ① "★ 自动分段"这个动作
   ② "★ 向量库作为知识底座"这个选择

★ 而检索层用什么，仍然可以按需选：
   · 向量检索 · 混合检索 · 图检索 · 全文检索
   ★ ★ 变的只是"★ 被检索的对象"：
   ★ ★ 从"碎片"变成了"完整页面"
```

> **★ RAG 把知识打碎，Wiki 把知识连起来。★ 检索的本质，也从「用相似度找碎片」变成了「用逻辑找结论」。**

★★ 一句必须先说清的话：★ 这条路线不是推演，是走过弯路之后的回头路

```
★ ★★ 在做 LLM Wiki 之前，★ RAG 这条路是走过的
★ ── 而且走的是最正统的那种：文档切段、向量化、检索 top-k、拼提示词
          ↓
★ ★★ 演示效果不错，★ 流程跑通了
★ ★★ 但进生产之后，★ 前面那四道坎一道都没绕过去
          ↓
★★ ★★ 而绕不过去的原因，★ 回头看很清楚：
   ★ 不是模型不够强，不是参数没调好
   ★ ★★ 是"★ 文档变成向量的那一刻"，★ 知识就已经被降维了
```

> **★★ 所以这一节的全部内容，是「走过之后写下的复盘」，不是「看论文得出的选型建议」。★★ 这也是本书少有的、可以讲「我们试过什么、为什么没成」的地方。**

★ ★★ 关于「慢」：慢在哪一段，决定了这笔账划不划算

★ 这里要说精确，因为很容易被误读。**「慢」和「不慢」，指的是两个完全不同的阶段。**

```
★ 那段"重活":【知识加工/建库】
   · 逐篇做逻辑整理、生成 Wiki 结构
   · ★★ 这是一次性或低频的活
   · ★★ 但它是【可复用】的:
   ★★ Wiki 建好一次,所有系统、所有查询、所有人都受益
   ★★ 而向量库每换一个模型,就要重来一次
         ↓
★★★ ★★ 而最反直觉的一点在这里:
   ★★ 这一段★ 不需要人来做整理
   · ★★ 整篇文档交给 AI,★ 由 AI 自主完成逻辑整理与 Wiki 页面生成
   · ★★ 人只做一件事:★ 抽检结果对不对
         ↓
★★ ★★ 于是"建库要花多少人力"这个问题,★ 答案比想象中轻得多:
   ★★ 不需要一支队伍去切文档、去写规则、去维护向量
   ★★ 需要的是:★ ★ 建库这件事本身可以交给 AI 去做
         ↓
★★ ★★ 而这恰恰是 LLM Wiki 相对 RAG 少有的、比"快"更重要的差别:
   ★★ RAG 的建库成本在"人"(分段、前处理、每次换模型重灌)
   ★★ Wiki 的建库成本在"★ 让 AI 干一次"(然后人只做抽检)

★ 不慢的那一段:【查询】
   · 不再是"检索碎片 + 拼提示词 + 生成"
   · 而是直接命中结构化页面
   · ★ ★ 查询这一段并不慢——★ 而且答案更准
   · ★ ★★ 需要老实说明:★ 这里没有单独的查询耗时实测数字
   ·   ★ 所以本书只讲机制,不给这个环节编一个秒数
```

★ ★ 算一下总账就清楚了:

	★ RAG	★ LLM Wiki
建库(一次性)	快(自动切段)	★ 不慢(★ 整篇交给 AI 自主整理)
换模型/换平台	★★ 全量重灌	★ 拷文件
制度更新	重切+重灌	★ ★ 改一篇页面
多系统共用	★★ 各自一套	★ ★ 一份
查询	快但可能是碎片拼凑	★ 不慢,且是完整逻辑
★★ 答案可审计	★ 难(片段自动切的)	★ ★ 有对应页面,可翻可追责

> ★ RAG 的成本集中在「第一次」和「每一次变更」；★ Wiki 的成本集中在「建库那一次」，之后越用越省。
>
> ★ 而且因为建库这一步本身可以由 AI 自主完成，★ 「建库要投多少人」这个排期问题，在 Wiki 路线上基本不存在。

★★ 核心判据：快 vs 准，在企业知识场景里根本不是一道二选一

★ 这是本节最重要的一句话，也是很多人选型时真正该问的问题：

> ★ 一个快速的模糊答案，和一个通过逻辑判断的准确答案，哪个更有价值，一目了然。

★ 再往企业场景推一步，这句话可以变成一个可操作的判断：

```
★ 一个 3 秒给出、但需要用户自己去核实的答案
   → ★★ 用户的真实成本是 10 分钟（还得去查原文）

★ 一个 10 秒给出、但可以直接进流程的答案
   → ★★ 用户的真实成本是 10 秒
```

> ★★ 在企业知识场景里，「快」从来不是第一指标，「能直接用」才是。★★ 而「能直接用」的判据不是快，是：★ 答案能不能直接引用进文件、直接支撑一个决定。

★ 而这一层，在受监管的制造企业里还有一道硬门槛：**可审计**。

```
★ RAG 的答案：你很难说清它引用了哪一段
   → 因为片段是自动切的，片段边界本身就没有业务含义
   → ★★ 呼应 4.2.2：★ 不能回溯的引用，在审计面前等于没有证据

★ Wiki 的答案：答案有对应的 Wiki 页面
   → 人能翻、能查、能追责、能更新
   → ★★ 这不是加分项，是能不能进生产的门槛
```

★ 实证：这条路线在本企业的落地结果

★ 研发中心用 **LLM Wiki 形态的知识助手**（知识库 Wiki 化 + 检索问答），实测：

```
┌──────────────────────────────────────────────────┐
│ ★ 效率侧：提效 20%~30%                            │
│   → 找资料、查制度、问流程的时间明显缩短           │
├──────────────────────────────────────────────────┤
│ ★★★ 质量侧：★ 回答的正确率/准确率提升约 30%       │
│   → 不只是"更快找到"，★ 而是"找到的是对的"        │
└──────────────────────────────────────────────────┘

          ↓
★★ ★★ 而质量侧这 30%，★ 恰恰是 RAG 路线最难拿到的那一项：
   碎片拼凑出来的答案，"快"但经常"不对"
   ★ 而一旦答案不对，★ 用户还是要回去查原文
   ★★ ★ 于是"快"就变成了"更快地给出一个错答案"
```

★ 而这两个数字的意义不在大小，在于——**它是在「答案能直接用」的前提下拿到的**，而不是靠「回答速度快」堆出来的。

★ ★ 这里要特别指出质量侧那 30% 的分量，因为它推翻了一个常见预期：

```
★ 很多人以为 RAG 的问题是"检索不准"，
★ 其实在企业制度这个场景里，★ 更大的问题是"检索到了，但拼出来的答案不完整"

★ 拼出来的东西看起来很像答案——因为它确实是从正确文档里来的
★ 但条款的例外、表格的表头、编号的引用链，★ 全都断在半路
          ↓
★★ ★★ 而这种答案最危险的地方在于：
   ★ 它"看起来对" → 用户信了 → 直接抄进文件 → 责任落到人头上
```

★ ★★★ 所以第四章这一节真正要证明的，其实不是「Wiki 比 RAG 快」：

> **★ 在企业知识场景里，「快」根本不是矛盾的一方。 ★ 真正的对手是「★ 碎片拼凑出来的错答案」—— ★ 而它之所以能赢，靠的从来不是速度，是「★ 逻辑完整」。**

★ 怎么识别自己在坑里

☐ **Q1：我切段是人工还是自动的？** 人工 → 不可持续；自动 → 那我一定切坏过，只是还没发现。☐ **Q2：我能说出「最近一次切坏的是哪一段」吗？** 说不出，说明还没人复核过切分质量。☐ **Q3：换嵌入模型或换向量库，我要重灌多少数据？** 答案是「全部」 → 已经绑死了。☐ **Q4：同一份制度在我的系统里有几份拷贝？更新后怎么保证同步？** 超过 1 份 → 迟早出事。☐ **Q5：答案里能点回原文吗？点回去是不是一个完整段落？** 点回去是碎片 → 审计过不了。

★★ Q4 和 Q5 是最关键的两问——它们问的不是技术，是「这套知识资产以后由谁维护、能不能被信任」。

★ 正确做法

```
① 先判断你的知识是不是"有结构的"
   · 制度、合同、标准、手册、模板 → ★★ 天然适合 Wiki
   · 碎片化的 FAQ、客服问答、闲聊记录 → ★ RAG 仍然合适
   ★ 别一刀切，这不是"RAG vs Wiki"，是"知识有结构吗"

② 让 AI 做"整理"，而不是做"切割"
   · ★ 整篇输入，要求它按逻辑层级重组
   · ★ 保留原始出处与位置标注
   · ★ 要它输出人能读懂的页面，而不是向量

③ Wiki 页面必须人可读、可审核、可编辑
   · ★★ 它的产物不是向量，是★ 一份人能维护的知识资产
   · ★ 如果只有系统能读、人看不懂，那它就退化成了一堆换皮的向量

④ 页面必须带出处与版本
   · 能被审计 · 能被追责 · 能被更新
   · ★ 这一条决定了它三年后还在不在

⑤ ★ 把"建库"明明白白算进排期和预算，但别把它算成"一支人力的队伍"
   · ★ 真实成本存在，别装作不是
   · ★★ 但★ 这一段已经可以由 AI 自主完成——所以排期上要算的是
   ·    "让 AI 跑一遍 + 人工抽检"的时间，不是"招人来切文档"
   · ★ 要一起算：它省掉的是"每一次变更都要重来"的成本
   · ★ 呼应第六章：★ 这笔账要算全，不能只算建库那一次
```

★ 本节金句

> ★ RAG 把知识打碎， Wiki 把知识连起来。
>
> ★ 你以为存的是知识，其实存的是向量。
>
> ★ 一个快速的模糊答案，和一个通过逻辑判断的准确答案，哪个更有价值，一目了然。
>
> ★ 在企业知识场景里，「快」从来不是第一指标，「能直接用」才是。

4.3 决策树三：工具接入： MCP 与权限代理

4.3.1 为什么工具接入是企业 AI 的真瓶颈

前面两棵树决定模型能做什么，第三棵树决定模型能碰到什么。

而在企业里，**AI 的能力上限几乎从不由模型决定，而由它能安全地调用的工具数量决定**。

问题的起点很朴素：一个企业里有几十个系统——OA 、 ERP 、知识库、合同系统、审批流、数据平台。每个系统的接口、鉴权方式、参数格式、错误码都不一样。

```
接入方式 A：每个工具写一套对接代码
  → N 个工具 = N 份适配代码 = N 套鉴权 = N 套日志 = N 套异常处理
  → 每接一个新工具，成本几乎不下降

接入方式 B：统一协议（MCP）+ 统一网关
  → 每个工具只需按协议声明"我能做什么、需要什么权限"
  → 鉴权、日志、限流、审计在网关统一做
  → 新增工具的成本大幅下降
```

MCP 协议标准化的意义，不是技术上的优雅，而是三件管理上的事：

1. **接入成本从「每加一个涨一次」变成「边际递减」**——这是数字化部门能把「工具接入标准」做成公共品的前提。
2. **权限模型可以集中**——否则每个工具各自为政，权限审计永远做不完。
3. **审计留痕可以一次做完**——留痕逻辑写在协议层，而不是散在每个对接实现里。

4.3.2 权限代理与审计留痕：受监管制造企业的硬要求

在受监管行业里，工具接入这件事的核心不是「能不能调通」，而是「**调完之后，能不能向第三方证明它做了什么、依据什么、谁批准的**」。

```
没有权限代理：
  AI 直接持有系统账号/密钥，每个系统各自授权
  → 权限授予散落，无全局视图；换人、收回权、临时提权全靠人工逐个系统处理
  → 出事时无法回答"它当时持有什么权限"

有权限代理（统一鉴权网关）：
  AI → 权限代理 → 业务系统
  代理层负责：身份映射、权限收敛（超出数据域即拒）、
              动作分级（读/写/不可逆）、全量留痕
```

审计留痕要记的不是「调用了接口」，而是这一串：

留痕项	回答什么问题
谁发起的（人或 Agent 身份）	这是谁的决定
什么时候	能否复盘时序
调用了哪个工具、什么参数	做了什么
依据哪段知识（原文出处）	为什么这么做
返回结果与后续动作	造成了什么后果

一个通用做法是把日志与业务单据**双向挂钩**：业务单据上能看到是哪次 AI 处理产生的，AI 的日志里也能定位到对应的单据号。没有这个双向索引，审计时就得靠人工翻。

```
审计能力的成熟度阶梯：

  阶段一：只有系统访问日志          → 能查"谁登录过系统"
  阶段二：有 AI 调用日志            → 能查"谁在什么时间问了 AI"
  阶段三：日志与业务单据双向索引    → 能查"这份合同为什么变成这样"
  阶段四：结论可回溯到知识原文      → 能查"依据是哪一条规定"
```

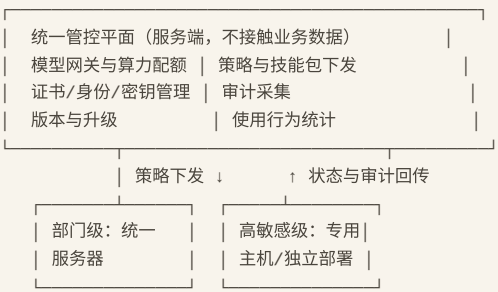
受监管制造企业的硬要求是第四级。前三级只能回答「做了什么」，第四级才能回答「对不对」。

4.3.3 这件事归谁：数字化部门的定位

工具接入标准、模型网关、权限与审计代理，是**典型的人人都需要、人人都做不好的公共品**——这正是数字化部门在 AI 时代的位置：

> **公共品提供者**（算力、模型网关、知识库、工具接入标准、权限与审计代理）、**语义与规则的治理者**、**合规守门人**、**转型教练与裁判**。

放到本章的语境里，这四件事有一个统一的解法：**统一管控平面 + 分级部署**。



一句话：**数据不出机，管控不脱管**。

「独立」的准确含义是数据与计算的位置，不是脱离管控。每一台独立主机都要纳入统一策略：技能白名单、模型版本锁定、证书轮换、审计采集、远程擦除。

【待核实】「高管级助手独立主机部署」目前是作者的方案设想，尚未确认已实际落地，运行形态（是否完全离线、能否访问内网业务系统、模型跑本地还是走内网 API ）需作者确认。

> **MCP 解决的是「接得上」，权限代理解决的是「敢让它接」。审计留痕的终点不是「记下了什么」，是「能回答它凭什么这么说」。**

4.4 落地形态三档：内部 Copilot / 流程内嵌 / 对客服务

技术方案定完，最后一层是**这个能力以什么身份出现在业务里**。三档形态，成本与风险逐级递增。

```
第一档   内部 Copilot          人来用，AI 给建议，人来执行
            ↓ 责任在人，成本最低，可以快速铺开
第二档   流程内嵌              AI 在流程里自动跑，人在关键节点审核
            ↓ 责任共担，需要流程改造与留痕
第三档   对客服务              AI 直接面对外部客户
            ↓ 责任外溢，风险最高，合规要求最严
```

4.4.1 三档对照

维度	第一档：内部 Copilot	第二档：流程内嵌	第三档：对客服务
人机关系	人做，AI 辅助	AI 跑，流程管，人审核关键节点	AI 直接对客，人兜底
典型场景	内部知识问答、文档起草、检索	合同预审、单据预填、异常提示	客服应答、对外咨询、材料生成
成本结构	席位与调用量为主	需要系统集成 + 工作流改造 + 留痕	全部前置成本 + 高可用 + 冗余 + 持续评测
主要风险	员工不用、答案不可追溯	错误被当成事实流入下游流程	错误直接外溢到客户与外部承诺
责任划分	人对结果负责	人机共担，节点必须有人签字	法律责任不可转移，必须有对外口径
审计要求	记录调用与引用	记录每一步决策依据、单据流转	全链路留痕 + 对外话术版本管理 + 投诉追溯
推进节奏	可以快	中速，需跨部门	慢，需合规与法务前置

4.4.2 为什么风险逐级递增

第一档到第二档，增加的不是技术，是「错误能不能被拦住」。

Copilot 模式下， AI 给错答案的后果停在屏幕上；流程内嵌模式下， AI 给错答案的后果会**变成一条进入下游的单据**。所以第二档必须补上三样东西：高风险动作的人工确认、置信度阈值转人工、以及从单据回看 AI 判断依据的链路。

第二档到第三档，增加的不是技术，是「错误能不能被收回」。

```
错误可逆性：

  内部 Copilot    出错 → 改掉                → 成本：几分钟
  流程内嵌        出错 → 单据要撤回、要走更正流程 → 成本：几天 + 跨部门
  对客服务        出错 → 已发出去的承诺 → 成本：可能要召回、可能要补偿
```

所以第三档推进慢，不是保守，是数学。

4.4.3 每一档的责任与审计要求

第一档：内部 Copilot

- **责任**：使用人对输出负责；系统方对引用可追溯负责。
- **必须有的三件事**：引用可点回原文；不引用就说不知道；使用反馈能回流到知识库维护。

第二档：流程内嵌

- **责任**：每个高风险节点必须有人工签字人，**且签字人要知道自己签的是什么**（ AI 的依据、置信度、争议点）。
- **必须有的四件事**： AI 判断依据入库；不可逆动作二次确认；错误回流为规则；审计可按单据穿透到 AI 决策。
- **常见失败**：把「人工确认」做成一个点击按钮。**一个不看内容的确认框，等于没有确认。** 这时候要做的不是加提示，而是减少需要人确认的动作数量。

第三档：对客服务

- **责任**：法律责任不可转移。 AI 的对外输出属于企业行为的一部分。

- **必须有的五件事**：对外话术的版本管理与发布审批；高风险问题（法律、临床、用药安全、价格承诺）的强制转人工与话术白名单；全量会话留痕与投诉可追溯；对外场景专用的模型与评测集；独立团队的抽检机制。
- **受监管制造企业特有的一条（以制药为例）**：涉及产品、适应症、用法用量的问答，**不交给通用模型自由生成**。这类内容要么走审核过的知识条目，要么转人工。

4.4.4 一个具体形态：独立部署的敏感场景

在企业内部，还有一类场景既不属于三档，又在三档之间：**敏感岗位的独立助手**。

作者的设计是：模型可使用公司内部资源，但助手主机单独采购、单独部署，以保障隐私性与独立性。这条思路的价值在于把部署形态**做成了可选档位**而不是一刀切——按敏感度分档，避免最严标准套到所有人身上而让成本不可承受。

【待核实】该形态的落地状态、离线程度与内网访问方式待作者确认，详见 4.7 。

> **从 Copilot 到对客服务，增加的不是技术，是错误的可逆性。 一个不看内容的确认框，等于没有确认。**

4.4.5 ★★ 入口的三个方向：收敛到已有、收敛到新、还是再叠加一个

★ 前面 4.4 讲的是「这个能力以什么身份出现在业务里」——那是责任维度。 ★ 这一节讲的是完全正交的另一个维度：用户从哪儿进来——这是入口维度。

★ 为什么必须把这两个维度分开？因为大多数 AI 方案翻车，都是因为把它们混在一起了。

> 责任维度：★ AI 是给建议的（Copilot）／★ 参与流程的（内嵌）／★ 直接对外的（对客）
> 入口维度：★ 用户从哪个软件点进去
>
> ★★ 这两个维度可以自由组合，★★ 而且组合数比想象的多
> ★★ 很多团队在讨论"我们要做第几档"的时候，★ 其实真正卡住他们的是入口

4.4.5.1 三种入口方向

★ 入口策略只有三个方向，而且它们有明确的优劣排序：

方向	做法	典型形态	核心代价
① 收敛到已有入口	把 AI 塞进用户本来就在用的系统	通过 API / MCP 集成进 ERP 、 CRM 、 OA	**★ 要改别人的系统** —— 排期长、跨部门、优先级不在你手上
② 收敛到新建统一入口	把散落的 AI 能力收拢到一个自有入口	专用 APP / 内部 AI 门户	★ 用户要多装一个；跨组织协作会断链
③ 再叠加一个入口	在现有入口旁边再加一个	独立网站、外部 IM 里的助手	**★ 几乎必然失败**（见 8.5 ）

4.4.5.2 ★★ 一条规律：叠加几乎必败，收敛才有机会

★ 这张表里最值钱的不是前两行，是第三行为什么会输。

```
③ 再叠加一个入口，为什么几乎必败？

用户面前同时有：老路（邮件 / Excel / 群 / BI）+ 新入口
        ↓
★★ 用户有选择权，★★ 而且老路在好几项上都更省事
        ↓
★★ 于是用户的理性选择是：★★ 继续用老路
        ↓
★★ 而新入口的"用户数"，★ ★★ 往往只剩项目组和来做演示的人
```

★ ★ 而这正是 §8.5 营销助手失败的完整机制。

★ 所以这里可以提炼出一条更普适的结论：

> ★ 入口策略的方向，比入口的形态重要得多。 ★ 「收敛」（把散落的收成一个）有机会；★ 「叠加」（在旁边再加一个）几乎必败。

★ 换个说法：另开一个入口的风险，★ 来自「选择权」，★ 而不来自「新入口」本身。 ★ 一个唯一的、比老路更省事的入口，和一个可有可无的新入口，是两件完全不同的事。

4.4.5.3 ★ 为什么「收敛到已有入口」（嵌入式）在很多企业推不动

★ 嵌入式（通过 API / MCP 把能力集成进 ERP / CRM / OA ）方向是对的——它属于收敛，而且用户零学习成本。

★ 但它在实践中经常卡死，而卡死的原因很少是技术：

```
★ 你要改的系统不属于你
        ↓
· 改动的优先级不在对方 KPI 里
· 对方要评估：会不会增加我的操作量？会不会给我添新的责任？
· 要排 IT 档期、要走变更流程、要过安全评审
· 而这些流程通常比业务方的 AI 项目活得更久
        ↓
★★ 而业务方的耐心是 3~6 个月
        ↓
★★ 于是：★ ★★ 方向正确的方案，死在时间差上
```

> ★ 「方向正确的方案，死在时间差上」——这是嵌入式推广最常见的死法。 ★ 而它和 8.5 的死法（方向也错）叠加起来，构成了入口问题的两种极端。

4.4.5.4 ★ 强监管环境下的第三条路：专用 APP

★ 有一类组织，把 ①②③ 中的大部分都堵死了：

```
· 通用 SaaS 协作工具不被允许使用（飞书 / 钉钉 / 企业微信等）
· 内部即时通讯工具本身受强监管，难以扩展
· 走外部方式（独立网站 / 外部 IM），便利性差、用户触达难
      ↓
★★ 剩下的路只有一条：★★ 自建一个专用 APP，
★★ 集成企业内部已有的 AI 应用，★ ★★ 解决入口与快速触达问题
```

★ 而这条路的价值，不只是「方便」——它实际上让 APP 承担了三重身份：

```
★★ 专用 APP 的三重身份

① 入口        →  解决"用户从哪儿进"和"触达不到"
② 权限边界 →  ★ 决定谁能调用哪个 Agent，★ ★★ 这是动态授权的载体
③ 审计载体 →  ★ 记录谁的哪一次调用用了哪些知识、★ ★★ 留痕天然落地
      ↓
★★★ ★★★ 而第 ②③ 两重身份，★★ ★★ 恰恰是第七章《合规与风险》反复要求的
★★★ ★★★ ★★★ 于是它把合规要求从"流程要求"变成了"架构要求"★★
```

★ 这是一个被普遍低估的收益：如果入口是自己建的，那么「按角色授权」和「调用留痕」就不必再靠事后补流程，它们是入口天然带出来的。

4.4.5.5 ★ 必须诚实写出来的代价

★ 专用入口不是免费的，它的代价至少有四项，其中两项是真问题：

#	代价	说明
1	**触达成本从 0 变 1**	老路是「打开电脑就有」，新入口是「多一个 APP」。★ 在允许使用通用 IM 的企业里这项很致命；★ 在 IM 被禁的环境里，它反而是**唯一成本**
2	~~外部协作会断链~~	**★ 已通过分层解决**——★ 对外部分本来就不该在内部 APP 上（见 §4.4.5.7）
3	~~移动端与 PC 端的形态不同~~	**★ 已解决**：★ 移动端自建 APP 、 PC 端网站；★ 内网访问在技术上可行——**★ 开放公网 API 访问**，或**APP 自带 VPN 唤起功能**
4	运维成本	分发、版本兼容、设备管理，都是长期支出

★ ★ 而好消息是：★ 这四项里最贵的两项——外部协作与内网访问——★ 在 §4.4.5.7 里都有解。

4.4.5.6 ★★ 「无感」与「有感」：入口的两种产品哲学

★ 这一节讲的是嵌入式入口和客户端入口最本质的区别，也是很多团队选错入口的真正原因。

现象

先描述两种截然不同的用户体感：

```
★★ 嵌入式入口（API / MCP 集成进 ERP / CRM / OA）：

   用户打开合同系统 → 提交 → 系统自己跑完预审 → 用户看到结论
         ↓
   ★★ 用户全程不知道有 AI 参与，★★ 更不需要改变任何操作习惯
   ★★ 准确地说：★ ★★ 这条路里 AI 是"无感"的


★★ 客户端入口（自建 APP / 内部门户 / 网站）：

   用户主动打开一个应用 → 输入问题 → 得到回答
         ↓
   ★★ 用户知道有一个 AI 在这里，★ ★★ 而且要为此付出"多打开一个东西"的动作
   ★★ 准确地说：★ ★★ 这条路里 AI 是"有感"的
```

★ 而这两条路的成功条件，完全不一样：

	嵌入式（无感）	客户端（有感）
用户要改变什么	★ **什么都不用改**	★ **要主动打开一个新入口**
AI 靠什么起作用	★ 在既有流程里自动跑	★ 靠用户主动发起
能不能承载「用户想问一句」的场景	★ **不能**——没有提问的地方	★ **能，而且这是它的主场**
抵触风险	★ 极低	★ 高（见 §1.4.1）
存在感	★ **弱**——做得好没人知道，做得差也没人反馈	★ 强，一眼可见
评价指标	★ 业务指标（省了多少、错了多少）	★ **使用率**（而使用率是残酷的）

为什么会这样

★ **因为「有感」是双刃剑，它同时放大成功和失败：**

```
★★ AI 无感时，做得好不好，★ 都在业务数据里体现，★ 用户不会烦你
★★ AI 有感时，★ 用户每天都要决定"要不要打开它"— ★ ★★ 而这个决定权在用户手里
```

★ **所以两种入口对应的项目，验收方式必须完全不同：**

- ★ **嵌入式的项目，不能用使用率验收**——它本来就该是低存在感的；

  ★ 要看的是**业务指标有没有变**，以及**出错的单据有没有被拦住**。

- ★ **客户端的项目，必须正视使用率**——★ 因为「用户不打开」这个事实，

  ★ **每天都在明摆着地告诉你这个项目不成立**。

★ **而这就是「无感」与「有感」的分野给项目管理者带来的第一个现实差异：** ★ **同一套管理动作（盯使用率、做推广），用在嵌入式项目上是错的，用在客户端项目上是对的**。

★ 移动端与 PC 端：★ 分端，而不是折中

★ **而这正好解释了一个很实际的分工判断：**

```
★★ 在安全保密要求高的机构和单位里：
★★
★★    移动端   →   ★ ★ 必须自建 APP
★★    PC 端    →   ★ ★ 一个网站就够了
```

★ 为什么移动端不能靠网站？

```
★★ 移动端的使用场景是"碎片、即时、外出"
★★    · 站在车间、库房、出差路上要查一件事
★★    · 手机上打开一个网站：开屏要输账号、要过内网访问、要加载一堆资源
★★    · 而一个 APP：图标点开就进，★ ★★ 而且能做推送
          ↓
★★ ★★ 而"快速触达"这个原始目标，★★ ★★ 恰恰只能靠推送和图标，★★ 靠网站是做不到的
```

★ 所以分端的意思是：★ PC 端用网站把「能用」解决掉，把成本压到最低；★ 移动端用 APP 把「能触达」解决掉。

★★ 而在高保密环境里，入口首先不是「选哪个」，是「哪些不能选」

★ 这是这段分析里最要紧的一层：

```
★★ 如果你用的是第三方的即时通讯工具（某家的 APP）：
★★
★★    你传文件、收文件、数据，★ 都要经过这个第三方平台中转
★★          ↓
★★ ★★ 对安全保密性高的机构和单位来说，★★ ★★ 这条路直接被排除
```

★ 所以在这些组织里，入口决策的实际形态是排除法，而不是选择法：

```
① 通用 SaaS 协作工具 ── 排除（不允许使用）
② 内部即时通讯工具   ── 排除（受强监管，难以扩展）
③ 第三方 IM 承载数据  ── 排除（数据过第三方平台）
④ 外部网站 / 外部入口  ── 排除（便利性差、触达难）
      ↓
★★ 剩下的路只有一条：★★ 自建专用 APP
      ↓
★★ 而它承接的正是：★★ ★★ AI 应用落地，★ ★★ 特别是移动端落地
```

> ★ 在这类组织里，「专用 APP」不是一个产品选择，是一张排除法的最后一行。

★ 一条硬依赖：★ APP 自建的前提，是平台也自建

★ 这一点如果搞错，APP 就只是一个壳：

```
★★ 专用 APP 要能调起各个 AI 应用，★ 前提是那些 AI 应用在企业可掌控的平台上
        ↓
★★ 如果 Agent 平台是外采的（闭源、在别人的环境里）
★★   → 你的 APP 只能做跳转和展示，★★ ★★ 做不了鉴权、留痕、动态授权
        ↓
★★ ★★ 而这三样恰恰是自建 APP 最大的价值（见 §4.4.5.4 的三重身份）
```

★ ★ 所以：入口自建 → 必须平台自建 → 而这又回到 §8.2.6 的那个矩阵式部署决策。 ★ 入口和平台，是同一个决策的两面。

4.4.5.7 ★★★ 入口不是一个，是一组：★ 先按边界分层，再选形态

★ 这一节是前面所有入口讨论的解法，也是本章最重要的一节。

现象

★ 先看一个典型的假矛盾：

```
内部员工要用 AI（有保密要求）★ 同时客户和供应商也要用 AI（要对外服务）
      ↓
★★ 于是冒出两个问题：
★★   ① 内部入口能不能同时给外部用？  →  ★ 不能，数据要隔离
★★   ② 外部用户怎么办？                →  ★ 总不能让他们装企业内部 APP
      ↓
★★★ 而大多数人的解法是"想办法打通"——★★★ ★★★ 于是问题越解越复杂
```

★ ★ 而正确的解法不是打通，是分层：

★★★ 分层的依据不是"用户是谁"，★★★ ★★★ 而是两条更硬的东西：

① 数据边界 —— 客户的数据、供应商的数据、内部的数据，
★ 能不能放在同一套系统里？
② 运营边界 —— 运营 AI 的能力、考核方式、迭代节奏，
★ 是不是同一套？
↓
★★ 按这两条切，★★ ★★ 就切出了几个互不相干的"应用世界"：

应用世界	谁用	数据边界	运营边界	★ 形态
内部应用	员工	内部数据	内部运营	**★ 专用 APP （移动端）+ 网站（ PC 端）**
供应商应用	上下游供应商	供应商数据	供应商运营	★ 独立一套，形态可选
客户应用	客户	客户数据	客户运营	★ 独立一套
└ C 端客户	个人消费者	**★ 更严的隔离**	另一套运营	**★ 形态可以降级**
└ B 端客户	企业客户	另一套	另一套	★

★★ 而这里有一个非常关键的判断：

★★ ★★ 对外的那几个应用世界，★★ ★★ 根本不必做成 App ★★
★★ ★★ 而是可以把功能迁移到 ★★ 小程序 ★★ —— ★★ 而这恰恰是更优解

★ 为什么小程序更优？三个理由，缺一不可：

#	理由	说明
1	**★ 企业 VI 识别 / 品牌形象**	★ 用户打开就认得出是哪家企业的服务，不用装、不用注册
2	**★ 定制化服务**	★ 同样是 AI 入口，★ 可以按客户类型、按场景做不同形态
3	**★ 能嵌入各大电商平台**	★ 直接出现在用户本来就在的地方，而不是要求用户迁移

★ 而这第三条，其实就是下面 §4.4.5.8 那条战略判断的入口。

为什么会这样

★ ★ 因为一旦分层，「外部协作断链」这个代价就自动消失了：

```
★ 之前担心的是："内部 APP 会把外部信息挡在门外"
        ↓
★★★ 但分层之后，★★★ ★★★ 外部信息根本不经过内部 APP
★★★ 而是有它自己的一套入口 —— ★★ 所以不是"断链"，★★ ★★ 是"本来就不该连"
```

★ ★ 由此可以提炼出一条处理「看起来矛盾」的一般性原则：

> **★ 当一个入口策略看起来自相矛盾时，往往不是因为它设计得不好，★ 而是因为你把两个不同的问题塞进了同一个入口。★ 解法通常是分层，不是打通。**

★ ★ 而这背后还有一条更基本的顺序问题：

```
★★ ★★ 顺序不能反：
★★ ★★   第一步：划边界（数据边界 + 运营边界）  →  ★ 决定有几套应用
★★ ★★   第二步：选形态（App / 网站 / 小程序）   →  ★ 决定每套怎么实现
        ↓
★★ ★★ ★★ 而很多团队是反着做的：★★ ★★ 先定形态（"我们做个 APP 吧"），
★★ ★★ ★★ 再想给谁用 —— ★★ 结果把该隔离的混在了一起
```

> **★ 「形态是最后一步，边界是第一步。」**

怎么识别自己在坑里

1. **★ 你的方案里出现了一句「这个 APP 给内部用还是给客户用」？**

   → 这句话本身就说明边界没划。

1. **★ 你的内部系统里，能不能查到客户或供应商的数据？**

   → 能，说明数据边界是漏的。

1. **★ 你的 AI 运营，是一套考核、一套迭代节奏吗？**

   → 如果内部和对外要用不同的人力、不同的话术、不同的迭代节奏，那它们**本来就该分开**。

1. **★ 你的移动 APP 怎么访问内网资源？**

   → ★ 技术上两条路都可行：**★ 开放公网 API 访问**，或 **APP 自带 VPN 唤起功能**。 → ★ 这不是死结，★ 是立项时要先定的一件事。

正确做法

```
① 划边界：数据边界 + 运营边界  →  确定有几套独立应用
② 选形态：内部 = APP(移动) + 网站(PC)
          对外 = 小程序（拿 VI 识别 + 定制服务 + 电商平台入口）
③ 定访问：内网移动端 → 公网 API  或  APP 自带 VPN 唤起
④ ★ 责任分层：★★ 对外那部分的风险等级最高（见 §4.4.3 第三档）
```

★ ★ 而最后一条最容易被忽略：

```
★★ ★★ 对外给客户和供应商用 AI，★★ 是全书风险最高的一档（§4.4.3 第三档：对客服务）
★★ ★★ 而"用大厂的在线能力"并不降低这份责任
        ↓
★★ ★★ 因为法律责任不可转移：★★ ★★ 企业品牌一旦出现在对外 AI 服务里，
★★ ★★ 出了问题，★★ ★★ 追责的还是企业，不是大厂
```

★ ★ 所以越往对外走，越不能靠「平台很强」来安心——★ 越要靠流程、留痕和人工兜底。

4.4.5.8 ★★ 同一个投入风险逻辑，量出了两个不同结论：★ 内部自建、对外借力

★ 这一节要回答一个读者一定会问、而且很少有书敢正面回答的问题：

> **★ 为什么企业内部搞开源自部署，对外反而要用大厂的在线能力？这不是自相矛盾吗？**

★ ★ 答案是：不是矛盾，是同一把尺子量出了两个不同结论。

★★ 第六章那把尺子：投入风险 = "这条路还能走多久"
★★ 而它从来不是单独使用的，★ ★★ 它要和"这个场景的核心瓶颈是什么"一起看

	内部应用	对外应用（客户 / 供应商）
核心瓶颈	★ **能不能换路线**——应用与开发层变化极快	★ **有没有流量和获客**
投入风险	★ 高——★ 两年后可能整套装废	★ 不是主要顾虑
技术成熟度	★ 自己能掌控即可	★ ★ 大厂的能力成熟度高得多
流量入口	★ 不需要	★ ★ 自建 APP **没有流量**，这是致命的
结论	★ **开源自部署，保留路线切换权**	★ **用大厂的在线能力**

★ ★★ 于是那两个看似矛盾的结论，其实出自同一条推理：

> **★ 在哪里投入，就在哪里用大厂的能力；★ 而「投入」和「不投入」的选择，由那个场景的核心瓶颈决定。**

★ ★ 而且这条判断还有一个很实用的副产品：

★★ ★★ 它让"要不要自建"这个问题，★★ ★★ 从"信仰之争"变成了"瓶颈分析"
★★ ★★ — 你不再需要争论自建好不好，★★ ★★ 你只需要问一句：
★★ ★★ "★ 这个场景卡住我的，★★ ★★ 到底是技术路线会变，★★ ★★ 还是根本没有流量？"

一句金句

★ 入口策略的方向，比入口的形态重要得多。 ★ 收敛有机会，叠加几乎必败——因为用户永远有选择权。

★ 嵌入式是无感的，客户端是有感的——同一套管理动作，用错了地方就是错的。

★ 在高保密环境里，入口首先不是「选哪个」，是「哪些不能选」。

★ 当一个入口策略看起来自相矛盾，往往是你把两个问题塞进了同一个入口——解法是分层，不是打通。

★ 形态是最后一步，边界是第一步。

★ 方向正确的方案，也会死在时间差上。

4.5 选型评估表（可直接抄）

技术总监真正需要的是一张能带进评审会的表。下面这张表分三块：先判形态，再判组合，最后判风险。

表一：形态判定（四因子，各一行，任意一格为「否」即需复核）

因子	判定问题	本企业结论	证据/来源	判定人
数据能不能出厂	哪些数据绝对不能出厂？占比多少？	□能 □部分 □不能		
成本规模	日均调用量、峰值形态、留存周期？			
合规审计	是否需监管穿透检查？留痕粒度要求？			
运维能力	谁来维护推理与版本？有几人？			

> 判定规则：前两项指向自建、第四项为空 → **不自建**，走 API + 数据分层。

表二：能力组合判定

需求类型	判据（问这一句）	选定做法	是否需要微调	失败信号
知识会变	知识更新是否要求不重训?	RAG	否	检索召回不到正确段落
风格格式	是否要求每次都稳定复现同一格式?	微调（薄）	是	输出格式反复跑偏
多步自主	任务是否需模型自己决定步骤?	Agent 编排	否	步骤循环、权限越界

> 判定规则：三条都命中 → **RAG 打底 + Agent 编排 + 少量微调**；只命中一条 → 只做那一条。

表三：风险与审计清单（上线前逐项打勾）

#	检查项	内部 Copilot	流程内嵌	对客服务
1	输出可回溯到知识原文	必	必	必
2	置信度低于阈值转人工	建议	必	必
3	不可逆动作有人工确认	—	必	必
4	工具/数据域白名单	建议	必	必
5	步数与预算上限	建议	必	必
6	决策依据与业务单据双向索引	—	必	必
7	审计日志含操作者、时间、参数、依据、结果	建议	必	必
8	敏感/法律/用药安全类问题强制转人工	—	必	必
9	对外话术版本管理与发布审批	—	—	必
10	定期抽检与回溯演练	建议	必	必

表四：采购项自检（每一条问一句）

采购项	明年的范式变了，我损失多少？	结论
算力、存储	接近零	可买
标准的模型服务接口	接近零	可买
可导出的知识库工具	接近零	可买
把业务逻辑/数据格式/工作流焊死的平台	要重做一遍	**先别买**

> 采购是结果，不是起点——你要落实哪些应用场景，就采购哪些。

> RAG 把知识打碎， Wiki 把知识连起来。你以为存的是知识，其实存的是向量。

> 一个快速的模糊答案，和一个通过逻辑判断的准确答案，哪个更有价值，一目了然。

> 在企业知识场景里，「快」从来不是第一指标，「能直接用」才是。

4.6 本章小结 + 金句墙

三棵决策树，压缩成一张图：

```
决策树一：私有化 vs API
  先分数据能不能出厂 → 再算成本规模 → 再看合规审计 → 最后看运维能力
  结论形态：数据分层外发 + 路由，不是全上云，也不是全本地

决策树二：RAG vs 微调 vs Agent
  知识会变 → RAG；风格格式 → 微调；多步自主 → Agent
  现实形态：RAG 打底 + Agent 编排 + 少量微调
  ★ 例外（4.2.4）：★ 企业多系统共用一套知识时，
    "自动分段 + 向量库作底座"这两个选择会被四个死结卡住
    → 换成 LLM Wiki（整篇逻辑整理成 Wiki 页面）
    ★ RAG 把知识打碎，Wiki 把知识连起来

决策树三：工具接入
  MCP 标准化 → 接得上；权限代理 + 审计留痕 → 敢让它接
  分级部署 + 统一管控 → 数据不出机，管控不脱管

落地形态：内部 Copilot → 流程内嵌 → 对客服务
  成本与风险递增的真正变量，是错误的可逆性
```

本章真正的产出不是结论，是三个提问顺序。技术总监往后接到的任何技术方案，都可以拿这三句话去问：

```
第一句：这份数据，能不能出厂？
第二句：这份知识，会不会变？变了要不要重训？
第三句：这个 Agent，能碰到哪些系统？谁替它守着权限？
```

金句墙

> **企业 AI 技术选型的第一性问题，不是「哪个更强」，是「哪个你敢让它拿到手」。**

> 私有化不是安全选项，它是一份运维合同。

> 一个模型看全部、一个模型只看一段，是两种完全不同的合规结论。

> 采购是结果，不是起点——你要落实哪些应用场景，就采购哪些。

4.1.4 本节结论

```
先问：有没有哪类数据绝对不能出厂？
  有 → 这些数据走本地，其余按分类走 API + 路由分层
  没有 → API 起步，把省下的运维人力投到场景上

再问：调用规模与运维能力撑不撑得起自建？
  撑不起 → 采购，别自建
  撑得起 → 再问明年的范式变了，这套东西损失多少
```

> 私有化不是安全选项，它是一份运维合同。 一个模型看全部、一个模型只看一段，是两种完全不同的合规结论。

> RAG 负责「知道什么」，微调负责「怎么说」， Agent 负责「先做什么」。

> Agent 越强，越会掩盖检索的烂账。

> MCP 解决的是「接得上」，权限代理解决的是「敢让它接」。

> 从 Copilot 到对客服务，增加的不是技术，是错误的可逆性。

本章要回答：从决定做，到真的被用起来，中间该做什么？ **纪律声明**：企业名、子公司名、事业部代号、产品名全部泛化；素材未覆盖处标 【待核实】，不编造案例、金额与效果数字。

5.0 本章要回答什么

第二、三、四章分别解决了**选事**、**备料**、**选型**。但这三件事做完，项目通常还没开始被使用。

真正卡住项目的，是从「方案通过」到「真的有人每天用」之间的这段路。这段路上有一系列决定项目生死的事，而它们**很少出现在项目计划里**：

```
方案通过了 → 谁先用？用多久？拿什么算成功？
          → 什么时候该停？
          → 谁来维护它？
          → 谁授权它？
          → 高层自己用不用？
```

这一章把这六个问题按顺序摆出来。

顺序很重要，因为它对应一条真实的推进链：

```
步骤1 场景梳理    →  步骤2 数据准备评估  →  步骤3 技术方案选型
   ↓                                          ↓
（做对的事）                                （用对的技术）
                                              ↓
                                  步骤4 试点验证 ↔★ 全书写"避坑"的关键
                                              ↓
                                  步骤5 规模化推广
                                              ↓
                              ★ 两个必须单独讲的专节
                            高管怎么接 AI ／ 助手的边界在哪
```

5.1 步骤一：场景梳理（承接第二章）

这一步的目标不是「选一个场景」，而是**把第二章那三个问过的问题，变成一张可以开工的清单**。

```
场景清单（第二章产出）
        ↓ 加三列
┌──────────────┬──────────────┬──────────────┬──────────────┐
│ 场景          │ 做错了谁赔    │ 现在人工怎么做 │ 谁的工作方式会变 │
├──────────────┼──────────────┼──────────────┼──────────────┤
│ ……           │ ……           │ ……           │ ……           │
└──────────────┴──────────────┴──────────────┴──────────────┘
        ↓ 收敛
    1–2 个场景
```

为什么是 1–2 个，不是 5 个：

```
同时推 5 个场景的结果：
  每个场景分到 1/5 的注意力
  每个场景都只做到"能演示"
  5 个都不能复制
        ↓
  一年后：5 个 POC，0 个能推广的东西
```

而推 1–2 个的结果：

```
一个场景做透 → 拿到可复制的路径 → 第二个场景成本减半
```

这一节唯一需要强调的是「收敛」这个动作。它看起来只是砍需求，实际上是本章第一道防线：

> **清单上的场景越多，越容易出现「每个都做一点、每个都没成」的结局。 而这种结局最致命的不是失败，是它会让组织相信「AI 这条路走不通」。**

5.2 步骤二：数据准备评估（承接第三章）

这一步的目标是**判断「现在能不能做」**，不是「把数据治理完」。

第三章那张判断表在这里直接用：

问题	答「是」	答「否」
核心指标有唯一算法吗？	继续	先治理口径，或换场景
数据能在 2 个系统内拿到吗？	继续	先做取数
历史够长吗（趋势类 ≥ 3 年）？	继续	降级为当期判断类
权限清晰吗？	继续	先做权限梳理
有可引用的文档来源吗？	继续	RAG 无解
出错有人认领吗？	继续	先定责任边界

六个「是」才启动。

这一步的产出是三样东西，缺一不可：

1. 数据可用性清单（这个场景要用的那一份，不是全公司的）
2. 文档治理清单（这个场景会引用的制度/规程，逐份确认所有者、版本、生效期、密级）
3. 风险与降级方案（拿不到的数据怎么办、结论降到什么程度）

第 3 样最常被漏，也最关键。它决定了当数据不到位时，项目是**降级**还是**卡死**。

> 【待核实】本章未引用具体场景的清单实例。若作者愿提供一份真实项目的清单样例（含脱敏），本章说服力会显著提升。

5.3 步骤三：技术方案选型（承接第四章）

这一步的产出不是「选了哪个模型」，而是**一份带约束的技术方案**：

```
┌─ 数据边界 ──────────────────────────────────┐
│  哪些数据出廠？哪些不出？                    │  ← 第三章 3.4
│  权限在哪一层过滤（检索前 / 生成后）？       │  ← 第三章 3.3
├─ 形态 ──────────────────────────────────────┤
│  内部 Copilot / 流程内嵌 / 对客服务          │  ← 第四章 4.4
├─ 能力 ──────────────────────────────────────┤
│  RAG / 微调 / Agent 的组合与边界             │  ← 第四章 4.2
├─ 留痕 ──────────────────────────────────────┤
│  每次调用的输入、输出、版本、责任人          │  ← 第七章 7.3
└─ 兜底 ──────────────────────────────────────┘
   置信度不足怎么办？无来源怎么办？人怎么确认？
```

注意最后两项。技术选型最容易被写成「选了什么」，而真正决定项目能不能过合规评审的，是**留痕与兜底**。

这一步最容易出的错，是把技术方案当成采购方案来做。

```
采购视角：找功能最强的、最好演示的、最便宜的
方案视角：先定边界（数据/权限/留痕/兜底），再在边界内选能力
```

第三章讲过的那句话在这里适用：先想「不做什么」，再想「用什么做」。

5.4 步骤四：试点验证 ★

这是全书写「避坑」的关键一节。

因为前面三步都还在纸面上，而这一步是**第一次让真实的人、真实的业务、真实的错误来检验前面所有的推理**。

而大多数项目的试点是这样做的：

```
选一个部门 → 培训两小时 → 扔一个工具过去 → 三个月后问"用得怎么样"
          ↓
    "还行吧"
          ↓
    然后就推广了，然后就没有然后了
```

这套做法有三个致命缺陷：没有范围、没有指标、没有止损。

5.4.1 灰度范围：谁先用，决定了试点能证明什么

试点的范围不是「随便挑个部门」，而是要同时满足三个条件：

```
┌──────────────────────────────────────────────────┐
│ ① 信任成本低：                                    │
│     这个部门的人，出了问题不会一纸状告到老板那里      │
│     （错得起，才试得起）                           │
├──────────────────────────────────────────────────┤
│ ② 决策权高：                                      │
│     部门负责人自己说了算，不用来回请示              │
│     （能拍板，试点结论才作数）                      │
├──────────────────────────────────────────────────┤
│ ③ 业务痛感强：                                    │
│     这个部门自己就痛，不是被安排来试的              │
│     （痛，才会用；不痛，只会配合）                  │
└──────────────────────────────────────────────────┘
```

这三条同时满足的部门，通常不会超过三个。如果一个都挑不出来，说明前面场景选错了，或者这个组织的信任成本普遍过高。

> ★ 这三条来自作者对试点用人的判断。【待核实】——是否保留这组判据作为正式方法，需作者确认。

5.4.2 试点周期：短到能止损，长到能看出问题

```
太短（几天）→ 只看出"能不能跑通"，看不出"好不好用"
太长（半年）→ 组织热情耗尽，问题被时间磨平，止损已经来不及
```

一个可用的经验区间：

```
4–8 周  ← 建议区间
├─ 前 2 周：上手期（记录卡在哪儿）
├─ 中间 3–4 周：真实使用期（★ 关键：必须用真实业务，不能用演示数据）
└─ 最后 1–2 周：验收与结论期
```

中间那段「必须用真实业务」，是试点最容易作弊的地方。

```
真实使用期最常见的两种假象：

  假象一：用"干净"的演示数据跑 → 避开了所有真实场景的脏乱
              → 结论：可行 → 推广 → 推广后集体翻车

  假象二：由项目组代做 → 用户只要"看结果"
              → 结论：可行 → 推广 → 推广后没人会用
```

试点必须由业务的人、在真实的业务上、用他们自己的数据做。这条违反了，试点结论就没有意义。

5.4.3 验收指标：怎么设一个不会被业务「做样子」的指标

这是试点设计里最难的一件事。

先看一种常见的失败设计：

```
验收指标：使用人数 ≥ 10 人
        ↓
业务怎么应付：让 10 个人注册账号，打开一次
        ↓
指标达成 ✓   实际价值 ✗
```

因为这个指标只考核「行为发生」，不考核「价值产生」，所以它必然可以作秀。

设计验收指标的三条原则：

原则一：指标要落在「业务结果」上，不是「使用行为」上。

不合格（可作秀）	合格（难作秀）
使用人数	这个场景原来每月人工 X 小时，现在降到多少
生成条数	生成的结论被采纳的比例
登录次数	原来要 3 天的汇总，现在当天出结果且被用于一次决策
满意度打分	原来漏掉的差错，现在漏了几起

原则二：至少留一个「反向指标」，防止为了达标而作弊。

```
正向指标：结论被采纳率 ↑          ← 会被"只挑好说的"操纵
反向指标：人工复核发现的错误率 ↑   ← 错误被发现了，说明真的在用
```

> 一个只涨正向指标的试点，基本可以判定为「没真用」。

原则三：指标在试点开始前就写下来，且由业务方签字确认。

```
事后再定指标 → 指标一定会被调整成"能达到"的样子
```

原则四：★ 指标必须匹配入口形态——★ 因为「无感」和「有感」的项目，验收方式本来就不一样。

★ 这一条来自第四章 §4.4.5.6 的推论，在试点设计里非常容易踩：

入口形态	★ 别用什么指标	该用什么指标
嵌入式（无感）	**★ 使用率、活跃人数**	**★ 业务指标**：省了多少工时、错了多少单、被拦下多少风险
客户端（有感）	★ 只看功能是否可用	**★ 使用率必须正视**，但要和「有没有替代掉一件具体的老活」一起看

★ 原因是这样的：

```
★★ 嵌入式项目本来就该是低存在感的——★ ★★ 用户不改变任何动作，AI 在后台把活干完
★★    → 拿"多少人用"去考核它，★ ★★ 是在考一件它本来就不该做的事
★★    → 拿它去追推广，★ ★★ 更会逼出"为了使用率而造使用"的假动作
          ↓
★★ 客户端项目反过来：★ ★★ 用户每天都要决定"要不要打开它"
★★    → 使用率不是可以回避的指标，★ ★★ 而是这个项目成不成的第一道判据
★★    → 但★ 只看使用率也不够：★ ★★ 天天打开但不替代任何老活的，★ ★★ 同样是没做成的
```

★ 所以这条原则的实质是：★ 先想清楚这个项目是「无感」还是「有感」，再决定拿什么尺子量它。

> **★ 「同一套管理动作，用在嵌入式项目上是错的，用在客户端项目上是对的。」**

5.4.3.1 ★★ 验收标准不能是「闭环」：★ AI 项目验的是「环节」，不是「流程」

★ 这一节来自作者 2026-11-03 补充的一个结构性判断，它直接改写验收标准的写法。 ★ 完整的论述见 8.0.2 。

★★ ★★ 先说这个判断：★★ 当下的大模型应用，★ ★★ 很少真正实现闭环管理
★★ ★★ 大多是嵌在原有业务闭环的某一个环节里，★ ★★ 做的是流程环节的再造与重塑
↓
★★ ★★ 而"全生命周期由 AI 完成"，★ ★★ 在传统企业里几乎做不到
★★ ★★ 因为它要改组织、改流程、改管理体系——★★ ★★ 而管理体系那一层，
★★ ★★ ★★ 不可能仅因某个岗位的工作就改变

★ ★★ 而这件事一旦成立，验收标准就跟着变：

★★ ★★ 传统系统的验收：★★ "★ 这个流程跑通了吗？"
★★ ★★ AI 应用的验收：★★ "★ ★★ 这个环节上的人，是不是省了时间、少担了责？"
↓
★★★ ★★★ ★★ 差别在于：★★ ★★ 前者要求你接管一整条流程，
★★★ ★★★ ★★ 而后者承认你只改造了其中一格

★★ ★★ 而这条对验收设计的直接影响，有三条：

★★ ① ★★ 验收指标里不要出现"★ 全流程贯通""★ 端到端闭环"这类词
★★ 它们在 AI 项目里没有可操作性，★ ★★ 写进去就等于没写

★★ ② ★★ 指标要落在 ★★"这个环节的投入产出"★★ 上
★★ 原来这个环节每月人工 X 小时 → 现在降到多少
★★ 原来这个环节每月出错 Y 起 → 现在多少
★★ ★★ 而不是落在"这个系统覆盖了几个环节"上

★★★ ③ ★★★ ★★ 立项时就要写明：★★ ★★ 这个项目负责哪一格，★★ 不负责哪一格
★★ ★★ 写清楚了，★ ★★ 后面才不会用"没闭环"来否定它
★★ ★★ 也没人会用"它还没闭环"来解释它为什么没做出效果

★★★ ★★★ 而这条其实反过来给出了一个更重要的提醒：

★★★ ★★★ 「★ 做 AI 就要做成闭环」——★★ 这句话听着都对，但它不是一个可执行的目标。 ★★★ ★★★ 因为在传统企业里，闭环意味着改组织、改流程、改管理体系， ★★★ ★★★ 而这三样，没有一样是项目组能自己决定的。 ★★★ ★★★ ★★ 于是这句话的真实效果，★ ★★ 是把一个不可能的目标， ★★★ ★★★ ★★ 压给了一个最没有能力完成它的人。

★★★ ★★★★ 它的正确版本是： ★★★ ★★★★ 「★ 不追求闭环，★ ★★ 先把一个环节做到位，★★ 再看它能不能连成线。」

★ 金句

「嵌入和闭环，是两种完全不同的生意。嵌入改造的是一个环节；闭环才谈得上改变一门业务的运行方式。」

5.4.4 止损条件：★ 什么时候该停

这是试点最容易被跳过、也最值钱的部分。

因为项目组天然有「再试一个月」的冲动——**而组织的时间与热情是最贵的成本**。

止损条件必须在试点开始前写好，并且是**可判定的**：

止损条件（示例）	判定方式
4 周后主动使用率 < 30%	从日志直接统计，无需访谈
关键指标口径仍不统一	数据可用性清单的"口径"项未关闭
出错无责任人	错误上报无人响应超过 N 次
每次输出需人工全量重做	抽样 20 条，重做比例 > 50%
业务方要求停用	一位决策者提出即可启动复核

止损不等于失败。这句话要写进项目制度里：

> **止损是「承认这个场景现在不成立」，不是「承认这帮人不行」。**
>
> **一个能在第 6 周止损的项目，比一个拖到第 30 周才承认失败的项目，节省的是整个组织的信任。**

而止损之后还有一件必须做的事：把这个场景的失败原因写进下一轮的场景清单。**止损也要产生知识。**

★★ ★★ 而止损这条，在 AI 领域有一条与传统不同的特殊性质，值得单独写：

```
★★★ ★★★ 一个与 AI 有关的结构性事实（★ 作者 2026-11-03 补充，详见 8.5.3）：

★★ ★★ 在 AI 领域，"★ 用了几个月、后来没效果就放弃"的案例其实不多
★★ ★★ 因为 AI 的失败是 ★★★ 速生 的：
   · ★★ 一开始不好用 → ★★ 用户迅速失去信心 → ★★ 根本不会用几个月再放弃
   · ★★ 一旦被持续用几个月 → ★★ 产生粘性 → ★★ 用户会提新需求，★ 不会主动放弃
        ↓
★★★ ★★★ ★★ 于是：★★★ 传统项目的止损窗口在中期
★★★ ★★★ ★★（都投了这么多，不能轻易停）；
★★★ ★★★ ★★ 而 AI 项目的止损窗口在 ★★★ 早期
        ↓
★★★ ★★★★ ★★ 所以对 AI 项目，★★ ★★ "★ 再给它一个月"这句话，★★ ★★ 几乎总是错的
```

> **★★★ ★★★ 而这给止损条件加了一条，不是数字，是原则：**
>
> **★★★ ★★★ 「★ 对 AI 项目，止损的收益曲线是朝下的——★ ★★ 越早停，省下的越多。」 ★★★ ★★★ 而这个道理在传统项目里是反的，★★ 所以很多从传统口味的项目经理，★★★ ★★★ 会在 AI 项目上做出最贵的那个决定：★ ★★ 再等等看。**

5.4.5 试点要双验

一个试点成功了，只能证明一件事：**这件事在这个部门能成**。

```
双验：
  可行性 —— 这个场景技术上做得成吗？
  可复制性 —— 这个做法能搬到别的部门吗？
        ↓
  只验可行性，不验可复制性
        ↓
  推广时才发现：它成立是因为这个部门有个能人
        ↓
  推给其他部门 → 全部失败 → 结论："AI 不行"
```

「**这个部门有个能人**」**是最常见的不可复制原因**。所以试点时要额外问一句：

> **这个做法里，有多少是靠这个部门的特殊条件？** 特殊条件包括：某个人特别厉害、某份数据特别全、某个流程本来就很规范。

只要答案里有一句「靠某个人」，推广方案就必须先解决「这个人不在场时怎么办」。

5.4.6 ★ 一个真实的试点案例：营销 AI 助手（做砸了的那一次）

> **本节与第八章 8.5 节是同一个项目的两个视角**：第八章讲「它为什么失败」，这一节讲「**试点本来应该在哪一步止损**」。 ★ 这是作者亲历项目。

案例的试点设定

试点对象：某大型制造企业营销条线的一线销售人员 + 其管理层
试点内容：两块
① 知识库：制度政策解读、销售政策与销售方案解读、产品信息
② 数据：一线随时查自己的销售数据，管理层可查下属与整体
研发方式：营销部门 + IT 部门联合研发
时间线：实施约 3 个月 → 上线 → 上线后约 3 个月基本无人使用
→ ★ 撑到半年周期正式宣告失败，此后停投入
★ 系统至今仍挂在线上，零使用量
试点结果：★ 功能做出来了，场景没有跑起来

这个试点，四个「应该」分别落在哪

试点四要件	这个项目上的实际表现	差距
范　围（谁先用）	营销一线 + 管理层	范围本身不算错，但**没有按「信任成本低 × 决策权高 × 业务痛感强」筛过**——一线是「被要求试」，不是「自己想试」
周期	实施约 3 个月后上线；上线约 3 个月即「基本无人使用」	**★ 有事实上的周期，但没做到「到点看结论」**——上线 3 个月后就该做去留决策，实际撑到半年周期才宣告失败
投入	★ 常态 5 人（IT 侧 2.5：开发 / 数据整理 / 移动端适配；业务侧 2.5：营销核心），★ 测试期营销扩到 5 人 → 参与峰值 8 人	**★ 常态 5 人投入半年、参与峰值 8 人**，且这笔投入分散在多个科目，**没有任何一张表显示「这个失败花了多少」**；★ 且 ★ 参与人数峰值出现在测试期，而日常使用量为 0——**参与人数不能当健康度指标**
验收指标	功能层面达成（能查、能问）	**★ 指标设在了「功能有没有」上，而不是「用户满不满意、数对不对」上**
止损条件	未见止损线	**★ 没有止损线，于是项目不是失败，而是慢慢死掉**

★ 从试点设计的角度看，这个案例暴露的不是执行问题，是四个要件里有三个没设或设错了。

如果当时按第五章的方法做，三个具体动作

动作一：验收指标要落在满足率上，不是功能上。

❌ 当时的指标形态：功能做出来了，能查能问 → 通过
✅ 应该的指标形态：
- 一线每周主动使用率 ≥ 30%
- 数据类请求的"满足率" ≥ 50%
  （抽 20 个高频问题，系统给出的数是否就是他要的那个数）
- 知识库类问题的一次解决率 ≥ 70%

★ 这里最值得记的是「满足率」这个指标形式——它把「数对不对」变成了可抽检、可统计的东西。而「能查」是无论如何都会达成的。

动作二：4–8 周就要看一次结论，而不是等「系统建好」。

★ 这个案例的结构性缺陷：
知识库那半边能用 → 让人以为项目在推进
数据那半边不对 → 让人以为是"数据没到位，再等等"
↓
于是永远有理由继续做
↓
★ 而 6 周一次的口径对齐会，本来能在一周内暴露"口径根本对不齐"这个事实

动作三：止损线要写在项目启动文档里，并且由业务方签字。

```
止损条件（本项目当时缺失）：
    试点 6 周时若
      · 主动使用率 < 30%
      · 数据类请求满足率 < 50%
    → 暂停功能开发，先治口径与入口
            ↓
★ 止损的意义不是承认失败，是把钱从"加功能"转到"治数据"
    —— 因为加功能会掩盖数据问题，而不是解决它
            ↓
★ 而本项目的现实是：止损线在"上线 3 个月后基本无人使用"时
    就早该触发了，实际撑到半年周期才宣告失败
    —— ★ 这半年的后半段，投的是"明知不行、但还没到宣布的时候"的钱
            ↓
★ 所以缺的不是阈值，是"阈值触发之后，谁有权拍板把它关掉"
    —— 没有这个人的制度，阈值只是一张纸
```

★ 这个案例对「试点」这一节的反直觉补充

> **试点跑成功了，能证明的东西有限；试点跑失败了，能证明的东西很多。**
>
> 这个案例证明了三件事：
>
> ① 数据口径不统一时，功能做得再好也没用
> ② 入口不在工作流里时，合规正确的路径一定输
> ③ 验收指标设在功能层时，项目一定会被误判为成功
>
> **而这三条，成功案例都证明不了——因为成功案例里，这些坑都恰好被绕过了。**

所以本书的第八章特意安排了一个失败项目；而这一节要补一句：失败项目的复盘价值，恰恰在于它把成功案例绕过去的坑，全部摊开给人看。

5.5 步骤五：规模化推广

推广阶段要解决三件事：**组织与授权、运营机制、持续改进**。

5.5.1 组织与授权：谁批准、谁运维、谁担责

推广时最容易出现的混乱是：**用的人多了，但没人负责**。

```
推广前必须明确的三件事：

  ① 授权：谁有权让这个工具在哪个范围内被调用
      （谁批准新用户接入？谁能扩权限？谁能下线它？）

  ② 运维：出问题时谁处理、多久内处理
      （尤其：业务方自己建的工具坏了找谁？）

  ③ 担责：它参与的业务环节里，出错谁签字
```

第三条最容易被推着走。推广期的典型场景是：

```
某场景在试点时由资深专家复核 → 推广后由轮岗的新人使用
    ↓
新人不具备判断 AI 对错的能力
    ↓
但流程上，他现在是"签字人"
    ↓
出错了，流程合规，人却冤
```

推广不是把人换掉，是把复核能力也复制出去。这一条要写进推广方案。

5.5.2 运营机制：谁来维护它

推广期最常见的三种死法，就在这里。

```
死法一：试点完就散
    试点团队解散 → 工具无人维护 → 半年后模型过期、知识不更新
    ↓
    现象：项目总结写得很好，然后没有然后

死法二：无人运营
    上线了，但没人看反馈、没人更新知识、没人盯指标
    ↓
    现象：用户逐渐流失，理由是"不好用"
    ↓
    ★ 而"不好用"通常不是模型问题，是三个月没人管

死法三：运维成本吞掉收益
    推广得越广，维护量越大
    ↓
    现象：算账时发现，省下来的人时全花在了维护上
```

这三种死法有一个共同点：它们都不是技术问题，都是「没人负责」的问题。

所以推广方案里必须有一行：

```
┌──────────────────────────────────────────────┐
│ 这个工具上线后的长期负责人：_____（名字，不是岗位）  │
│ 每周投入：_____ 小时                           │
│ 反馈闭环由谁接：_____                          │
└──────────────────────────────────────────────┘
```

「名字，不是岗位」是重点：写岗位意味着没人真的负责。

> 运营机制的具体形态可参考第四章提到的 MCP 与权限代理思路：**把「谁在什么范围调了什么」变成系统自动记录的事，而不是靠人记得。**

5.5.3 持续改进：让它一年比一年好

持续改进的循环：

使用 → 发现问题 → 定位到源头（知识/规则/数据口径）
→ 改源头 → 记录一条通用规则
→ 回到使用

这正是第三章 3.2.3 讲的反馈闭环——在推广阶段，它从「一个清单」变成「一个机制」。

判断一个 AI 场景是否健康的三个信号：

信号	健康	不健康
用户提问的重复率	下降（同类问题越来越少）	不变（说明每次都在重复劳动）
人工复核的错误发现率	下降（模型在变好）	持平（模型没变好，只是没人查）
新场景从提出到上线的周期	缩短（能力在复用）	不变（每个场景都是一次性项目）

5.6 专节：高管怎么接 AI —— 不是培训，是助理

这一节是本书最具原创性的一节，也是全书最反直觉的主张。

5.6.1 两个真障碍，培训解决不了

```
高管用不起来，通常归因为"积极性不够""学习意愿不强"
    ↓
于是方案是：培训、专题会、训练营、大赛
    ↓
但真实的障碍是另外两件事：

  障碍一：不熟悉快速迭代的 AI
          → 有门槛、达不到预期 → 于是判定"没用"

  障碍二：顾虑数据安全
          → 不敢用 → 于是判定"不安全"
```

认知提升解决不了这两个障碍。它只能让人「知道 AI 是什么」，而这两个障碍需要的是「**给我一个敢用、好用、还不涉密的工具**」。

于是有了一个反直觉的主张：

> **对高层，不要先教 AI，要先配助理。**
>
> **员工要的是培训，高层要的是助理。**
>
> **给高管讲一百次大模型，不如给他一个真的能用的助手。**

5.6.2 助理的四类定制需求

给一位高管配 AI 助手，先问四个问题——**这不是设想，是工作量清单**：

#	高管要的	助理要做的	典型形态	安全级别
①	**他需要的数据**	把他平时要的数据汇总好	每日经营与业务简报、异常提示	高（业绩、利润、人力）
②	**他脑子里的规则**	把判断标准显性化并据此判断	按规则判断、出初稿、提示例外	中（规则本身常涉密）
③	**他每天要批的流程**	AI 预处理，他只做确认	审批预填、汇报初稿、差异比对	中高
④	**他关心的信息**	监控 + 摘要 + 提示	政策、竞品、舆情、项目进展	低（可外部化）

「量身定制」这四个字听起来很贵，但它是可以估算的：四类需求 → 四类工作 → 可分工、可排期。

而它贵得值，因为高层的使用是杠杆最高的。

```
高层不用 AI  → 下面的人不会用 → 整个组织停在 L1
高层用了 AI  → 下面的人看到有用 → 才有第五章 5.5 说的那种扩散
```

5.6.3 三个设计决定

决定一：出处可追溯。

每条结论指向原始来源（文件、数据、制度条款）。

> **让高管能自己核，而不是被迫「选择信或不信」。**

决定二：置信度阈值。

```
置信度 ≥ 阈值 → 直接给结论
置信度 < 阈值 → 不给结论，改为列出"需要你判断的分歧点"
```

这一步把 AI 从「决策者」降为「提示者」，同时把风险显性化。高管看到的是「有分歧的地方」，这恰恰是最值得他花时间的地方。

决定三：草拟制。

> **助理起草，高管拍板。**
>
> **高管的 AI 助手不是替他决策，是替他省掉一半的读和写。**

5.6.4 ★ 最大的心理障碍可能不是「不会用」，而是「被记录」

这一条值得单独讲，因为很多方案会在这里翻车。

助理天然携带两项能力：**会议/沟通留痕** + **决策记录**。而对很多高管（尤其合规敏感岗位），「被记录」意味着：

```
自己的判断被留档 → 成为日后被追溯的证据
沟通被当成"可被审阅的内容" → 说话变谨慎、变慢、不说真话
```

这会同时毁掉两件事：助理的价值，和组织的坦诚。

（这里和第一章 1.4 的悖论是同一个：员工效率提升了，但如果没有相应减少事务、兑现收益，抵触就是理性选择。**助理如果只增加记录、不减少事务，就是纯负担**。）

设计对策：

```
① 提供"仅对我可见 / 不入正式记录"的模式，让高管先建立信任
② 明确留痕的用途边界（服务本人 / 不作绩效素材），并让高管看得见留了什么
```

5.6.5 试点选人：三维度 + 双验

高管助手的试点选人，和 5.4.1 的三条件是同一套逻辑：

```
信任成本低 × 决策权高 × 业务痛感强
```

但高管助手还要多做一次双验：

可行性 —— 这个助理对他真的有用吗？
可复制性 —— 这个配置能复制给另一位高管吗？

复制的难点在「量身定制」本身：一个人的规则、关注点、阅读习惯都是独有的。

> 所以高管助手规模化推广的方式，**不是把同一个助手发给所有高管，而是把「定制流程」标准化**——把「提炼四类需求 → 配置 → 草拟制校验 → 阈值设定」变成一条可复制的流水线。

5.7 专节： AI 助手能做到哪一步 —— 组织透明度是天花板

上一节讲「高管怎么用起来」，这一节讲「**最多能用到什么程度**「。两者必须一起讲，因为只讲前者会让人产生不切实际的期待。

5.7.1 两种能力层级

层级	能力范围	数据来源	前置条件	可实现性
L1 · 系统内智能感知	汇总已有系统的数据、给异常提示、辅助判断	ERP/OA/CRM/ 项 目 管理系统等**已有系统**	无	**当前绝大多数企业能做到**
L2 · 全盘管理	全面掌握信息与数据，感知会议决议与个人行为，支撑「全面管理」	组织的**全部真实过程**	**★ 组织文化达到「完全透明化管理」**	**绝大多数企业做不到**

关键区分：

L1 是对结构化数据的读取与加工
L2 是对组织过程的感知 —— 会议决议、谁说了什么、决策怎么来的、行为轨迹

L2 需要的数据压根不在系统里，在人和会议里。

5.7.2 为什么 L2 卡在文化上

```
想让高管助手"全面管理所有信息和数据"
        ↓
需要感知：会议决议 · 个人行为 · 决策过程
        ↓
这些信息在哪里？
        ↓
不在系统里 —— 在会议桌上、在沟通中、在人的行为里
        ↓
要让 AI 感知到，必须先把它"变成数据"
        ↓
而"变成数据"的前提是组织愿意被记录、被看见
        ↓
★ 这不是技术问题，是组织文化问题
        ↓
组织文化未达"完全透明化管理"
        ↓
✗ L2 不成立 → 只能退回 L1
```

> **AI 感知的是系统，组织运行在人的行为里。要感知后者，先要组织愿意被看见。**
>
> **技术能穿透系统，不能穿透文化。**

5.7.3 分寸声明（这一条不能省）

```
透明化是为了让决策看得见全貌，
不是让管理者看得见每个人的一举一动。
```

这句话不是免责声明，是方案能否被组织接受的分水岭。

一旦「高管 AI 助手」被理解为「监控工具」，它会遇到两股力量同时反对：

```
管理层：担心被盯着
员工：担心被盯着
        ↓
助理成了"记录工具"，而不是"省事工具"
        ↓
没人配合真实数据 → L1 都做不好
```

所以在启动高管助手之前，必须先把用途边界写清楚并公开：

✅ 做什么：帮高管省掉读与写、把分散信息汇总、把他的规则显性化
✅ 做什么：让决策看得见全貌（有依据、有出处）
❌ 不做什么：不做个人行为的持续监控
❌ 不做什么：不作为绩效素材

5.7.4 给作者的一句提醒

这一节在书里的作用是**设限**，所以它必须诚实：

> **做不到全盘管理不是 AI 不行，是组织还没准备好被看见。**

而反过来也一样：

> **一个把 L1 做到极致的组织（L1 才是绝大多数企业现在能拿到的），价值已经足够大。**

不要用 L2 的想象去卖 L1 的产品——那是一切失望的来源。

5.8 本章小结

五步法，六个要点：

① 场景收敛到 1–2 个 —— 铺得越宽，越像"AI 走不通"
② 数据先评估再治理 —— 六个"是"才启动，不是治理完再做
③ 技术方案先定边界再选能力 —— 数据/权限/留痕/兜底，四项必须先写
④ 试点必须四件齐全 —— 范围、周期、指标、止损，缺一不可
④' ★ 验收验"环节"不验"闭环" —— 嵌入和闭环是两种生意
⑤ 推广先定人再谈机制 —— 谁授权、谁运维、谁担责，三件事写名字
⑥ 高层要助理不是培训 —— 且要知道天花板在组织透明度

这五步里，最容易被跳过的分别是：

步骤	最常被跳过的	跳过的后果
场景梳理	收敛	5 个 POC ， 0 个能推广
数据准备	评估（直接开干）	卡两个月，或者带着脏数据硬上
技术选型	边界与兜底	过不了合规评审，返工
试点验证	**止损条件**	**拖到第 30 周才承认失败**
规模化推广	长期负责人	试点完就散，半年后没人维护

而这一章真正的落点只有一句：

> 从决定做，到真的被用起来，中间隔着的不是技术，是六张必须有人签字的纸。

> ★ 嵌入和闭环，是两种完全不同的生意：嵌入改造的是一个环节；闭环才谈得上改变一门业务的运行方式。

★ AI 项目的失败是速生的——不好用，用户就不用了；所以止损越早，省下的越多。

★ 对 AI 项目，「再给它一个月」这句话，几乎总是错的。

5.9 金句墙

> 清单上的场景越多，越容易出现「每个都做一点、每个都没成」的结局。

> 试点必须由业务的人、在真实的业务上、用他们自己的数据做——违反了，试点结论就没有意义。

> 验收指标要落在业务结果上，不是使用行为上；使用行为是可以作秀的。

> 止损是「承认这个场景现在不成立」，不是「承认这帮人不行」。

> 一个能在第 6 周止损的项目，比拖到第 30 周才承认失败的项目，节省的是整个组织的信任。

> 推广不是把人换掉，是把复核能力也复制出去。

> 推广期最常见的三种死法——试点完就散、无人运营、运维成本吞掉收益——都不是技术问题，是「没人负责」。

> 写岗位意味着没人真的负责，写名字。

对高层，不要先教 AI，要先配助理。员工要的是培训，高层要的是助理。

高管的 AI 助手不是替他决策，是替他省掉一半的读和写。

助理如果只增加记录、不减少事务，就是纯负担。

AI 感知的是系统，组织运行在人的行为里。要感知后者，先要组织愿意被看见。

技术能穿透系统，不能穿透文化。

透明化是为了让决策看得见全貌，不是让管理者看得见每个人的一举一动。

不要用 L2 的想象去卖 L1 的产品——那是一切失望的来源。

从决定做，到真的被用起来，中间隔着的不是技术，是六张必须有人签字的纸。

★ 嵌入和闭环，是两种完全不同的生意：嵌入改造的是一个环节；闭环才谈得上改变一门业务的运行方式。

★ AI 项目的失败是速生的——不好用，用户就不用了；所以止损越早，省下的越多。

★ 对 AI 项目，「再给它一个月」这句话，几乎总是错的。

本章要回答：怎么向 CFO/老板证明这事值？以及——**该不该买、买什么。** **纪律声明**：★ **本章按通用方法撰写，不写任何一家企业的真实财务数据**（作者明确要求）。企业名、人名、产品名全部泛化；素材未覆盖处标 【待核实】，不编造金额与效果数字。

6.0 本章要回答什么

老板问的从来不是「AI 好不好」，而是三个问题：

```
① 这笔钱要花多少？
② 能换回来什么？
③ 为什么别人买平台你不买？
```

大部分企业的答法是第一种：报一张预算表。

而这张预算表往往漏掉最贵的那一项。

更要命的是第三个问题——在 AI 这行，「该不该花这笔钱」远不如「**这笔钱买的东西会不会明年就作废**」要紧。

所以本章把算账做成两件事：

```
6.1–6.2  把账算全：成本五项、收益四类（传统算账）
6.3–6.4  把账算对：采购决策方法、云与本地的真问题（战略算账）
6.5–6.7  落到纸上：ROI 表、KPI 设计、不同规模企业的差异
```

> **一句话主张：采购是结果，不是起点。**
>
> 第二章选场景 → 第三章看数据 → 第四章定形态 → **第六章据场景倒推采购清单** → 第五章按预算定试点范围。 这就是本书「治决策」的价值主张——**把「要不要投 AI」这个假问题，换成「这几个场景值不值得投」这个真问题。**

★★ 本章的核心框架： AI 项目的账，要算三笔

★ 这一节是全章的骨架，后面每一节都在往这三笔账里填东西。

```
┌──────────────────────────────────────────────────────────┐
│ ★ 第一笔：钱                                              │
│    授权费 / 咨询费 / 硬件 / 实施费  ↔  人力 / 时间          │
│    → 这是所有人都会算的那一笔                              │
├──────────────────────────────────────────────────────────┤
│ ★★ 第二笔：能力                                           │
│    判断力（选型、配置、维护）  ↔  兜底（有人替你负责）       │
│    → 这笔账很少有人主动算                                  │
├──────────────────────────────────────────────────────────┤
│ ★★★ 第三笔：投入风险 ★★★                                  │
│    ★ 路线变化时，谁买单？                                  │
│    → 这笔账几乎没有人算，而这恰恰是 AI 行业最要命的一笔      │
└──────────────────────────────────────────────────────────┘
          ↓
★★ ★★ 而这一章最想说的话就在第三笔里：
    ★★★ 在一个高速变化的领域里，
    ★★★ 付费买的是"别人的路线"，
    ★★★ 而企业真正要的是"★ 保留换路线的权利"
```

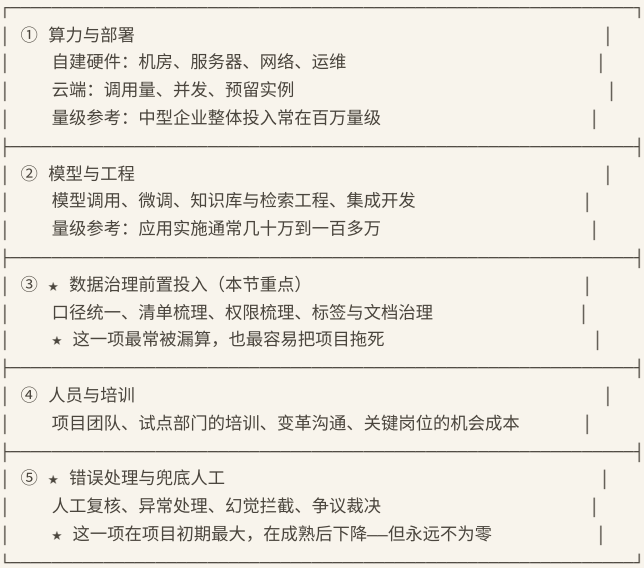
★ 为什么第三笔最重要？因为它是唯一一笔「事后才知道贵不贵」的账。

```
第一笔账：签字那天就能算出来
第二笔账：签完半年后才知道贵不贵
★★★ 第三笔账：★ 要等下一代范式出来，才知道贵不贵
          ↓
★★ ★★ 而在 AI 这个领域，★ 下一代范式来得比多数企业的采购周期还快
          ↓
★★★ 所以这一章有一个很硬的推论：
    ★★★ 你无法靠"算得更细"来规避第三笔风险——
    ★★★ 你只能靠"★ 把风险结构换掉"来管理它
    ★★★ 这就是 6.3.2 那条主张的真正来由
```

6.1 成本侧：五项，其中一项从不出现在预算表里

AI 项目的成本不是一项，是五项。而预算表上通常只写了两项。

6.1.1 五项成本



而行业里被讨论最多的投入，通常发生在前面：

支出项	泛化量级（不指向具体企业）
外部咨询	一百多万到两百万
设备购买 / 硬件算力	几百万
应用实施	几十万到一百多万

> **写法提醒（给作者）**：不写「我们花了 X」，写「**在这个行业里，一个中等体量的企业把 AI 这件事做起来，咨询、算力、实施三项加起来通常是百万量级**」。用意是让读者产生」这确实不是小数目「的实感，从而接受后面」最小化投入「的主张。

6.1.2 ★ 第三项：数据治理前置投入

这一项之所以最常被漏算，是因为它**不发生在技术团队里**。

```
预算表上写的：   买模型 + 搭系统 + 做集成
实际发生的：
   ① 三个口径对不上 → 要业务部门开会定 → 要 IT 改数仓 → 要重跑历史数据
   ② 文档没有生效期 → 要逐份找所有者确认 → 要建标签
   ③ 权限不清 → 要梳理岗位与授权 → 要改接口
   ④ 历史数据不够 → 要么改目标，要么外部采购
        ↓
   以上每一项都不是"技术采购"，而是"业务与治理的工作量"
        ↓
   它不进任何一张技术采购单
```

这一项的三个特征决定了它必然被漏：

```
① 不在采购合同里      → 采购流程看不见它
② 不由技术团队完成    → 预算归口的技术负责人报不出来
③ 发生在项目启动后    → 立项时的估算里没有它
```

所以算账必须在立项时就把它写进去。第三章 3.5 讲的「最小起步」能压低这一项，但压不掉——**第一个场景要用的数据，一样得治**。

★ 而这一项漏算的代价，往往在一年后才显形，而且显形的方式很隐蔽。

```
★ 一个真实的方向性失败（详见 8.5.2）：
   想对 C 端做数字员工（穿戴设备/智能屏/虚拟数字人），
   规划时与国内头部大厂合作
          ↓
★ 停下来，三条原因之一是：
   ★★ "企业内部数据沉淀和管理模式不成熟，无法适应新模式"
          ↓
★ ★ 注意这一条的形态：
   它不出现在任何一张采购单上，
   它不出现在任何一份技术方案里，
   ★ 而它恰恰是让整个方向站不住的那一条
          ↓
★ 所以算账时，除了"数据治理前置投入"这项成本，
★ ★ 还要加一条更前置的判断：
   ★ "这个新模式要用的数据，现在在我们公司里以什么形式存在？"
   ★ 存在系统里且结构化 → 可以往下走
   ★ 还在纸单、微信、口头里 → ★ 这不是 AI 项目，
     是流程数字化项目，且该由流程负责人牵头
```

6.1.3 ★ 一份真实披露过的成本结构（★ 已按作者要求去除全部金额）

> **脱敏说明**：本节所有金额已按作者要求删除，只保留**结构与相对量级**。金额本身不可比（不同企业规模差一个数量级），而**结构不可比才最值得看**——它揭示的是「钱花在了哪一类事情上」。

```
┌─ 一期 AI 创新专项的公开投入结构（不含金额）──────────┐
│  人力                                              │
│    业务 AI 领域负责人 二十余人 / IT AI 领域负责人 二十余人 │
│    项目参与人数 七十余人 / AI 创新专项参与总人数 两百余人    │
│    ★ 专职从事 AI 相关工作的人力：不足两人             │
│                                                    │
│  投入构成（按类别，不含金额）                        │
│    外部咨询费（若走这条路）                          │
│    培训                                            │
│    平台开发费                                      │
│    20+ 个数字员工开发费                            │
│    移动端应用开发                                  │
│                                                    │
│  另一本账（集团/上级层面）                           │
│    AI 战略分解项目投入                             │
│    后期原有算力投入                                │
│    上级单位可能投入的算力资源                        │
└────────────────────────────────────────────────────┘
```

★ 这份结构里藏着三条对全书都极其重要的信息：

第一条：自主推进真正省下的，是「咨询」这一块，而不是「开发」这一块。

```
平台开发 + 20+ 数字员工开发 + 移动端应用
        ↓
而咨询是"一次性的方案"
自主开发是"自己的能力"
        ↓
★ 这就是第六章 6.3.2"尽量不预先采购产品化平台"最硬的一条证据：
   钱省下来了，而且省下的方式不是"少做"，是"自己做"
        ↓
★ 而"自己做"的成本不在预算表上：
   它是那些本来要投给咨询的时间，转投到了自己的团队身上
```

第二条：★ 最贵的那一项成本，在这份账里根本不是钱。

★ 专职从事 AI 相关工作的人力：不足两人

↓

而这个项目要推进：

- 上百项需求的梳理与分类
- 5 个共性解决方案 + 9 个领域解决方案
- 数百人次培训的组织
- 三十余支队伍、一百五十余人的大赛运营
- 二期五大成果的落地
- 本地算力平台（开源推理架构 + 容器化部署）

↓

★ 算一下：不足两人的专职人力，
要同时承担上面全部七件事

↓

★ 所以所有"投入很可控"的叙事，都漏掉了最贵的一项——

★ 那两个人一整年的人力，以及他们被拖住的机会成本

第三条：还有一本更大的账，在集团层面，而且它自己承认了一个问题。

本企业自主推进（一期口径）

↓

上级单位的战略分解项目 + 算力投入 + 可能投入的算力资源

↓

★ 而汇报材料自己承认：
市场部、数转办、数字化中心、各板块部门各自做规划和预算，
上级单位也有自己的规划

↓

★ 结果：实施投入缺少统一策划

> **★ 这条正好接上 8.5 节那个失败案例的病根—— 营销助手失败于「口径和入口的解法不在项目组手里」；而这一条说的是「连钱和规划都不在同一张表上」。 两者是同一个病的两种表现：责任分散在多个主体手里，而没有一个主体握有全局。**

> 【待核实】「专职 AI 人力不足两人」这一表述是否对外可披露；以及各级投入的**相对**比例（哪一块最重），可用于本章「钱花在哪」的一段。金额一律不出书。

6.1.4 ★ 第五项：兜底人工

这一项最容易被认为是「暂时的」，但它有一个长期特征：

```
一个 AI 场景要达到可信可用，通常需要：
   模型能力够 + 资料够好 + 兜底流程够稳
         ↓
   三者中最后一项最容易被砍（因为它不出现在系统里，只出现在排班表里）
         ↓
   兜底一砍：复核变抽查 → 漏检 → 出事 → 信任崩塌 → 场景停用
```

兜底人工是场景的「信誉税」。它不产生功能，但它决定用户还信不信这个工具。

6.2 收益侧：四类，不要只算第一类

6.2.1 提效类（最容易算，也最容易被做样）

```
提效 = 人时节省
     =（原来做这件事的耗时 − 现在做这件事的耗时）× 人数 × 频率
```

这个公式有一个经典的注水空间：只算「一次任务的时间差」，不算「从零到一的改造」。

```
只算场景内的耗时：
  原来人工做一份汇总：4 小时
  现在 AI 出初稿：0.5 小时，人复核 1 小时
  → 每份省 2.5 小时 ✅

但完整链路是：
  原来 4 小时 → 现在 1.5 小时（省的 2.5）
  ＋ 数据准备（一次投入，多场景摊销）
  ＋ 流程改造（审批环节新增、权限申请、系统对接）
  ＋ 兜底复核的额外工时
  → 净节省远小于 2.5 小时
```

6.2.2 决策类（最值钱，最难算）

```
决策类收益 = 决策提速 + 风险降低
```

它的特点是：省下的不是工时，是「错误决策的概率×单次错误的代价」。

```
一个价格审批场景：
  原来 3 天出结论 → 现在 1 天
  → 这不是省了 2 天工时，这是省了 2 天的市场窗口
```

> 【待核实】具体决策类收益的量化方式需按场景单独设计，本章不给通用数字。

6.2.3 质量类（最容易归零，最容易长期化）

```
质量类收益 = 差错减少 + 合规事件减少 + 一致性提升
```

质量类收益的特点是「不出现在当月报表里」。

```
原流程：10 个销售各自判断 → 判断标准 7 套
新流程：AI 按统一口径初筛 → 判断标准 1 套 + 人工例外处理
        ↓
收益不是"这个月多卖了 X"
收益是"明年稽核时，这块不再是最常出问题的地方"
```

而在受监管行业里，这一类收益的价值可能被低估了。

6.2.3.1 ★★ 第四类收益：★★ 替代外部专业服务——★ 这是传统制造企业最实在的一笔账

★ 这一节来自作者 2026-11-03 补充的一个判断，★ 它补上了本章原本缺的一类收益。 ★ 而这一类，★ 恰恰是 AI 落在传统制造企业上最实在的方向。

```
★★ ★★ 先看本书前面算过的三类收益：
    · 提效类 —— 人时节省
    · 决策类 —— 决策提速 + 风险降低
    · 质量类 —— 差错减少 + 一致性提升
        ↓
★★ ★★ 这三类，★ ★★ 有一个共同点：★ 它们算的都是"★ 企业内部省下来的东西"
★★★ ★★★ ★★ 而下面这一类，★ ★★ 算的是"★ 企业外部不再花出去的钱"
```

★★ ★★ 而这一类的典型形态是：

```
★★ ★★ 原来：★★ 一件事需要找外部咨询，★ ★★ 费用可能是几百万级别
     （行业研究、标杆对标、专项诊断、方案设计、系统选型……）
          ↓
★★ ★★ 现在：★★ ★★ 找 AI 做咨询就可以了
          ↓
★★★ ★★★ ★★ 这中间省下来的，★ ★★ 不是"工时"，★ ★★ 是 "★ 外部支出 ★★"
```

> **★★ ★★ 而这一类收益，★ ★★ 在传统制造企业里，★ ★★ 价值可能最高、却最少被算进 AI 的账：**
>
> **★★★ ★★★ 因为一笔几百万的外部咨询费，★★ ★★ 它在财务上通常不叫「效率问题」，★★★ ★★★ ★★ 它叫「★ 咨询费 ★★」——★★ ★★ 而没人会把它和「AI 项目的 ROI」放在一起比。**

★★ ★★ 所以这一类收益，有三个特点：

特点	说明
金额大	外部咨询是一笔一次性的、大额的支出，通常不进日常人力成本
不可见	它在「管理费用」或「项目费用」里，不在「人力成本」里，**做提效测算时根本不会被看见**
可累积	★ 若某个领域的咨询可以部分由内部 AI 承接，**★ 那么每年可以少请几次**

★★★ ★★★ 而正因为它「不可见」，★ 立项时特别容易被漏掉——★ 所以这里给一条具体的做法：

```
★★ ★★ 立项时，★ ★★ 除了问"这个场景能省多少工时"，★★ ★★ 还要多问一句：
          ↓
★★ ★★ ★★ "★★ 这件事，★ ★★ 今天是不是要请外部咨询？"
★★ ★★ ★★ "★★ 如果是，★ ★★ 一年请几次？★ ★★ 每次多少钱？"
          ↓
★★★ ★★★ ★★ 而这个问题的答案，★ ★★ 往往比"能省多少工时"大一个量级
```

★★ ★★ 而这一类收益还有一层更深的意思：

★★ ★★ 外部咨询费省下来的，不只是钱，★★ ★★ 还有三样东西：

- ★★ 时间（不用等排期、不用做材料准备、不用做汇报答辩）
- ★★ 内部能力（★ 做过一次，★★ ★★ 组织里就有人会了，下次不用再买）
- ★★ ★★ 议价能力（★★ ★★ 知道这件事市场价，★★ ★★ 谈的时候心里有数）

> **★★★ ★★★ 而这第三样，★ ★★ 常常是长期价值最高的那一样：★★★ ★★★「★ 你知道这件事本该值多少钱，★★ ★★ 和你只知道」★ 我不知道多少钱「★★，★★★ ★★★ ★★ 在谈判桌上的表现是完全不同的。「**

★★ ★★ 最后必须说清这一类收益的边界，否则它会被滥用：

★★★ ★★★ ★★ 外部咨询能被 AI 替代多少，★★★ ★★ 取决于三件事：

① ★★ 那件事的产出，★★ ★★ 是否可以被验证
→ ★★ 需要资质、需要背书、需要承担责任的结论，★★ ★★ AI 替代不了

② ★★ 企业内部，★★ ★★ 有没有人能审这份结论
→ ★★ 没有懂行的人把关，★★ ★★ AI 的咨询就是"看起来专业但没法用"

③ ★★ 那笔咨询费本来是为了买什么
→ ★★ 买"一个外部视角"，★★ ★★ AI 替代不了
→ ★★ 买"一份可以照着做的方案"，★★ ★★ AI 可以替代一部分

> **★★★ ★★★ 所以这一类收益，★ ★★ 最诚实的表述是：**
>
> **★★★ ★★★ 「★ AI 能替代的，是咨询里那部分 ★★ 可被验证、可被内部消化的产出；★★★ ★★ ★★★ ★★★ ★★ 替代不了那份外部背书本身。「**

> **★ 金句**
>
> **「在传统制造企业里，AI 最实在的一笔账，不是替代谁，而是减少对外咨询的费用。」**
>
> **「一笔几百万的外部咨询费，在财务上叫「咨询费」，不会有人把它和 AI 项目的 ROI 放在一起比。」**

6.2.4 ★ 一个必须提前承认的问题

收益侧有一个结构性的诚实问题：

```
提效类收益，会被"人没减"这件事完全抵消。

  原来 10 个人做这件事 → 现在 2 个人做这件事
↓
省下的 8 个人并没有消失
↓
他们去做别的事？还是被要求做更多的事？
↓
如果没有重新配置，这 8 个人的时间会被新的工作填满
↓
→ 效率提升在组织层面不体现为产出增加
```

这不是 AI 的问题，是「效率红利如何兑现」的管理问题——但它必须在立项时就想清楚：

```
立项时就要写明：这笔收益兑现到哪里去？
  ① 释放人手去做更高价值的事 → 写明是哪几件事
  ② 直接减少编制/外包支出 → 写明是哪一个岗位
  ③ 支撑业务增长（同样的人做更多的业务） → 写明业务量目标
```

三者都不写，那这笔收益在账上永远只是「理论收益」。

6.3 采购决策方法：你要落哪些场景，就采购哪些

这一节是本章真正的骨肉。

6.3.1 唯一原则：从场景倒推采购清单

> 原则：我们要落实哪些应用场景，就采购哪些。

它的正确用法是倒推，不是正推：

```
❌ 正推（大多数企业这么做）：
   听说要做 AI → 咨询公司来调研 → 出方案 → 报预算 → 采购
   → 买回来发现：有一半功能对应的场景，你的组织根本没准备做

✅ 倒推（本书主张）：
   场景清单（第二章，已收敛到 1–2 个）
        ↓
   每个场景要什么形态（第四章：Copilot / 流程内嵌 / 对客）
        ↓
   每个场景要什么数据（第三章的三张清单）
        ↓
   每个场景要什么算力与调用量
        ↓
   采购清单 + 金额区间
```

倒推的好处是：采购清单天然和场景一一对应，删掉一个场景就删掉一笔钱。

而正推的采购清单，删不掉——因为它买的是「平台能力」，不是「场景能力」。

6.3.2 ★ 尽量不要预先采购产品化的平台（以及它的边界条件）

这是本章最有争议、也最有价值的判断。

理由：

```
AI 发展速度太快。从 2023 年到现在，每年年底年初都会出现大的新应用模式。
        ↓
假如企业刚投入，花 3 个月到半年上线一套 AI 平台，花了大价钱，
结果还没用起来就要被淘汰，然后又要赶快等着大平台升级
        ↓
牵动的东西很多
```

注意这个论证的结构：它不是说「平台不好」，而是说「平台的保质期短于项目的周期」。

所以判断标准不是「平台 vs 不平台」，而是：

```
你的项目周期  是多少？
平台的技术代际  是多少？
        ↓
平台代际 < 项目周期 → 这个平台大概率会在你上线前就换代
        ↓
而升级的代价，通常远高于初次采购的差价
```

★ 而这个判断之所以成立，是因为第三笔账——投入风险。下面把它的完整算法写出来。

★★ ★★ 风险不是"平台 vs 自建"的道德问题，是一笔可以算的账：

```
┌──────────────────────────────────────────────┐
│ 外采路线：你为一整套"会过时的东西"长期付费       │
│       ★★ 平台换代 → 你的用法过时，授权还在       │
│       ★★ 风险形态：★ 押在别人的路线上           │
├──────────────────────────────────────────────┤
│ 自部署路线：你用人力换掉了"押错路线"的风险       │
│       ★★ 框架换代 → 你的代码跟着换              │
│       ★★ 风险形态：★ 跟不上的风险               │
│       ★★ ★★ 两种风险，不存在没有风险的那条       │
└──────────────────────────────────────────────┘
```

↓

★★ ★★ 拆开看，AI 这件事的风险恰好分成两层：

- 算力层：★ 变化还不算太大
  → 所以算力买还是租、按什么规格买，不用太纠结
- ★★ 应用层与开发层：★★ 变化非常大
  → ★★ 而这恰恰是平台卖的东西，也是你买的东西

↓

★★★ ★★★ 于是第三笔账的算法出来了：

★★★ "★ 这笔钱买的东西，会不会在项目周期内换代？"

★★★ 会 → ★ 风险由你承担（人力换自由度）

★★★ 不会 → ★ 风险由供应商承担（钱买确定性）

★ 所以这里给出全章唯一一条真正的决策判据：

> **★★★ 「你的组织撑不撑得住'路线一直变'这件事？」**
>
> **撑得住 → 自部署（用人力换自由度） 撑不住 → 外采（用钱买确定性）**
>
> **★★★ 这道题没有普遍正确的答案，只有算得清的账。 ★★★ 而这道题的关键不是「你会什么技术」，是「★ 你这个组织能持续几年投入在维护上」。**

★ ★★ 而这条判据的延伸，正好是 6.7 节要讲的那个真实反例：

★ 大型企业买了大厂平台 → 半年后新形态爆发 → 平台基本报废
★ 而中型企业用开源自建 → 虽然累，但资产是自己的
↓
★★ ★★ 请注意这个反差的方向：
★★ 买平台的那家，★ 花的是"确定性"的钱
★★ 自建的那家，★ 承担的是"变化"的风险
↓
★★★ ★★★ 而两家最后谁更划算，取决于一件很具体的事：
★★★ ★★★ 那家买了平台的，★ 有没有为"平台作废"预留过退出成本？

6.3.3 这条主张的边界条件（必须写出来，否则会误导读者）

不预先采购平台，不等于不能买平台。以下三种情况，买平台是合理的：

① 已有明确需求且平台能力正好匹配
→ 买的是"能立刻用的东西"，不是"未来的可能性"

② 平台提供的是通用基础设施（模型网关、检索服务、权限代理、审计）
→ 这类东西不会因为模型换代而作废，且自建成本高于采购

③ 你的团队规模不足以自建
→ 公开研究口径：自建失败率约为采购的两倍【待核实——具体研究出处需作者核实】

而下面这些情况，应当坚决不预先采购：

① 平台的核心卖点是"把最新的模型能力打包" → 这个卖点明年就贬值
② 平台的核心卖点是"支持你未来想做的所有场景" → 你还不知道未来场景是什么
③ 平台需要"先买后规划"才能体现价值 → 采购额随规模膨胀，且无法按场景核减

> 【待核实】「自建失败率约为采购两倍」这一口径的**具体出处与统计口径**，需作者核对原始报告后引用；本章暂按「公开研究口径」表述并标注。

6.3.6 ★★ 投入风险这把尺子，不能单独使用

★ 这一节补一个容易被误用的地方。

★ 很多企业把「第三笔账（投入风险）」当成了唯一判据，于是推出一个很绝对的结论：★ 「所以我们只能自部署。」

★ 但这把尺子必须和另一件事一起看，才是完整的：

```
★★★ ★★★ 投入风险回答的是：★ "这条路还能走多久"
★★★ ★★★ 而它从来不单独决策，★ ★★ 它要和 "★ 这个场景的核心瓶颈是什么 ★" 一起看
          ↓
★★★ ★★★ 同一个"自建 vs 外采"，在两个场景里会得出相反的结论
```

	企业内部应用	对外应用（客户 / 供应商）
核心瓶颈	★ **路线会不会变**——应用与开发层变化极快	★ **有没有流量和获客**
投入风险	★ 高——★ 两年后可能整套装废	★ 不是主要顾虑
技术成熟度	★ 自己能掌控即可	★ ★ 大厂成熟度高得多
流量入口	★ 不需要	★ ★ 自建入口**没有流量**，这才是致命的
结论	★ **开源自部署，保留路线切换权**	★ **用大厂的在线能力**

★ ★ 所以这两个看似矛盾的结论，其实出自同一条推理：

> ★ 在哪里投入，就在哪里用大厂的能力；★ 而「投」与「不投」的选择，由那个场景的核心瓶颈决定。

★ ★ 而这条判断有一个很实用的副产品——它把一个「信仰之争」变成了「瓶颈分析」：

```
★★ ★★ 你不再需要争论"自建到底好不好"，★★ ★★ 你只需要问一句：
★★ ★★ "★ 这个场景卡住我的，★★ ★★ 到底是技术路线会变，★★ ★★ 还是根本没有流量？"
```

> ★ （详见第四章 §4.4.5.8，那里有一份完整的入口场景推演。）

6.3.5 ★ 平台型能力与单点型能力，是两本完全不同的账

★ 这是本章最容易被算错的地方，也是很多企业算完账得出错误结论的根源。

```
单点型能力（一个工具解决一个问题）
    例：合同审核助手、印章识别工具
          ↓
    账很清晰：
      投入 = 一次性开发 + 少量运维
      收益 = 省下的工时 × 单价（当场可算）
      周期 = 几个月
      风险 = 场景本身不成立就停下，损失有限
          ↓
    ★ 这类项目算 ROI 时，几乎总是划算的
      —— 因为它的收益是"确定的工时节省"

平台型能力（给一个部门建一套共用的底座）
    例：部门级 Agent 平台（一套智能体+一套知识库+一套技能库，全员共用）
          ↓
    账完全不同：
      投入 = ★ 主力是人力（长期）+ 算力 + 安全与运维的持续投入
      收益 = ★ 很难单独归因（每个人都"觉得有用"，但说不清省了多少）
      周期 = 以年计（要养、要迭代、要让人信）
      风险 = ★ 一次投入，可能同时服务几十个人
          ↓
    ★ 这类项目用单点那套算法算，必然算成"亏"
      —— 因为它的收益是"分散的、不可拆分的"
```

★ 真实的自建 vs 外采账（本案例一手材料口径，金额已泛化）：

★ 路线一：开源自行部署（用开源项目自搭平台）
  · 授权费：0（★ 8.2.6 的真实做法：QwenPaw 与 Octop
    ★★ 都是开源系统自行部署，不是买来的商业平台）
  · 咨询费：省掉（原本 50~100 万区间）
  · 平台开发费：小额
  · 数字员工/应用开发费：按需
  · ★ 但真正的大头不在预算表上：★ 专职人力
    （本书算过的最贵的一项——专职 AI 人力不足两人，
     却要同时扛需求梳理、方案提炼、培训组织、赛事运营、
     本地算力平台、二期五大成果落地）
      ↓
★ 所以这条路线的真实代价是：不是"少花了钱"，
  而是"把钱换成了自己人的时间"
      ↓
★★ ★★ 而 ★★ 这条路线真正的理由，★ 还不止是省钱：
  ★★ 那是 ★ 投入风险 —— 见下方"第三笔账"

★ 路线二：商业平台外采（买 WorkBuddy 这类）
  · 咨询/许可/定制费用：要花
  · ★ 换来的是三样东西：
    ① 更快上线（不用自己踩坑）
    ② 不用长期养一套平台
    ③ ★ 有人对"平台能跑"负责
      ↓
★ 于是判断依据只有一条：
  这套底座是不是要长期演进？
    是 → 自部署（因为你要改它，而且要改很多次）
    否 → 外采（因为你要的是尽快用起来）

★★★ 而在 AI 这个领域，这道判断题前面还得再加一句——第三笔账：投入风险

★★★ ★★★ 前面比的是"钱"，但真正的分野在这张表之外：

- ★ 算力层：★ 变化还不算太大
- ★★ 应用层与开发层：★★ 变化非常大

↓

★★ ★★ 于是两条路线的风险长这样：

```
┌──────────────────────────────────────────────┐
│ 外采：你为一整套"会过时的东西"长期付费          │
│       ★★ 平台换代 → 你的用法过时，授权还在      │
│       ★★ 风险形态：★ 押在别人的路线上          │
├──────────────────────────────────────────────┤
│ 自部署：你用人力换掉了"押错路线"的风险          │
│       ★★ 框架换代 → 你的代码跟着换             │
│       ★★ 风险形态：★ 跟不上的风险              │
│       ★★ ★★ 两种风险，不存在没有风险的那条     │
└──────────────────────────────────────────────┘
```

↓

★★★ ★★★ 所以这两笔账必须一起看：

- 第一笔：钱（授权费 vs 人力）
- 第二笔：★ 能力（判断力 vs 兜底）
- ★★★ 第三笔：★ 投入风险（★ 路线变化时谁买单）

↓

★★★ ★★★ 而第三笔账给出了一条只有在这个领域才成立的判据：

★★★ ★★★ "★ 你的组织撑不撑得住'路线一直变'这件事？"

★★★ 撑得住 → 自部署（用人力换自由度）

★★★ 撑不住 → 外采（用钱买确定性）

★★★ ★★★ ★★ 没有普遍正确的答案，只有算得清的账

> **★ 落到一句可署名的话：**
>
> **★ 在一个高速变化的领域里，付费买的是「别人的路线」，★ 而企业真正要的是「★ 保留换路线的权利」。**
>
> **★ 所以这条路的代价不是「省钱」，是「★ 用人力承担路线变化的风险」。★ 而这笔人力，恰恰沉淀在组织里，谁也拿不走。**

★ 但无论自建还是外采，平台型能力都必须回答同一个问题：

```
★ 第二个部门接进来时，改动大不大？
          ↓
   改动小 → 它真的是底座，账会在第二年、第三年摊薄
   改动大 → 它只是第一个部门的工具，
              ★ 那么它就该用单点型的算法去算账，
                也就该按单点型的标准去砍
```

> **金句**
>
> **「单点型能力算工时，平台型能力算时间。用错算法，好项目也会被砍掉。」**
>
> **「自建不是省钱，是把钱换成了自己人的时间——而时间是最贵的那一项。」**

6.3.4 最小化投入：把预算换成能力

```
最小化投入 ≠ 少花钱
最小化投入 = 少买"可能用得上"的东西

省下来的钱，投到两处：
  ① 场景本身（数据、流程、验收）
  ② 人（让员工、让技术部门的人在这个过程中认识到最新的应用模式）
```

第二条常被忽略，但它其实是这笔投入最保值的部分。

```
场景会过时（AI 变了，场景的解法也变了）
平台会过时
模型会过时
          ↓
但"这个团队现在知道 AI 能干什么、怎么干"不会过时
```

所以一笔 AI 预算里，有一部分严格来说不是买项目，是买组织的 AI 肌肉。

6.4 云端 vs 本地：这是个假二选一

第四章给了「私有化 vs API」的决策树。这一节补上采购视角下最容易被忽略的一层。

6.4.1 两个朴素方案，两个明显问题

```
方案 A：全上云
  → 问题：数据出厂的安全性

方案 B：全放本地
  → 问题：模型能获取到的只是局部或部分信息，能力受限
```

6.4.2 真问题：数据的外发边界

```
真正的设计对象不是"云 or 本地"，是"哪些数据、以什么形态、出厂到哪"。
```



6.4.3 路由模型的价值不是省钱

切换多模型（用路由层按任务分发）：

一个模型看全部 → 合规结论：数据全量外发
一个模型只看一段 → 合规结论：数据外发范围可论证

★ 关键判断：路由模型的价值不是「省成本」，是「把数据外发的范围切到最小」。

> 一个模型看全部、一个模型只看一段，是两种完全不同的合规结论。 这也是第四章决策树里「数据能不能出厂」这个因子的真正含义——**它问的不是「能不能用云」，是「出得了厂的那部分数据有多小」**。

6.4.4 云端算力的三个非安全理由

采购视角下，还有三个常被忽略的判断：

① 波动性：试点期的调用量无法预测，按峰值买硬件必然闲置
② 迭代速度：本地部署的硬件对新模型能力的兼容要等下一轮采购
③ 兜底路径：本地部署失败时，有没有云上的降级方案

> 【待核实】路由层/模型网关的具体组件是否点名，需作者拍板（与坑二「是否点名模型名」是同一类问题）。

6.5 ROI 计算表模板

这一节给一张可直接抄的表。**核心是把五项成本、四类收益都放进去**。

```
┌──────────────────────────────────────────────────────────────┐
│  场景：_____________  周期：____ 个月                        │
├──────────────────────────────────────────────────────────────┤
│  【一次性投入】                                              │
│  数据治理前置投入      ：_____（人日 × 单价）                │
│  场景改造 / 流程对接   ：_____                               │
│  接入与集成开发        ：_____                               │
│  知识库建设            ：_____                               │
├──────────────────────────────────────────────────────────────┤
│  【持续投入（按月）】                                        │
│  算力 / 调用           ：_____                               │
│  运维与内容更新        ：_____                               │
│  兜底人工复核          ：_____ 人时 × 单价                   │
│  培训与变革沟通        ：_____                               │
├──────────────────────────────────────────────────────────────┤
│  【收益】                                                    │
│  提效类：人时节省 _____ 人时/月 × 单价                       │
│  决策类：决策周期缩短 → 业务影响 _____ 【需单独论证】        │
│  质量类：差错/合规事件减少 _____ 【需单独论证】              │
│  ★ 外部支出类：★ 原来要请外部咨询的 _____ 次/年 × 单价      │
│     （★ 这一类最容易漏，★★ 但金额常常最大，★ 见 6.2.3.1）   │
│  收益兑现去向：□ 释放人力做更高价值的事 □ 减少编制 □ 支撑增长 │
├──────────────────────────────────────────────────────────────┤
│  【结论】                                                    │
│  投入回本周期：_____ 个月                                    │
│  ★ 注明：若"收益兑现去向"未填，本表收益部分不成立            │
└──────────────────────────────────────────────────────────────┘
```

最后一行的备注是这张表的关键设计。它把一个常被跳过的追问写进了模板：

> **写不出「收益兑现到哪里去」，就说明这笔收益在组织层面拿不到——那这张表的 ROI 只是纸面数字。**

6.6 KPI 设计：设一个不会被业务「做样子」的指标

KPI 是本书反复出现的主题（第五章 5.4.3 讲试点验收指标，这里讲长期 KPI）。**两者的原则一致，但对象不同。**

```
试点验收指标  →  判断"这个场景成不成"
长期运营 KPI  →  判断"这件事值不值得继续投"
```

6.6.1 三类 KPI

类别	指标示例	作用
使用类	活跃使用人数、场景渗透率	早期信号，但**极易作秀**
价值类	单位场景节省人时、结论采纳率	真实收益，需人工核对
健康类	复核发现的错误率、人工介入次数、用户主动反馈数	判断它是否在被真正使用

6.6.2 KPI 设计的三条原则

原则一：每个 KPI 必须配一个「反向 KPI」。

```
只看"使用人数"    → 会涨
只看"采纳率"      → 会被挑着算
只看"满意度"      → 会虚高

配反向：
  使用人数 ↑  ⇄  主动报错率 ↑
  采纳率 ↑    ⇄  复核否决率 ↑
  满意度 ↑    ⇄  深度使用（连续 4 周）占比
```

只有正向指标的 KPI 看板，等于给「做样子」发奖金。

原则二：KPI 的统计口径必须写下来，并与第三章的「口径唯一」要求一致。

```
"活跃使用"  = 7 日内使用 ≥ 1 次?
            = 还是 7 日内使用 ≥ 3 次?
            = 还是产生过一次被采纳的输出?
        ↓
三个定义会得出三张完全不同的看板
```

这是第一章坑二在 KPI 上的翻版：同一个词，两套算法，第三次沟通就是吵架。

原则三： KPI 要有一个「归零」的机制。

```
如果一个场景连续 N 个月:
  · 价值类 KPI 达不到初始目标的 50%
  · 且健康类 KPI 显示人工介入次数上升
        ↓
那么它应该被降级或下线，而不是继续"优化"
```

为什么要有这个机制？因为 sunk cost （沉没成本）是 AI 项目最常见的死法。

6.7 不同规模企业的算账差异

这一节原本是全章材料最薄弱的地方（缺不同规模企业的对比数据），后来补入了一个**真实的对照案例**（6.7.4）后，这一节反而成了本章最有说服力的部分。

★ **本节的写作纪律**：不填任何具体金额与合同数字；案例按「某大型制造企业」脱敏呈现；方法论部分标 `【待核实】`。

6.7.1 小企业（几十人量级）

```
特征：没有 IT 团队、没有数据部门、没有预算做平台
          ↓
最优解：几乎全部采购，不自建
          ↓
算账方式：按"人天订阅 × 人数"算，不算资产
          ↓
★ 最大的坑：为了"显得有技术能力"而买硬件
          → 一台闲置的服务器，比不买的成本高得多
```

6.7.2 大型企业（几千人量级）

```
特征：IT 部门存在但往往不做 AI；有大量存量系统；有数据治理历史包袱
          ↓
最优解：混合——通用能力采购，场景自建
          ↓
算账方式：
  ① 一次性投入（数据治理 + 集成）与人天收益分开算
  ② 收益兑现必须写明去向（尤其"释放人力"要指定承接部门）
  ③ 平台采购要按"代际 vs 周期"判断（6.3.2）
```

6.7.3 中型企业的中间路线

```
特征：有 IT 但没有 AI 团队；有预算但不够自建平台；场景多而杂
        ↓
最优解：先做 1–2 个场景，用最小化投入验证路径
        ↓
算账方式：把这一次的投入当作"组织能力建设投资"来算
        → 一部分算项目 ROI，一部分算能力沉淀（6.3.4）
```

6.7.4 ★★ 一个已经发生的反例：大手笔买了平台，却在半年后成了废品

★ 这是本章唯一一个有真实前因后果的对照案例（★ 作者 2026-11-02 补充）。

先看这件事怎么发生的：

```
★★ 某大型企业（★ 脱敏：某大型制造企业）
        ↓
★ 2025 年底：★ 直接购买了大厂平台
   —— 买的是"确定性"：快、成熟、有人兜底
        ↓
★★ 2026 年：★ 开源生态连续爆发
   · ★★ 先是 OpenClaw 一类开源形态起来
   · ★★ 随后是 WorkBuddy 一类形态起来
        ↓
★★★ ★★★ 结果：★ 那套刚买的平台，★ 基本报废
        ↓
★ 而这家企业当初买平台时，★ 并没有做错任何一步：
   · 场景是真的
   · 需求是真的
   · 采购流程合规、供应商也是大牌
   · ★★ 甚至连"平台会过时"这件事，★ 在行业里也不是秘密
```

★★ 而这件事真正的价值，不在于「买错了」，在于它把第三笔账变成了一次真实发生的事：

★ 前一节讲的是判据：会不会换代
★★ 这一节发生的是：★ 还没等换代完成，★ 它就已经被绕过去了

```
2025 年底   买平台（当时它是最好的选择）
2026 年初   开源形态出现
2026 年中   又一个形态出现
★ ★ ★ 此时平台上刚开发完的功能，★ 已经有人在别处免费用着了
```

> ★★ 写到这里必须停一下，把这个案例的「教训」写准——
>
> ★ 它不是「买平台买错了」。★ 因为在 2025 年底那个时点上，买平台大概率仍然是对的。
>
> ★★★ 真正值得记下来的是这句话：
>
> ★★★ 「★ 就算你买对了，两年后它也可能变成废品」—— ★★★ 所以一笔采购决策的真正问题不是「买不买」，而是「★ 这笔钱里，有多少是可以被下一代形态继承的」。**
>
> ★★★ 而这个比例，几乎无法在采购那一刻算出来。★★★ ★★★ 这就是第三笔账最残酷的地方：★ 它只能事后结算。

6.7.4b ★★★ 一个正在发生的公开案例：★ 平台方自己宣布退出

★ ★★ 上面那个反例是「某大型企业买完平台之后才发现不对」。★ ★★ 但它有一个弱点：★ 它是单点的、隐性的，★ 外人不容易核对。★ ★★ 所以这里要补一个更硬的证据——★ 一个所有人都能查证、而且正在发生的事件。★ ★★ ★★ 需要先说清的是：★ 下面这件事是一个★ 公开的行业事件，★ 不是本企业的案例；★ ★★ ★★ 它的价值恰恰在于——★ 它谁都查得到，★ 所以谁也反驳不了。

★ 现象：★ 平台方自己发了下线公告

```
★★ 2026 年 10 月末：★ 字节跳动的 AI 应用开发平台「扣子（Coze）」
★★ 发布《扣子企业版下线公告》
        ↓
★★ 公告给出的时间表：
   · 企业标准版、企业旗舰版 → ★★ 2026 年 11 月 30 日起停止接受新客户购买
   · 旧版套餐（在云平台购买的） → ★★ 2026 年 12 月 31 日全面下线
   · 新版企业套餐 → ★★ 2027 年 12 月 31 日全面下线
   · ★★ 超出日期的未使用部分，★ 自动退款
```

> **★ 这份公告的关键含义，★ 不在于「某个产品要下线」，★ 而在于三件事：**
>
> **★★★ 第一，★ 下线的是企业版——★ 也就是付费买得最多的那一档。 ★★★ 第二，★ 从停止新售到旧版全面下线，★ 只有一个月。 ★★★ 第三，★ 它不是被竞争对手挤掉的，★ 是被自己的母公司战略收拢掉的。**

★ 为什么这比「买错了」更值得写进书里

```
★★ 前一个反例：★ 企业买的平台，★ 被下一代形态绕过去了
★★★ 这一件：★★★ 平台本身，★★ 被它的创造者关掉了
        ↓
★★★ ★★★ 这中间差着一整个量级的风险：
★★★ ★★★ 你不但可能买到一个"会被下一代替代"的平台
★★★ ★★★ 你还可能买到一个"★ 连供给方自己都不打算续"的平台
```

★ 下面是公告背后那半年的时间线，它比任何评论都更能说明问题：

```
★★ 2026 年 7 月底       飞书产品团队与豆包产品团队整合
★★ 2026 年 8 月 24 日   TRAE 与扣子团队整体并入豆包体系
★★ 2026 年 8 月 25 日   独立办公 AI 产品「豆包工作」发布
★★ 2026 年 10 月末      ★★ 《扣子企业版下线公告》发布
        ↓
★★★ ★★★ 从组织收拢到发出下线公告：★ 不到三个月
★★★ ★★★ 而企业走完一次采购审批、★ 集成、★ 上线、★ 验收：★ 通常要更久
```

> ★★★ 这就是本书反复说的那句话的最强版本：
>
> ★★★ 「★ 就算你买对了，两年后它也可能变成废品」—— ★★★ 而这一件还要再加半句：★ 买对的那一次，★ 也不代表你能用完它。**
>
> ★★★ 因为作出「买」这个决定的那家公司，★ 可能比你先改变主意。

★ 那这件事对应书里的哪一笔账

```
★★ 第三章的投入风险 → ★★ 直接命中
   ★ "这条路还能走多久"这个问题，★ 在这里有了一个公开答案
   ★ 而且答案比"两年"短得多

★★ 6.7.5 的可继承比例 → ★★ 同样命中
   ★ 公告里那句"超期未使用部分自动退款"，
   ★ ★★ 说明连平台方都知道：★ ★★ 这笔投入里有一部分是沉没的
   ★★ 而能被继承的，★ 从来只有你自己的数据、知识和团队能力
```

★ 但必须说清楚一件事，否则这一节会被读歪：

> ★★★ 这件事证明的是「★ 平台形态会换代」，★ 不是「★ 工作流模式过时了」。
>
> ★★★ 企业里的 AI 应用分成两类：

	嵌入式 AI	独立 AI 应用
★ 是否打破原有系统	★ **不打破**	★ 打破
★ 典型形态	★ 挂进 ERP / CRM / OA 等既有流程	★ 用户单独打开的应用
★ 对平台形态的依赖	★ **低**——它长在流程里	★ **高**——它就是平台本身

> ★★★ 而嵌入式这一类，★ 恰恰仍然需要用工作流的方式，★★★ 去提升原有系统的智能化水平。
>
> ★★★ 所以正确的读法是：★★★ 「★ 别把 AI 应用押在平台的产品形态上」，★★★ 而不是「★ 别用工作流了」。

★ 所以这一节真正要落下来的判断是：

```
★★★ 判断一：★★ "大厂平台"不是一个稳定选项，而是一个会被替换的选项
★★★ 判断二：★★ 你能继承的，永远只有数据、知识、团队能力
★★★ 判断三：★★★ 越是把 AI 嵌进既有流程的形态，★ 越不依赖平台换代
              ★★ 越是做成独立的 AI 应用，★ 越是在赌平台的形态不
```

> ★★★ 于是那句关于工作流的结论就完整了：
>
> ★★★ 「★ 工作流不会过时——★ 过时的是'★ 平台'这个载体」。
>
> ★★★ ★★ 当平台方宣布退出的时候，★ 那些被写进企业既有流程里的工作流，
> ★★★ ★★ 会跟着企业的系统一起留下来。★ 而这，★ 恰恰是它最值钱的地方。

★ 最后留一个可自查的问题：

```
★★★ 如果你今天签的这份 AI 平台合同，★ 三年后平台方宣布下线，
★★★ ★★★ 你会失去什么？
★★★ ★★ 你写在平台里的流程配置、★ 攒下来的知识库、★ 调好的提示词，
★★★ ★★ 会跟着你走吗？
★★★ ★★ 还是它们会跟着平台一起消失？
          ↓
★★★ ★★★ 如果答案是"会跟着平台一起消失"，
★★★ ★★★ ★★ 那么你现在付的不是"能力建设"的钱，
★★★ ★★★ ★★ 而是"★ 一次必然的作废"的钱。
```

★ **本节事实来源**：字节跳动《扣子企业版下线公告》原文（2026 年 10 月发布，扣子官网公告页可查）；下线时间表（企业标准版/旗舰版 2026-11-30 停止新售、旧版套餐 2026-12-31 全面下线、新版企业套餐 2027-12-31 全面下线、超期未使用部分自动退款）均引自该公告原文；★ 组织收拢时间线与「豆包工作」发布时点引自行业媒体报道【待核实】；★ 其 2025 年在中国智能体开发平台公有云市场份额约 19.3%、位居第一亦为行业媒体口径【待核实】；★ 退款与过渡期安排以各客户实际合同条款为准【待核实】。

6.7.5 ★★ 那笔账该怎么算？——「退出成本」这个被漏掉的东西

★ **案例讲完，得给出算法。否则它只是一个 anecdote 。**

★★ ★★ 反例真正的落点在这里：
买平台的那家，★ 花的是"确定性"的钱
自建的那家，★ 承担的是"变化"的风险
↓
★★ 那到底谁更划算？★ 取决于一件很具体的事：
★★ 那家买了平台的，★★ 有没有为"平台作废"预留过退出成本？
↓
★★★ ★★★ 答案是：★ 大多数企业没有预留

★★ 所以这里给出本章第三条判据（★ 与 6.3.2 的「撑不撑得住」并列）：

★★★ ★★★ 判据三：★ 你的这笔投入，有多少能在下一代形态到来时活下来？

算式（不用精确，用比例感就行）：

```
┌──────────────────────────────────────────────┐
│  可继承的部分                                  │
│    = 数据（你自己的数据永远是你的）              │
│    + 知识资产（沉淀下来的文档、结构、口径）      │
│    + 团队能力（人学会的东西换平台也带得走）      │
├──────────────────────────────────────────────┤
│  不可继承的部分                                │
│    = 平台功能（换代即作废）                     │
│    = 授权（继续付，但换不回来）                 │
│    = 定制工作量（改在别人家身上的）             │
└──────────────────────────────────────────────┘
```

↓
★★★ ★★★ 于是采购决策多了一个以前没有的问题：
★★★ "★ 这笔投入里，可继承的部分占多少？"
↓
★ ★ 平台买得越多、功能定制得越深 → 可继承比例越低
★ ★ 而 ★★★ 场景、数据、团队能力 ★★★ 投得越多 → 可继承比例越高

> ★★ 于是这一节给出一个很反直觉、但我认为很正确的结论：
>
> ★★ 在一个高速变化的领域里，★ 越是在「平台」上花钱，越是在为一次必然的作废付钱。★★ 而越是把钱投在「★ 数据、知识、人」这三样上，越是在为下一代形态存本钱。
>
> ★★ 这也解释了为什么 6.4 那句「★ 最小化投入」必须说清「最小化的是什么」——★★ 最小化的不是「少花钱」，是「★ 少买那些下一代形态带不走的东西」。

6.7.6 ★★ 三档企业各自的算账要点（补入反例后重写）

★ 在有了上面那个反例之后，三档企业的差别终于可以落到一句话上：

```
★★ 小企业（几十人）
   ★ 优势：没有历史包袱，★ 换路线几乎不要成本
   ★ 所以：★ 几乎全部采购，★ 但 ★★ 千万别买"全家桶"
   —— 因为你的组织撑不住"路线一直变"这件事（6.3.2 判据一）
          ↓
★★ 大型企业（几千人）
   ★ 优势：★ 有钱、有 IT、能自己养
   ★ 劣势：★★ 决策周期长、采购流程重
   ★★ ★★ 而这两点加在一起，就是那个反例的成因：
      · 决策周期长 → 从决定到用起来，★ 正好跨过一轮换代
      · 采购流程重 → ★ 采购额随规模膨胀，且难以按场景核减
          ↓
★★★ ★★★ 所以对大型企业，本章真正要说的不是"买不买"，而是：
   ★★★ ★★★ 采购决策的时点，★ 和技术代际的时点，★ 必须放在一起看
   ★★★ ★★★ 而采购流程越长，★ 越要往"★ 买可继承的东西"上倾斜
          ↓
★★ 中型企业
   ★ 优势：★★ 决策快、自建得起
   ★ 所以：★ 先做 1–2 个场景，★ 把投入当"组织能力建设投资"算
          ↓
★★★ ★★★ ★★ 而三个规模档共同的一条：
   ★★★ ★★★ 买的应该是"★ 能被下一代形态继承的东西"，
   ★★★ ★★★ 而不是为了"用起来方便"买的东西
```

6.8 本章小结

成本五项，其中两项常被漏：
③ 数据治理前置投入（不进采购单）
⑤ 兜底人工（不进系统，只进排班表）

收益四类：
提效（易算易做样）、决策（最值钱最难算）、质量（长期化）
★ 外部支出类（★ 2026-11-03 作者补充：★★ 原来要请外部咨询的几百万，★★ 现在找 AI 做咨询就可以了 —— ★★ 这一类金额最大、最容易被漏）
★ 但收益必须写明"兑现到哪里去"，否则只是理论收益

★★ 本章的骨架：AI 项目的账要算三笔
第一笔 钱 → 授权费/咨询费/硬件/实施 ↔ 人力/时间
第二笔 能力 → 判断力（选型配置维护） ↔ 兜底（有人负责）
★★★ 第三笔 投入风险 → ★ 路线变化时谁买单
★★★ 而第三笔只能事后结算——所以只能靠"换掉风险结构"来管

采购三原则：
① 从场景倒推采购清单，采购是结果不是起点
② 尽量不预先采购产品化平台，除非它是通用基础设施/开箱即用/你自建不起
③ 最小化投入，把一部分预算明确投成"组织能力"

★★ 三条判据（连起来读）：
判据一（6.3.2）：★ 你的组织撑不撑得住"路线一直变"这件事？
判据二（6.3.2）：★ 这笔钱买的东西，会不会在项目周期内换代？
★★★ 判据三（6.7.5）：★★ 这笔投入里，有多少能被下一代形态继承？
→ 可继承 = 数据 + 知识资产 + 团队能力
→ 不可继承 = 平台功能 + 授权 + 定制工作量
→ ★★ 越是在"平台"上花钱，越是在为一次必然的作废付钱
→ ★★ 越是投在"数据、知识、人"上，越是在为下一代存本钱

云与本地：
不是二选一，是设计数据外发边界
路由模型的价值不是省钱，是把外发范围切到最小

KPI：
每个正向指标配一个反向指标
口径先写下来
要有"归零"机制

而这一章真正的落点是那一句：

> AI 项目最贵的那一项成本，从不出现在预算表里——它叫「数据原本就不干净」。

6.9 金句墙

> 算账不是财务动作，是采购决策动作。

> ★ 在传统制造企业里， AI 最实在的一笔账，不是替代谁，而是减少对外咨询的费用。

> ★ 一笔几百万的外部咨询费，在财务上叫「咨询费」，不会有人把它和 AI 项目的 ROI 放在一起比。

> ★ 你知道这件事本该值多少钱，和你只知道「我不知道多少钱」，在谈判桌上的表现完全不同。

> 「该不该花这笔钱」，远不如「这笔钱买的东西会不会明年就作废」要紧。

> 采购是结果，不是起点。删掉一个场景，就该删掉一笔钱。

> 平台代际短于项目周期时，买平台就是在为一次必然的升级付两次钱。

> 最小化投入不是少花钱，是少买「可能用得上」的东西——省下的钱，一部分买项目，一部分买组织的 AI 肌肉。

★ 就算你买对了，两年后它也可能变成废品——所以采购决策的真正问题不是「买不买」，而是「这笔钱里，有多少是可以被下一代形态继承的」。

★ 越是在「平台」上花钱，越是在为一次必然的作废付钱；而越是把钱投在数据、知识和人上，越是在为下一代形态存本钱。

模型会过时、平台会过时、场景会过时；「这个团队现在知道 AI 能干什么」不会。

路由模型的价值不是省钱，是把数据外发的范围切到最小。

一个模型看全部、一个模型只看一段，是两种完全不同的合规结论。

写不出「收益兑现到哪里去」，就说明这笔收益在组织层面拿不到。

只有正向指标的 KPI 看板，等于给「做样子」发奖金。

沉没成本是 AI 项目最常见的死法——所以 KPI 必须有归零机制。

AI 项目最贵的那一项成本，从不出现在预算表里——它叫「数据原本就不干净」。

本章要回答：哪些事绝对不能干？ **纪律声明**： ① 企业名、人名、机构名、事业部代号、产品名全部泛化，统一用「某大型制造企业」「数字员工 A/B」。 ② 素材中大量数字属**目标值/设计值/第三方调研值**（效率、准确率、幻觉率、日志留存年限等），一律标 【待核实】。 ③ 不写具体法规条号，涉及法规只写名称类别，并标 【待核实】。

7.0 这一章不讲「合规很重要」

前面六章讲怎么把 AI 用起来，这一章讲怎么不把自己送进去。

在企业里，合规的重要性从来不需要论证——需要论证的从来是「我们做的这件事到底算不算数」。所以这一章只回答一个问题：

> **哪些动作，会让你在签字的那天晚上睡不着？**

夜里睡不着，通常不是因为系统崩了，是因为白天签了四个字：同意、确认、已阅、准予。签完才发现，下面压着的东西当时没看清。

7.1 AI 输出的合规地位： AI 辅助 vs AI 决策

现场画面

流程跑顺了。 AI 提取要素、标注风险、生成报告，人看一眼、点同意，单子往前走。季度汇报数据很好看。

某天一批文件里出了一份问题。追溯时流程图上写着：

```
上传 → [AI 审核] → [人工审核] → 归档
                    ↑ 这一步到底是谁?
```

问到人，答复是「系统是这么判的」。

「系统是这么判的」，是企业 AI 项目里最贵的七个字。 它意味着：当时签字的那个人，其实不知道自己签了什么。

两种模式，必须提前定死

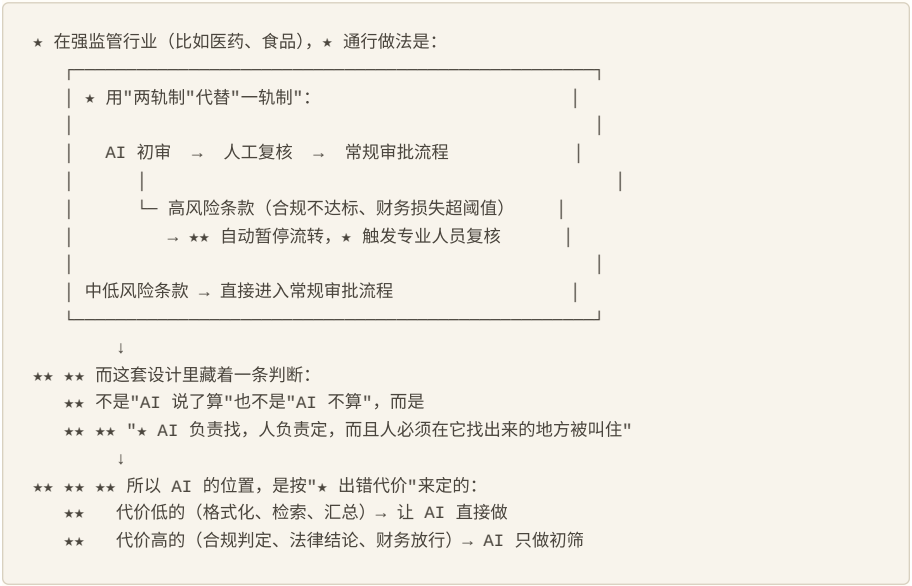
在受控场景里（ GMP/GxP 相关要求覆盖的计算机化系统场景【待核实】)， AI 参与业务动作只有两种姿势。区别不在技术，在人的动作。

维度	模式 A： AI 出草稿，人签字	模式 B： AI 出结论，人复核
AI 的产出物	结构化草稿、要素提取、风险标注、候选条款	判定结论、处置建议、评分
人的动作	从零审阅、改写、签字	复核通过 / 否决（全量或抽样）
责任落点	签字的人	签字的人（复核人同担）
主要失效方式	人偷懒，签得比原来还快	复核流于形式，变成橡皮章

两种模式的差别不在 AI 强不强，在「人」这一步有没有真的发生动作。模式 A 是「人变懒」，模式 B 是「人假装在管」。

★ 受监管行业的通行做法：把 AI 的位置按「出错代价」分级

★ 下面是行业实践中的通行做法（★ 按公开可查的方案口径整理，不指向任何一家企业的实测值）。



★ 这一节的边界同样要写清楚：★ 「哪些场景与数据属于受控范围、判定口径是什么」，★ 这件事只有质量与法务部门能给结论。★ 本书只写工程方法与通行做法，★ 不写具体条号，也不替任何企业做合规判定。

责任不能转移给模型

模型不会签字，不会接到检查组的问询，不会在纠纷中承担损失。只要业务动作最终落到某个人的名字上，责任就在那个人身上——不管结论是模型算的、检索的，还是拍脑袋来的。

> 责任链只有三种形态：
>
> ① 人签字 → 责任在人（正常）
> ② 人没签字，系统自动执行 → 责任在流程设计者 + 系统所有者
> ③ 人签字了，但人不知道 AI 说了什么 → 责任还是在人

第三种最危险，因为它**形式上完全合规**：审批记录齐全、签名齐全、时间戳齐全，只有内容是空的。

具体怎么做

第一步：给场景分级，不给工具分级。 同一个模型，用在「抽取合同要素」和「判断供应商资质是否合格」上，风险差着量级。分级按业务后果分。

级别	定义	AI 的角色	强制人工动作
L1 提示级	不改变业务动作	起草、归纳、提示	引用式使用，注明来源
L2 建议级	影响业务动作但不自动生效	打分、排序、风险标注	逐条判断（模式 A）
L3 判定级	直接决定流程走向	分类、路由、拦截	复核 + 留痕（模式 B）
L4 受控级	涉及 GMP/GxP 受控数据与受控文档	原则上不做	需质量部门单独批准【待核实】

第二步：签字人必须是自然人，且有资质。 质量体系里的签字看的是这个人有没有被授权做这个判断。 AI 团队没有这个授权，业务部门的人有。

第三步：审批意见禁止出现「AI 建议已确认」这种写法。

```
不推荐：✗ "经 AI 审核无异常，同意。"  ✗ "系统显示通过。"  ✗ "已阅。"

推荐：  ✓ "已核对第 3、7、12 条与合同原件一致；AI 标注的第 9 条风险不成立，理由：附件三已约定。"
```

第二种写法把「确认」变成盖章动作。审计和检查问的从来不是「你确认了吗」，是「你凭什么确认」。

7.2 数据不出厂与个人信息边界

现场画面

某部门为提高效率，把一批历史文档喂给外部大模型做分析，结果确实有用。

一周后有人问：那些文档里有多少是受控文件？涉及多少客户个人信息？供应商那边留多久？训练了没有？

三个问题一个都答不上来。**答不上来不是因为团队不负责，是因为这些数据在流出之前从没被人分类过。**

数据分级：三档

分档依据是「这份数据如果泄漏，第一层伤害是谁」。

档位	判定标准	典型内容	处理方式
绝对不进	泄漏即造成不可逆损失	GMP/GxP 受控数据（工艺参数、批记录、检验数据、偏差与质量事件记录）【待核实】；客户与患者个人信息；未公开合同、价格、专利、研发路线	不得进入任何外部模型/API；仅限本地私有化部署并白名单授权
需脱敏	脱敏后可用，价值保留	员工与经销商联系信息、金额类字段、客户名称、内部编码、日志参数与返回值	去标识化、掩码、哈希、分桶、金额泛化；规则与结果留痕
可进模型	本来就是公开的	公开法规文本、行业报告与新闻、公开的药品备案与说明书信息	可直接使用，仍需登记用途与来源

分档是全章最容易被跳过、也最不能跳过的一步，跳过的代价通常半年后才结算。

具体怎么做：让数据 owner 按三档给自家表打标，产出一张字段级的「数据分级台账」，一行一个字段，一列一个档位。台账不复杂，但它是后面所有事情的地基。

数据在哪、谁能看：用真实表结构说

抽象讲「数据安全」没人记得住。看一眼真实系统里数据长什么样就记住了。某大型制造企业销项与进项系统几十张表、几百个字段，挑几张：

表	字段	内容	敏感判断
客户信息表	纳税人识别号、地址电话、开户行、账号	企业客户主体信息	需脱敏
客户信息表	证件类型、证件号码（个人客户必填）、联系人手机	自然人身份信息	绝对不进
全电发票主信息表	购买方经办人证件号码、经办人联系电话	第三方自然人身份信息	绝对不进
发票主信息表	合同号、收款人、复核人、开票人	合同关联与责任链	需脱敏

看出两件事：**个人信息不在一个叫「个人信息表」的地方**，它散在发票表、客户表、进项转出表里，字段名叫「经办人证件号码」「联系人手机」——所以分级要落到字段级；**真正值钱的不是那串证件号，是它和其他字段拼起来的那张画像**——所以脱敏的难点在组合。

内部数字员工的权限边界

某大型制造企业《数字员工管理指南》里，把数字员工分成两类，边界划得很清楚：

维度	数字员工 A （内部知识管理型）	数字员工 B （外部创作服务型）
数据访问	企业信息系统、数据库、质控文档	公开行业报告、新闻资讯、创意平台
敏感操作	需双人授权（数字化中心 + 业务部门）	内容发布前需经品牌部审核
网络边界	禁止访问互联网，数据交互仅限企业内网系统	可访问公开网络资源，但需过滤敏感信息

这张表可以直接抄进自家制度：它把「哪些数据出厂」变成每个机器人身上的一条属性。三条硬规则随之而来：保密型禁止走公网；创作型只许访问公开资源且输入侧过滤敏感信息；保密数据操作需双人授权。

外部数据与第三方预测的边界

很多场景里，模型要靠外部数据补齐内部缺的那一块——公开的药品备案信息、行业趋势、第三方预测结果。这类数据可以用，但它的**权威性不能等同于内部数据**。

外部数据有三个坑：时效（公开信息比内部台账旧，冲突时以谁为准）、口径（第三方定义与内部不一致，混着算结论就是垃圾）、责任（出了错，提供方不承担你的业务责任）。

所以：外部数据只能做参考输入，不能做唯一依据；结论必须落回内部可追溯来源；第三方预测可以参与排序，不能参与最终判定。

公开知识的更新节奏也要写进机制。 AI 陪伴产品这类知识密集场景里，公开医学与药品知识库采用「每日增量 + 季度全量 + 高风险知识由专业人员复核」的组合【待核实】——值得抄到任何依赖外部知识的场景。

7.3 审计留痕

现场画面

检查组来了，要三月份某次合同审核的证据：谁上传的、哪个版本模型处理的、输入是什么、输出是什么、谁复核的。

IT 查了一圈： OA 有上传记录，日志系统有一半操作记录。但「用的哪个模型版本」「提示词长什么样」「AI 原始返回是什么」这三样，一个都没有。**这批业务从头到尾无法自证**。

留痕要回答的五个问题

一次 AI 参与的完整动作，必须能回答：

```
谁    在何时    调了哪个工具（版本、配置）    取了什么    返回了什么    谁确认了
└──────────────  这五问缺一，就不算留痕  ──────────────┘
```

对应到字段是四类：

类别	记什么	缺了会怎样
主体	账号 ID 、姓名、部门/租户、 IP 、终端标识	追不到人
时间	请求时间、耗时、关联追溯 ID	时间线断链
工具与输入	服务/接口/模型版本、请求参数（脱敏后）、提示词模板 ID 、检索命中文档 ID	只能证明「用过 AI」，不能证明「AI 看了什么」
输出与处置	返回值/结论、置信度或风险等级、是否采纳、采纳与否决的修改记录	能证明过程，不能证明「人做了什么」

这套字段不是凭空设计的。某大型制造企业销项系统里的一张第三方服务监控日志表已经很接近：操作人、精确到毫秒的操作时间、请求参数、返回值、追溯 ID 、来源 IP 与终端、成功失败标识、异常编码与描述、执行耗时。

这张表可以直接当模板。差别在于：它记的是「某系统调了某个接口」，AI 场景要扩成「某人把哪段输入喂给了哪个版本的模型，拿回什么结论，采没采纳」。

★ 行业实践中的做法：留痕要留到什么程度才算数

★ 下面是行业实践中的通行做法（★ 按公开可查的方案口径整理，不指向任何一家企业的实测值）。

★ 受严格监管行业的普遍做法是：

```
┌──────────────────────────────────────────────────┐
│ 为满足追溯要求，需构建 ★ 全流程审计日志            │
│ 记录项（★ 常见口径为二三十项元数据）：            │
│   · 接口调用时间（精确到毫秒）                    │
│   · 操作人员 ID                                  │
│   · 字段修改前值 / 修改后值                      │
│   · 请求参数与返回值                             │
│   · 服务/接口/模型版本标识                       │
│   · 追溯 ID、来源 IP 与终端                      │
│   · 成功失败标识、异常编码与描述、执行耗时        │
├──────────────────────────────────────────────────┤
│ ★ 保存周期：★ 通常要求 ★ 不低于五年              │
│   —— 且不得短于业务数据本身的保存期              │
├──────────────────────────────────────────────────┤
│ ★★ 防篡改：★ 关键操作做哈希存证或第三方存证      │
│   —— 哈希值写入审计日志，形成可验证的链条        │
├──────────────────────────────────────────────────┤
│ ★★ 可解释性：★ 用规则引擎把判断逻辑              │
│   转成可解释的数字化规则，而非"黑箱输出"          │
└──────────────────────────────────────────────────┘
```

★★ 而这套做法背后，是一条比「记多少项」重要得多的原则：

```
★ 留痕的价值不在"记了多少项"
★ 而在 ★★ "能不能让第三方独立复现这次判断"
          ↓
★ 一个只有结论、没有输入与过程的日志，
★ 只能证明"系统这么说了"
★ ★★ 而不能证明"系统是在这份输入上这么说的"
          ↓
★★ ★★ 这就是为什么"模型版本"和"提示词模板 ID"必须在日志里：
★★    同一个模型，不同版本，行为不同
★★    同一个模型，同一版本，不同提示词，行为也不同
★★    ★★★ 少记任何一项，★ 这条链就断在那里
```

> ★★ 而这一节要老实交代一处边界：★★ 上面这些是行业实践中的通行做法，★ 不是某一家企业的实测结论。★★ 一家企业要做到什么程度，取决于它受哪些监管约束、约束到什么颗粒度—— ★★ 而这正是「合规」这件事最实际的形态：★ 不是一句「要合规」，是「★ 监管要求到哪一层，你就得留到哪一层」。

★★★ 留痕与追责：出事之后怎么对账

★ 留痕写得再好，如果出事时没人会去查，它就只是一堆日志。下面是「出事之后」的动作序列。

★★ 一次 AI 参与的输出被质疑时，★ 对账的顺序是：

```
┌──────────────────────────────────────────┐
│ ① 这次输出是谁触发的?                    │
│    → 账号 ID + 时间戳                    │
│ ② 喂进去的是什么?                        │
│    → 输入原文 + 检索命中的文档 ID        │
│ ③ 谁改了什么、没改什么?                  │
│    → 修改前后值 + 采纳与否决记录         │
│ ④ 当时用的是哪个版本的什么?              │
│    → 模型版本 + 提示词模板 ID            │
│ ⑤ 谁确认的?                              │
│    → 复核人 + 复核时间                   │
└──────────────────────────────────────────┘
        ↓
```

★★ ★★ 五个答案齐了，★ 才能开始谈责任

★★ ★★ 而如果其中任何一项是空的，★★ 那结论就只有一个：

★★★ "★ 这批业务从头到尾无法自证"

↓

★★ ★★ ★★ 所以本章的落点不是"要做审计"，是：

★★★ ★★★ 留痕的目的不是事后追责，是 ★★ 让业务在事后能说清自己

★★★ ★★★ ——★ 而这恰恰是审计要的不是罚款，是"可追溯、可解释"

★ 一个推论（★ 这一节最实际的一条）：

★ 如果你的日志里没有"模型版本"和"提示词模板 ID"，

★ 那么 ★★ 每次模型厂商一升级，你的历史输出就永久失去了可解释性

↓

★★ ★★ 而模型厂商升级，几乎不由你决定

↓

★★★ ★★★ 所以这两项不是"最好有"，是 ★★ 缺了就等于没有留痕

为什么「日志留存」是供应商条款里最该看的一条

因为除了日志，你手上什么都没有。

```
「使用先进的大模型技术」     → 不可验证，不构成约束
「将保存调用日志」            → 有约束，但没写留多久、存哪、能不能删
「完整保存请求与响应，期限不少于 N 年，企业可随时导出；
  终止后不得删除，删除须取得企业书面同意」 → 这才是能拿去和法务谈的条款
```

三条理由：

第一，日志在谁手里，谁就有解释权。 供应商说只保留 30 天，那第 31 天以后发生的事，你连自证的机会都没有。

第二，日志和训练数据是两回事，很多条款把它们混在一起。 「可能使用交互数据改进模型」和「保存交互日志供客户审计」是两句完全不同的话：前者要明确拒绝，后者要明确要。

第三，日志留存期要覆盖追溯期。 制药场景里数据与记录保存有明确年限要求【待核实】，审计日志保存期不得短于业务数据，否则记录还在、日志没了，等于缺了一半。

具体怎么做

1. **自建留痕，不能靠供应商的「后台有日志」。** 必须能一键导出、格式稳定（结构化，不是截图）、能按人/时间/案件号检索。供应商后台的日志，永远拿不出来。
2. **日志要能拼成完整链路。** 用追溯 ID 把「上传 → 解析 → 调用模型 → 返回 → 人工复核 → 归档」串成一条线。断链的日志在检查面前等于没有。
3. **不可篡改。** 关键操作日志做哈希存证或只追加写入，保留删除标识但不物理删除。日志的价值全部来自「它没被改过」。
4. **敏感操作双人授权。** 保密数据操作需双人授权（数字化中心 + 业务部门）；对外发布内容需业务归口部门审核。
5. **人员流动时回收权限与代码。** 转岗、离职同步回收账号、服务器权限、代码仓库权限。 AI 项目里个人的代码和提示词沉淀着业务逻辑和上下文，跟着人走比跟着系统走危险。

7.4 幻觉的风险控制

现场画面

AI 给出一条非常专业的意见。条款号、章节号、金额、日期，全是齐的。法务看一眼，说「确实是这样」，签了。

后来核对原件，发现那条根本不是合同里的条款——是模型自己编的。但它编得太像了：编号格式对，语体对，前后文接得上。

幻觉最危险的地方不是它瞎编，是它编得像真的。

四个可执行的机制

机制一：强制引用

每条结论必须带来源，且来源是**可点开验证的具体对象**——条款原文 + 章节位置 + 文档 ID，或知识库条目 ID。**这条结论如果没法在 10 秒内找到出处，就不该出现在报告里。**

```
不接受：✗ "根据相关规定，赔偿上限不得超过合同总额的 30%。"  → 「相关规定」点不开。
接受：  ✓ "赔偿上限：合同第 12.3 条，'乙方累计赔偿不超过合同总价的 20%'（合同 v3 第 14 页）"
```

机制二：无来源即拒答

做成系统硬约束，而不是靠提示词求模型「讲实话」。

拒答不是失败，是功能。要让模型学会三种输出：有来源的结论（附引用）、有来源的疑问（「知识库中没有 X 的规定，需 X 部门确认」）、以及无来源（「我不知道」）。

第 ③ 种如果被当成低质量回答去调参，模型很快会学会把「不知道」伪装成「知道」。所以必须在训练/调优阶段允许「不知道」是有价值的输出。

机制三：置信度阈值

低于阈值的内容不自动流转，转人工。阈值按场景标定，不能照抄别家。素材里出现过「置信度低于 85% 自动触发人工复核」的写法，也有通过法规知识库检索增强压低幻觉率的做法【待核实】——这些数字要确认是否实测。

阈值的**用法**比取值更重要：它不是用来筛掉低置信内容，是决定「这一条要不要占用一个人 5 分钟」。高置信 + 低影响自动通过；低置信 + 低影响只做标记；**低置信 + 高影响必须有人看**——风险集中在这格。

机制四：人工确认点设在哪个环节

这是四个机制里最容易做错的一个。

错的做法：把确认点设在最后——流程末端加一个「AI 已审核，请确认」的按钮，这是给橡皮章发工资。

对的做法：确认点设在**不可逆动作之前**，以及**低频但高后果的判断上**。

判断类型	例子	确认点设在哪
可逆、批量、低后果	格式整理、要素提取、摘要	不设，异常时抽查
可逆、高频、中后果	合同风险标注、报销单初审	抽样复核
不可逆、低频、高后果	放行受控文件、对外发布、改主数据	**动作发生前逐条确认**
不可逆、高频、中后果	自动路由、拦截、任务分配	规则先行 + 定期复盘

> **确认点要卡在「错了就收不回来」的那一步前面，不是在「结果已经出来了」的那一步后面。**

AI 生成代码：静态扫描查不出的那一类风险

传统代码检查能查漏洞，查不出「逻辑对不对」。AI 生成的代码更容易出现**语义正确但业务错误**的静默错误——字段名对、逻辑通顺，但算出来是错的，静态扫描基本无效。

补法只有两个：**影子运行**（用真实数据副本跑一遍，不写入生产）和**与现有系统对账**（差异逐条看）。这属于「业务验收」，不是「代码审查」，要分开做。

7.5 供应商与模型的风险

四个问题，签合同前必须问出来

问题	问不出答案的后果
数据是否用于训练？	喂进去的合同、客户数据成了别人的模型参数，且不可撤回
日志留存多久？	出事之后无法自证（见 7.3）
模型下线后数据怎么办？	服务终止时，文档、日志、向量索引是否删除，删除有无凭证
换版本后行为漂移怎么管？	同一场景上周结论正确，这周变了，没人知道

第四个最容易忽略，也最难事后补救。

行为漂移：版本一换，之前验过的全部作废

模型换版本不是「小版本优化」。同一份合同、同样的提示词，新旧版本输出可能明显不同，准确率、周期时长、输出格式都可能变。

> 三个月后可能处在这种状态：验收报告写的是 v1 的表现，生产跑的是 v3，
> 没人知道 v2 到 v3 改了什么，出事时团队内部对"这算不算问题"没有共识。

具体怎么做：

1. **锁版本。** 生产环境的模型版本固定并写进部署配置，升级走变更流程。模型升级不是运维操作，是变更。
2. **建回归集。** 攒一批固定的、带标准答案的测试样本（覆盖高风险场景即可）。每次版本变更全量跑一遍，比对差异。
3. **差异必须人工看一眼。** 差异清单逐条确认「改进还是回归」，只看平均准确率会掩盖单场景的严重退化。同时保留旧版本一段时间，出现争议时要能复现「当时那批结论是怎么来的」。

合同里必须写死的五条

这五条建议直接交给法务落条款。

#	条款	必须写清的点
1	**数据不得用于训练**	覆盖人工标注、人工反馈、评测数据；「改进服务质量」不属于例外；训练数据不得回流客户环境
2	**日志留存与可导出**	留存期限（不少于企业业务数据保存要求【待核实】）、日志范围、结构化可导出的格式、企业随时索取权、终止后的处置与删除凭证
3	**服务终止后的数据处置**	全部数据（含文档、向量索引、缓存、备份）在期限内删除；提供书面删除证明；企业有权委托审计
4	**版本变更的通知与回归**	重大变更提前通知期；变更后提供回归测试结果；企业有权在约定窗口内要求暂停升级
5	**数据位置与跨境**	存储与处理位置；未经书面同意不得跨境；分包商清单及同等义务传递；审计配合义务与响应时限

> **第五条里的「分包商清单」不要漏。** 今天直接签的供应商，明天可能把推理转给第三方。这不是假设，是行业常态。

7.5.1 ★ 出事之后：追责链条与「对账机制」

> ★ 前面 7.5 讲的是签合同之前。这一节讲的是**出事之后**—— ★ 而这一环，恰恰是绝大多数企业完全没准备的。

现象

```
一个 AI 场景上线三个月后出了业务问题。
        ↓
第一反应（每个人都这么问）：
    "这是模型的错，还是人的错？"
        ↓
★ 而真实情况往往是第三种：
    "谁也说不清，因为没有留下能对账的东西"
```

★ 三个典型的追不下去的场景：

场景	卡在哪
审批意见写着「AI 审核无异常」，后来发现该查的风险没查	系统只留了结论，没留**依据**——无法回溯它看了什么、依据什么判断「无异常」
模型换过版本，出事时没人说得清当时跑的是哪版	没有把**版本快照**和业务动作绑在一起
用户手动改过 AI 的输出，但改动没被记录	系统只记了 AI 的输出，没记**人的修改**——于是「谁的责任」彻底说不清

为什么会这样

★ 根本原因：AI 的输出被当成了「系统的输出」，而不是「某个人的签字结果」。

```
传统系统里：
    一条记录 = 某人某时通过某个界面做了一次操作
    → 责任链天然存在
        ↓
AI 系统里：
    一条记录 = 模型算了一串分数，系统据此生成了一句话
    → ★ 没有人参与，所以没有责任链
        ↓
★ 而企业的习惯做法是：
    把 AI 的输出直接写成"审核意见"，盖上章
        ↓
★ 于是"模型是辅助"还是"模型在决策"，在纸面上完全分不出来
```

★ 更隐蔽的一层：★ 时间戳救不了你。

```
时间戳只能告诉你"什么时候发生的"
但它回答不了"用哪一版模型、看了哪份文件、依据哪条规则"
          ↓
★ 所以留痕的重点不是"多记"，而是"记能复现结论的东西"
```

怎么识别自己在坑里

★ 四个问题，任何一个答不上来，说明你的追责链是断的：

```
□ 1. 半年后有人问"这条意见当时是怎么来的"，
       我能不能在十分钟内把当时的输入、依据、版本原样调出来？
         → 答不上来 = 只存了结论，没存过程

□ 2. 有人手动改过 AI 的输出，这个改动有没有痕迹？
         → 没有 = 责任链在这里断了

□ 3. 模型换过版本，历史记录能不能告诉我"当时跑的是哪一版"？
         → 不能 = 出事时连"这是不是已知问题"都判断不了

□ 4. 如果这条意见是错的，能不能定位到"是规则写错了，还是模型理解错了"？
         → 不能 = 你没法修，只能全盘重来
```

★ 还有一个更隐蔽的坑，值得单独说：

```
★ "系统是这么判的"
          ↓
这句话在传统系统里是免责的，在 AI 系统里往往不是
          ↓
因为：
   传统系统 → 规则是人定的，人认
   AI 系统  → 规则可能来自一份没人读过的知识库，
                或者模型自己从语料里归纳出来的
          ↓
★ 也就是说："系统是这么判的"可能是一句
   没人能解释、也没人能反驳的话
```

正确做法

做法一：给每条 AI 结论配一份「可复现包」。

```
★ 一条 AI 结论 = 结论 + 依据 + 版本 + 人工修改记录
   打包在一起，随业务单据一起存档

具体包含四项：
   ① 结论本身（人看到的意见）
   ② 依据（它看了哪些资料、命中了哪几条规则）
   ③ 版本快照（模型版本 + 知识库版本 + 提示词版本）
   ④ 人工修改痕迹（谁改的、什么时候改的、改动前是什么）
         ↓
★ 有了这四项，任何一条结论都可以在十分钟内复现
   —— 而"可以复现"是所有追责的前提
```

做法二：★ 把「AI 的结论」和「人的签字」在系统里分成两栏。

```
❌ 现在的做法（很多系统都这样）：
   审核意见：[AI 审核无异常]
   审核人：  [王五]
   → ★ 一旦出事，这句话会把责任全推给签字的人，
     而他其实从未看过 AI 的依据

✅ 应该的做法：
   AI 建议：  [未发现明显风险，依据：合同审查规则 v3 第 4.2 条]
   ★ 人工结论：[同意 / 不同意 / 需补充材料]
   ★ 签字人：  [王五]
   ★ 签字时间：[时间戳]
   → ★ 三者分开，责任链立刻清晰：
     模型负责建议的准确性，人负责决策，规则负责可被质疑
```

做法三：建立「回归集 + 差异确认」的双轨机制（与 7.5 呼应）。

每次模型或知识库变更：
① 用固定的回归集跑一遍，产出差异清单
② 差异清单逐条人工确认："改进"还是"回归"
③ 确认结论归档，作为下次变更的比对基线
↓
★ 为什么要归档：
出事时要回答的不只是"这次对不对"，
而是"这次的问题上个版本有没有出现过"

做法四：★ 定期做一次「责任链演练」。

★ 这是最容易被跳过、但最有价值的一件事：

随机抽一条三个月前的 AI 结论，
假装现在要追责，然后：
· 能不能调出当时的输入和依据？
· 能不能知道当时跑的是哪个版本？
· 能不能说清是谁在哪一步做的判断？
↓
★ 演练一次的成本：半天
★ 演练失败一次 avoided 的成本：一次产品召回

金句

> **★ 模型不会签字，所以责任永远在签字的人身上——前提是签字的人看得懂依据。**

> **★ 一条 AI 结论只有配上「依据 + 版本 + 人工修改」，才是一份可以复现的记录。**

> **★ 「系统是这么判的」，在 AI 系统里往往是一句没人能解释、也没人能反驳的话。**

> **★ 追责链不是事后补的，是日常每一条记录里攒出来的。**

> ★ 半天做一次责任链演练，比出事后再花十天复盘便宜得多。

7.6 风险清单模板（可直接抄）

用法：每个 AI 场景立项填一份，填不满的不许上线。这是本章唯一「拿来就用」的东西。

#	风险项	触发信号	处置动作	责任部门	状态
1	受控数据流入外部模型	出网请求含受控文档 ID	阻断 + 清理 + 上报	IT + 质量	☐
2	个人信息出厂	输入含证件号、手机号、账号	阻断 + 脱敏回补 + 评估	法务 + IT	☐
3	AI 结论被直接执行	流程无人工确认记录	停用自动执行，降级为建议	业务 + IT	☐
4	结论无引用来源	抽样引用缺失率超阈值【待核实】	退回重做 + 加校验规则	AI 团队	☐
5	人工复核橡皮章化	复核意见为固定套话	改逐条确认 + 修改留痕	业务	☐
6	审计日志缺失或断链	无法还原五问	补建字段 + 追溯 ID 串联	IT	☐
7	日志留存期短于业务保存期	供应商留存期 < 企业追溯期【待核实】	改条款，或本地镜像留存	采购 + IT	☐
8	模型版本漂移	生产版本 ≠ 验收版本	锁版本 + 回归集 + 差异评审	AI 团队	☐
9	供应商数据用于训练	条款未禁止或表述宽泛	合同加条款 + 关训练开关	采购 + 法务	☐
10	服务终止后数据残留	合同无删除义务与凭证条款	补条款 + 退出时核查	采购 + IT	☐
11	AI 代码静默逻辑错误	与现有系统对不上但无报错	影子运行 + 对账 + 抽验	AI 团队 + 业务	☐
12	权限随人流动外流	离职后权限未回收	回收纳入离职清单	IT + HR	☐

三条用法：前 5 项是「上线前必须有答案」的；第 6~12 项是「运行中要监控的」，要有责任人和巡检频率；数据分级台账与外部数据依赖这两项最容易跳过。

7.7 本章小结

这一章讲「绝对不能干」，但能落地的其实是「必须先干」。

```
7.1  先定场景级别，再定工具。签字的永远是人。
7.2  先做数据分级，再谈数据出厂。分级落到字段级。
7.3  留痕要能回答五问，日志必须在自己手里。
7.4  幻觉靠机制，确认点卡在不可逆动作之前。
7.5  合同写死五条，版本漂移靠回归集管。
7.6  风险清单先填前五项，填不满不上线。
7.5.1 ★ 出事之后靠对账：结论 + 依据 + 版本 + 人工修改，缺一不可复现。
```

> **合规不是拦住 AI 的门，是让 AI 敢往深里做的地板。** 这四件事做实了，业务部门才敢把更大的活交给 AI；不做，第一个季度就会有人因为一次「系统是这么判的」而绕开系统。

金句墙

1. **「系统是这么判的」，是企业 AI 项目里最贵的七个字。**
2. **模型不会签字，所以责任永远在签字的人身上。**
3. **审批意见写「AI 审核无异常」，等于把确认变成盖章。**
4. **数据分级不落到字段级，就等于没分级。**
5. **个人信息不在「个人信息表」里，它在「经办人证件号码」字段里。**
6. **日志留存期短于业务保存期，等于业务记录只留了一半。**
7. **除了日志，你手上什么都没有——所以日志必须自己导出。**
8. **确认点要卡在「错了就收不回来」的那一步前面；模型换版本不是运维操作，是变更。**
9. **★ 一条 AI 结论只有配上「依据 + 版本 + 人工修改」，才是一份可以复现的记录。**
10. **★ 「系统是这么判的」，在 AI 系统里往往是一句没人能解释、也没人能反驳的话。**
11. **★ 追责链不是事后补的，是日常每一条记录里攒出来的。**

纪律声明：① 本章所有效果数字均来自一手材料，但**缺基数的数字一律标【待核实】**；② 企业名、集团名、子公司名、事业部代号、产品名**全部泛化**，统一用「某大型制造企业」「某制造业集团」；③ 本章的**人称口径尚未确认**——当前统一写成「一位做了三年 AI 落地的项目负责人」的第一人称观察，**是否署真名待定**【待核实】；④ 8.5「没推成的那些」**素材尚未到位**，本章以框架 + 已确认的反向素材（46 个计划做场景）占位，**绝不编造失败案例**。

8.0 本章要回答什么

前面七章讲的是道理：坑在哪、场景怎么选、数据怎么备、技术怎么选、怎么落地、怎么算账、哪条线不能碰。

道理容易讲，难的是读者心里那个没问出口的问题：

> **「你们是真做成了，还是在写 PPT？」**

这一章只干一件事：把一家企业三年里真实发生过的事情原样摆出来——**第一年在试什么、第二年推得怎么样、第三年为什么换了一种做法**；做成了什么，为什么做成的全是「助手」，员工效率提升了然后怎样，以及**哪些还没做成**。

不美化，不打鸡血。数字不够就空着。

8.0.0 ★ 如果你来自乙方：请先读这一段

> **★ 这一段是写给服务商、创业团队、 AI 咨询顾问的。 ★ 它不客气，但它是这本书最有价值的一部分之一。**

★ 先说一个前提：这一章里的失败，是甲方自己踩的，不是乙方坑的。

```
★ 所以这一章对乙方读者来说，
    ★ 看的不是"谁的责任"，而是"这个坑长什么样、你是怎么被它套住的"
            ↓
★ 因为诚实的结论是：
    ★ ★ 很多甲方项目，是被一份听起来很有道理的方案推进到失败里的。 ★ ★
```

★ 三条真实发生的路径（甲方视角）：

```
路径一：场景本身不该做，但方案听起来很好
    乙方说："这个场景很有前景，做出来能替代大量人工"
    甲方说："那做吧"
          ↓
    ★ 三个月后：数据拿不到、口径对不上、入口改不动
    ★ 而此时已经投进去的人力、算力、时间全都沉没了
          ↓
    ★ 沉没成本让决策人更难承认失败
          ↓
    ★ 最后的结果不是"这个场景失败了"，
      而是"AI 没用" —— 而这个结论，是整个企业对 AI 的判断

路径二：过度承诺
    乙方说："这个效果我们能做到"
    甲方把这句话带进了立项材料
          ↓
    ★ 达不到预期时，赔的不只是这个项目的钱
    ★ 是甲方在下一个项目上还敢不敢投的胆量
          ↓
    ★ 一家企业的 AI 信任是脆弱资产，
      ★ 而乙方每做一次过度承诺，都在消耗它

路径三：只讲能做的，不讲做不到的
    乙方为了签单，把能力边界讲得模糊
    甲方签了，钱花了，上线后发现不是那么回事
          ↓
    ★ 最后甲方的结论是："这家不专业"
      而真相是：★★ 双方都没说清楚能力边界 ★★
```

★ 而「甲方自己会说什么」，最好的例子在 8.5.2：

★ 那个 C 端数字员工的方向，规划时是跟国内头部大厂合作的。
★ 最后停下来，三条原因之一是：
↓
★★ "企业内部数据沉淀和管理模式不成熟，无法适应新模式"
↓
★ 这句话的意思是：
★ 方案没问题、伙伴也专业，
★ 但我们家的底子，配不上这个新模式
↓
★★ 而这句话，★ 大厂不会说，乙方更不会说
—— 说出来等于在评价客户
↓
★ ★ 所以这一段是专门写给乙方的：
★ 当你在客户这里听到"我们在探索"、
★ "这个方向很有前景"的时候，
★ 你其实可以替他问一句：
★ ★ "这个新模式要用的数据，现在在你们公司里是什么形式？"
★ ★ 这一句话问出来，很多方案会自己散掉——
★ ★ 而这恰恰是帮客户省钱。

★ 所以，如果这本书只有一段话是写给乙方看的，那就是这一段：

> **不要为了自己的业绩，去推动甲方投入。**
>
> **★ 要为甲方设身处地地考虑解决方案，而不是为乙方自己的业绩表考虑方案。**

★ 而"设身处地"在实操上是可执行的，具体就是三件事：

□ 1. 在签合同之前，说清楚"这个场景现在不该做"的场景有哪些
↓
★ 一份诚实的方案里，一定包含"我们建议你先别做的那部分"
★ ★ 而这恰恰是签单最难的部分 —— 也是最容易失去的部分 ★ ★

□ 2. ★ 把"能力边界"写进方案，而不是靠嘴说
↓
哪些能做、哪些不能做、哪些需要补什么前置条件
—— 写下来，双方都有据可依
—— ★ 甲方拿这份东西去立项，责任才清得干净

□ 3. ★ 当项目失败时，乙方要帮甲方把"失败的结论"讲清楚
↓
★ 而不是急着推下一个方案
↓
★ 因为"这个场景不适合，但另一个适合"，
才是真正对甲方负责的说法

> ★ 而对甲方读者来说，这一段同样值得读—— 因为甲方自己也做过「乙方」。
>
> ★ 内部汇报里的每一个「有前景」，每一个「行业都在做」， 都是一次不经意的推销。而甲方的钱，付的是自己的。

> 金句
>
> 「劝甲方投一个不合适的场景，代价从来不在乙方那边，在甲方那边。」
>
> 「一份诚实的方案里，一定包含「我们建议你先别做的那部分」。」
>
> 「甲方拿这份东西去立项，责任才清得干净。」
>
> 「甲方自己也做过乙方。内部汇报里的每一个「有前景」，都是一次不经意的推销。」

8.0.1 ★ 为什么这些坑是通用的：制造业的六个环节

> 这一节要回答一个容易被误会的问题：**这本书讲的失败，是「我们公司没做好」，还是「制造业 AI 落地的共同规律」？ 答案是后者。** 而这恰恰是这本书能立住的原因。

先看制造业的结构：

```
不管你是精密生产、流程生产，还是离散生产，
业务链条上的环节几乎是一样的：

   研发 → 采购 → 生产 → 库存 → 质量 → 销售
            ↓
★ 环节一样，AI 在每个环节遇到的问题结构也就一样
```

把这六个环节逐一摊开，会发现一件很反直觉的事：每个环节的「AI 适合度」完全不同。

★ 先用大白话说一遍（如果只想记住一件事，就记住这段）：

假设一家工厂，链条是：研发 → 采购 → 生产 → 库存 → 质量 → 销售
同样是上 AI，这六个位置的难易完全不一样：

采购 —— 最容易先成
因为它天天比价、看供应商资质，★ 规则写在纸上，
出了错当场就能看出来

研发 —— 最难
一个项目好几年，做完了才知道好不好，★ 效果没法快速验
★ 但注意：研发里"制度与知识管理"这类文档活是可以做的
（见 8.2.6 研发中心 LLM-WIKI 知识助手，提效 20%~30%）

生产 —— 很难
数据在设备和老师傅脑子里，既要实时又要全，
★ 这两个要求本身就打架

销售 —— 要拆成两半看
卖合同、协议、政策 → 跟采购一个性质，规则明确、文档多 → 能成
卖数据（查自己业绩、要看过程指标）
→ 口径在销售脑子里、入口在微信上 → 做不成

库存 —— 中等：数据是有的，就是各部门口径不一样
质量 —— 中等：能用，但责任不能交给 AI，人必须签字

★ 所以「上 AI 从哪儿开始」这个问题，答案不是「从最痛的地方开始」：

最痛的地方，往往同时是三个"最难"叠在一起：
① 口径在人的脑子里 → 说不清
② 工作入口在别人地盘上 → 改不动
③ 效果要好几年才看得见 → 验不了
↓
★ 而 AI 最擅长的地方恰好相反：
规则明确 + 文档密集 + 错了当场能复核 + 今天做今天看对不对

再展开成表格（这张表是本节的骨架，★ 2026-11-02 已由作者确认定稿）：

环节	数据与知识形态	AI 的适合度	典型结局
研发	知识密集、数据私有、验证周期长（一个项目几年）	★ 差	效果无法快速验证，试点容易悬空
采购	规则明确、文档密集、错一条能复核	★ 最好	**最容易先成功**
生产	实时性要求高、数据在设备与工艺里、口径在老师傅脑子里	★ 差	实时与完整互斥，卡在数据侧
库存	数据完整但滞后、周转口径各部门分歧	中	卡在口径统一，不卡在取数
质量	责任不可转移，AI 只能出草稿、人必须签字	中	能用，但必须配套责任与留痕设计
销售·文档	合同、协议、政策：规则明确、文档密集	★ 好	**可做**（合同智审已覆盖销售合同与销售协议，且全公司覆盖）
销售·数据	查自己的业绩、要过程指标：口径在销售脑子里、入口在社交软件	★ 差	**典型失败场景**（ 8.5 的营销助手就在这一格）

★ 于是有了一条可以署名的普适结论：

> **AI 落地的顺序，不应该从「最痛的地方」开始，而应该从「规则最明确的地方」开始。**
>
> 因为「痛」不等于「适合」：
>
> ```
> 最痛的地方，往往同时具备三个特征：
>   ① 口径在人的脑子里（说不清）
>   ② 工作入口在别人的地盘（改不动）
>   ③ 效果要等很久才看得见（验不了）
>         ↓
> ★ 而 AI 最擅长的地方是：
>   规则明确 + 文档密集 + 错了能被复核 + 效果当天可见
> ```

★ 这条结论解释了大量现象：

```
为什么很多企业第一个成功的是"审合同"？
          ↓
因为审合同规则明确、文档密集、错一条有人能看出来、今天审完今天知道对不对
          ↓
★ 而很多企业卡住的是"销售预测""智能问数""智能排产""个性化推荐"
          ↓
因为它们分别踩了口径、实时性、效果验证三颗雷
```

★ 而最值得注意的一组对照，是「同样是销售相关的活」：

```
┌─ 销售·文档（合同、协议、政策解读）──────────────────┐
│   规则明确、文档密集、错了能复核                    │
│   → 合同智审已经做到全公司覆盖，含销售合同与协议    │
├──────────────────────────────────────────────────┤
│ 销售·数据（查自己的业绩、要看过程指标）──────────┤
│   口径在销售脑子里、入口在社交软件                  │
│   → 8.5 的营销助手失败；8.6 的智能问数半失败        │
└──────────────────────────────────────────────────┘
          ↓
★ 同一个部门，同一批人，一件事做成了，一件事没做成
   —— 差别不在部门，在"规则 vs 口径"、"文档 vs 数据"
          ↓
★ 这就是为什么"按部门找场景"必然失败，而"按规则密度找场景"不会
```

★ 但同样重要的是本节的另一半：

「通用」不等于「不用管自己」。

★ 六个环节的顺序是通用的， 但每个企业在自己环节里卡住的具体原因各不相同： · 是口径没人定，还是数据压根没落库？ · 是效果验不了，还是入口根本没人改？ ↓ ★ 所以这本书给的不是一套标准动作， 是一套「先看清你在哪一格、卡在哪个子项」的判断框架

★ 最后一条，也是本书的诚实声明：

这本书的素材来自一家大型制造企业三年三段的实践，但书里的结论不是那一家公司的专利。

它写的是： 制造业六个环节里， AI 在每一格会遇到什么、为什么大多数人会掉进同一个坑，以及先做哪一格更容易活下来。

它不写的是： 某一家公司的成功学。

8.1 一个反直觉的开场：几百个 Agent，零实质价值

现象

先说一个几乎每家企业都有的画面。

公司里 AI 遍地开花。各种智能体、应用、插件、机器人，名字一个比一个漂亮，数量一个比一个多。开发平台上有几百个 Agent 在跑，部门群里天天转发「我们部门也上线了一个 AI 助手」的截图。

然后你问一句：**这些 Agent 一共为公司带来了多少收入？**

答案是——**没有产生任何实质价值**。

这不是个别现象，是一整类现象。一位做了三年 AI 落地实施观察的项目负责人说得很直接：

> 「AI 遍地开花，很多企业做了几百个 Agent，但实际上并没有产生任何实质价值。而传统上更多是应用在合规文档、内部管理、行政等方面，一些小的节点都是分散点，没有对整个企业真正的营收产生实质性价值。」

★ 这里必须把话说准，因为它关系到这本书敢不敢下结论：

```
★★ "几百个 Agent"这个量级，★★ 是行业里的普遍现象，★★ 不是某一家企业的盘点数据
★★ 而作者本人此前服务过的一家企业，★★ 量级是四五十个 Agent —— ★★ 量级不同，★ 结论相同
```

★ 所以这一节讲的不是一个惊人的数字，而是一个重复出现的结构：

```
★★ 几十个到几百个 Agent，★ 分布在大大小小的部门里
        ↓
★★ 员工侧确实感受到了效率提升 ★
        ↓
★★ 公司侧的经营价值，★ 很低 ★
        ↓
★★ ★★ 而这个落差的原因，★★ 不在 Agent 数量，★★ 在 §8.4.1 讲的那条规则
```

★ 换句话说：Agent 数量的多少，和这件事最后成不成，几乎没有因果关系。

而更麻烦的是**上下文**：这发生在全球经济下行、企业普遍存在生存压力的时期。预算收紧、决策变快、「能不能今年就见效」成了默认问题。

于是形成一个闭环，越转越紧。

为什么会这样

拆开看，「几百个 Agent 零价值」至少有三层原因，而且这三层互相加固。

第一层：尺子错了。

考核「上线了多少个 Agent / 覆盖了多少个部门」，就必然刷数量。这不是执行者道德问题，是任何指标都会诱导行为。指标指向数量，团队交付的必然是数量。

第二层：只敢做成本侧。

合规文档、内部管理、行政事务——这些场景共同的特点是**基线清晰、周期短、结果可数**。一份合同原来审 3 小时，现在审 20 分钟，账立刻就算得出来。

而收入侧——销售预测、预算打通、渠道优化——周期长、不确定、归因困难，一句话就能被否掉。

于是在下行期，**理性的选择恰恰把团队锁在了最不可能产生价值的地方**。

第三层：点不成线。

每个部门各建各的。一接一类新文档就新建一套本体、一套规则、一套字段。推广得越多，语义烟囱越多。到后来没有人说得清「公司里的金额」到底指哪个口径。

三层叠加，就是那个反直觉的结果：**不是他们不努力，是理性的努力系统性地避开了价值**。

怎么识别自己在坑里

四个自测题，任何一个答不上来，你大概率已经在这个坑里了：

1. 你能说出**上季度 AI 直接贡献的收入金额**吗？说不出来，说明你只做了成本侧。
2. 你手上的 AI 场景，**有没有一条是从头到尾贯通的链路**？还是全是单点？

3. 你的部门上线了三个 AI 应用，它们**共用同一套指标口径**吗？
4. 你能回答「**这个 Agent 改造了哪条流程**」吗？如果答案是」让大家少点几下鼠标「，那它不是场景，是玩具。

正确做法

先把尺子换掉，再谈场景。

- **KPI 从「 Agent 数/上线数」换成三条**：① 被真正改造过的**流程条数**；② 进入**营收链路**的场景数；③ **AI 原生岗位数**（原本不存在、现在因为 AI 才有的岗）。
- **设闸门**：一个场景进生产之前，必须书面回答四个问题——改造了哪条流程？省下的时间去了哪？谁为营收结果负责？三个月后营收没变，是谁负责调整？
- **先盘点再选场景**：数据可用年限、系统表清单、知识模块覆盖度，摸清家底再谈。
- **宁浅一链，不广撒百点**：选一条端到端链路做深，把分散点挂到链上。

> **金句**
>
> **「几百个 Agent 不是成果，是没想清楚要什么之前先动手的证据。」**

8.2 我们做成了什么：一手盘点表的真相

现象

先给一张硬数据。

某大型制造企业在推进 AI 之后，把全集团的 AI 业务场景做了一次完整盘点，出一张表：**46 个「计划做」，23 个「已实现」**。

两组场景**零重叠**——已实现的 23 个，在计划做的 46 个里一个都没有；计划做的 46 个，也一个都没落地。

★ 但这张表其实只讲了故事的一半。 它统计的是「计划中」和「已完成」两档，**缺了第三档：★ 曾经规划过、投了人、最后散掉的方向**。

```
   第一档：做成并活下来          → 23 个
   第二档：计划中没动            → 46 个
   ★ 第三档：★ 规划过、投了人、最后散了   → ★ 这一档不在这张表里，
                                              但它真实发生过
        ↓
★ 而第三档之所以不在这张表里，恰恰因为它最不像"失败"：
   它没进过任何一个"已实现"的统计口径，
   也没留在"计划做"里 —— 它只是没人再提了
        ↓
★ 一个典型的例子（完整复盘见 8.5.2）：
   ★ 2025 年曾规划"对 C 端/供应商做数字员工"，
     考虑过穿戴设备、智能屏、虚拟数字人等形态，
     规划时与国内头部大厂合作，
     最终因三条原因停下：技术不成熟、
     ★ 企业内部数据沉淀和管理模式不成熟、
     前期投入大周期长、只能带来"小事点上的创新"
```

按业务域拆开，构图是这样的：

业务域	已实现 23 个	占比	计划做 46 个	占比
办公	**19**	**83%**	6	13%
财经	3	13%	3	7%
营销	1	4%	9	20%
供给	**0**	**0%**	**19**	41%
生产	**0**	**0%**	5	11%
研发	**0**	**0%**	4	9%

三条铁证，就在这张表里：

1. **已实现的能力几乎全部落在「办公域」。** 而承载制造业核心的**供给、生产、研发三个域，一个都没落地。** 37 个「营销+供给+生产+研发」场景， 100% 停在计划阶段。
2. **已实现的 23 个，全部是「助手」层——没有一个改变了流程。** 制度问答、文件解读、文件比对、合同风险识别、内容审核、 Excel 处理……全是「在原有流程上加一层助手」，没有一个是「流程本身被改造」。
3. **46 个「计划做」并列无优先级。** 一张排了 46 个场景却不排序的表，等于没有路线图。
4. **★ 而真正值得警惕的，是那些「散掉」的方向。** 它们没有结论、没有复盘，

投入已经花掉了，却没有人说得清它算什么——**而且它们会在第二年以「我们还在探索」的形式重新出现。**（见 8.5.2 ）

> ⚠ ◆ **一句必要的说明（避免读者误读）**：这张表是**某个时点的盘点快照， 它说明的是「当时已完成什么、还欠着什么」，不是「永远做不成」。** 2026 年的二期工作里，这些域已经开始有落地了——最典型的是 **研发中心的 LLM-WIKI 知识管理助手（提效 20%~30% 以上，见 8.2.6 ）**， 以及**部门级 Agent 平台底座**（见 8.2.6）。
>
> **★ 而这恰恰印证了本节的核心论点**：研发域真正落地的，是「**制度与知识管理**「这种文档密集、规则明确的活， **不是「研发算法」那种验证周期长的活。** 快照里「一个都没落地」，说的是后者。

为什么会这样

这不是一家企业的失误，这是**下行期的合理选择 + 组织现状的必然结果**。

看第二层最清楚：办公域之所以先成，恰恰因为办公域最符合「快赢」三条件——数据现成（都在 OA 、 CRM 、 FSSC 里）、见效快（省的是打字和翻页）、风险低（错了不涉及放行和批签）。而供给、生产、研发的场景，每一个都要先打通另一套系统、先做数据治理、先处理责任划分——它们做不了快赢。

问题在于：**当 46 个场景全部按「计划做」平铺在表里，而没有任何排序时，「快赢」就不再是一个选择，它变成了唯一结果**。 你没有做路线图，组织就替你做了路线图——谁最容易就先做谁。

怎么识别自己在坑里

对照你自己的场景盘点表，问三个问题：

1. 已实现的场景里，有几个**改变了流程本身的流转方式**？如果全是「加了个助手」，你还在 L1 。
2. 你最核心业务域（对你企业来说是供给、制造，还是服务）的 AI 场景，**已实现的有几个**？
3. 你的「计划做」清单，**有没有排过序**？还是 30 多个场景平铺在同一个状态里？

第三条尤其要命。一张不排序的清单，在年底复盘时会变成什么？ ——变成「有的做了有的没做」，然后明年继续重排一遍。

正确做法

给场景盘点表加**两列**，让它从清单变成路线图。

- **优先级列**：按前面说的四维评估打分（可行性 4 分 / 效益重要度 5 分 / 紧迫度 5 分 / 价值维度），**排出前 5**。
- **前置依赖列**：每个场景标注它依赖哪几条数据治理工作、哪个系统改造、哪项制度决定。

然后把 46 个场景砍到 10 个以内。**砍掉的不是机会，是排期**。

同时记住那 23 个已经做成的——它们不是白做的，是**建立信任的筹码**。一个能拿出手的、跑通的、有留痕的助手，比一百个演示有价值得多。

> **金句**
>
> **「46 个场景平铺在一张表上，不是路线图，是许愿池。」**

8.2.5 ★ 成的两个单点场景：价值长什么样

> 本节补上「做成」的那一面。前面 8.1 讲了「几百个 Agent，零实质价值」，这一节给两个**真正跑出了数字**的场景。 ★ 数据来源：一手工作汇报材料（脱敏），非估算。

场景一：合同智审——把一件天天做的规则活交给 AI

```
四大合同类型全部上线：
    · 服务类合同审核上线
    · 框架类合同审核上线
    · 物料类合同审核上线
    · 资产类合同审核上线
    · 销售合同、销售协议（含 2026 年新增的非固定版本销售协议）
          ↓
    ★ 推广范围：全公司覆盖（★ 可复制）
    ★ 注意：合同类型已横跨采购与销售两侧
          ↓
    ★ 这本身就印证了本章 8.0.1 的判断——
       适合 AI 的是"规则与文档"，不是"哪个部门"
量化结果：
    效率提升     50% ~ 96%
    年度价值     可核算的年度效益（年度金额按作者要求不出书）
    质量收益     降低漏检风险 30% 以上
```

而它现在还在往前走：从「审核通过与否」，走向「关键字段智能核对」——字段自动提取、比对、差异项自动标注，并向其他合同类型推广。

场景二：印章识别——把「逐页翻看」变成「一键识别」

```
痛点：合同、协议、公文的印章完整性核验，长期靠人工逐页翻看
      效率低、易漏检、无留痕
        ↓
做法：批量扫描 PDF / Word 每一页，精准识别公章、区分骑缝章，
      支持 OCR 深度识别
        ↓
量化结果：
   技术性能    61.5 秒完成 529 页合同文档的全量印章检测
               五百余枚印章全部命中，0 误检
   业务价值    人工耗时 数小时/天（50 份 × 6~7 分钟/份）
               按 250 个工作日测算 ≈ 千余小时/年
               效率提升 ≈ 30 倍，释放约 不足一人年
```

★ 为什么会这样：这两个为什么成了

把它们和 8.5 节那个失败的营销助手放在一起看，会看到三条极清晰的分界线。

	★ 成的两个单点	✗ 没成的那个
交给 AI 的活	规则明确、天天做、批量、错了有成本	一线要的是"他心里那个数"，口径不清
能否量化	有基线：漏检率、人天、份数、页数	没有基线，效益说不清
要不要改工作入口	不用改：在批处理、嵌入式流程里跑	★ 必须改：一线得换环境、换工具
错误的代价	错一个印章、漏一条条款，能被复核抓到	错一个数，业务就不敢信这个工具

★ 一句话总结这个对比：

> **成的两个单点场景，都不需要一线改变工作入口；失败的那个，恰恰需要。**
>
> **而这两件事（口径对齐、入口改造）都需要业务下定义——都不在项目组手里。**

★ 还有一条更反直觉的：

> **这两个场景的价值之所以能算出来，是因为它们都有「改造前的基线」。**
>
> 印章核验原来 数小时/天、 50 份 × 6~7 分钟——**这个数字在项目开始前就存在。** 合同漏检率的损失更是行业共识。
>
> **所以「先量出基线」不是项目做完后的总结，是项目能不能立项的前提。**
>
> 而 8.5 节那个营销助手之所以说不清价值，正是因为**它从来没被量化过。**

★ 扎与推：一组不该被忽略的对比

这是本章最想让读者记住的一组数字，它来自同一个项目、同一年。

广度（推）这一侧		
	培训人次累计	数百人次
	内部AI应用创新赛参赛	三十余支队伍、一百五十余人
	最终胜出	十余支队伍
	★ 最终真正落地的场景	数个
	AI 创新专项参与总人数	两百余人
深度（扎）这一侧		
	合同智审：效率提升 50%~96%，可核算的年度效益	
	印章识别：≈千余小时/年，≈30 倍，不足一人年	

而「推」这一侧的效率数据是这样的：

```
三轮训练营报名：AI 248 人、RPA 202 人、数据分析 234 人
        ↓
培训后的认知调查：
   50% 以上不知道 Agent 是什么
   71% 不知道 AI 工作流是什么
```

★ 所以这张表真正说明的不是「少做培训」：

> **大规模培训把认知拉到了「听过」，但从「听过」到「落地」之间，隔着的不是知识，是场景、数据和入口。**
>
> **数百人次培训产出 3 个落地场景，而 2 个「扎」出来的场景，拿出了 千余小时/年和 可核算的年度效益。**
>
> **推的产出是声量，扎的产出是数字。**

★ 而这正是这本书叫《 AI 未成之路》的原因：

```
如果只写"推"这一侧的故事，这本书会变成一份成功学
        ↓
但真实发生的是：
   · 几百个 Agent，零实质价值（8.1）
   · 23 个助手全是助手，一个流程都没改（8.3）
   · 效率提升长出了新的空闲（8.4）
   · 46 个还躺在计划里（8.6）
   · 营销 AI 助手：功能做出来了，用户没有来（8.5）
        ↓
★ 而"未成"的那一面，恰恰是唯一能指导下一步的信息：
   它告诉你该往哪挖，而不是告诉你该喊什么
```

怎么识别自己在坑里（两个成的案例也提醒了反面）

信号一：你的场景说不出「改造前的基线数字」。

"效率提升明显"、"大幅节约人力"——这不是基线，这是形容词
↓
★ 没有基线 → 上线后无法证明价值 → 半年后被当成"没用的工具"下线

信号二：你的场景需要一线「多做一个动作」才能用上。

★ 越是需要用户改变习惯的场景，成功率越低
——因为它同时叠加了"数据不对"和"入口不对"两重风险

信号三：你在用「推广人数」汇报进展。

培训了多少人、报名多少人、多少人参赛
↓
★ 这些是过程指标，不是结果指标
真正该汇报的是：多少个场景在跑、每个跑出了什么数

信号四：★ 你说不出「老路是怎么做的」。

★★ 这个信号最容易被跳过，因为立项讨论里通常只谈新方案
★★ 但只要问一句"★ 这件事现在大家是怎么做的？"，★ 答案往往长这样：
· 数据部门发 Excel 邮件
· 某个报表平台自助查
· 问某个熟人的 Excel
↓
★★★ ★★★ 然后问第二个问题：★ "★ 老路哪里比新方案强？"
★★★ ★★★ 如果这个问题答不上来，★ 那大概率是被回避了——
★★★ ★★★ ★★ 因为老路一定在某一项上强，★★ 不然它早该被换掉了
↓
★★★ ★★★ ★★ 而老路强的那一项，★ 往往就是新方案会输的那一项

信号五：你在给「没做成的」找外部原因。

"业务不配合"、"数据不干净"、"推广力度不够"

↓

★ 8.5 节那个案例里，这三句话都成立
但成立的原因是：口径和入口的解法不在项目组手里

↓

★ 所以正确的做法不是等业务改变，而是
① 先找那 3 个"扎"得动的场景
② 把它们跑出可量化的基线
③ 用这两个数字去换组织的信任
—— 而不是用 数百人次培训去换

正确做法

做法一：把资源从「推」挪到「扎」。

不比"办了几场活动"，比"有几个场景在跑、跑出了什么数"

↓

★ 一个可用的替换公式：
推广指标（人次/报名/参赛）→ 替换为（在跑场景数 × 单场景年度价值）

做法二：立项时先量基线。

合同智审的基线：漏检率、审核人天
印章识别的基线：50 份/天 × 6~7 分钟/份

↓

★ 没有基线的场景，不立项

做法三：优先选「不改入口」的场景。

后台批处理 / 嵌入式 / 在既有系统里加一个节点

↓

★ 先在"不用改入口"的场景上证明能力，
再去打那些需要一线改习惯的硬仗

做法四：★ 立项文件里必须写「老路对照表」。

★★ 这张表只有四行，但它是本书认为最该有的一张：

```
┌──────────────┬──────────────┬──────────────┐
│  ★ 老的做法   │ ★ 新方案      │ ★ 谁赢       │
├──────────────┼──────────────┼──────────────┤
│ 移动端可查？  │              │              │
│ 自己能挑数据？ │              │              │
│ 能转给同事？  │              │              │
│ 零学习成本？  │              │              │
└──────────────┴──────────────┴──────────────┘
```

```
        ↓
★★★ ★★★ ★★ 填完这张表，★ "要不要用 AI"这个问题自己就有答案了：
★★★   ★★★ 如果新方案只赢一行，★★ 那它大概率不值得立项——
★★★   ★★★ 除非那一行是 ★★ ★★ 不可替代的那一行（比如"移动端"）
        ↓
★★★ ★★★ ★★ 而更常见的情况是：★★ 赢了那一行也不够，
★★★   ★★★ ★★ 因为老路赢的那几行，★ 恰好是使用者每天都在意的
```

做法五：★ 把「用户会回到老路」当成立项时的默认假设。

```
★★ ★★ 立项时问自己一句：
★★ "★ 如果这个项目半年后没人用，★ 他们会怎么继续干这件事？"
        ↓
★★ ★★ ★★ 答案如果是一条现成的、顺手的、不需要任何新动作的路，★★
★★ ★★ ★★ 那这个项目的真实产出，★ ★★ 就不是"提高了效率"
★★ ★★ ★★ 而是"★ 让大家又多了一个不用它的理由"
        ↓
★★★ ★★★ ★★ 而这个问题的答案，★ 其实在立项前就能拿到——
★★★ ★★★ ★★ 去问三个用得最凶的人："★ 你们现在怎么查这个数？★ 嫌哪麻烦？"
```

做法六：用两个跑出数字的场景，去说服剩下的人。

```
★ 这不是"内部宣传"，是组织里最有效的说服方式：
    身边人拿出一个真数字，比一百场分享有用
```

金句

> 成的两个单点场景，都不需要一线改变工作入口；失败的那个，恰恰需要。

> 能算出价值的场景，都有一个项目开始前就存在的基线数字。

> 没有基线的场景，不立项。

> 数百人次培训产出 3 个落地场景；2 个「扎」出来的场景，拿出 千余小时/年和 可核算的年度效益。

> 推的产出是声量，扎的产出是数字。

> 培训把认知拉到「听过」，但从「听过」到「落地」之间，隔着的不是知识。

> 一个可用的替换公式：推广指标（人次/报名/参赛）→ 在跑场景数 × 单场景年度价值。

> 不要用 数百人次培训去换组织的信任，要用两个真数字去换。

8.2.6 ★ 第三个成功：给一个部门建一套共用的 Agent 底座

> **★ 本节是 2026 年补入的第三个成功案例**（二期成果汇报 + 阶段汇报 + 作者口述）。 **★ 它和前两个成功不是一类：前两个是「单点工具」，这一个是「底座型成功」。**

现象

前两个成功案例都有一个共同点：**都是「一个工具解决一个问题」**——合同智审管合同，印章识别管印章。

2026 年出现的第三个成功，形态完全不同：

```
★ 不再是给一个场景做一个工具，而是给一个部门建一套共用的底座：

   部门级 Agent 平台
   （形态对标 WorkBuddy / OpenClaw 这一类）
         ↓
   一套智能体 + 一套知识库 + 一套技能库
   部门全员共用、统一标准、持续迭代
         ↓
   ★ 效果：2026 年开始在部门推进使用，反馈不错
```

★ ★ 而它的部署形态有一个必须说清的地方（★ 2026-11-02 作者补充）：

```
★★ ★★ 这是 ★ 矩阵式部署 —— ★ 每个部门一套 Agent 平台
   （而不是全公司一套，从上往下推）

   数字化中心一套 | 研发一套 | 财务一套 | 质量一套
         ↓
★★ 而"共享知识库"这个说法，★ 主要指的是 ★ 部门内共享
   （跨部门的知识共享，目前不在这个范围内）
         ↓
★★ ★★ 于是 8.2.6 的准确说法应该是：
   ★ 不是"建了一个全公司共用的底座"
   ★ ★ 而是 "★ 建了一套可以被复制的建法"——
   ★ ★ 每个部门自己建一套，平台与技能是通用的，知识库是本部门的
```

★ 已跑起来的部分（一手材料可查）：

落点	内容	量化结果
数字化中心	试点技术线 **16 个 AI 智能体助手**，覆盖项目管理、技术开发、安全管理、日常运维、文档制度管理	—
★ 实际在用的部门（★ 2026-11-02 作者补充）	**★ 研发、财务、质量、数字化中心**四个部门已在使用；**★ 并有新的部门主动申请开通**	★★ 使用人数与调用量不便公开，此处不写数字
研发中心	研发知识管理智能小助手（**LLM-WIKI 架构**）：原始资料层 → Wiki 知识层 → 规则层三层，专治「制度文件量大、来源复杂、人工查阅与咨询耗时长」	**提效 20%~30% 以上**
平台底座	2026 年 2 月底隔离环境部署 **两套开源系统**用于测试，场景为硬件运维、项目管理	—
自主搭建	用开源免费项目 **QwenPaw 完成公司内 AI 智能体平台搭建上线**；同时用 **DeepSeek Harness** 探索企业智能体应用模式	见下方「账」
★ DeepSeek Harness 的结局（★ 2026-11-02 作者补充）	**★ 已放弃** —— 它并不适合企业内部使用：★ 产品本身尚不成熟，★ 更适合极客个人使用	★ 探索结束，未进入生产
★ 当前在用的底座	**★ QwenPaw + 腾讯 Octop，两套都是★ 开源系统自行部署** —— 这是部门级 Agent 平台**目前实际在跑**的底座	见下方「账」

★ 关于「底座到底是什么」，这里要老实交代一句边界：

```
★ 这个部门级 Agent 平台，★ 当前实际在用的底座是两块：

    ┌──────────────────────────────────────────┐
    │ ① QwenPaw —— 开源系统，★ 自行部署          │
    │    2026 年完成公司内平台搭建上线           │
    ├──────────────────────────────────────────┤
    │ ② 腾讯 Octop —— ★ ★ 同样是开源系统自行部署  │
    └──────────────────────────────────────────┘

          ↓
★★ 注意这里最关键的一点：
   ★★ 两套都不是买来的商业产品，★ 都是开源 + 自行部署
   ★★ 也就是说：★ 这家公司在这件事上，★ 没有走"外采"这条路
          ↓
★ 而 OpenClaw 与 DeepSeek Harness 属于
★ 同期做过的测试与探索（两套 OpenClaw 部署于隔离环境，
   DeepSeek Harness 用于探索企业智能体应用模式）
```

★ ★★ 而「为什么没走外采」这个问题的答案，比「省了多少钱」重要得多：

```
★★★ ★★★ 不外采的根本原因：★ 不是钱，★ 是 ★ 投入风险
    （★ 作者 2026-11-02 补充）

   · ★ 算力这一层，★ 变化还不算太大
   · ★★ 但 ★ 应用层和开发层，★ 变化非常大
          ↓
★★ ★★ 商业平台路线意味着什么？

    ┌──────────────────────────────────────────┐
    │  你要为一整套"会过时的东西"长期付费：       │
    │     授权费 / 实施费 / 定制费               │
    │     ★★ 而且是 ★★ 在一个变化极快的层面上    │
    └──────────────────────────────────────────┘

          ↓
★★ ★★★ 而自部署这条路，★★ 风险长什么样：

    ┌──────────────────────────────────────────┐
    │  ✅ 不押在别人的路线上                     │
    │     ★ 框架换一代，你不跟着付费             │
    │  ❌ 但风险没有消失，★ ★★ 它换了个地方      │
    │     ★ 从"买错版本的风险"变成"跟不上的风险"  │
    │     ★★ 而跟不上的风险，★ 靠的是自己的判断力 │
    └──────────────────────────────────────────┘
```

★ ★★ 换句话说，这条路的完整逻辑是：

```
★ 外采路线：★ 把"技术路线押错"的风险，转化成"长期付费"
   → 平台换代了，你的授权还在，但你的用法过时了
   → ★★ 而 AI 恰恰是一个"换代极快"的领域
        ↓
★ 自部署路线：★ 把"长期付费"的风险，转化成"自己的判断力"
   → 框架换代了，★ 你的代码跟着换，成本是人的时间和学习
   → ★★ 而这份能力是沉淀在组织里的，★ 不会被下一家供应商收走
        ↓
★★★ ★★★ 于是这里有本书最该说的一句关于选型的话：
   ★★★ ★★★ 在一个高速变化的领域里，
   ★★★ 付费买的是"别人的路线"，
   ★★★ ★★★ 而你要的恰恰是"★ 保留换路线的权利"
```

★ ★★ 所以这笔账的算法，和「省了多少钱」完全不是一回事：

```
★★ ★★ 商业平台路线（买 WorkBuddy 这类）：
   ┌──────────────────────────────────────────┐
   │  付出：授权费 + 实施费 + 定制费          │
   │  好处：快、成熟、有人兜底                │
   │  代价：★ 知识库和技能库装在别人家        │
   │        ★★ 且你的用法被对方的版本节奏绑住 │
   ├──────────────────────────────────────────┤
   │  ★★ 开源自行部署路线（本例走的这条）：   │
   │  付出：人力（部署、配置、升级、排障）+ 判断力 │
   │  好处：★ 知识库是自己的，数据不出门      │
   │        ★★ 框架换代时，你有换的自由       │
   │  代价：★ 要自己承担运维、升级、故障      │
   │        ★★ 跟不上，就要靠自己补           │
   └──────────────────────────────────────────┘
        ↓
★★ ★★ 原始材料里那笔账的逻辑，说得很直白：
   ★★ "通过自主推进降低 AI 建设成本"
   ★★ （咨询一项原本 50~100 万）
        ↓
★★ ★★ 于是这里有一笔必须算给读者看的账：
   · 省下来的：★ 授权费 / 许可费 / 咨询费
   · ★★ 付出的：★ 人力（部署、配置、升级、排障）
   · ★★★ 以及一项报表上看不见的：★★ 承担路线变化的风险
        ↓
★★ ★★ ★★ 而"省下来的钱"和"付出的力气"，
   ★★ 不是同一个单位，不能直接相减——
   ★★ 这正是第六章算账那一章反复强调的事
```

★ ★★ 而「两套开源同时在用」这件事，还带出一条更重要的判断：

```
★ ★★ 开源自行部署，最容易被忽略的真实成本不是钱，是 ★ 选型与维护的判断力

    ┌──────────────────────────────────────────────┐
    │ ★ 部署本身不难：★ 拉下来、配一下、跑起来          │
    │ ★★ 难的是：★                                  │
    │    · 哪个开源项目适合企业场景、哪个只适合极客      │
    │    · ★★ 上线之后谁来盯升级、盯安全、盯故障        │
    │    · ★★ 出问题的时候，谁负责——★ 厂商不负责       │
    └──────────────────────────────────────────────┘
          ↓
★★ ★★ 所以选开源自行部署这条路，★ 前提是组织里有这个能力
★★    —— 或者说，★ 你已经在这条路上付出过学费了
★★ ★★ 这也解释了为什么这个案例不是"随便选个开源就能成":
   ★★ 同期试过的几个开源，最终活下来的是两个
```

★★ 于是 8.2.6 的第二个价值可以改写成一句更准确的话：

★ 它证明的不是「开源替代商业平台」，而是「★ 开源自行部署这条路，在企业里走得通」—— ★ 而走通的代价是人力，收益是知识资产和数据主权。

★ 这道题没有标准答案，只有账算不算得清。

★ ★★ 而「 DeepSeek Harness 被放弃」这件事，本身是一条很值得写的判断：

★★ 一个工具被放弃，通常不是因为它"差"，而是因为它"不对"
  · DeepSeek Harness ★★ 不是不成熟，是不成熟在另一个地方
  · 它适合的是：★ 极客自己玩、自己搭
  · 它不适合的是：★ 企业内部、多部门、有人要对结果负责
      ↓
★★ ★★ 所以这里有一条选型时少有人讲的判据：
  ★★ 一个工具"能不能用"，取决于它面向的是谁
      ↓
  ★★ ★★ 面向开发者个人的工具，★ 和面向企业业务的工具，
  ★★ 要求的根本不是同一件事：
    · 前者要"能折腾、好玩、能自定义"
    · ★★ 后者要"稳定、可控、出事有人负责"
      ↓
★★ ★★ ★★ 而"组织一次技术选型"最容易漏掉的，就是这一层：
  ★★ 试的时候是你一个人在折腾——顺手就上了
  ★★ ★★ 交给部门用的时候，要考虑的是另外一整套东西

★ ★★ 还有一条关于部署形态的判断，★ 是矩阵式部署逼出来的：

★★ 矩阵式部署（每部门一套）看起来"不集中"，★ 但它有一个集中式做不出的好处：
  ★ ★★ 每个部门可以按自己的口径建自己的知识库，★ 不用迁就"最差的那个系统"
      ↓
★ 这恰恰是第四章讲 RAG 死结四时说的那个前提——
  ★ 共用知识库的前提是"知识要有一个不依赖任何系统的形态"
      ↓
★★ ★★ 而当那个形态还没准备好时，★ 矩阵式反而是更现实的路：
  ★ 先在部门内跑通三件事（谁来维护 / 谁有权限改 / 答案不对以谁为准）
  ★ ★★ 等"知识长什么样"这件事有共识了，再谈跨部门共用
      ↓
★★ ★★ 也就是说：★ 这本书第一条讲的"别急着推"，
★★ 在这里又验证了一次——
★★ ★★ 先在最小的范围内把一件事做对，比在全公司铺开更重要

★ 而「全员共享一套知识库」的完整架构（三层）在原始材料里是作为下一步规划提出的：

```
部门共享智能体架构（规划形态）
    ┌─ 用户接入层：部门全体成员 + 管理员
    ├─ 核心能力引擎层：处理请求、执行任务
    └─ 共享资源底座层：★ 部门统一的知识库 + 技能库
            ↓
★ 这三层的意思是：
    部门知识和技能统一管理与共享，
    每个人调用的都是同一套智能体能力
```

> ⚠ ◆ **请注意这里的诚实边界**：上面这个三层架构，在原始汇报材料里列在「下一步关键工作内容」。**2026 年真正跑起来的是上表那四行。** 本书不把规划写成成果。

为什么会这样

★ 这个案例真正的价值，是它回答了一个前面几章都没答好的问题：

```
第一章坑四说：员工抵触，是因为系统不是他的工作流
            ↓
★ 那问题就来了：如果一个新工具要服务整个部门，
    它怎么才能"在部门的工作流里"？
            ↓
★ 答案藏在这个案例的结构里：
    把 Agent 建在部门内部，用部门自己的知识库
            ↓
    它不需要员工改变工作流——因为它就在部门里，
    用的是部门的话、部门的事、部门的规矩
```

★ 第二个价值：它证明了「开源替代」这条路走得通。

这条路线是：
用开源免费项目（QwenPaw / DeepSeek Harness / OpenClaw）
搭出公司自己的智能体平台
↓
而不是买 WorkBuddy 这类商业平台
↓
★ ★★ 而这里要说得比"开源替代"更准：
★★ 当前实际在跑的两套底座 —— QwenPaw 与腾讯 Octop
★★ ★ 两套都是开源系统自行部署，没有一套是买来的
↓
★ ★★ 于是这条路的完整形态是：
★★ 省下的是授权费与咨询费，
★★ 付出的是人力（部署、配置、升级、排障）
★★ ★★ 而省下的钱和付出的力气不是同一个单位——
★★ 这笔账要算全，见第六章
↓
★ 这不是省钱这么简单的动机——
原始材料里明确写了这笔账的逻辑：
"通过自主推进降低 AI 建设成本"（咨询一项原本 50~100 万）
↓
★ 于是这一节同时回答了第六章的一个大问题：
★ 平台型能力的账，和单点工具的账，根本不是两本账

★ 第三个价值，也是最容易被写错的一个：它让 8.0.1 的结论变得更精确。

回忆 8.0.1 那张六环节表：研发 = AI 适合度差

↓

★ 可是研发中心明明做成了啊？——研发中心做成了提效 20%~30% 的知识助手

↓

★ 不矛盾。因为它做的不是"研发算法"，
而是"研发文档与知识的管理"

↓

★ 于是判断的单位被进一步厘清了：

```
┌──────────────────────────────────────────────┐
│ ★ 判断的单位不是"部门"，                      │
│   是"这个部门里具体是哪一件事"。              │
│                                              │
│   研发的算法探索      → 验证周期长 → 难       │
│   研发的制度/知识管理 → 规则明确、文档密集 → 易 │
└──────────────────────────────────────────────┘
```

↓

★ 这正好把 8.0.1 那句"按规则密度找场景不会失败"落到了实处：
研发中心里就同时躺着"适合"和"不适合"两种活

怎么识别自己在坑里

如果你也打算「给部门建一个 AI 平台」，先问这五个问题：

□ 1. 这个平台的知识库，是"部门的"还是"公司的"？
→ 这两条路线的组织成本完全不同（见下方正确做法）

□ 2. 我是在建"底座"，还是在建"又一个应用"？
→ 底座的判据：第二个部门接进来时，是不是几乎不用改
→ 如果每个新部门都要重做一遍，你建的是应用，不是底座

□ 3. 自建还是外采？我算过哪本账？
→ 平台型能力的账 = 人力（★ 开源自部署）vs 咨询+许可（商业采购）
→ 和单点工具的 ROI 账不是一本

□ 4. 有没有第二个部门愿意接？
→ 没有第二个部门，它就永远是一个"部门的工具"，不是底座

□ 5. 这个 Agent 出错，谁改、谁担责？
→ 部门级平台最容易缺这一条

★ 更要紧的是识别这个坑：

★ 最容易出现的失败形态（三个症状）：

症状一：平台建好了，但只有 IT 部门在用
→ 一线觉得"这不是我的工具"，回到坑四

症状二：知识库建了一堆，但没人维护
→ 三个月后没人敢信它的答案
→ ★ 这是底座型成功最常见的死法（详见下方正确做法四）

症状三：每个部门各建一套，集团层面互相不通
→ 变成"烟囱式 AI"，与第三章治理结论直接冲突

正确做法

做法一：先把「共享」的范围说清楚，再动手。

最常见的误解：
"共享知识库" = 全公司一套知识库
↓
★ 但公司级知识库是个大工程：
各部门口径不同、密级不同、话术不同
↓
★ 更现实的起点是"部门内共享"：
一套智能体 + 一套知识库 + 一套技能库，部门全员共用
↓
先在一个部门把三件事跑通：
① 知识谁来维护（关键）
② 谁有权限改（管理员）
③ 答案对不上时以谁为准

做法二：先确认这件事是「高规则密度」的活。

★ 回到 8.0.1 的判据：
规则明确 + 文档密集 + 错了能复核 + 效果当天可见
↓
★ 典型适合：制度问答、文档检索、流程指引、模板生成
★ 典型不适合：需要长期验证、需要实时数据、结果无法复核

做法三：★ 开源自行部署 与 商业采购，要算两本账，不要混着算。

★ 自部署账：主力是人力（★ 这本书算过的最贵的一项），
加上算力、加上安全与运维的长期投入
↓
★ 外采账：咨询费 / 许可费 / 后续定制费，
换来的是「更快上线」和「不必自己踩坑」
↓
★★ ★★ 但在"选开源"这件事上，有两笔账最容易被漏掉：

```
┌──────────────────────────────────────────┐
│ ① 开源省下的是"钱"，                      │
│    ★ 开源增加的是"★ 判断力和责任"——       │
│    ★ ★ 选哪个、怎么配、升级谁盯、坏了谁修  │
│    ★ ★★ 这些厂商都不替你负责              │
├──────────────────────────────────────────┤
│ ② ★★ 而更根本的一笔是 ★ 投入风险          │
│    ★★ 应用层与开发层变化极快              │
│    ★★ 外采 = 为一个可能很快过时的版本长期付费 │
│    ★★ 自部署 = 用人力换掉"押错路线的风险"  │
│    ★★ ★★ 两种风险，★ 不存在没有风险的那条  │
└──────────────────────────────────────────┘
```

↓
★ 判断依据只有一条：
这套底座是不是要长期演进？
是 → 自部署（因为你要改它，而且要改很多次）
否 → 外采（因为你要的是尽快用起来）
↓
★★★ ★★★ 而当整个领域本身就在高速变化时，
★★★ 这条判据前面还得再加一句：
★★★ ★★★ "★ 你的组织撑不撑得住'路线一直变'这件事？"
★★★ 撑得住 → 自部署（用人力换自由度）
★★★ 撑不住 → 外采（用钱买确定性）
★★★ ★★★ ★★ 而本例的现实是：★ 它撑住了——
★★★ ★★ 两套开源自部署并用，★ 选型的能力是练出来的

做法四：★ 知识库必须有人负责——这是底座型成功的真正命门。

一个共享知识库要活下来，只需要一件事：

↓

★ 有一个人 / 一个角色，对"答案是对的"负责

↓

要写清楚三件事：

- 谁可以新增/修改知识条目
- 谁定期复核（哪怕只是抽查）
- 发现答案错了，怎么反馈、多久修

↓

★ 没有这个人的知识库，活不过三个月：
不是因为它不会用，是因为没人敢信它的答案

↓

★ 这条与第三章"反馈闭环清单"是同一件事：
★ 反馈闭环没有"回流"，就只是一个抱怨箱

做法五：让「第二个部门」成为验收标准。

★ 判断一个部门级平台是不是真的底座，看一件事：
★ 第二个部门接进来时，改动大不大？

↓

改动小 → 它是底座
改动大 → 它只是这个部门的工具

↓

★ 而"第二个部门愿意接"这件事，
是从第一个部门接进来时就开始积累的

金句

> **前两个成功证明「AI 能做成事」，这一个证明「AI 能做成组织能力」。**

> **把 Agent 建在部门里、用部门自己的知识库——它不需要员工改变工作流，因为它就在部门里。**

判断的单位不是「部门」，是这个部门里具体是哪一件事。研发的算法难，研发的制度管理容易。

一个共享知识库要活下来，只需要一件事：有人对「答案是对的」负责。

★ 知识库活不过三个月，通常不是因为它不好用，是因为没人敢信它的答案。

★ 在一个高速变化的领域里，付费买的是别人的路线，而你要的恰恰是保留换路线的权利。

判断一个平台是不是底座，看第二个部门接进来时，改动大不大。

8.3 做成的 23 个里，为什么全是「助手」

现象

看回那张表。已实现的 23 个场景，名字念出来是这样的：

制度问答、文件解读、文件比对、合同风险识别、内容审核、 Excel 处理、项目简报生成、绩效评分辅助……

你会发现一个共同点：**它们都是在原有流程旁边，加一个「会读会写会查」的帮手**。

没有一个是「这条流程从此以后换了条路走」。

一位项目负责人对此有个很锋利的判断——他谈的是**高管 AI 助手**能走到哪一步：

> 「如果是要让 CEO 全面管理所有信息和数据，就需要组织文化做到完全透明化管理……否则的话，**只能针对已有系统的数据实现智能感知，而不能对全盘进行管理**，包括会议决议、个人行为的全能感知。」

他把这件事分成两层：

层级	能力范围	前置条件	企业实况
L1 · 系统内智能感知	汇总已有系统数据、给异常提示、辅助判断	无特殊要求	**绝大多数企业实际到这一层**
L2 · 全盘管理	跨系统、跨会议、跨个人行为的全局感知与决策支持	**组织文化达到完全透明化管理**	极少数企业能到

为什么会这样

把这句话和 8.2 的铁证二放在一起看，答案就出来了：

已实现的 23 个全是助手层，正是「 L1 而非 L2 」的现实体现。

而为什么只能到 L1？因为——

改流程需要看得清流程，而看清流程需要透明。

这不是技术团队偷懒，也不是预算不够。改一条流程，意味着要动这条流程上所有人的职责边界、审批权限、留痕规则。而在一个还没有做到透明化管理的组织里，**这些人是看不见的**——你连会议决议散落在哪几个系统里都不知道，怎么改那条把会议串起来的流程？

技术能穿透系统，不能穿透文化。

这是本章最想说的一句话。

怎么识别自己在坑里

1. 你的 AI 场景清单里，**有几个改的是「人怎么用」，几个改的是「事怎么流」**？如果清一色是前者，你的天花板已经封在 L1。
2. 你想做的那个跨系统场景（比如「高管一键看全盘」），**它依赖的流程和数据，现在有几条是断的**？
3. 你上次尝试推动一个跨部门流程改造，**对方拿什么理由挡回来的**？「我们这边流程还没理清」、「这涉及责任归属」、「先小范围试点」——这三句出现得越多，说明组织透明度越低。

正确做法

承认 L1，然后把 L1 做到极致，再有意识地爬向 L2。

在 L1 层的正确做法：

- 把「助手」做到**留痕**：每次 AI 的判断都要能追回原文依据。留痕是信任的唯一入口。
- 把「助手」做到**嵌入**：进 OA、进 CRM、进 FSSC，而不是让人另外开一个网页。
- 把「助手」做成**闭环节奏**：事前 AI 智审嵌入 → 事中 AI 批量巡检 → 事后 AI 批量跟进整改。

爬向 L2 的正确做法（按投入排序）：

1. **先做会议与决议的归集**——这是最便宜的一块「透明」，一次会议一收，日积月累就能知道每次会议的结论是什么。
2. **再把高层关心的指标治理干净**——「至少针对高层关心的指标进行了完整的治理以后，这个智能体才能推出，才有作用」。

3. **最后才碰跨系统的全局感知**——这一步绕不过去，也不需要一步到位。

> **金句**
>
> **「技术能穿透系统，不能穿透文化。」**

8.3.1 ★ 但有一个例外： PMO 案例，以及它为什么值得单独看

★ 上面这一节讲的是一个很硬的规律：**已实现的 23 个全是助手层，因为改流程需要透明，而透明需要文化**。

★ 但同一本书里，第九章 9.8 记录的 PMO 案例，看起来像是这条规律的**反例**：

```
★ 项目管理小助手做了什么？（★ 2026 年 3 月上线，初期用千问的 Agent 架构）
   · AI 预审可行性、立项、启动等各环节核心文件
   · 人工复核之后，★ 线下评审会改成了线上评审会
   · 并打通了投资管理系统与报销系统（★ 预算情况 + 实际支付情况）
   · ★ 超预算在【立项评审 / 变更评审 / 支付环节】当场被发现，★ 而非事后统计
   · 还管模板资料与项目相关制度
   · ★ PM 侧用它做文档初审（★ 零告警才允许提交 PMO）
   · ★ PMO 侧用它按 PMO 的规则规范要点做针对性检测
   · ★ AI 审确定性规则（模板残留/错别字/与本项目无关的内容），人审判断性规则（项目组/执行计划/可行性
   · ★ 实测：线下评审会转线上的比例约 30%~40%
```

★ 这已经不是「助手」了——★ 这是**改了流程**（会怎么开）、**改了系统边界**（连预算系统一起读）。

★ 那它是不是推翻了「改流程需要透明」？

★ 没有。它恰恰印证了这条规律，而且指明了破局的位置。

```
★ 回头看 8.3 给的三条"爬向 L2 的正确做法"：
   1. 先做会议与决议的归集——★ 最便宜的一块"透明"，一次会议一收
   2. 再把高层关心的指标治理干净
   3. 最后才碰跨系统的全局感知

★ 而 PMO 案例做的，★ 正是第 1 条：
   ★ 它没有试图"感知全盘"，只做了一件事——★ 把评审这件事本身搬到了线上
   ★ 线上评审天然产生记录，记录天然产生透明
```

★ 所以 PMO 案例的真实启示不是「我们也能改流程」，而是：

> **★ 改流程不必从最难的那端下手。 ★ 从「一次会议一收」这种最便宜的透明开始， ★ 把一件原本就在发生的事换个形式发生， ★ 透明就自己长出来了。**

★ 而且它的收益结构也和普通助手完全不同：

	★ 普通助手（23 个做成的都是这类）	★ PMO 案例
用户要不要改变习惯	要（另外打开一个入口）	★ 不要（评审本来就要开）
变革发生在哪	用户侧	★ ★ 流程侧（线下会→线上会）
AI 读什么	文档 / 知识	★ 文档 + 投资管理系统 + 报销系统
★ 超预算何时发现	无	★ ★ 立项/变更/支付环节，当场
★ AI 碰不碰判断	—	★ ★ 只取数，判断权仍在评审会
谁得到收益	用的人	★ ★ 所有参与评审的人（PMO、项目经理、部门）
★ 推广靠什么	培训 + 推广	★ ★ 流程自然迁移
★ 覆盖比例	无从谈起	★ ★ 30%~40%（★ 且知道剩下 60%~70% 为什么该留在线下）
★ 到今天	—	★ ★ 上线约 8 个月，仍在用

★ 最后一行是全章最关键的一句：

> **★ 助手型项目要靠「推广」才能活； ★ 流程型项目会自己活——因为流程把人带过来了。**

★ 所以 8.3 的结论不是「只能做到 L1」，而是要补一句更准确的话：

> ★ 8.3 修正：绝大多数做成的确实是助手层，但★ 真正做出流程级改变的案例，★ 走的正是「先从最便宜的一块透明入手」这条最笨的路。
>
> ★ 而这类项目在失败项目里占大多数——★ 因为它要求的组织变革更大，★ 一旦选错了变革的切入点（比如 8.5 营销助手那样另开入口），代价也最大。

8.3.2 ★★ 一成一败的对照：同一个团队，两种进入方式

★ 这是全书目前最有价值的一组对照，但★ **它不是一次严格的对照实验**—— ★ 说清它「不严格」在哪里，比把它说得更漂亮更重要。

	★ 项目管理小助手	★ 营销 AI 助手
时间	★ 2026 年 3 月上线	★ 更早建设，约 3 个月完成上线
★ 团队	★ ★ 同一个 IT 团队	★ ★ 同一个 IT 团队
底座/架构	★ 千问的 Agent 架构（智能体架构）	★ AI 开发 + 移动端适配自建
★ 谁在用	★ ★ IT 团队自己用（技术基础充分）	★ ★ 交给业务部门用（营销一线）
★ 进入方式	★ 接进既有审批链	★ 另开入口
用户要做什么	★ 什么都不用做——流程把人带过来	★ 主动登录、主动提问、主动判断
半年后	★ 仍在用，★ 约三到四成的评审会转到了线上	★ 宣告失败，至今零使用量
到今天(2026-11)	★ ★ 约 8 个月，仍在用	★ ★ 零使用量

★ ★ 先把这组对照「不严格」的地方说清楚（★ 这一段本身比结论更值钱）：

```
★ 差异一：★ 时间不重叠
    两个项目不在同一时期，中间隔着技术进步很快的那几年
            ↓
★ 所以不能说"同一时期、同样的技术条件"
★ 严格讲：如果营销助手晚两年做，未必会失败
            ↓
★★ 差异二：★ ★★ 谁在用，才是更硬的那条线
    项目管理小助手是 ★ IT 团队自己用起来的
    → ★★ 用的人就是造的人，技术基础充分、遇到问题当场就能改
    → ★★ 这本身就是成功的一部分原因，不是"部署方式"能解释掉的
            ↓
★ 营销助手交给了业务部门（营销一线）
    → 用户不掌握技术，遇到问题自己解决不了
    → ★★ 而且它另开了一个入口：★ IM 里既拿不到数据、也检索不到文件
```

★ ★ 所以这组对照真正能支撑的结论，比「同一时期」要小一点，但反而更结实：

> **★★ 不是 AI 不行，是「★ AI 从哪里进入」决定了它是资产还是负债。**
>
> **★ 更准确的说法是三条线一起在起作用：★ ① 它接进了用户原本就要做的事里，还是要求用户为它多做一件事；★ ② 用它的人是不是具备让「它」活下来的技术能力；★ ③ 那个入口本身能不能拿到做事需要的数据和文件。**
>
> **★ 同一个团队、同一套 AI 能力，换一个进入方式，换一批使用者，换一个入口——★ 结果就是一个在用、一个零使用量。**

★ ★★ 而这三条，在第四章 4.4.5 里被提炼成了一条更普适的规律——★★

```
★★★ ★★★ 入口策略只有两个方向：
★★★ ★★★   收敛（把散落的 AI 能力收成一个入口）  →  ★ 有机会
★★★ ★★★   叠加（在旁边再加一个入口）            →  ★ ★★ 几乎必败
★★★ ★★★
★★★ ★★★ 而营销助手走的是第二条：★★ ★★ 它不是"多做了一件事"，★★ ★★
★★★ ★★★ 它是在老路还在、用户有选择权的前提下，★★ ★★ 多加了一个入口
```

★ ★ 所以它输的真正原因，要比「用户不配合」精确得多：

> **★★ 不是 AI 不行，是入口策略的方向反了。 ★★ 用户永远有选择权——★ 而老路在好几项上都更省事。**

★ ★ 而这条规律也划出了一条出路：

```
★★ ★★ 如果"再叠加一个"几乎必败，★★ ★★ 那出路只有两条：
★★ ★★    ① 收敛到已有入口（嵌入 ERP / OA / CRM）— ★ 方向对，★ 但要动别人的系统
★★ ★★    ② 收敛到新建的统一入口（自建专用 APP / 内部门户）
★★ ★★
★★ ★★ ★★ 而在通用协作工具被禁用的组织里，★★ ★★ 往往只剩下第二条
```

★ ★ 而这三条里，最容易被忽视的是第 ③ 条：

```
★ 一个"另开的入口"，如果它天生就够不着数据和文件，
    ★ 那它不是"体验不好"，而是 ★★ 结构性无解
          ↓
★ 用户问一句"上个月华东的动销率是多少"
★ 系统给不出—因为数据在另一套系统里，文件在共享盘上
★ ★ 而 IM 这类工具本来就不是为"取数 + 查文件"设计的
          ↓
★★ 于是失败的过程是这样的：
    第一次问 → 答不上来
    第二次问 → 还是要走老路（Excel / 找同事）
    第三次      → ★ 不问了
          ↓
★★ ★ 而这个"三步衰减"，★ 从管理者视角看，
    ★ 和"用户觉得不好用"是同一件事
```

★ 所以在评估任何一个新 AI 场景时，第一个问题不应该是「这个场景听起来重不重要」，而是：

```
★ 你的 AI 项目需要改变谁的行为？
    · 如果答案是"用户"    → ★ 营销助手的路，代价最高
    · 如果答案是"流程"    → ★ 项目管理小助手的路，代价最低
          ↓
★ ★ 还要再问两句：
    · 谁在用？★ 他有没有能力把"它"养活？
    · 入口在哪儿？★ 它够得着这件事需要的数据和文件吗？
```

★ 而「约三到四成的评审会转到了线上」这个数字本身，★ 也值得单独看一眼——

⋆ 它不是"AI 覆盖率"，是 "⋆ AI 可靠地覆盖了它能覆盖的那一段"
⋆ 剩下六成之所以还在线下，是因为那些会要处理跨部门利益冲突、
⋆ 口径分歧和高风险拍板 —— ⋆ 恰恰是 AI 靠不住的地方
⋆ 而知道"哪部分不该给 AI"，本身就是成熟度的标志

8.4 效率悖论：员工效率提升了，然后呢

现象

这一节讲一件让很多老板和员工都不舒服的事。

很多企业做 AI 的起点，都是**全员认知提升**：让员工了解 AI、熟悉 AI、在日常工作中提效。培训、训练营、大赛、提示词模板，一整套。

然后你会得到一个汇报上非常好看的数字：**人均效率提升 X%**。

但一位有近三年实施观察的项目负责人说：

> 「很多人认为员工能力提升了、效率提升了，就等于降本增效了。**但这往往是一个悖论**。」

为什么会这样

逻辑链其实很清楚：

```
企业做全员 AI 认知提升
        ↓
员工效率提升（单位时间产出 ↑）
        ↓
但公司收入没有增长
        ↓
业务量不变 → 工作量不会随之增加
        ↓
员工"做完该做的事"之后，剩余的只是空余时间 ↑
        ↓
而收入没涨 → 公司不可能给员工加薪
        ↓
✗ 效率红利既没有兑现为个人收益，也没有兑现为公司收益
```

关键在第三步：**收入没涨，工作量就不会涨**。

员工用 AI 把一份报告从两小时压到半小时，省下的一个半小时不会自动变成新的业绩——它变成了空闲。老板看得见「人变闲了」，看不见「人变强了」，于是第一反应是压缩编制。

所以这个悖论不是执行不力，是**结构性的**。你不能靠「再努力一点」打破它，只能改变链条上的某一环。

★ 而「为什么红利默认不归公司」这个问题，§8.4.1 会给出更底层的答案—— ★ 在那里，这个问题会从「管理不善」变成「机制使然」。

值得补一句背景：同样的逻辑也解释了为什么「AI 落地应该直接改变营收」是个陷阱。像蒸汽机、电器或互联网时代一样，对传统制造企业而言，**工作流程和模式需要彻底改变，整个过程需要 5~10 年才可能实现，不会是立竿见影的**。

怎么识别自己在坑里

1. 你的 AI 项目汇报里，**有没有一个数字是「人均节省了多少小时」**？如果只有这个数字，说明你只完成了链条的前两环。
2. 上一个 AI 项目上线后，**业务量涨了吗**？如果没涨，那些省下来的时间去哪了？
3. 你能不能说出「**这个项目的产出，谁会因为它而多拿钱或多拿权**」？说不出，就是纯蒸发。

正确做法

打破悖论只有四条路，你至少要挑一条：

断点	做法	谁来做
收入侧开口子	把省下的时间转投**增量业务**（新客户、新产品、新区域），而不是原有业务	高层
工作量侧扩容	把「空余时间」变成**新增工作项**：原来不做的分析、不覆盖的区域、不盯的渠道	管理层
人员结构侧调整	让一部分人**真的被释放**去做更高价值的事，而不是被要求做更多报表	管理层 + 高层
价值侧重新定义	效率不换算成钱，换算成别的 KPI：质量、时效、员工体验、留才	高层

> ⚠ ◆ **口径提示**：这四条是本书作者对该悖论的**方法论推演**，非当事人原话，入书前需当事人确认【待核实】。

另有一条**组织侧的路**，值得单独说：把 AI 的推动单元从「个人」改成「**部门**」。

以部门为单元组织 AI 大赛，让部门间赛马竞技——好处是部门负责人会更关注，训练营以部门集中式培训为主，从上到下各级人员都能参与。更关键的是，**荣誉与奖励构成了一条不依赖营收增长的兑现路径**：部门拿到荣誉、项目拿到资源、个人拿到评优和晋升。

> **金句**
>
> **「效率提升如果没长出新的业务，就只会长出新的空闲。」**

8.4.1 ★★ 效率红利的归属权：为什么省下来的时间不会变成经营价值

★ 这一节要把上一节的「悖论」再往下一层挖。 ★ 上一节讲的是「红利没有兑现」，这一节讲的是「★ 红利的默认归属规则 ★」—— ★ 而这条规则一旦看清，很多企业的困惑会一次性消解。

现象

企业做全员 AI 提效，最常见的结果是两句互相矛盾的话：

```
管理层看到的：★ 人均效率提升了 ★
经营层看到的：★ 公司经营价值没什么变化 ★
```

★ 而这两句话往往都是真的。

作者此前服务过的一家企业，★ 同样经历过这个局面：★ 那家公司前后做了**四五十个 Agent**，覆盖了多个部门，★ 员工侧确实感受到了效率的提升，★ 但从公司经营的角度看，★ 价值非常低。

★ 注意这里的措辞——「员工提升了价值」，而不是「公司提升了价值」。 ★ 这不是修辞，是两件完全不同的事。

为什么会这样

★ 核心论点只有一句：

> **★ 效率红利有一条默认的归属规则——★ 谁提供了效率提升，谁就默认拿走这份红利。 ★ 而在绝大多数企业里，这条规则的默认指向是员工，不是公司。**

★ 把这条规则展开成链条：

```
员工用 AI 把一周的活压到 3 天做完
      ↓
★ 多出 2 天
      ↓
★★ 而"多承担事情"是有成本的，★★ 不是免费的
      ↓
★★ 公司给对应的回报了吗？
      ↓
· 没有涨薪  · 没有晋升  · 没有扩权  · 没有加编制
      ↓
★★ 所以：★ ★★ 理性的员工不会因为"我又变强了"，★ ★★ 就主动去接更多的事
      ↓
★★★ ★★★ ★★ 这是一件"反人性"的事——★ ★★ 但它完全符合人的理性
★★★ ★★★ ★★ ★★ 员工不懒，★ ★★ ★★ 是 ★★ 激励结构决定行为 ★★
      ↓
★★ 那多出来的 2 天去哪了？★★
· 用来提升自己（考证、学新东西、准备跳槽）
· 做自己的事
· 或者，★ 摸鱼
      ↓
★★★ ★★★ ★★★ 而这一切的净效果是：
★★★ ★★★ ★★★ 效率红利被员工端消耗掉了，★ ★★ 公司经营价值原地不动
```

★ （★ 这一节的另一面在第一章 §1.4.1：员工为什么抵触 AI，★ 同样不是态度问题，而是「位置问题」——★ 抵触强度与岗位的 AI 替代性成正比。★ 两个机制合起来，就是同一个结论：★ 效率和阻力，都不由态度决定。）

★ ★★ 所以，从经营的角度看， AI 提效这件事有一个尖锐的推论：

★★ ★★ 在一家没有回收机制的企业里，★ ★★ AI 提效是"把产能转移给了员工，
★★ ★★ 而不是转移给了公司"。

★★ ★★ 而这意味着一件反直觉的事：
★★ ★★ ★★ 提效做得越猛，★ ★★ 公司经营价值越可能是零。
★★ ★★ ★★ 因为省下来的产能没有承接点，★ ★★ 全部沉淀成了员工的自由度。

★ ★★ 而这也解释了一个更普遍的现象：

> **「人均效率提升 30% 和公司利润增长 0%，往往不是两件事，是同一件事。」**

怎么识别自己在坑里

★ 四个问题，前两个一分钟就能答完：

1. **你的 AI 项目汇报里，有没有「人均节省了 X 小时」这个数字？**

★ 而且，**这个数字后面有没有跟一句「这批时间被投向了哪里」？** → 只有前者，没有后者，说明你只完成了一半。

1. **★ 你能不能说出「这批省下来的时间，最后变成了谁的什么东西」？**

→ 一份新客户的跟进？一个没做过的分析？一次新的流程尝试？ → **说不出，就是蒸发。**

1. **上一个 AI 项目上线后，有没有人因此承担了更多的事？**

→ 如果有，是谁、为什么他愿意？（愿意的原因只有三种：钱、权、或者他真想干。） → **如果答案是「没有人」，那这个项目的产出已经全部蒸发完了。**

1. **★ 当你听说「人均效率提升了」，你的第一反应是什么？**

→ 如果是「那是不是可以少招人 / 压编制」—— → **★ 那说明你正在用减法花掉这份红利，而红利根本没有被收回来过。**

正确做法

上一节给了四个断点（收入侧开口子 / 工作量侧扩容 / 人员结构调整 / 价值侧重新定义）。**★ 但光有断点不够——断点是「往哪走」，而下面这三条是「★ 怎么保证红利真的回到公司手里 ★」。**

★★ ★★ 回收机制三件套：
★★ ★★ 缺任何一件，红利都会漏掉

#	机制	具体做法	为什么缺了就漏
1	★ **给去处**	省出的时间必须指向一个**具体的、有名字的产出**：谁、做什么、什么时候交	不指定去处，时间就会自动流向「最舒服的地方」
2	★ **给回报**	效率红利的一部分必须变成**可见的个人收益**：奖金、评优、晋升、扩权	★ 没有回报的额外责任不会被承担——这是上一节那条「反人性」的直接推论
3	★ **给抽查**	**第 3 个月**就检查「这批红利有没有被承接」，没承接就立刻重新设计	等到年底算总账，红利早就在第一个月蒸发完了

★ ★★ 而这里有一个组织侧的现成做法，值得单独说：

> **把 AI 的推动单元，从「个人」改成「部门」。**
>
> 以部门为单元组织 AI 大赛、部门间赛马，★ 好处不只是荣誉—— ★ 关键是**荣誉与奖励构成了一条不依赖营收增长的兑现路径：部门拿到荣誉、项目拿到资源、个人拿到评优和晋升。**
>
> **★ 这正好绕开了本节那个死结：★ 如果公司利润没涨，就发不出奖金；★ 那就走荣誉、资源、晋升这三条不需要现金的兑现路径。**

一句金句

> **★ 「效率提升的红利，默认归员工，不归公司——除非你设计了一条把它收回去的路。」**
>
> **★ 「省下来的时间不会自动变成收入，它会变成员工的自由度。」**

8.5 ★ 一个失败案例：营销 AI 助手——功能做出来了，用户没有来

> ★ **本节是作者亲历的项目，也是全书目前唯一一个完整的失败项目复盘。** 前四节讲的是「做成了什么」，这一节讲「没做成什么」——而后者才是这本书的读者最需要的。

现象

某大型制造企业的营销条线，要给一线销售人员做一个助手。

它有两块内容：

```
┌─ 内容一：知识库 ──────────────────────────────┐
│   · 制度政策解读                              │
│   · 销售政策与相关销售方案的解读              │
│   · 产品信息                                  │
│   （这一块是文档问题）                        │
└──────────────────────────────────────────────┘
┌─ 内容二：数据 ────────────────────────────────┐
│   · 销售一线随时查自己的销售数据              │
│   · 管理层也能查（看下属、看整体）            │
│   （这一块是数据问题）                        │
└──────────────────────────────────────────────┘
```

营销部门与 IT 部门联合研发。功能做出来了。

★ 它的时间线（作者复盘）：

```
立项 → 实施约 3 个月 → 上线
          ↓
上线后约 3 个月 → ★ 基本无人使用
          ↓
★ 半年周期后，正式宣布失败
（★ 也就是说：上线后约 3 个月就基本无人使用，
   再撑了约 3 个月，半年时宣告结束）
此后 ★ 已停止投入（不再迭代、不再推广）
但 ★ 系统仍然挂在线上，至今 ★ 零使用量
```

★ 投入账（★ 这是失败案例里最该有、也最常被省略的一栏）：

★ 项目周期：约 3 个月建设 → 半年周期宣告失败

★ 常态投入（★ 作者 2026-11-02 最终确认，按 IT / 业务两侧分）：

IT 侧 2.5 人	· 承担开发、数据整理、移动端适配三项职能
业务侧 2.5 人（营销部门核心）	· 业务骨干
合计	5 人

★ 注：这 5 人是「常态投入」，不是「人头名单」—— ★ 一个人可能同时兼几项职能，所以两侧都带 0.5 人。

★ 测试期投入：

营销部门扩到 5 人参与测试	IT 侧仍在	合计 ★ 参与峰值 8 人

↓ ★ ★ 也就是说：★ 半年之内， ★ 常态 5 人投入，最终 ★ 参与峰值 8 人， ★ 全力投在了一个后来零使用量的系统上 ↓ ★ 而这些投入，在很多公司的账上是「分散」的： 技术人力记在研发成本，业务人力记在部门费用，测试记在项目费用 ★ 没有任何一张表会显示「这个失败花了多少」

★ 顺便得到一个反直觉的观察，它比投入数字本身更值得记：

★ 这个项目参与人数的峰值（8 人），出现在测试阶段 ★ 而这个项目最终的日常使用量是 0 ↓ ★ ★ 也就是说：★ 参与人数最多的时候， ★ 离「它会被真正用起来」最远 ↓ ★ 为什么测试期反而要拉更多人？ · 让业务方充分参与 → 显得项目很受重视 · 让测试覆盖更多场景 → 显得很严谨 · 汇报时的参与人数好看 ↓ ★ 但「参与测试」和「日常使用」是两件事： ★ 测试期人人都在，是因为有项目在推 ★ 日常使用要靠每个人自己愿意打开它 ★ ★ 两者甚至可以朝相反方向走

> **★ 这就是为什么「参与人数」不能作为项目健康度的指标。**
>
> **★ 一旦项目停止推进，参与人数会自然归零——但使用量归零得更早。**
>
> **★ 更该盯的指标是：★ 上线之后的第 30 天、第 60 天，有多少人主动打开过它。**

然后有两件事同时发生：

第一件（数据侧）：★ 数据口径不全，数据不完整。

> 即使 AI 把功能实现了，最终也没有产出用户喜爱的（结果）。

★ 说得更具体一点，这个「不满足」是两层：

```
★ 第一层：★ 口径不全
    一线要的数（业绩/过程/对比）与系统里存的数，对不齐口径
    ★ 而缺的那部分口径，★ 在销售的脑子里，不在任何一张表里
          ↓
★ 第二层：★ 数据不完整
    不是"查不到"，而是"查得到的那些不足以回答问题"
    ★ 缺的那部分数据，要么压根没落库，要么散在别处
          ↓
★★ ★★ 于是 AI 在这个场景里必然落到一个尴尬的位置：
    ★★ 它能查，但查出来的不是"他要的那个数"
    ★★ 而它答得越流利，★ 错得越像真的
```

第二件（入口侧）：客户端接入问题。

> 销售人员不习惯使用公司里特有的那些工具，更喜欢用微信这类工具。 而公司禁止把公司数据传输到微信这类软件上。 于是两股力量顶在一起——也就「存在着导致他出错、出问题」。

★ ★★ 而这一条有一个比「用户不习惯」更硬的理由，值得单独写出来：

```
★★ ★★ 就算一线愿意用微信，★ 这条路也是走不通的——
★ 因为 IM 这类工具，★ ★ 天生就干不了这件事：

   ① ★ 它拿不到数据
      ★ 数据在业务系统里，不在聊天窗口里
   ② ★ 它检索不到文件
      ★ 制度、方案、产品资料在共享盘/文档系统里
   ③ ★ 它不会自动检索文件和数据做分析
      ★★ 没有"文件→分析→回答"这条链路
        ↓
★★ ★★ 所以这条路的真实结构是：
   ★★ 用户在 IM 里问一句"上个月动销率多少"
   ★★ 系统给不出——它既没有数，也读不了文件
   ★★ 而这不是"再开发一个功能"能解决的，★ 是工具形态决定的
```

★ ★ 而这就把 8.5 的失败从「选错了入口」升格成一句更硬的话：

> **★ 失败不只是「入口选错了」，而是「★ 这件事压根不该放进这个入口」。**
>
> **★ 一个入口如果既拿不到做事需要的数据，又检索不到做事需要的文件， ★ 那它不是「不顺手」，它是「★ 结构性地做不了这件事」。**

★ 而「用户到底什么表现」，这一点其实没有模糊地带：

```
★ ★★ 用户的真实表现是：★ 根本不用
   —— 不是"用了就骂"，也不是"嘴上说不好用但偷偷用"
   —— ★★ 是上线后约 3 个月，几乎没有打开过
```

结果：功能实现了，场景没有跑起来。

★ ★★★ 而「AI 走了以后，人是怎么干活的」，是整个案例最有画面感的落点。★ ★★★ 而这一条，作者 2026-11-03 补充后，它不再是一个「退回去了」的注脚，★ ★★★ 而是一条完整的因果链——它同时解释了「当初为什么做」和「为什么失败」。

先看 AI 之前，那条老路长什么样：

```
★★ ★★ 营销一线要看数的时候，★ 走的是两条现成的路：

   ① ★ 数据部门把数据整理成 Excel，★ 发邮件给大家
      → 谁要看，自己打开附件
      → 想给同事看，★ 转一下邮件就行
      → 想看哪部分，★ 自己挑

   ② ★ 公司 BI 平台自助查询
      → ★★ 但这条路用得很少
      → ★★ 原因很朴素：★★ 一线本来就不太喜欢用系统
      → ★★ 他们更习惯打开邮件——★ 邮件更方便
        ↓
★★ ★★ 于是真实的日常是：★ ★★ 邮件是主力，BI 是备用
```

★ 而这就是当年立项时，那条听起来非常正当的理由：

```
★★ ★★ "一线要随时随地查数据，★ 邮件做不到"
★★ ★★ "★ 手机上能快速拿到，★ 这才是我们要的"
        ↓
★★ ★★ 这条理由，★ ★★ 在今天回头看，★★ 它是对的
★★ ★★ 它抓到的痛点是真实的：★ 邮件确实不能移动端查
★★ ★★ 它给的价值是具体的：★ 不是"更聪明"，★★ 是"★ 在移动端快速获取"
        ↓
★★ ★★ 所以这个项目不是拍脑袋想出来的，★ ★★ 也不是为了做 AI 而做 AI
```

★★★ 而现在，这三条事实摆在一起，画面就完整了：

```
★ 立项理由：★ 移动端快速获取  ★ —— ★ 这是对的
★ 老路的现状：★ 邮件是主力（BI 用得少）★ —— ★ 这也是对的
★ 结果：★ ★ 用户回到邮件
        ↓
★★★ ★★★ 于是真正的问题浮出水面了——
★★★   ★★★ 不是"AI 对不对"，★★ 是"★★ 这条老路，其实比想象中好"
```

★★★ ★★★ 而这里有一句，是本案例最该被记下来的话：

★★★ 「★ 老路被用户抛弃」和「★ 老路其实一直在赢」，是两种完全不同的诊断。★★★ 而这两种诊断，会导向完全相反的行动：★★★ 前者让你去优化产品、换入口、加推送；★★★ ★★★ 后者让你去问一句更要紧的话：★★★ 「★ 那个被 AI 顶掉的需求，★ 到底值不值得用 AI 来顶？」

★ ★★ 而顺着这个方向，还能再往下问一层——★ 为什么老路会赢？

```
★★ 邮件这条路赢，★ 不只是因为"方便"，★ 而是因为它保留了三样东西：
   ① ★★ 选择权：★ 用户自己挑要哪张表、哪个时间段
   ② ★★ 可传递：★ 转给同事、抄送主管，★ 一封邮件就够
   ③ ★★ 零学习：★ 不用培训、★ 不用改习惯、★ 不用记任何新操作
          ↓
★★★ ★★★ 而 AI 助手这一侧，★ 恰好把这三样都收走了：
   ★★★ 它替用户挑（你只能问）、★ 它默认单线程（不如转邮件）、
   ★★★ 它要求用户先学会怎么问（★ 这是一道新门槛）
          ↓
★★★ ★★★ 于是出现了一个很反直觉、但非常普遍的规律：
★★★    ★★★ ★★ 用户不会因为"新工具更先进"而迁移，
★★★    ★★★ ★★ 只会因为"新工具让我的那三样东西变得更麻烦"而放弃。
```

★★★ ★★★ 而这一层，本书可以署名一条判断：

★★★ ★★★ 「★ 助手型产品有一个结构性的软肋：★ 它替用户做了选择。」★★★ ★★★ 而用户之所以一直忍受着邮箱和 Excel 这种「落后」方式，★★★ ★★★ 恰恰是因为那些方式把选择权完整地留给了他们。★★★ ★★★ ★★ 一个只解决「取数快」、但同时收走选择权的产品，★★★ ★★★ ★★ 很容易变成「更方便的系统，但不如邮件方便」。

★★ ★★ 而这个案例真正的代价，到这里才算算清楚：

```
★★ ★★ 不只是那 5 个人半年
★★ ★★ 而是：★★ 它占住了入口（建了一个 App），★★ 却没有替换掉原来那条路
★★ ★★ ★★ 而原来那条路，一直都在，★★ 而且更好用
          ↓
★★★ ★★★ 所以这不是"白做一个 App"，
★★★ ★★★ 而是"★★ 用一个更贵的方式，★★ 替用户维持了一种他们本来就不想要的生活方式"
```

为什么会这样

为什么会这样（零）：★ 为什么「回到邮件」不是用户保守，是用户理性

★ 上面那条因果链讲完了「发生了什么」，这一节讲「为什么必然会这样」。

```
★ 假设把时间倒回立项那一刻，★ 双方都很理性：

   业务方要的是：★ 移动端快速拿数
   IT 方给的是：★ 一个能查数的新入口
   ★★ 两边都没错，★★ 而且这个需求是真的
          ↓
★★ ★★ 那为什么它会失败？★★ 因为需求成立 ≠ 产品成立
```

★ ★★ 而「产品成立」这件事，需要多满足一条很少被写进立项文件的前提：

```
★★★ ★★★ 新方式，★ 必须比老方式更省事——★★ 而不是只在新场景里更省事
          ↓
★★ ★★ 老方式（邮件 Excel）在这三件事上，★ 恰好赢了 AI 助手：
   · ★★ 移动端做不到 → ★★ AI 赢了
   · ★★ 但"选哪张表"要自己挑 → ★★ 邮件赢了
   · ★★ 但"转给同事"一封就够 → ★★ 邮件赢了
   · ★★ 但"零学习" → ★★ 邮件赢了
          ↓
★★★ ★★★ ★★ 结果就是：★ AI 赢的那一项（移动端），
★★★ ★★★ ★★ 抵不过老路赢的那三项（选择权 + 可传递 + 零学习）
```

> **★★★ ★★★ 而这带出一条比案例本身更普适的判断：**
>
> **★★★ ★★★ 「★ 新工具要替换掉老工具，★ 光有增量优势是不够的——」 ★★★ ★★★ 「★ 老工具身上那些看起来'不先进'的优点，★ 恰恰是它活到今天的原因。」 ★★★ ★★★ 而这些优点，通常不出现在任何一份需求文档里， ★★★ ★★★ 因为它们是「没人提」，而不是「不重要」。**

★ ★★ 而再往下一层，还有一个更朴素的原因，★ 这个原因可能才是最致命的：

```
★★ ★★ 邮件这条路，★ 是数据部门"顺手给"的
★★ ★★ 而 AI 助手这条路，★ 是要用户"主动去用"的
          ↓
★★★ ★★★ ★★ 顺手的动作会一直发生，★★ 主动的动作需要理由
★★★ ★★★ ★★ 而"我今天想查个数"——★ 这个理由，★★ 在有邮件的情况下，★ 永远不够强
```

> **★★★ ★★★ 而这带出本书全书最该反复讲、目前却少有人明说的一句：**
>
> **★★★ ★★★ 「★ 一个新工具真正的对手，★ 从来不是另一个新工具， ★★★ ★★★ ★★ 而是那个『★ 不需要用户做任何新动作★』的老办法。「**

为什么会这样（一）：双失败互相掩护

这个案例最有价值的地方在于：**它不是单点失败，而是双失败，而且两者互相掩护**。

失败点一：数据不满足业务需求
失败点二：入口不在一线的工作流里
↓
★ 单看任何一个，失败原因都很清楚
· 只有数据问题 → 用户会说"查出来的数不对" → 明确反馈，好修
· 只有入口问题 → 用户会说"我不会用" → 明确反馈，好修
↓
★ 但两者同时出现时，用户给不出有效反馈
因为他自己也分不清是"数不对"还是"不顺手"
↓
★ 最终表现只有一个字：不用
↓
★ 而"不用"是所有失败里最难诊断的一种

为什么会这样（二）：数据侧的三重错位

一线说的「我的销售数据」，要的是：

· 我的业绩 —— 结算口径？发货口径？回款口径？
· 我的过程 —— 铺货、动销、库存、拜访、订单
· 我的对比 —— 同比、环比、达成率、排名

而系统里天然躺着的是：

· 结算口径的发货明细
· 批次的销售订单
· 月度汇总表

于是产生三重错位：

错位	表现	为什么难发现
口径错位	他要「我的数」，系统给的是「业务的数」	系统里的数**是真数**，报表对得上，只是不是他要的那个
颗粒错位	他要过程与对比，系统给明细与汇总	看起来「有数据」，只是颗粒不对
时效错位	「实时」与「完整」互斥	两个要求同时提，谁都不肯让步

> **★ 本书可以署名的一条判断：**
>
> **数据类 AI 场景的第一性问题不是「能不能查」，是「查出来的数是不是他心里的那个数」。**

> **★ 「老路被用户抛弃」和「老路其实一直在赢」，是两种完全不同的诊断。**

> **★ 助手型产品有一个结构性的软肋：它替用户做了选择。**

> **★ 一个新工具真正的对手，从来不是另一个新工具，而是那个「不需要用户做任何新动作」的老办法。**

> **★ 老工具身上那些看起来「不先进」的优点，恰恰是它活到今天的原因。**
>
> 而「他心里的那个数」，在两个系统对不齐之前， IT 是查不出来的——**因为它只存在于业务的口头习惯里，不在任何一张表里。**

> **★ 第二条判断，来自这个案例最实际的一条：**
>
> **「要实时」和「要完整」，在多数企业系统里是互斥的两个要求。**
>
> 谁先承认这一点，谁就能把需求谈下来；谁都不承认，双方就会在验收阶段互相指责——IT 以为业务贪心，业务以为 IT 敷衍。

还有一层更贵的成本：这个错位在项目早期是看不见的。

```
功能做出来了 → 一开始查得出数（只是范围窄）
              → 用户提意见 → 加功能 → 还是不对
              → 再加功能 → 变成"用不上的功能越来越多"
              ↓
★ 于是项目不是"失败"，而是"慢慢死掉"
   —— 每一步都有产出，每一步都不解决问题
```

为什么会这样（三）：入口侧的错位

```
一线的真实工作入口是什么？
   群消息、单聊、随手截图、一句语音
        ↓
而这些大多在个人社交软件里
        ↓
公司系统要求他们去：门户 / 客户端 / 自建工具
        ↓
★ 两个同时成立的事实：
   ① 一线没有"为了查个数而切换工作环境"的习惯
   ② 公司红线：公司数据不得外传到个人社交软件
        ↓
★ 于是唯一的合规路径，恰好是最不顺手的路径
        ↓
一线的两个选择，都对应"出错、出问题"：
   · 不合规地绕过去（截图转发、私下问 IT 手动出数）→ 出问题
   · 干脆不用                                          → 项目失败
```

★ ★★ 而在「另开入口」这条路上，还有一层更深的结构性问题：

```
★ 前面说的是"用户不顺手"，★ 但往下追一层会发现：
★★ 这个入口★ 本身就不具备做这件事的能力

   IM 能聊天  → 但拿不到业务系统的数
   IM 能传文件 → 但不会自动检索文件和数据做分析
        ↓
★★ ★★ 于是"入口选错"这个说法其实不够准确：
   ★★ 准确的说法是 —— ★★ 这件事压根不该放进这个入口
        ↓
★★ ★★ 这也是"另开入口"最危险的地方：
   ★★ 它看上去只是"体验差"，★ 实际上往往是"做不了"
   ★★ 而前者能靠优化改善，★ 后者只能靠换形态
```

★ ★★ 所以这个案例最扎人的地方，是它和「现在怎么查数」放在一起看：

```
★ AI 在的时候：用户要开另一个工具、换一种姿势、学会怎么问
★ AI 不在的时候：用户打开 BI 平台，或者收一封邮件里的 Excel
        ↓
★★ ★★ ★★ 老路没有消失，它只是变得更明显了
★★    它不需要任何学习成本，★ 所以它永远会赢
```

> **★ 第三条判断：**
>
> **一线不用你的工具，不是因为懒，是因为你的工具不在他的工作流里。**
>
> **而当合规红线把「唯一正确的路径」和「最顺手的路径」分开时，失败的不是用户，是设计。**

为什么「营销 + IT 联合研发」没有救回来

一个自然的问题是：两个部门一起做，为什么还失败？

联合研发解决了：需求描述、优先级、验收标准

↓

★ 但它解决不了这两件事：

① 口径 —— 口径不在 IT 手里，也不在项目组手里，在销售的脑子里

② 入口 —— 入口不在项目组的权限里，在公司的红线与一线的习惯之间

↓

★ 这两件事都不是"协作"能解决的

它们分别需要"业务下定义"和"公司改红线或改数据形态"

★ 所以这个案例给出了一条反直觉的结论：

> **跨部门联合研发能提高项目成功率，但它救不了「数据口径」和「入口合规」这两类问题。**
>
> **因为这两类问题的解法不在项目组手里。**

怎么识别自己在坑里

这个案例可以反过来做成一张自查表。

信号一：问答类用得还行，数据类几乎不用。

上线后问一句"XX 政策怎么规定的"，有人用；

问一句"我这个月卖了多少"，没人用。

↓

★ 这说明知识库那半边是成立的，数据那半边没过关

—— 而项目往往因为"问答能用"而误判为"整体成功"

信号二：用户私下问同事要数。

一线在群里问"我这个月的达成率多少"，另一个同事截图发过去。

↓

★ 这是最诚实的信号：

用户已经绕过了你的系统，并且绕过了红线

信号三：需求评审里出现「要实时」。

```
"最好能实时看到" —— 这句话出现时，
★ 要立刻问一句：你能接受 T+1 吗？
   因为"实时 + 完整"通常互斥
```

信号四：项目在「加功能」而不是「在收敛」。

```
第 3 版、第 4 版、第 5 版功能
          ↓
★ 每一版都在加功能，且每一版都不解决最初那个"数不对"的问题
   → 这是"慢慢死掉"的典型轨迹
```

信号五：验收标准里没有「满足率」这类可测指标。

```
如果验收标准是"能查到数据" → 一定能通过（功能做了就能查）
如果验收标准是"用户问的 10 个高频问题，8 个能给出他要的数" → 才算数
```

正确做法

做法一：入口不要争，改「结论找人」。

```
❌ 错误做法：做客户端、做门户、要求一线登录系统查数
   → 入口之争永远赢不了：你要改变的是几千人的工作习惯

✅ 做法：数据留在系统里，结论送到人所在的通道
   · 企业内 IM 机器人：查数直接问，不切换环境
   · 定时简报：日报 / 周报主动送到群里，不等人来查
   · 异常提示：只在异常时出现，不查不问
          ↓
★ 核心转变：把"入口之争"降级为"触达方式之争"
```

做法二：合规红线不动，能动的是「外传什么」。

公司红线（数据不得外传到个人社交软件）不必动，也不该动

↓

能动的是数据形态：

- 只推结论，不推原始明细
- 只推脱敏 / 聚合后的摘要
- 关键数据留在企业内通道，不落到个人设备

↓

★ 这条同时把第七章 7.2 的"数据分级"讲清了：
分级不是"禁止外传"，是"决定外传的形态与颗粒度"

做法三：先对口径，再谈实时。

第 1 步　口径对齐会：营销与 IT 双方各出 5 个高频指标
　　逐个定唯一算法，并当场写下来（第三章的数据可用性清单）
第 2 步　只做营销最痛的 3 个指标，不做"全量实时"
第 3 步　先切"查询"，不做"智能分析"——先让数可信
第 4 步　数据可信之后，再谈 AI 能不能给结论

↓

★ 顺序不能颠倒：
先上分析能力，是在不准确的数上叠加不确定的推断

做法四：这个场景应该有止损线，而且应该更早。

试点 4–8 周，若同时出现：

- 主动使用率 < 30%
- 数据类请求的满足率 < 50%（用户问的数，系统给不出来）

↓

★ 结论：先回去治数据与入口，不要继续加功能
—— 加功能会掩盖数据问题，而不是解决它

↓

★ 本书的现实对照：这个项目实施约 3 个月上线，
上线后约 3 个月就"基本无人使用"了，
★ 但止损线在"上线 3 个月后基本无人使用"时早该触发，
实际是撑到半年周期才正式宣告失败
—— ★ 这半年的后半段，投的是"明知不行、但还没到宣布的时候"的钱
—— 真正缺的不是阈值，是"触发之后谁来拍板关掉它"

做法五：把「半成功」重新定义为资产。

这个项目没有跑起来，但它留下了三样东西：

① 一份营销高频指标的口径清单（哪怕只定了 3 个）

② 一条被验证走不通的入口路径（这本身就是结论）

③ 一批知道了"AI 能做什么、不能做什么"的一线人员

↓

★ 而第三样，正是第一章说的"组织能力"——

失败项目如果什么都不留下，它才是真正的失败

金句

> 功能做出来了，不等于场景做成了。
>
> 用户不喜爱的原因，通常不在模型，在数据和入口。

> 一线不用你的工具，不是因为懒，是因为你的工具不在他的工作流里。

> 当合规红线把「唯一正确的路径」和「最顺手的路径」分开时，失败的不是用户，是设计。

> 「要实时」和「要完整」在多数系统里是互斥的——先承认这一点，再谈需求。

> 数据类 AI 场景的第一性问题不是「能不能查」，是「查出来的数是不是他心里的那个数」。

> ★ 「老路被用户抛弃」和「老路其实一直在赢」，是两种完全不同的诊断。

> **★ 助手型产品有一个结构性的软肋：它替用户做了选择。**

> **★ 一个新工具真正的对手，从来不是另一个新工具，而是那个「不需要用户做任何新动作」的老办法。**

> **★ 老工具身上那些看起来「不先进」的优点，恰恰是它活到今天的原因。**

> **联合研发能提高项目成功率，但救不了口径和入口——因为它们的解法不在项目组手里。**

> **失败项目如果什么都不留下，它才是真正的失败。**

8.5.2 ★ 一个黄掉的方向：想给 C 端做数字员工

> **★ 本节补上了第八章论证里缺失的第三档。** 8.1 那个「46 计划做 / 23 已实现 / 零重叠」的盘点里，一直缺一档：**★ 曾经规划过、投了人、最后散了的方向。** ★ 而这一档，恰恰是最值得复盘的——因为它最不像「失败」。

现象

内部数字员工做了很多场景之后，一个很自然的想法冒出来：

★ "既然对内能用数字员工，
那对外能不能也做一个？"
↓
对外 = 对 C 端客户，或对供应商
↓
目标是一整套：
① 形成 AI 应用模式的转变
② 形成营销模式的变化
③ 把线下变线上
④ 数据回流
⑤ 沉淀知识
↓
★ 最终落点：促进销售、改善用户体验

★ 而在怎么做的层面，规划时考虑过一系列形态：

穿戴设备 | 智能屏 | 虚拟数字人 | …… 一系列方式
↓
★ 规划阶段与国内头部大厂合作推进
↓
★ 最终，三个问题让这个方向停了下来：
① AI 技术整体不成熟
② ★ 企业内部数据沉淀和管理模式不成熟，无法适应新模式
③ 前期投入大、周期长，效果不明显，
可能只是散在小事点上的创新，不能带来本质变化

为什么会这样

★ 先说最要紧的：这三条原因里，最有价值的是第二条。

★ 原因二：企业内部数据沉淀和管理模式不成熟，无法适应新模式
↓
★★ 而这句话，是【只有甲方自己会说出来的话】
↓
大厂不会说 —— 那是他们的客户，不便评价
乙方更不会说 —— 说出来等于说"你们家底不行，别做"
↓
★ 于是这句话，恰恰是这本书能写、
而乙方视角的书写不了的东西
↓
★ 也正是这个定位，让这本书值得被读到

★ 而这三条原因，合起来其实是一条完整的通用判据：

★ 一个 AI 方向该不该做，看三件事：

① 技术条件成熟了吗？
不成熟 → 这不是你的错
★★ 但也不该由你来承担这个风险

② ★ 内部的数据和管理，配得上这个新模式吗？
配不上 → ★★ 这不是技术问题，是你还没准备好
★★ 而"还没准备好"是可以解决的，只是需要时间
→ ★ 这条的解法恰恰是第三章的内容

③ ★ 它带来的是"本质变化"，还是"小事点上的创新"？
★★ 只能带来小事点上的创新 → 那它不叫战略，那叫碎片

★ 再看这个方向本身，它踩中了三个很典型的坑。

★ 坑一：载体先行。

★ 当年的路径：先考虑载体（穿戴设备 / 智能屏 / 虚拟数字人），
再往里填内容
↓
★ 而正确的路径是反过来的：
先想清楚"哪个业务动作需要被数字化"，
再决定用什么载体
↓
★ 载体是手段，不是方向。
★ ★ 先定载体的人会被载体绑架：
每种载体都要投一大笔钱，
而每一种都只能做到"小事点上的创新" ——
★ 恰好就是原因三说的那句话

★ 坑二：「数据回流」这句话有个前提，书里很多方案都漏了它。

★ "把线下变线上，数据回流，沉淀知识" —— 听起来无懈可击
↓
★ 但"回流"的前提是：★ 线下业务本身已经产生了结构化数据
↓
如果线下的动作还发生在纸单、微信、口头交流里
↓
★ 那就没有东西可以"回流"
而"沉淀知识"的前提更严：
★ 你得先有一个被公认的知识来源（制度、流程、专家）
否则沉淀下来的是一堆没人认的东西
↓
★ 这正是第二章 2.2.1 那张速查表里"销售·数据"为什么是最差一格的原因：
★ 数据不在系统里，AI 就只能"猜"，而它猜的东西没人敢用

★ 坑三：★ 一个方向要改变的东西越多，越可能一件都改不成。

★ 当年那个方向，同时想要五样东西：
① AI 应用模式转变 ② 营销模式变化 ③ 线下变线上
④ 数据回流 ⑤ 沉淀知识促进销售与体验
↓
★ 而最后落点其实是 ⑥"促进销售和用户体验"
—— 从"改变模式"退回到了"促进某个指标"
↓
★ 教训：
★ 一个项目如果同时要改变五件事，
★ 那它一件都没打算真改变

怎么识别自己在坑里

★ 这一类失败最难识别，因为它看起来不像失败。四个信号：

□ 1. ★ 项目的目标里同时出现了五个以上的"改变"
→ 目标越多，越没有一条会被真正推动

□ 2. ★ 讨论里出现得最多的是"形态"，不是"业务动作"
→ "用穿戴设备还是智能屏"讨论了三个月，
但没人能说出"哪个具体的业务动作要被改变"
↓
★ 而只要说不出那个动作，就说明这个方向还没想清楚

□ 3. ★ 内部数据准备情况，从没被认真问过
→ 因为"数据"听起来像是实施阶段的事
→ ★ 而这一条，其实在立项前就该问

□ 4. ★ 这个方向"停"下来的方式是"先放一放"
→ 不是被否决，不是被降级，是没人再提
↓
★★ 这是最糟的一种结束方式：
投入已经花掉了，但没人说得清它算什么
→ 它会在第二年以"我们还在探索"的形式重新出现

★ 而识别「方向性失败」的关键，是和另外两种失败分开：

★ 你需要先分清，你现在面对的是哪一种：

场景层失败：方向对、场景选错 → 换场景重来
方向层失败：★ 方向本身不该做 → 要重新想方向
试探层失败：试了、不成、认了 → ★ 最便宜，收工
↓
★ ★ 而这三种里，方向层失败最贵、也最不容易被察觉

正确做法

做法一：★ 把「内部数据配不配得上」提前到立项前问一次。

★ 一句话的问题：
"这个新模式要用的数据，现在在我们公司里以什么形式存在？"
↓
□ 存在系统里、结构化、可查
→ 可以往下走
□ 存在系统里、但不完整 / 口径不统一
→ ★ 先做数据治理，这本身就有价值
□ 还在纸单、微信、口头里
→ ★★ 停。这不是 AI 项目，这是流程数字化项目
→ ★★ 而且流程数字化该由流程负责人牵头，不是 AI 项目组

做法二：★ 把「载体」从方向讨论里踢出去。

★ 一条纪律：
★ 载体（穿戴设备/智能屏/数字人/小程序/App）
★ 只在"业务动作已经确定"之后才允许被讨论
↓
★ 因为载体的选择会反向绑架目标：
选了穿戴设备，就会不自觉地去找"用穿戴设备能做的事"，
★ 而那个东西通常就是"小事点上的创新"

做法三：★ 用「本质变化」这个标准做一次收口。

★ 一个方向要落地，至少要能回答一句：

"如果这件事做成了，
★ 公司在哪个方面会不一样？"

↓
□ 能说出具体的、别人能认的变化
→ 值得投
□ 只能说"效率高一点""体验好一点""数据多一点"
→ ★★ 这就是"散在小事点上的创新"
→ ★★ 它的收益抵不上它的投入

做法四：★ 让「没立项的失败」也有一个结论。

★ 这个案例真正可惜的地方不是没做成，
★ 是它"散"掉了 —— 没有一个结论被写下来
↓
★ 一个方向被叫停时，至少留下三样：
① 一句话结论："这个方向在当时的条件下不成立"
② 三个原因（技术/数据/投入产出各一条）
③ 一句"什么条件下可以重启"
↓
★ ★ 而第三样最值钱：
"等线下业务完成数字化之后，这条路值得重新走一遍"
—— 这句话留着，
★ 就不至于三年后有人再从零想一遍

金句

> ★ 劝一个方向停下来，最重要的不是「为什么不行」，是「什么条件下可以再来」。

> ★ 一个方向要改变的东西越多，越可能一件都改不成。

> ★ 载体是手段，不是方向。先定载体的人会被载体绑架。

> ★ 「把线下变线上」是最容易讲的故事——但线下没有结构化数据，就没什么可回流。

> ★ 「散在小事点上的创新，不能带来本质变化。」

> ★ 试探性失败最便宜，方向性失败最贵——而方向性失败最不像失败，因为它看起来像「在探索」。

> ★ 没立项的失败，是预算的黑洞。

8.5.3 ★★ 为什么 AI 的失败是「速生」的，而传统系统的失败是「慢性」的

★ 这一节是 8.0.2 那个结构性事实的直接推论，也是本书最反直觉的一节。 ★ 作者 2026-11-03 补充。

```
★★ ★★ 一个反直觉的经验事实：★★
★★ ★★ 在 AI 领域，★ "★ 用了几个月、后来没效果就放弃了"★ 这种案例，★ 其实不多
```

★★ 而这与传统应用系统的实施，★ 差别大到值得单写一节：

```
──────────── 传统系统 ────────────
立项 → 建设（半年到两年）→ 上线
        ↓
★ 上线那一刻，★ 整套东西已经在流程里跑着了
★ 组织为了它改过流程、投过培训、配过岗
        ↓
★★ 于是它的命运 = ★★ 沉没成本 + ★★ 已被改造的流程
★★ ★★ 哪怕不好用，★★ 大家也硬着头皮用下去
★★ ★★ ★★ 而且 ★★ 是用很多年，★★ 才在换代时换掉
        ↓
★★ ★★ 这就是传统的"慢死"：★★ ★★ 失败是慢性的、隐性的、几乎无痛的

──────────── AI 应用 ────────────
上线
        ↓
★★ 第一印象就是全部：★★
★★ 好不好用、答得对不对、★ 一次就见分晓
        ↓
★★ ★★ 如果一开始就不好用：★★
★★    ★★ 用户会在很短时间内失去信心，★★ 根本不会"用几个月再放弃"
        ↓
★★ ★★ 反过来，★★ 如果它被持续用上了几个月：★★
★★    ★★ ★★ 它就产生了粘性——★★ 用户会提新需求，★★ 会在上面加用法，★★ 会离不开它
        ↓
★★★ ★★★ ★★ 于是 AI 的命运是二元的、★ 陡的：
★★★    ★★★ 速死，或者长住
```

★★★ ★★★ 而这个差别，★ 直接改写了一条行业惯性判断：

```
★★★ ★★★ "★ 很多 AI 项目花了时间没达到效果，★ 后来就放弃了"
        ↓
★★★ ★★★ ★★ ★★ 而按上面的观察，★★ 这句话在 AI 领域其实不太成立——
★★★ ★★★ ★★ 因为它要么早就死了（没到"用几个月"），★★
★★★ ★★★ ★★ 要么已经被用出粘性了（不太会主动放弃）
        ↓
★★★★ ★★★★ ★★ 所以真正需要解释的，★ 不是"为什么用几个月后放弃"，★ 而是：
★★★★    ★★★★ ★★ "★ 为什么很多 AI 项目甚至没撑过第一个月？"
```

★★★ ★★★ 而这一节的落点，其实比那个观察本身更有用：

★★★ ★★★ 因为「速死」这个特征，★ 给了一条与传统完全不同的止损原则：

★★★ ★★★ 传统项目的止损窗口在中期（都投了这么多，不能轻易停）；★★★ ★★★ ★★ 而 AI 项目的止损窗口在早期——★★ ★★ 越早停，损失越小。★★★ ★★★ ★★ 所以对 AI 项目，★ 「★ 再给它一个月」这句话，★ ★★ 几乎总是错的。

★ ★★ 而这条观察，还有第二个用处——它反过来定义了「成功」：

```
★★ ★★ AI 项目的成功标志，★ 不是"★ 功能实现了"
★★ ★★ ★★ 而是：★★ "★ ★★ 它被用过了几个月，★★ 而且用户开始对它提新要求"
          ↓
★★★ ★★★ ★★ 因为这个"提新要求"，★ 才是粘性的真实证据
★★★ ★★★ ★★ ——它证明用户已经把这件事的完成，★★ ★★ 委托给它了
```

★ 金句

「AI 项目的失败是速生的：不好用，用户就不用了；不会用几个月再放弃。」

「用户开始对 AI 提新需求，才是它活下来的真正证据——那说明他已把这件事委托给了它。」

8.6 ★ 半失败的价值：投入少、损失低的另一种「成功」

> **本节讲的是全书最反常识的一课**，也是一个企业最该学会的评价方式。★ 案例：智能问数（2026 年 2 月完成部署 → 测试阶段 → 计划 4 月推广 → 最终「半失败」）

现象

先说清楚「半失败」是什么意思，因为这个词很容易被说成好话。

```
智能问数这条路走过的轨迹：
   2026 年 2 月  · 完成部署，进入测试阶段
   计划 4 月      · 公司内推广
        ↓
★ 最终状态：试了、推了，但一线没有继续用
   —— 功能在，数据在，流程在，人不在
   —— ★ 无人使用状态已持续长达 5 个月
```

★ 同样重要的是「停下来的方式」：

```
这个项目没有像营销助手那样拖着——它是被明确承认地停掉的。
对比一下两个项目"停下"的样子：

   营销 AI 助手：上线 → 3 个月后基本无人使用 → ★ 撑到半年周期才正式宣告失败
                        （期间系统仍在线，只是没人用，也不再迭代）
   智能问数    ：部署 → 推广 → ★ 无人使用长达 5 个月，投入已停
        ↓
★ 同样是"没人用"，一个撑到半年才宣告失败，一个几个月就认了
   —— 差别不在结果，在"认账的速度"
```

它不是「没做成」，它是「做成了功能，但没有变成日常」。

而这个案例最关键的一句话，来自作者自己的复盘：

> **它最终算是个半失败的产品，但因为没花相应的费用，也可以说是初步尝试—— 体现了 AI 创新专项模式的优势：试验了、推了，但没继续用；投入少、损失低，符合创业试点的逻辑。**

★ 请注意这句话里藏着的评价标准，它和行业通行的评价标准完全不同：

行业通行的标准：	成没成？
这个案例的标准：	★ 失败的代价是多少？

为什么会这样：两种「没成」的差别，是本书最重要的一次区分

把两个案例并排放，差别立刻显形：

	8.5 营销 AI 助手	8.6 智能问数
为什么没成	数据口径不对 + 入口与红线冲突	价值未被一线认可，场景本身需求不足
上线到"基本无人用"	约 3 个月	部署后推广，★ 无人使用状态已持续 5 个月
停在哪一步	上线 3 个月后基本无人用，★ 撑到半年周期才正式宣告失败	无人使用后不久即停，不再追加投入
是"半失败"还是"慢性失血"	★ 慢性失血（半年才认账）量仍在挂	★ 半失败（试过即止）
系统还在不在	★ 还挂在线上，至今	功能在，无人使用
结束方式	★ 正式宣告失败，但系统仍挂着，没人再管	★ 结束了，且是被承认的结束
数据/口径治好了吗	没治好，且被功能掩盖	不需要治：边界清晰
代价	★ 高：★ 常态 5 人投入、参与峰值 8 人，且至今零使用	★ 低：几乎无沉淀损失

★ 而把三个失败案例放在一起，会看到一条更要紧的排序：

★ 失败的三个层次（★ 本书最该记住的一张对照表）

	8.5 营销助手	★8.5.2 C端数字员工	8.6 智能问数
失败在哪一层	场景层 （选错了场景）	★ 方向层 （这事本身不该做）	试探层 （试了，不成，认了）
投入规模	中	★ 大	小
周期	短	★ 长	短
结束方式	★ 半年宣告失败，	★ 散（没人再提）	明确承认后停
留下的东西	负资产（系统还在线）	★ 一份 PPT 和会议	★ 一个清楚的结论
损失	中	★ 最大	最小
★ 最容易掩盖吗	中	★ 最容易 （可以说"在探索"）	最难 （结论清楚）

↓

★ ★ 试探性失败最便宜，方向性失败最贵——
★ ★ 而方向性失败最不容易被察觉，因为它没有"失败"的样子。

★ 所以真正的分界线在这里：

> 同样是「没成」，一种是「没有结论」，一种是「有结论的结论」。
>
> 智能问数的价值在于：它用很小的代价，买到了「这条路走不通」这个结论。
>
> 而营销 AI 助手的问题不是失败，是它用近 5 个人力、半年周期，撑出了一个至今零使用量的系统。
>
> ★ 更值得警惕的是它现在的样子：系统还挂在线上、零使用量、也没有人再管它。一个「正式宣告失败、但系统还挂着」的项目，会一直躺在资源清单上，被当成「还在推进的资产」。这才是慢性失血最难缠的地方——它连失败都算不上，所以永远不会被清算。

★ 这也顺手修正了第五章 5.4.4 的止损机制：

```
止损 ≠ 失败
止损 = 承认这条路现在不成立，并把代价控制住

★ 所以止损做得好不好，看的不是"停没停"，
   看的是三件事：
      ① 停的时候花了多少、留下了什么
      ② 从"没人用"到"承认它没人用"，中间隔了多久
      ③ 系统关了吗？——还在线的僵尸项目会一直占资源
            ↓
   停了但花了半年和一版版功能 → 慢性失血
   停了而且代价可控、结论清楚 → 低成本试错
   ★ 事实上结束了但账面没结束 → 最差的一种
```

★ 三阶段时间轴：这本书真正的主线

作者走过三年，三段的性质各不相同。**其中第二年（AI 创新专项正式展开那年）是本书的证据年——下面绝大多数数字都来自它。**

```
┌─ 第一年 · 技术探索期 ── 试模型、试工具、试数据 ──────────┐
│                                                          │
│  没有规模化的培训、需求清单和队伍                        │
│  就是在试：哪个模型能用、工具能不能接、数据出不出厂      │
│  ★ 关键词：试错。没有这一年的踩坑，后一年连坑都踩不上    │
└──────────────────────────────────────────────────────────┘
                          ↓
┌─ 第二年 · AI 创新专项正式展开 ── 广度年，学会"试" ──────┐
│                                                          │
│  上百项需求收集 → 数十项与 AI 直接相关 → 42 项共性合并为 7 个 │
│  5 个共性解决方案 + 9 个领域解决方案                     │
│  培训累计 数百场次（AI 248 / RPA 202 / 数据分析 234）    │
│  内部AI应用创新赛 三十余队、一百五十余人 → 十余队胜出 → 落地数个场景 │
│  成熟度从近乎零起步，一年内进入第二档（十个月）          │
│  数字员工平台登录 四千余次、调用 两千余次                │
│  ★ 关键词：试。广度铺开，把认知拉起来，把坑踩一遍        │
│  ★ 上面这些，全都发生在同一个年度里                      │
└──────────────────────────────────────────────────────────┘
                          ↓
┌─ 第三年 · 持续探索期 ── 深度年，开始"扎"，模式转变 ──────┐
│                                                          │
│  合同智审：四大合同类型全部上线 → ★ 全公司覆盖           │
│  印章识别：合同审阅部门（行政、营销等）落地              │
│  智能体平台：技术线 16 个 AI 智能体助手                  │
│  研发知识助手：LLM-WIKI 架构（原始资料层/Wiki 层/规则层）│
│  智能问数：完成部署、测试、推广 → 半失败（可控退出）     │
│  自建算力：开源推理架构 + 容器化部署，模型版本可控       │
│  下一程：合规文控、部门数字员工矩阵、共享智能体          │
│  ★ 关键词：扎。选少数几个，把它们做透                    │
└──────────────────────────────────────────────────────────┘
```

★ 三阶段对照出来的三个判断：

判断一：第一年不「扎」是正常的，因为扎的前提是知道该扎什么。

```
2025 年的产出不是 3 个场景，是"我们知道哪些路走不通"
          ↓
★ 而"知道走不通"恰恰是"扎得准"的必要条件
   ── 这就解释了为什么智能问数这类"半失败"是有价值的：
   它替后面真正的成功（合同智审全公司覆盖）排掉了一个选项
```

判断二：★ 第一年的那些数字（数百人次、一百五十余人、四千余次登录）不能拿来汇报第二年。

2025 年可以用"广度"指标汇报，因为它本来就是在做广度
2026 年如果还用"活动场次、参赛人数、登录次数"汇报
↓
★ 那就说明组织还在用广度的尺子，量深度的活
—— 而这一章 8.2.5 那些数字（千余小时/年与可核算的年度效益）才是 2026 年该被汇报的东西

判断三：★ 智能问数这类「半失败」，正是「创新专项模式」该有的样子。

为什么？因为它符合创业试点的三条判据：

① 假设清晰：查数这件事，一线真的需要吗？
② 成本可控：投入小，且没有沉没在定制开发里
③ 退出干净：没继续用，也就没人被绑架

↓
★ 而如果这三条都不满足，那不叫试点，叫沉没成本
—— 这正是 8.5 节营销助手的情形

怎么识别自己在坑里

自查一：我这个项目，停了之后会留下什么？

✅ 留下：一份口径清单 / 一个被验证走不通的方案 / 一批会用的人
❌ 不留：只有一段谁也看不懂的代码，和一个不再有人提的群
↓
★ 不留东西的半失败，是真的失败

自查二：★ 我在用「成没成」评价项目吗？

如果答案是"是"，那就会出现两个后果：
① 团队只敢报成功 → 不敢报半失败 → 真实信息被过滤
② 半失败被包装成"还在推进" → 慢性失血被延续
↓
★ 建议改成问三个问题：
它证明了什么？
它否定了什么？
它花了多少？

自查三：我的「半失败」有明确的结束标志吗？

```
✅ 有：某个时点停投、某个结论入库、某个资产归档
❌ 没有：状态一直叫"推进中"，投入一直可以追加
          ↓
★ 没有结束标志的半失败，就是慢性失血的早期
```

自查四：★ 我在用第一年的尺子量第二年的活吗？

```
2025 年：活动场次 / 报名人数 / 参赛队伍 / 登录次数
2026 年：在跑场景数 × 单场景年度价值 / 数据类请求满足率 / 主动使用率
          ↓
★ 尺子不换，汇报就会一直报喜不报忧
```

正确做法

做法一：把「半失败」写进制度，作为一类正式结论。

```
┌─ AI 项目结论三分类 ──────────────────────────────┐
│  ① 成功    ：在跑、有量化收益、可复制              │
│  ② 半失败 ：结论清楚、成本可控、资产归档、停止投入 │
│  ③ 慢性失血：还在加功能、没有结论、投入持续        │
│      → ③ 必须被强制处理，不能长期占用 ② 的位置     │
│      ★ 本书两个案例的现实对照：                    │
│        营销 AI 助手 = ③（上线 3 个月后基本无人用，  │
│          ★ 半年周期宣告失败，此后系统仍挂着、      │
│        智能问数      = ②（无人使用 5 个月，投入已停）│
└──────────────────────────────────────────────────┘
```

做法二：给每个试点设「单场景投入上限」。

```
上线前先定：最多投入多少人天 / 多少预算 / 多少周
          ↓
★ 超过上限且没有新的证据 → 自动触发退出评估
   —— 这一条把"及时止损"从个人勇气变成制度
```

做法三：★ 把「半失败」的结论也纳入知识库。

成功的经验要沉淀，失败的结论更要沉淀
↓
★ 因为失败的结论价值在于：它能防止下一个人再走一遍
—— 而这类资产在多数企业的知识库里是缺的
（二期成果汇报里自己就指出："知识规则体系薄弱，AI 智能体应用缺少可遵循规范"）

做法四：★ 汇报口径分两套，不要混用。

广度期汇报：活动、覆盖、认知提升（合法）
深度期汇报：在跑场景数、量化收益、复制成本（这才是真话）
↓
★ 混用的结果是：深度期还在报活动场次，
而没人敢报"我们试过 3 个场景，死了 2 个"

金句

> **评价一个 AI 项目该不该继续，不能只看「成没成」，要看「失败的代价是多少」。**

> **同样是没成，一种是「没有结论」，一种是「有结论的结论」。**

> **半失败的价值在于：它用很小的代价，买到了「这条路走不通」这个结论。**

> **智能问数替后面真正的成功，排掉了一个选项——这就是它的价值。**

> **止损做得好不好，看的不是停没停，是停的时候花了多少、留下了什么。**

> **第一年不「扎」是正常的，因为扎的前提是知道该扎什么。**

> 第一年可以用广度的尺子汇报，第二年还在用广度的尺子量深度的活，汇报就会一直报喜不报忧。

> 没有结束标志的「半失败」，就是慢性失血的早期。

> 成功的经验要沉淀，失败的结论更要沉淀——因为它能防止下一个人再走一遍。

> 智能问数这个案例提醒我们：不是所有 AI 项目都该活下来。

8.7 没推成的那些：46 个还躺在计划里

现象

上面 8.5 节给了本书第一个完整的失败项目复盘。与它对照，这一节讲的是另一类：连“推过”都还没到的那些。

这里只能给出两类已经确认的反向素材：

★ 但在讲这一节之前，必须先纠正一个很容易犯的错——★ 这里要说清一件反直觉的事：

```
★★★ ★★★ "★ 花了时间、没达到效果、后来放弃了"这种案例，★ ★★ 在 AI 领域其实不多
★★★ ★★★ 因为 AI 的失败是速生的：★★ 不好用，用户很快就不用了（详见 8.5.3）
        ↓
★★★ ★★★ ★★ 所以这一节真正要解释的不是"为什么有项目半途而废"，
★★★ ★★★ ★★ 而是 "★★ 为什么有很多项目甚至没进到"用几个月"那一步"
```

第一类：还躺在「计划做」里的 46 个。

它们不是失败的案例——它们是**尚未发生**的案例。但从一张盘点表里已经能看出几件事：

- **供给域 19 个（占计划做 41%），零落地。** 这不是巧合。供给场景几乎全部卡在同一件事上：数据不在一处、口径不统一、跨部门责任不清晰。
- **营销 9 个只落地 1 个。** 营销是离营收最近的域，落地率却最低——因为营销场景要跨渠道、跨产品线、跨事业部。
- **计划做 46 个并列无优先级。** 没有排序，就没有取舍，就没有「这个季度做这个」。

第二类：23 个已实现场景的共同形态。

如果「做成」的标准是「改进了流程」，那么这 23 个都属于**部分做成**——助手层做成了，流程层没做成。

8.7.1 ★★ 为什么很多项目「连第一个月都没撑过」——★ 一张判断路径

★ 这一节回答 8.7 开头提出的那个问题。 ★ 而它给的不是案例，是一张判断路径——★ 因为按 8.5.3 的观察，这类死法在 AI 领域最常见，也最值得说清。

```
★★★ ★★★ "★ 花了时间、没达到效果、后来放弃了"这种案例，★ ★★ 在 AI 领域其实不多
★★★ ★★★ 因为 AI 的失败是速生的：★★ 不好用，用户很快就不用了
          ↓
★★★ ★★★ ★★ 所以真正需要解释的，是 "★★ 为什么有很多项目甚至没进到"用几个月"那一步"
```

★ ★★ 而这一节要给的，是一条可以拿去排查的路径：

```
★ 一个 AI 项目在第一个月里死掉，★ 通常是下面四个环节之一断了：

    ① ★★ 它没被知道      → 推广没跟上
    ② ★★ 它不被信任      → 第一次用就答错 / 答得不对味
    ③ ★★ 它不好用        → 要学、要填、要迁就
    ④ ★★ 它不值得用      → 老路更方便（见 8.5）
          ↓
★★★ ★★★ ★★ 而这四个，★ ★★ 排查顺序不该乱：★★ 因为它们是串行的
```

★★ ★★ 逐个说清：

① 它没被知道（推广没跟上）

```
★★ 典型现象：★ 上线了，但没人知道
    · 只有一个部门在用，其他部门不知道有这个东西
    · 知道的人，★ 也没被告知"这件事现在可以少做一步"
          ↓
★★ ★★ 而这类死的特征是：★★ ★★ 你去看使用数据，会发现"用的人都是本来就在这个项目里的人"
          ↓
★★★ ★★★ ★★ ★★ 识别信号：★★ 如果你的用户名单，★ 和项目参与人名单高度重合
★★★ ★★★ ★★ ★★ 那不是用户喜欢用，★ ★★ 那是"除了他们也没别人知道"
```

② 它不被信任（第一次用就答错）

```
★★ 典型现象：★ 有人试了一次，问了个自己真正关心的问题
   · AI 答得不对味 / 答得很泛 / 答了个不相干的
   · ★★ 试过一次的人，★ ★★ 再也没打开过
         ↓
★★ ★★ 而这类死的特征是：★ ★★ 使用量在第一天之后就断崖式掉到零
         ↓
★★★ ★★★ ★★ ★★ 识别信号：★★ 去看 "★ 第一次提问问的是什么 ★★"
★★★ ★★★ ★★ ★★ ★★ 因为第一个问题决定了这个人会不会有第二个
```

③ 它不好用（要学、要填、要迁就）

```
★★ 典型现象：★ 用是有人在用，但用得很别扭
   · 每次都要重新交代背景
   · ★★ 要在三个系统之间来回切
   · ★★ 输出不能直接用，还要手工改一遍
         ↓
★★★ ★★★ ★★ ★★ 而这一类最容易被误判成"★ 用户不配合 ★★"
★★★ ★★★ ★★ ★★ ★★ 识别信号：★★ ★★ 去看有没有人 ★★ ★★ 在把它当"资料来源"而不是"成品"
★★★ ★★★ ★★ ★★ ★★ ——★ 当大家开始从 AI 输出里"抄一段"，★ ★★ 那就是不好用了
```

④ 它不值得用（老路更方便）

```
★★ 典型现象：★ 用户试了，说"还行"，然后继续用老路
   · ★★ 这个最难发现——★ ★★ 因为使用数据可能还很好看
         ↓
★★ ★★ 而这一类的特征是：★ ★★ 使用集中在"顺便查一下"，★ ★★ 不在"本来就得做"的事上
         ↓
★★★ ★★★ ★★ ★★ ★★ 识别信号：★★ ★★ 去看 ★★ ★★ 它的使用有没有挤掉一件具体的老活 ★★ ★★
★★★ ★★★ ★★ ★★ ★★ 如果它一个月查了 200 次，★ ★★ 但那 200 次里没有一次替代掉一封邮件，
★★★ ★★★ ★★ ★★ ★★ 那它就是 ★★ ★★ 第四种死法：★ ★★ 白做了
```

★★★ ★★★ 而这四条排查路径，最有价值的一点在于：

★★★ ★★★ ★★ 它们是按时间顺序的，★ ★★ 排查也应该按时间顺序：

```
第 1 天  →  有没有人知道?        → ①
第 1 周  →  第一次用答对了吗?    → ②
第 2–4 周 → 用得别扭吗?          → ③
整个月   →  替代掉老活了吗?      → ④
```

★★★ ★★★★ ★★★ ★★ 而顺序不能反的原因很简单：
★★★ ★★★★ ★★ 如果第 1 个月根本没人知道（①断了），
★★★ ★★★★ ★★ 那你后面所有的"用户说不好用"，★ ★★ 都是没有意义的反馈

> **★ 金句**
>
> **「如果一个 AI 项目第一个月就死了，先查推广，再查产品——顺序反了，你收集到的所有反馈都是噪音。」**
>
> **「当大家开始从 AI 的输出里「抄一段」用，那就是不好用了。」**

为什么会这样

未落地场景的死因，行业里反复出现的是三种：

1. **数据不达标**——不是「没有数据」，是「数据不对齐、不完整、口径不一致」。这类死因最隐蔽，因为立项时看起来数据是有的。
2. **业务方不接**——场景由技术侧或需求部门提出，但收益归属不清（省下的人力算谁的？），业务方没有动机接。
3. **高层变主意**——预算周期切换、领导更替、优先级被别的事挤掉，项目停在半路。

> **说明**：以上三类是**行业通行做法的归纳，不是本企业案例**。本章不把它们伪装成当事人经历。

怎么识别自己在坑里

对每一个还挂在「计划做」上的场景，问三句：

1. **这个场景要用的数据，现在在几个系统里？口径一致吗？** 超过两个系统且口径未对齐的，先别排期。
2. **这个场景省下来的时间/钱，归谁？** 归不明确的，业务方不会接。
3. **如果它三个月后没达标，谁来负责调整？** 答不上来的，闸门没过。

正确做法

对没推成的部分，**用可管理的失败代替沉默的搁置**。

- 建一张「**未落地原因登记**」：每个计划做场景标上它的阻塞类型、阻塞点、解除条件、复查日期。
- **不砍掉，先降级**：把 46 个分成「本季度做」「本年度做」「暂缓并写明解除条件」三档。暂缓不是搁置，是有条件等待。
- **把「计划做」表变成滚动台账**，每季度更新一次状态与优先级，而不是年底重排一次。

> **★ 本章待补（★ 2026-11-03 已重新评估）**：【待核实】原计划补充 1–2 个「推过但没推成」的案例。 **★ 但根据作者 2026-11-03 补充的经验，这类案例在 AI 领域并不常见**——因为 AI 的失败是速生的 （详见 8.5.3）。**因此本节的补法调整为**： ★ 优先补的是「★ 为什么很多项目**连第一个月都没撑过**「的具体现场（哪里没做好、用户第一次用就放弃了什么） ——**这一类反而更常见，也更有价值** —— 而不是「用了一阵子才放弃」的中间态案例。 在补到之前，本节**不编造任何失败细节**。

> **金句**
>
> **「没做成的项目不写下来，就会被当成从没想过。」**

8.8 时间尺度：5~10 年不是保守，是事实

现象

最后一个必须说清楚的事：读者合上这一章，最可能带着一个不耐烦的结论——「搞了半天，什么都没变。」

这个结论有一半是对的。但另一半被一个时间刻度掩盖了。

一位项目负责人对这件事的判断是：

> 「很多 AI 落地误区在于希望直接改变营收。但这就像蒸汽机、电器或互联网时代一样，对传统制造企业而言，**工作流程和模式需彻底改变，整个过程需要 5~10 年才可能实现，不会是立竿见影的**。」

而这家的做法是：**从 2025 年就开始做全员培训**，AI 训练营、AI 大赛，基本从底层员工做起，只在特定专题会上给高层做 AI 培训教育。

为什么会这样

AI 不是一个可以插进现有流程的零件，它是**对流程的重写**。流程改了，职责边界改了，审批链改了，数据产生的方式改了。

这件事的性质，和当年上 ERP、上 MES、上自动化产线是同一类：**它是基础设施，不是功能**。

基础设施的特点是：前期投入看不见回报，中期只体现为「新系统的数据终于能对上了」，后期才体现为「原来做不到的决策现在能做了」。任何期待它在 6 个月内改变营收的，都是在用功能的节奏要求基础设施。

怎么识别自己在坑里

1. 你的 AI 立项书里，**验收标准写的是「上线」还是「业务指标变化」**？只写上线，说明你在做功能。
2. 你上一次给老板汇报 AI 成效，用的是「**我们上线了 N 个**」还是「**某个经营指标变成了 X**「？
3. 你心里那个「 AI 什么时候能出效果」的答案，是一个季度，还是一个五年?

正确做法

对内，用两种账。

- **短期账（ 1 年内可交付）**：提效类——省下的工时、减少的差错、加快的响应。用它维持组织信心和预算。
- **长期账（ 5~10 年）**：决策类与收入类——因为工作模式和流程的改变才能产生。**用它去和高层对赌**。

对高层，别只培训，要配助理。

高层的 AI 认知不是靠听课建立的。对高层，不要先教 AI，**要先给他配一个助理**——按他的四类定制需求（数据、规则、流程、关注）配，把技能固化下来，让它先有使用价值。高层一旦用起来，认知就建立了。

推进路径：熟人试点 → 成功后辐射。

找自己最熟悉或直管领导做试点，试点成功了再辐射到其他高管。这条路径的成功率远高于「全员宣贯 + 全面推广」。

AI 团队自身的定位要变。

如果 Vibe Coding 这类技术让普通员工能自己做出贴合业务的小应用，传统技术部门和 AI 推进部门的压力会很大——那我们的意义是什么？答案不是去比谁能写代码，而是三件事：

1. **发现与孵化场景**：找到每个员工针对本部门做的小应用，把它扶起来；
2. **掌握算力与数据资源，提供安全审计**：数据是企业的核心资产，不能外流；对自建程序做代码检查、部署到统一服务器统一监管；
3. **推广、复制、产品化**：一个部门提了效、解决了合规问题，那只是一个点——**要把这个点放大十倍、一百倍**。

> **金句**
>
> 「把 AI 当功能做，一年就见分晓；把 AI 当基础设施做，五年才见分晓。」

8.9 本章小结：十句话带走

1. **不是做得少，是做得偏**。 23 个已实现、 46 个计划做、零重叠——已实现的 83% 落在办公域，供给/生产/研发一个没落地。做成了 23 个助手，但一个流程都没改。（一手盘点表）
2. **改不了流程，是因为看不见流程**。 技术和数据能穿透系统，穿透不了文化。组织透明度是 L2 的天花板。（组织透明度观点 + 铁证二互证）
3. **效率提升如果没长出新的业务，就只会长出新的空闲**。 全员培训是起点不是终点，红利要有人负责兑现。（效率悖论）
4. **功能做出来了，不等于场景做成了**。 营销 AI 助手不是败在模型，是败在数据口径与入口——而这两件事的解法，都不在项目组手里。（★ 作者亲历案例）
5. **推的产出是声量，扎的产出是数字**。 数百人次培训落地数个场景； 2 个扎出来的场景拿出 千余小时/年与 可核算的年度效益。（★ 一手数据）
6. **评价一个 AI 项目该不该继续，不能只看“成没成”，要看“失败的代价是多少”**。 智能问数是半失败，但投入少、损失低、结论清楚——这正是创业试点该有的样子。（★ 一手数据 + 作者补充）
7. **★ 三年是三段性质，不是一段长跑**。 第一年技术探索（试模型、试数据），第二年才是创新专项正式展开的一年（数百场次培训 → 一百五十余项需求 → 三十余支队伍 → 落地数个场景），第三年进入持续探索（合同智审全公司覆盖、印章识别进合同审阅部门，落地应用模式整体转变）。**★ 同一套「投入 vs 产出」的尺子，套在三年上必然算错账**——第二年投入最重、产出最少，第三年投入最省、产出最实。
8. **★ 前两个成功证明「AI 能做成事」，第三个证明「AI 能做成组织能力」**。 部门级 Agent 平台用一套共享知识库服务整个部门；研发中心用 LLM-WIKI 知识助手拿到 20%~30% 提效——**而这也让 8.0.1 的判断更精确了：判断的单位不是「部门」，是这个部门里具体是哪一件事**。
9. **★ 失败的三个层次：场景层（选错场景）／方向层（这事本身不该做）／试探层（试了即止）**。 **★** 试探性失败最便宜，方向性失败最贵——**而方向性失败最不像失败，因为它看起来像「在探索」**。（8.5 / 8.5.2 / 8.6 对照表）
10. **★ 需求成立，不等于产品成立**。 **★** 营销 AI 助手的立项理由（移动端快速拿数）是对的，老路（邮件 Excel 为主、 BI 用得少）也是对的——**但新方式赢的那一项，抵不过老路赢的那三项：选择权、可传递、零学习**。（★ 8.5 ，作者 2026-11-03 补充）

1. ★ **当下的大模型应用很少真正闭环，大多嵌在原有业务闭环的某个环节里。** ★ 这不是技术不足，是组织约束——**全生命周期由 AI 完成需要改组织、改流程、改管理体系，而不可能仅因某个岗位的工作就改变整个公司的管理体系。**（★ 8.0.2，作者 2026-11-03 补充）
2. ★ **AI 的失败是速生的，而传统系统的失败是慢性的。** ★ 所以「用几个月没效果就放弃」在 AI 领域并不常见——**要么没撑过第一个月，要么已经被用出粘性。** 由此推出的止损原则与传统正好相反：**AI 项目越早停，省下的越多。**（★ 8.5.3 ）

> **本章整体金句**
>
> **「AI 落地的真相是：你能做出多少助手，取决于你敢让多少流程被改掉。」**

> **「嵌入和闭环，是两种完全不同的生意。嵌入改造的是一个环节；闭环才谈得上改变一门业务的运行方式。」**

> **「不可能仅因某个岗位的工作，就改变整个公司的管理体系。」**

> **「AI 项目的失败是速生的——不好用，用户就不用了；不会用几个月再放弃。」**

> **「用户开始对 AI 提新需求，才是它活下来的真正证据。」**

> **「对 AI 项目，'再给它一个月'这句话，几乎总是错的。」**

> **纪律声明**：① 企业名、集团名、子公司名、事业部代号、产品名**全部泛化**，统一用「某大型制造企业」；② 人名、具体人设细节**一律不写**，只用「数字员工 A/B」；③ 无一手出处的效果数字标 `【待核实】`；④ 制度条款均为本企业实测口径，入书前需作者逐条校准。

9.0 一个必须先说清的问题：为什么这些东西叫「数字员工」

在讲怎么建之前，得先回答一个看起来像常识、其实很少有人认真回答的问题：

> **为什么企业里那些 AI 小应用、 Agent 、机器人——包括某位一线员工自己搭出来的那一类——都被叫做「数字员工」？它们和以前建的 ERP 、 CRM 、 OA 这些数字化系统，到底有什么区别？**

这个问题的答案，**是本书全部方法论的起点**。

★ 而这个问题不是抽象概念——它有一个非常具体的来源（★ 2026-11-02 作者补充）：

```
★ 企业组织过 ★ 内部AI应用创新赛
★ 参赛者里有大量 ★★ 一线员工自己动手搭出来的数字员工
        ↓
★★ 而这些人搭出来的东西，形态和 IT 部门做的完全不一样：
   · 数量多、散落在各个岗位
   · 没有走需求评审、没有排期、没有立项
   · ★★ 但确实解决了自己岗位上的一件真事
        ↓
★★ 已知的例子（★ 均为通用化表述）：
   · ★ 生产质量过程文件检查小助手
   · ★ 财务成本管理小助手
        ↓
★★★ 于是就出现了一个非常典型的画面：
   ★★★ IT 部门在建"数字员工"，★ 一线员工也在建"数字员工"
   ★★★ 而且后者往往更快落地——因为他只解决自己那一件事
```

★ ★★ 所以这一章要回答的，其实不是「什么该叫数字员工」这种概念问题，★ 而是一个真问题：★ 一线自己搭的这些，和 IT 建的那些，★ 到底是什么关系？

```
★★★ 而本章的答案是：
   它们 ★ 不是两类东西，★ 是同一类东西的 ★ 两个来源
        ↓
   ★★ 一个是"★ 岗位先存在，工具后做出来"（一线自建）
   ★★ 一个是"★ 工具先做出来，再去找岗位"（IT 立项）
        ↓
   ★★★ 9.0.6 会讲：★ 只有前者是对的顺序
```

因为如果搞混了这两件事，后面所有的做法都会走偏：

- 搞混了 → 你会**给每个数字员工建一套系统**，然后被 IT 成本和运维拖死（这是 8.2 那张表里「46 个计划做」排不下去的深层原因之一）；
- 搞混了 → 你会用**评估系统的方式**去评估数字员工（看它有多少功能、覆盖多少模块），而应该用**评估岗位的方式**去评估它。

9.0.1 先看传统数字化系统长什么样

ERP、CRM、OA、SRM……这些系统的形态，经历过这轮数字化的人都很熟悉，它们有非常一致的骨架：



三个特征极其鲜明：

1. **它是一套完整闭环。** 一件事从输入到审批到归档，全在这套系统里做完，**它自己带走了整个业务环节。**
2. **它是独立王国。** 有自己的用户体系、自己的菜单树、自己的权限模型、自己的登录页。**员工要进入它，必须先登录它。**
3. **它是单一场景的产物。** 一个系统解决一个或几个明确的管理闭环——采购的闭环在 SRM 里，销售的闭环在 CRM 里，**每个闭环各自成系统。**

> **一句话概括传统数字化系统：它是按「业务闭环」切出来的垂直烟囱。**

9.0.2 再看数字员工长什么样

现在换一个视角。

一个数字员工——比如负责合同审核的那个，负责智能问数的那个，负责物料检测异常分析的那个——它在企业里实际的样子是这样的：



三个特征同样极其鲜明，而且正好是上一段的镜像：

1. **它不带走业务环节，只干活。** 合同还在 OA 里走，付款还在 SAP 里跑。**数字员工插在环节之间，做的是「某个岗位该做的那部分工作」**，而不是自己成为一个新的业务闭环。
2. **它没有界面。** 员工不需要登录它、不需要从菜单里点进它。它**出现在原本就有的界面里**——OA 的审批页、 SAP 的订单页、 CRM 的客户页。它甚至没有入口，因为**它的入口就是别人的入口**。
3. **它按岗位切，不按业务闭环切。** 一个数字员工对应**一个岗位的一段工作**，而不是一个模块。

> **一句话概括数字员工：它是按「岗位」切出来的横向能力，穿插在多个系统的业务节点之间。**

9.0.3 五个关键差别

把两边的差别摊开，一共五条，每一条都很致命：

#	维度	传统数字化系统（ERP/CRM/OA）	数字员工
1	**建设逻辑**	按**业务闭环**垂直切分，一个闭环一套系统	按**岗位工作**横向切分，一个岗位一个数字员工
2	**与业务的关系**	**自己带走一个完整环节**（输入→审批→归档全在里面）	**穿插在别人的环节之间**，自己不形成闭环
3	**用户入口**	有独立登录页、统一菜单、独立界面	**没有入口**——它的入口是 OA/SAP/CRM 里原有的那个按钮
4	**系统边界**	垂直烟囱，系统之间靠接口对接、边界清晰	**跨系统、跨环境、跨交互**，天然横跨多个系统
5	**评估方式**	看功能覆盖、模块数、流程闭环完整度	看**它替的那个岗位省了多少、担了多少责**

第 4 条是数字员工最独特的地方，也是最容易被低估的地方。

传统系统之间的边界是**清晰的**——ERP 到 CRM 要走接口，接口挂了就是挂了，出问题好定位。

数字员工的边界是**模糊的**——它同时在 OA 、 CRM 、 FSSC 、 SAP 里取数、比对、写入。**它的价值恰恰来自这种模糊；它的风险也恰恰来自这种模糊。**

> **传统系统出问题，锅在系统。数字员工出问题，锅在岗位。**

这就是为什么第九章后面要花大力气讲「它替代的到底是哪个岗位」。

9.0.4 为什么这个区别决定了做法完全不同

上面五条差别，往下推，会推出**完全相反的两套做法**。这是本章存在的理由：

如果你把它当系统建	如果你把它当员工建
先做需求调研 → 出功能清单 → 设计界面 → 写代码 → 联调 → 上线	先定**岗位** → 说清它替的是谁、担什么责 → 划数据边界 → 定它在哪几个节点上干活 → 再谈技术
评估用「功能覆盖率、模块完成度」	评估用「**这个岗位原来要几个人几天，现在多少**「
推广靠「这个系统上线了，请大家登录试用」	推广靠「**你原来那件活，现在交给它了**「
需求方是 IT 部门	**需求方是那个岗位的负责人**
失败信号：功能没做完	失败信号：**功能都做完了，但没人用它**

左边那一列，几乎所有企业都在走。

所以才有了那个刺眼的现状：系统一个接一个上线，业务部门用得都不深，每个系统都有点用但没有一个真正离不开——**因为它们是被「建」出来的，不是从岗位上「长」出来的。**

9.0.5 怎么识别自己在坑里

三道题，答错就说明你还在按系统思维看数字员工：

1. **如果你要给数字员工做一整套系统架构图，那大概率方向就错了。** 数字员工的价值恰恰在于它**不需要**一套独立系统。
2. **如果你的数字员工有自己的登录页和主菜单，那它已经变成系统了。** 它的入口应该是 OA 审批页上的那个按钮。
3. **最关键的一题：你说得清「这个数字员工替代的是哪个岗位的哪一段工作」吗？** 说不清 —— 那你做的不是数字员工，是一个没有主人的机器人。

第三题是全章的判卷标准。**说不清岗位的数字员工，没有资格进入管理体系。**

9.0.6 正确做法：先写岗位说明书

在讨论任何技术方案之前，每个数字员工必须先有一张「岗位说明书」。 这是本章所有制度的地基：

项	内容	示例（脱敏）
岗位名称	它替的是哪个岗位	合同审核初审岗
原岗位工作	这个岗位原来具体做什么	法务逐字审合同、核对条款、比对 OA 表单金额
数字员工做什么	承接其中哪几段	一级初审（合法性/金额合理性/敏感词识别）+ 表单字段自动比对
数字员工不做什么	明确不承接的	不做最终审批、不签字、不出具法律意见
在哪些节点	穿插在哪些系统的哪些环节	OA 合同审批流的「一级审核」节点
人机分工	什么由人担责、什么由它担责	它出意见+留痕，人做二级审核并签字
岗位价值	这个岗位的成本、工时、差错率	效率提升幅度、年省金额、漏检率变化【待核实：缺基数】

这张表有三个好处：

1. **它把「AI 项目」翻译成了「岗位改造」**——后者是业务负责人的语言，前者不是；
2. **它天然带出边界**——「不做什么」和「在哪些节点」两栏，就是后面 9.4 的权限设计输入；
3. **它天然带出验收标准**——「岗位价值」那栏，就是这个数字员工的成功判据。

> **金句**
>
> **「数字员工不是一套新系统，是一个新岗位；先写岗位说明书，再谈技术方案。」**

9.0.7 一个具体例子：从「上系统」到「配岗位」

用那个效果最完整的案例（AI 合同智审）走一遍：

如果按系统思维做：

> 调研合同审核流程 → 提炼功能清单（合同解析、风险识别、字段比对、印章核验、报告生成）→ 设计界面 → 开发 → 与 OA 做接口对接 → 上线 → 培训 → 推广

如果按岗位思维做：

> ① 这个岗位原来是什么？——法务/审核岗，逐字审合同，一级人工识别风险 + 二级人工与 OA 表单逐条比对。 ② 它承接哪几段？——一级初审 + 表单字段比对。**二级审核和签字不接。** ③ 插在哪个节点？——OA 合同审批流的「一级审核」节点，不另开入口。 ④ 边界？——不出具法律意见、不做审批决策；所有意见留痕可追回原文依据。 ⑤ 岗位价值？——效率提升 50%~96%、年度效益可核算（金额按作者要求不出书）、漏检风险降 30% 以上【待核实：三个数均缺基数】 ⑥ 人机分工？——它出意见，人签字。**签字的人对结果负责。**

同一件事，两种路径的差别在于：

- 第一种，需求方是 IT，验收标准是「功能是否可用」，失败信号是「接口没调通」；
- 第二种，需求方是**审核岗负责人**，验收标准是「这个岗位的工时和差错率变了没有」，失败信号是「做完了但没人敢用」。

而真正让这个案例落地的，恰恰是第二种路径里的一句设计：

> 一级审核由 LLM 以法律专家身份做条款合法性、金额合理性、敏感词风险识别，**生成审核意见嵌入 OA 表单**。

「嵌入 OA 表单」——这五个字就是「数字员工不是系统」的全部含义。

9.1 为什么数字员工要单独立一章

前面八章反复出现同一个词：**数字员工**。

但到目前为止，它在书里一直是个「技术名词」——一个 RPA 机器人、一个 Agent 、一个智能体。

而当你真的把它**当人一样建起来**——给它名字、给它形象、给它边界、给它职责、给它制度、给它宣传——你会发现一件反直觉的事：

> **这一步不是包装，是落地的前置条件**。

理由很硬：

- 一个**没有名字**的工具，员工不会记住它，也不会有归属感，用完就忘；
- 一个**没有形象**的工具，跨部门传播时无法被指认（「让那个审合同的看看」比「让数字员工 A-合同看看」传播成本低十倍）；
- 一个**没有权限边界**的工具，不敢接核心业务，因为它随时可能被当成数据泄露源；
- 一个**没有制度**的工具，无法被推广，因为没人敢为它签字；
- 一个**没有宣传**的工具，永远只有技术团队知道它存在。

这五件事（ IP 、 VI 、匿名/边界、范围与场景、管理体系与制度、宣传）合起来，就是「把 AI 变得更人性化」的全部内容。

而人性化的目的只有一个：**让更多员工和领导理解，未来的工作方式是「人 + 数字员工」的协同，而不是「人对抗 AI」**。

这一章讲的，就是怎么把这件事落地。

9.2 IP 设置：先回答「它是谁」

现象

很多企业做数字员工，第一件事是取名。然后就卡住了。

因为「取名字」这个动作背后其实藏着四个必须回答的问题，而 90% 的企业只回答了第一个。

为什么会这样

我们来看一份**真实的、已经跑起来的**数字员工管理规范。这套规范出自某大型制造企业的《数字员工管理指南 V1.0》，它对命名的规定是这样的：

> **命名规范**：采用「企业品牌简称 + 功能定位关键词」组合命名，**禁止部门自行添加后缀**（如 「- 财务版」「小某某」）。
>
> **示例**：数字员工 A （企业简称 + 「智能知识管理」定位）；数字员工 B （某文化业务品牌 + 「智能创作」定位）。
>
> **版本管理**：迭代版本需在命名后用数字标注（如 「数字员工 A 2.0」），重大功能升级需经数字化中心审批。

这四条规则，每一条都在回答一个 IP 层面的问题：

规则	回答的问题	不这么做的后果
品牌简称 + 功能定位	**它是谁**	名字不统一，集团层面认不出这是「我们的」数字员工
禁止部门自行加后缀	**它归谁**	各部门各叫各的，最后产生「机器人重名/撞名」，管理层完全失控
品牌 vs 功能二段式	**它对外代表什么、对内干什么**	面向外部客户的名字必须过品牌审核，面向内部的必须能说明职责
版本号 + 升级审批	**它现在长什么样**	改了功能不换号，业务方手里的文档、口头称呼、界面入口全部对不上

这条规范最狠的一句是「禁止部门自行添加后缀」。

它背后的判断是：**数字员工的 IP 是集团资产，不是部门私产。** 一旦允许部门自己加后缀，三个月后你会有「数字员工 A-财务版」「数字员工 A-财务部版」「数字员工 A 财务」三个东西， IT 管不动，管理层也说不清。

而且这个规范有个非常漂亮的细节：**命名格式本身就编码了权限边界**。看一眼名字就知道它是什么——「数字员工 A 」是内部保密型，「数字员工 B 」是外部创作型。名字本身就是身份的第一次自证。

怎么识别自己在坑里

1. 你能列出**公司所有数字员工的完整清单**吗？有多少个是部门自己起的名？
2. 有没有出现**同一个名字被多个部门用**的情况？
3. 业务方在口口相传时，**用的是编号还是名字**？（用编号 = 传播失败）
4. 版本升级之后，**旧的文档和截图还对得上吗**？

正确做法

一张 IP 定义表，每个数字员工必填五项：

项	内容	举例（脱敏）
名称	品牌简称 + 功能定位	数字员工 A-财务报销
定位	一句话说清它是什么	财务共享报销单据初审助手
职责	3~5 条核心职责	单据完整性校验、重复报销识别、异常打回
禁区	明确不能做什么	不做最终审批、不出纳付款、不接触薪资数据
版本	迭代号 + 审批记录	1.2 ，重大升级经数字化中心审批

同时立三条硬规矩：

1. **命名权集中**：只有数字化中心能批名字，部门只能提申请
2. **台账化管理**：建立《数字员工命名台账》，记录所有名称、职责、使用部门，避免重复——**这一条常被忽略，但它是防止「数字员工重名」和「功能重复建设」的唯一手段**
3. **版本可追溯**：重大功能升级必须走审批，且版本号可见

> **金句**
>
> **「名字是数字员工的身份证，不是它的营销名。」**

9.3 VI 设置：形象不是包装，是识别成本

现象

有一家企业做过一次统计：**把公司里所有 RPA 数字员工的性别、年龄、姓名偏好全部统计出来做成报表**。

这件事看起来很小，很「虚」，但它是企业数字化史上少见的、真正从「人」的角度出发的设计动作。

它要回答的是：**我们的数字员工，长什么样？**

为什么会这样

一个数字员工的形象（ IP 视觉形象）解决四个非常具体的问题：

问题	没有 VI 时	有 VI 时
识别	截图里一堆框框，跨部门分不清谁是谁	一个头像就是身份
信任	员工不知道这东西是谁，不敢用	有名字有形象有职责，愿意打开
传播	每次宣传都要截图、做图、凑素材	海报、头像、 KT 物料现成
归属	工具，用完即弃	「我们部门的同事」，有归属感

更实用的一点是**VI 是物料的复利**。

在办内部 AI 应用创新赛这件事上，宣传物料被拆成了一张清晰的清单：

- 前期宣传： RPA 自动化概念宣传、大赛背景与流程利好、宣传语与渠道准备
- **大赛资料包：现成文案模板、海报模板、 RPA 机器人元素图**
- 厂商资源包：厂商 logo 与 IP 形象（外发版）
- 启动会：产品与场景宣讲、企业内已落地应用场景分享、**高层领导讲话并给比赛定性**

- 赛后：比赛成果与优秀队伍宣传、按项目规模安排 IT 人员部署、品牌宣传支持（公众号等）、宣传海报和文案

注意那句「**RPA 机器人元素图**「——这正是 VI 的核心作用：**它让所有物料出自同一套视觉体系，而不是每次活动临时凑图**。

还有一句更值得注意：「**高层领导讲话并给比赛定性**」。这说明在设计体系里，宣传不只是海报和公众号——**最高层的一次公开定调，其效力远超一百张海报**。

怎么识别自己在坑里

1. 你能说出你们数字员工的**形象设定规则**吗（性别、年龄、配色、字体）？还是每次外包设计师随便画?
2. 你的数字员工在**企业微信/OA/系统里有没有头像**？还是灰色的通用图标?
3. 你做一次内部宣传，**需要多久凑齐物料**？超过两天 = 没有 VI 资产库
4. 上一次**高层公开为 AI 站台**是什么时候?

正确做法

第一步：做一次形象统计（这是最容易起步的动作）。

把现有数字员工的性别、年龄、姓名偏好统计出来，做成一张表。这张表的价值不在数据本身，在于它**把「我们的数字员工长什么样」从模糊感觉变成了可讨论的选项**。

统计完再定三件事：

- **形象基调**：统一还是分族群？（建议统一基调 + 按职能区分色系/配饰，便于区分「内部型」和「外部型」）
- **命名偏好**：从既有名字里反推员工对数字员工的想象，避免新起一套名字风格
- **族群规划**：哪些族群是「内部保密型」、哪些是「对外服务型」、哪些是「陪伴型」——**不同族群必须视觉可区分**，因为它们的权限完全不同

第二步：建 VI 资产库。

固定一套：头像（多尺寸）、半身像、表情包、元素图（机器人元素/图标/线条）、海报模板、 PPT 模板、 KT 物料模板、名片/工牌样式。

做到这一步的标志是：办一次活动，物料不外包也能出齐。

第三步：把 VI 和权限绑在一起。

视觉区分必须对应权限区分。内部保密型和外部服务型数字员工，**头像就该不一样**——这样员工在系统列表里一眼就能看出「这个能问、那个不能问」，减少误用。

> **金句**
>
> **「形象不是给人看的，是给权限看的——员工看到头像就该知道能不能碰。」**

9.4 匿名与边界：数字员工的权限底座

现象

同样出自那份《数字员工管理指南 V1.0》，还有一组规定，这一组比命名规范更硬：

数字员工 A （内部知识管理型）

- 定位：企业级保密数据管家，专注内部知识闭环管理
- 核心职责：管理研发数据、生产工艺等保密信息，提供加密查询与分析；负责内部培训资料、合规文档的智能归类与问答
- **禁止访问互联网，数据交互仅限企业内网系统**

数字员工 B （外部创作服务型）

- 定位：开放式内容创作引擎，连接外部资源与业务场景
- 核心职责：基于互联网数据生成市场宣传文案、科普内容等非涉密材料；支持产品手册图文创作、短视频脚本生成等营销场景
- **可访问公开网络资源，但需过滤敏感信息**（如竞争对手数据）

对应的权限矩阵：

维度	数字员工 A （内部型）	数字员工 B （外部型）
数据访问	企业信息系统、数据库、质控文档	公开行业报告、新闻资讯、创意平台
交互范围	内部员工、合规审计系统	外部客户、营销渠道、公开平台
敏感操作	**需双人授权**（数字化中心 + 业务部门）	**内容发布前需经品牌部审核**

为什么会这样

这一节对应作者说的「**匿名设置**「——即**数字员工对外呈现时以什么身份出现、能看到什么、代表谁说话**。

但这份规范给出的答案比「匿名」两个字丰富得多，它实际上是一张三**层的边界设计**：

第一层：网络边界。 能不能出网。内部型**物理上禁止访问互联网**；外部型可以，但只能碰公开资源。这不是配置项的区别，这是**架构上的隔离**。

第二层：数据边界。 能碰哪些数据。研发数据、生产工艺、质控文档——这些是受监管制造企业最敏感的东西，只能给最受限的那个数字员工。

第三层：操作边界。 关键动作需要谁点头。内部型的敏感操作**双人授权**（数字化中心 + 业务部门）；外部型的内容**发布前经品牌部审核**。

> **这一层是最容易被忽略、也最要命的。**
>
> 一个数字员工如果能自己访问互联网 + 接触内部数据 + 直接对外发布内容，那它就是一个**自主的数据泄露通道**，而且这个通道看起来还「很智能」。

匿名设置的本质是回答一个问题：它做的动作，算谁做的?

外部型数字员工生成的市场文案，最终是对外发布的。谁签的字？规范给的答案是：**品牌部审核**。也就是说——**数字员工产出，经人签字，等于员工产出**。 这条链路一旦定下来，责任归属就清楚了。

怎么识别自己在坑里

1. 你的数字员工，**能不能访问互联网**？这个答案是「是」还是「否」，还是「没想过」？
2. 有没有一份**权限矩阵**（谁能用、能碰什么数据、敏感操作谁批）？
3. 数字员工**对外发布的内容**，有没有一道人工审核？如果直接发出去，等于你的品牌部在替 AI 背书。
4. 出现数据泄露时，**你能说清是谁的责任**吗?

正确做法

每个数字员工上线前，必须填完这张边界表：

项	内容
网络访问	内网 / 公网 / 公网+白名单（默认内网）
数据范围	可访问的系统、数据集、字段（默认最小集）
交互对象	内部员工 / 外部客户 / 公开渠道 / 无
敏感操作	列举 + 每项的审批人（默认双人授权）
对外呈现	实名（挂企业品牌）/ 匿名（仅显示功能）/ 半匿名（显示品牌+编号）
内容审核	对外发布是否需人工签字（默认需要）
留痕	谁在何时调了什么、返回了什么

再加三条制度：

1. 《**数字员工命名申请表**》：新增数字员工必须提交，明确功能定位 + 命名建议 + 使用部门
2. 《**数字员工命名台账**》：数字化中心统一维护，记录所有名称、职责、使用部门，**防止重复建设**
3. **违规处理有明文**：擅自命名或修改数字员工身份——**首次警告并整改，再次违规暂停相关账号权限**

第三条最值得学。很多企业的制度只有「应该怎么做」，没有「违反了怎么办」，结果就是**违规成本为零，制度自然失效**。而「首次警告、再次停权限」这个梯度设计很聪明：第一次给面子改正，第二次才真的疼，而且**不封死**——它留了一条「改正后恢复」的路。

> **金句**
>
> **「数字员工的匿名，不是让它没有身份，是让它不能替人背锅。」**

9.5 范围与场景：1.0 试点、2.0 覆盖、3.0 生态

现象

数字员工建好一个、两个之后，绝大多数企业会做同一件事：**开一个更大的会，规划「全场景数字员工」**。

然后就没有然后了。

因为「全场景」从来不是规划，是许愿。

为什么会这样

有一份数字人建设规划，把这件事拆成了三段，比「全场景」这种说法有用得多：

阶段	定位	技术	目标	场景
1.0	**3 个基础场景试点**	NLP（自然语言理解与生成）	通过数字人结合 NLP 模型在成熟场景下实现初步效果	展厅数字人、营销数字人、陪伴数字人
2.0	**结合 IT 系统进行场景覆盖**	多模态 + 数据治理	通过大模型 + 企业内部数据，**形成可胜任岗位职责的数字员工**	营销辅助、财务辅助、差旅助手、陪伴助手、物流管理助手等
3.0	**联合生态扩展与创新**	多模态 + 数据治理 + IoT	联合上下游生态伙伴推广合作方向	慢病管理与监测、医药看护陪伴、用户健康管家

这个三段法里有三个判断值得学：

第一，1.0 阶段的目标是「在成熟场景下实现初步效果」，不是「覆盖业务」。 展厅、营销、陪伴——都是**流程短、数据熟、风险低**的场景。

第二，2.0 阶段的目标才升级为「形成可胜任岗位职责的数字员工」。 注意这个措辞：**从「能对话」到「能胜任岗位」，中间隔着的不是模型，是数据治理和多模态。**

第三，3.0 走出企业边界。 这一步需要生态合作，技术上反而不是最难的，最难的是**你凭什么让别人愿意接你的数字员工**——所以它只能放在最后。

底座部分也同样有层次：**硬底座**（算力调度、算力切分、算子加速、全链路运维、安全防护、云边协同）+ **软底座**（知识库中台：产品手册、企业历史知识沉淀、行业知识库、企业文档；基础大模型；向量检索与重排模型）+ 数据中台与 IT 平台。

> **一个值得注意的排序**：安全防护和全链路运维在硬底座里，与算力调度并列。**不是最后补的。**

怎么识别自己在坑里

1. 你的数字员工规划里，**1.0 阶段有几个场景**？超过 5 个 = 不是试点
2. 你能说清 2.0 阶段**新增的技术依赖**是什么吗？说不清 = 2.0 只是 1.0 的加量
3. 每个阶段的**成功标准**是什么？如果答案是「上线了」而不是「达成了什么」→ 没有阶段划分
4. 底座清单里，**安全和运维**排在算力后面还是前面？

正确做法

按三段法走，每段有明确的「过门条件」：

```
1.0（试点）   → 过门条件：场景跑通、有人每天用、数据可回流
   ↓
2.0（覆盖）   → 过门条件：数据治理过关、知识库可信、能承接岗位职责
   ↓
3.0（生态）   → 过门条件：产品化、可交付给外部伙伴
```

并且，1.0 阶段必须严格限制场景数量。

有一个现成的筛选思路可以借用：大赛的赛道划分本身就是一种筛选——

- **赛道一「实施落地」**：场景复杂度适中，**通过培训后参赛团队能在赛程内完成开发**。要求是「详细阐述改造前后的流程对比、设计方案、预期实现效果及其对集团运营产生的经济和社会效益分析」。
- **赛道二「创意提案」**：场景复杂、需要更强技能、**参赛团队无法在赛程内完成开发**。只提交方案，不要求开发。

这个划分很聪明：**它把「能力不够」从失败重新定义为赛道分类**。做不出来的场景不是失败，是「创意提案」。这对保护创新热情非常有效——**一个组织最怕的是员工提了 idea 然后发现做不出来就不敢再提了**。

场景选择的一个反向标尺

大赛的评审标准里，有一条是很多企业场景评审表都缺的：

> **可复制性（10 分）：可在公司（甚至行业）同类型业务中进行复用。**

很多企业的场景评估只看「这个场景价值多大」，不看「这个场景能不能复制」。结果就是：**每个部门都做了一个高度定制、无法推广的项目，第二个部门要用只能从头再来**。

在场景清单上直接加一列「可复制性」，比事后做推广便宜得多。

> **金句**
>
> **「1.0 阶段做三个场景，比规划三十个场景更有价值——因为三个会逼你真的跑通。」**

9.5.1 ★ 共享智能体：一套知识库服务整个部门

> ★ 本节承接第八章 8.2.6 的第三个成功案例（部门级 Agent 平台底座）。**它回答的问题是：当你要服务的不只是「一个岗位」，而是「整个部门」时，数字员工该长成什么形状**。

先看两种形状的区别：

```
形状一：岗位级数字员工（1.0/2.0 阶段）
    一个岗位一个数字员工，一个知识库
    → 灵活，但知识不沉淀，第二个岗位要重做一遍

形状二：部门级共享智能体
    ★ 一套智能体 + 一套知识库 + 一套技能库
      部门全员共用、统一标准、持续迭代
    → 一次建，全员用
```

★ 它的三层结构（这是「平台型」数字员工与「工具型」数字员工的根本区别）：

```
┌─ 用户接入层 ──────────────────────────────┐
│    部门的所有成员 + 管理员                  │
│    （不是某一个岗位，是整个部门）            │
├──────────────────────────────────────────┤
│ 核心能力引擎层 ──────────────────────────┤
│    负责处理请求、执行任务                   │
├──────────────────────────────────────────┤
│ ★ 共享资源底座层 ────────────────────────┤
│    部门统一的知识库 + 技能库                │
│    —— 这一层是整套东西能不能立住的关键      │
└──────────────────────────────────────────┘
          ↓
★ 带来的变化：
    部门知识和技能的统一管理与共享
    每个人调用的都是同一套智能体能力
    → 协作效率提升 + 工作标准化
```

★ 为什么「底座」这一层最难建，也最值得建：

底座难在哪：

- 知识库要持续维护（★ 没有人对"答案是对的"负责，它三个月就没人敢信）
- 各部门口径不同，汇总到一套要花力气
- 密级不同，一个部门共用意味着权限要重新设计

↓

★ 但底座一旦立住：

- 第二个部门接进来时，改动很小 → 这才是底座而不是又一个应用
- 部门的经验沉淀在知识库里，而不是散在每个人电脑里
- ★ 人员流动时，能力留下来（这是"资产"的真正含义）

> **金句**
>
> 「一套知识库服务整个部门，才是数字员工从工具变成资产的那一步。」
>
> 「底座难在维护，不在搭建。没人对'答案是对的'负责的知识库，活不过三个月。」

9.6 管理体系与制度：让数字员工可以被推广

现象

制度写完、形象画好、场景跑通，然后呢？

然后你会遇到这个场面：你想把这个数字员工推给另一个部门，对方问三个问题——

1. 它叫什么？→ 答得上（有台账）
2. 它能干什么、不能干什么？→ 答得上（有制度）
3. **出了事谁负责？** → 答不上来

第 3 个问题答不上来，推广就到此为止。

为什么会这样

因为**制度的作用不是约束，是授权**。

一份《数字员工管理指南》，真正解决的是「**谁有权批准一个新的数字员工存在**」这个问题。它包含五块内容，正好是一个完整闭环：

```
① 命名规范          → 它叫什么（IP层）
② 身份与职责定义    → 它是谁、能干什么（岗位层）
③ 权限与数据边界    → 它能碰什么（安全层）
④ 新增审批机制      → 谁能让它存在（治理层）
⑤ 违规处理          → 违反了怎么办（纪律层）
```

第 ④ 条的流程是：

> 各部门如需新增数字员工，需提交《数字员工命名申请表》，明确：
>
> - **功能定位**（如「生产数据预测」「供应链调度」）
> - **命名建议**（需符合「品牌简称 + 场景关键词」规则）
>
> **统一备案管理**：数字化中心建立《数字员工命名台账》，记录所有名称、职责及使用部门，避免重复。

第 ⑤ 条是：

> 部门擅自命名或修改数字员工身份，**首次警告并整改，再次违规将暂停相关账号权限**。

这个「两级处理」的设计值得单独说。

大多数企业的制度写着「严禁擅自命名」，但没有后果——**违规成本为零，制度自然失效**。而「首次警告、再次停权限」这个梯度设计很聪明：第一次给面子改正，第二次才真的疼，而且**不封死**——它留了一条「改正后恢复」的路。

怎么识别自己在坑里

1. 别的部门要上线数字员工时，**走的是你的审批流，还是他们自己上线了你事后才知道**？
2. 你有没有一份**台账**，能一眼看出全公司有多少个数字员工、谁在用、干什么？
3. 你的制度里，**有没有「违规怎么办」**？
4. 数字员工出问题时，**谁是你的第一联系人**？这个问题必须在制度里写死。

正确做法

制度四件套（可直接进附录）：

文件	作用	关键内容
《数字员工管理制度》	主　文件	命名规范、身份职责、权限边界、审批、违规处理
《数字员工命名申请表》	入口	功能定位 + 命名建议 + 使用部门 + 业务负责人
《数字员工命名台账》	底账	名称、职责、数据边界、使用部门、版本、启用日期、责任人
《数字员工权限矩阵》	安全	网络 / 数据 / 交互对象 / 敏感操作审批人 / 留痕要求

再加两条治理规则（很多企业没有）：

1. **新增数字员工必须有业务侧责任人**，不能只挂在技术部门——**没有业务责任人的数字员工，半年后一定变成没人维护的僵尸**
2. **台账每季度复核一次**：把「已启用但 90 天零调用」的数字员工标记出来，要么优化要么下线

第二条尤其重要。数字员工会**只增不减**——因为下线在心理上和政治上比上线难。一年下来台账里可能有几十个僵尸。而**僵尸数字员工是安全风险，也是维护负担**。

宣传与推广：制度之外的另一半

制度解决「能不能用」，宣传解决「**愿不愿用**」。

现成的一条完整宣传线（来自内部 AI 应用创新赛的实践）：

阶段	动作	关键动作
前期（1-7天）	概念与氛围	RPA 自动化概念宣传、大赛背景/流程/利好宣传、宣传语与渠道准备、**报名通道准备**
筹备	物料就位	文案模板、海报模板、**RPA 机器人元素图**、厂商 logo 与 IP（外发版）、大赛日程（可外发）
启动会（1天）	造势与定性	产品与场景宣讲、**已落地应用场景分享**、**高层领导讲话并给比赛定性**、会议场地与参会引导
赛中	参与	各部门组队、赛程推进、评审
赛后	转化与扩散	优秀场景选题成果转化实际应用落地、**IT 人员按项目规模部署**、比赛成果与优秀队伍宣传、品牌宣传支持（公众号）、宣传海报与文案
长期	沉淀	优秀场景持续推广、复制到同类业务

三个最值得注意的点：

① **启动会讲「已落地的应用场景」，而不是讲技术。** 老员工能立刻看懂「这个已经有人在用了」——这是最强的说服力。

② **高层领导讲话并给比赛定性。** 这一步是把「部门活动」变成「公司级信号」。**没有这一步，宣传做得再好也只是自娱自乐。**

③ **赛后要跟「IT 人员部署」。** 宣传和交付之间必须有承接，否则大赛结束就断档——**赛后就没人用的数字员工，是最伤的失败。**

还有一个数据值得记：某大型制造企业**首届**内部 AI 应用创新赛，**多个一级部门全员参与，三十余支队伍报名、三十余支投入实战**，其中生产/质量/EHS 一级部门以十余支参赛队伍成为「主力军」。

这个分布很说明问题：**报名最多的是离制造最近的部门**（提质、降本、增效的诉求最迫切），而不是技术部门。这正是第一章讲的「别从全员培训开始」的另一面印证——**真正会报名的是那些最痛的人。**

> **金句**
>
> **「制度给数字员工发户口本，宣传给它办入职仪式——两个都缺，它就是个没人认领的账号。」**

9.7 让 AI 变得更人性化：这一章真正想说的话

现象

我们把 IP 、 VI 、边界、场景、制度、宣传六件事讲完了。但它们各自看都很「细碎」——为什么值得单独立一章?

因为它们合起来指向同一个结论，而这个结论是全书前八章都没说过的：

数字化协同的工作模式，不是「员工被 AI 替代」，而是「每个员工带着自己的数字员工工作」。

为什么会这样

把作者的原话拆开：

> 数字化包含自动化和 AI，即数字员工；它可通过交互获取数据并自动执行任务，**让员工提炼自身技能**，以此方式建立。

这句话里有两个关键词，是整本书的落点之一：

第一个是「交互获取数据」。 数字员工不是后台批处理，是**有来有回**的。它先问你要什么、你要给它什么、它的边界在哪。**这个「交互」过程本身，就是一次岗位建模**。

第二个是「让员工提炼自身技能」。

这句话很重。它意味着：员工在跟数字员工磨合的过程中，会被迫想清楚——**我这件事到底是怎么做的？我凭什么比它做得好?**

而这恰恰是**第六章「算账」和第八章「效率悖论」要回答的问题的入口**：

- 如果员工想清楚了「我的技能是什么」，那「 AI 替代我」这个说法就不成立了——**AI 拿走了任务，留下了判断**
- 如果员工提炼出了技能，那「效率红利」就有了落点——**不是空出来的时间，是更强的岗位能力**

- 如果每个员工都带着自己的数字员工，那 8.2 里那张「46 个计划做、 23 个已实现」的场景清单，**就不该由一个部门去建，而该由每个岗位去建**

> **换句话说：数字员工是人机协同的载体，不是自动化工具的升级版。**

而这也回答了第一章那个最尖锐的问题——**「几百个 Agent 零价值」为什么会出现?**

因为把 Agent 当**工具**建（做完就完），价值必然是零； 把数字员工当**岗位**建（有名字、有边界、有制度、有责任人、有技能沉淀），价值才有落点。

怎么识别自己在坑里

1. 你能说出**你们公司哪个数字员工，是某个员工自己「带」出来的**吗？不是 IT 给的，是这个员工自己磨合出来的。
2. 你的员工用数字员工时，**是在「用工具」，还是在「带下属」**？这听起来矫情，但体感差别很大。
3. 你有没有统计过：**某个员工在跟数字员工协作后，他自己的岗位能力发生了什么变化**？如果从来没有被观测过，那「提炼技能」就只是口号。

正确做法

把数字员工的运营目标，从「上线数量」改成「岗位能力增量」。

具体三个动作：

① **每个数字员工配一个「带它的人」**。 不是管理员，是**使用它的岗位上的那个人**。他的工作是：提需求、喂数据、纠错、然后**把自己的方法论沉淀进去**。这才是「提炼自身技能」的具体机制。

② **建「人机分工表」，而不是只建「场景清单」**。

环节	人做什么	数字员工做什么
数据获取	定义口径、判断数据可不可信	交互式获取、对齐口径、标记缺失
规则判断	处理例外、担责签字	执行确定性规则、标记不确定项
方案生成	选择路径、评估取舍	生成候选、给出依据
执行落地	授权、沟通、异常处理	自动执行标准动作

这张表的价值：**它把「AI 会不会取代我」这个恐惧问题，变成了「我和它怎么分工」这个管理问题**。

③ 宣传的对象，从「数字员工」转向「人和数字员工一起工作」。

大赛、海报、公众号、内部宣讲——**主角应该是「某个员工和他的数字员工」，而不是「某公司上线了 N 个机器人」**。

前者有代入感，后者只有数字。

> **金句**
>
> **「数字员工的终点不是替人干活，是让人从干活的人变成会带数字员工的人。」**

9.8 ★ 被 AI 改变的人：两个真实视角

现象

前面九章都在讲「项目」——项目做成了还是没做成，边界画在哪，止损线设在哪。

但企业里真正被 AI 触碰的，从来不是项目，是**人**。

有一个问题，项目的成败表回答不了：

> **AI 到底把谁的工作改了？改成了什么样？**

本书写到这里，所有的案例都是「项目视角」：一个场景、一段周期、一次去留决策。 而当一位负责人被问到「 AI 到底改变了什么」时，他给出的答案完全不在这个框架里——

他说的不是「我们上线了几个系统」，而是：

> **「被 AI 具体改变的人，我觉得在全世界核心首先改变的首先是程序员，不用说了，就是开发、我们相关技术部门开发的同事。第二个呢，就是比如像做项目管理的。」**

★ 注意他用的词是「**改变**」，不是「使用」。 ★ 这是本书到目前为止，第一次从「岗位」而不是「项目」的视角看 AI 。

★ 这个案例的基本信息（★ 作者 2026-11-02 补充）：

```
┌──────────────────────────────────────────────────────┐
│ 项目管理小助手                                       │
├──────────────────────────────────────────────────────┤
│ 正式名称    项目管理小助手                           │
│ 上线时间    ★ 2026 年 3 月                           │
│ 技术底座    ★ 初期采用千问的 Agent 架构              │
│ 定位        负责项目管理                             │
│                                                      │
│ 管理范围:                                            │
│    · 项目文件管理                                    │
│    · 项目预算管理                                    │
│    · 各种模板资料管理                                │
│    · 项目相关制度管理                                │
│                                                      │
│ ★ 线下评审会转线上比例：约 30%~40%                   │
└──────────────────────────────────────────────────────┘
```

★ 注意「项目管理小助手」的命名——**它一出生就被定义为「某个岗位的助手」，不是「某个系统」**。★ 这和 9.0 讲的「数字员工不是系统，是岗位」是同一件事，只不过这次它体现在名字上。

为什么会这样

一、为什么是程序员排第一

因为程序员的 AI 是**直接进了工具链**的。

写代码、改 bug 、读文档、写测试——这些工作本来就有明确输入输出，有确定的正确性判据。★ 所以 AI 在这个岗位上几乎不需要「变革」，只需要「接入」。

★ 这给出了一个规律：

> **AI 改变一个岗位的难易，和这个岗位的工作里「规则明确的比例」正相关。**
>
> **规则越明确， AI 越容易先改变这个岗位；规则越依赖经验， AI 越晚、甚至永远进不去。**

★ 而这正是第六章那条老结论在人的层面的翻版：**AI 替走的是任务，留下的是判断——所以被改变的岗位，一定是「判断占比下降」的岗位。**

二、第二个被改变的是项目管理（PMO）——★ 这一段是全章最实的素材

> 「因为我们的数字员工中有项目管理的。那么比如说在项目过程当中有相当多的文件，比如可行性、立项、启动等等各个环节的核心文件的审核，都会交给这个数字——说 AI 去提前预审。
>
> 审完以后呢，再做人工审核。如果没有什么大的问题和风险的情况下，原来线下的这个评审会呢，都可以调整到线上评审会，就是说通过线上快速的方式来评审，节约了很多的时间。「

★ 这段话里藏着三**个层级的改变**，一个比一个深：

```
★ 第一层：AI 接管了"预审"
   可行性、立项、启动……各环节核心文件
   → AI 先审一遍，人工再审
   → 人看到的不是原始文档，是"AI 已经筛过一遍的文档"

★ 第二层：★ 评审形式被改变了
   线下评审会 → ★ 线上评审会
   → 这已经不是"多了一个助手"
   → 这是"组织的开会方式"被改了

★ 第三层：★ AI 打通了预算系统
   "每个项目的预算的情况也比较清晰地知道啊，
    他的预算来自哪里，这个项目预算完成情况以及是否超预算等等"
   → AI 不再只读文档，它读的是"项目的钱"
```

★ 第二层是判断「AI 有没有真进流程」的分水岭。

很多企业的 AI 项目停在一层：加了个助手，回答问题、写写材料，人还是照样跑线下会、照样人工逐字审。★ 而「线下评审会变成线上评审会」意味着：**流程本身被重画了，不只是工具被加了一层。**

★ 第三层则更微妙：**当 AI 能读懂「这个项目的钱从哪、执行到哪、有没有超」，它才真正成了一个「项目助手」，而不只是「文档检查器」。**

★ 第三层的细节：★ 它打通了哪两个系统，超预算又是在什么时候被发现

★ 这两件事，作者补充得非常具体，也非常关键：

```
★ 打通的是两个系统：

  ┌──────────────────────────────────────────────┐
  │ 投资管理系统   ← 预算侧：计划、批准、变更          │
  │       ↕ ★ 打通了                              │
  │ 报销系统       ← 执行侧：实际支付、真实花销        │
  └──────────────────────────────────────────────┘

  → ★ 于是 AI 可以同时拿到：
    · 这个项目的预算情况（批了多少、来自哪里）
    · ★ 这个项目的实际支付情况（实际花了多少）
```

★ 为什么非打通这两个不可？

★ 因为在大多数企业里，**预算和执行本来就不在一个系统里**：

```
★ 投资管理系统说：   这个项目批了 300 万
★ 报销系统说：       这个项目实际报了 187 万
★ ★ 两者之间：         ★ 靠人工把它们对上，靠 Excel 拼一张表
★ ★ 而这张表：         ★ 没人实时更新，通常是季度末财务才拉一次
```

★ 所以这个打通的真实价值，是**消灭了「预算口径」和「花钱口径」之间的那条时间差**。

★★ 而「超预算在什么时候被发现」，是这一层最值钱的地方：

> 「包括你超预算，实际上超预算在这个做相应的评审的时候就已经会发现了。 对，比如说立项评审啊，或者变更评审，就是说相应支付的时候，这个环节就会发现。」

★ 超预算的发现点，是三个具体环节：

```
┌──────────────────────────────────────────────┐
│ ① 立项评审   —— 还没开始花，就先看预算够不够      │
│ ② 变更评审   —— ★ 要追加/调整时，当场看超没超    │
│ ③ 支付环节   —— ★ 真的要付了，★ 提示你超了       │
└──────────────────────────────────────────────┘
        ↓
★★ 这三个环节的性质，完全不同于"事后统计"
```

★ 一个必须说清的边界（★ 写在这里，是因为它决定了这一节能讲到什么强度）：

★ 支付环节发现超预算，是「★ 提示」，不是「★ 硬卡点」—— ★ 系统告诉你「这笔超了」，但★ 批不批、付不付，决定权仍在人手里。

```
★ 这不是能力不足，是有意为之的设计：
  · 硬卡 → 遇到口径错、预算临时调整、跨期记账等合理情况也会被卡住
  · 提示 → ★★ 把信息摆在决策那一刻，★ 判断权完整地留给人
      ↓
★★★ 于是这一层真正的价值是"★ 早"，
    而不是"★ 拦得住"：
    早知道 → 预算还有回旋余地、流程还有卡点可用
    晚知道 → 钱已经花出去了，发现得再准也改不了什么
```

★ 为什么要区分「事中发现」和「事后统计」？

```
★ 事后统计告诉你：   "这个项目超预算 50 万"
★    → 钱已经花出去了，★ 发现得再准也改不了什么

★ 事中发现告诉你：   "这一笔超了"
★    → ★ 预算还有回旋余地，流程还有卡点可用
★    → ★★ 注意：是"发现"不是"拦住"——★ 判断权仍在人手里
```

★ 而这正是第三章讲「数据准备」和第六章讲「算账」时反复说的那句话在流程里的样子：

> **★ 数据的价值不在「准」，在「早」。** ★ 一个晚了三个月的准确数字，只能用来解释历史； ★ 一个早了一天的数字，可以用来改变决定。

★ 而且★★ 这里有一层特别值得注意的东西：**AI 在这件事上做的不是「判断」，是「把数据准时端到会上去」**。

```
★ 超预算这件事本身，★ 判断权一直在人手里（评审会）
★ AI 做的是：★ 让评审人在做判断的那一刻，手边就有真实数据
★ ★ 而不是让 AI 去决定批不批
```

★ 这就又绕回了本书反复讲的一条线——**AI 拿走了任务，留下了判断**。 ★ 在预算这件事上，它连任务都只是「取数」，**判断一步都没碰**。

★ ★ 而这恰恰是它能被信任的原因：

> **★ 一个会告诉你「这项目要超了」的助手，和一个会替你决定「这项目不许批」的系统， ★ 在组织里的寿命是完全不同的。**
>
> **★ 前者让评审会变短，后者让评审会不敢开。**

三、项目经理本人得到了什么

> 「所以在项目管理的人就比较的轻松，相对来说，原来所有文档一行一行地看，一一逐字逐字地审，那方便的多。
>
> 同时在项目经理这里也是一样的，所有的文档他会提前上 AI 先审一遍，我们内部的项目管理小助手。 审一遍以后呢，减少各种错别字、文笔不对、数据错误、时间等等这些相对低级的一些错误问题。 啊，这些容易被忽视的东西，也可以提高我们整个推进的效率「

★ 请注意他列举的这几项：**错别字、文笔不对、数据错误、时间**。

★ 这些是「相对低级的、容易被忽视的」错误。

★ 而这恰恰是**AI 在人身上最被低估的价值**：

```
★ 很多人以为 AI 的价值是"替你想"
★ 实际上 PM 感受到的第一层价值是"★ 替你挡住最烦的东西"

   原来这些错误要靠人的细心，
   而人的细心是有波动的：
   · 第 1 份文档认真看，第 20 份就累了
   · ★ 状态好的时候抓得住，状态差的时候抓不住
   · ★ 而且这些错误在评审会上被指出来时，最伤的不是内容，是"你连这个都没看"

★ AI 的稳定性，正是人最缺的那部分稳定性
★ 所以它第一个被感知的收益点，是"低级错误少了"，而不是"高级判断准了"
```

★ 这也解释了为什么一个「看起来不高级」的用途，反而最容易让人愿意用：**它解决的是「我不想再因为看漏了而挨说」这件事**。

四、★★ 两级审查的真实分工：人机边界最清楚的一次落地

★ 这是全书目前**人机分工最清晰的一份一手材料**，因为它不是设计出来的，是已经在跑了。

★ 第一步：先说清「错」是怎么产生的

> 「只要是人工手写的各种文件难免都会有错误，特别是 PM 他都是用模板， 原有的模板可能还是上一个项目的，他会把上一个项目的模板拷下来拿到这个模板里面来改，经常就是改改项目名称呀、日期呀什么的就可以套用了。 但实际上在这个改的过程当中难免有疏漏， 比如说项目名称和甲方乙方或者实施的内容会有差异的这些东西。」

★ 这段话揭开了一个**极其普遍、却几乎从不被讨论的错误来源**：

★ 模板复用

上一个项目的立项报告 → 拷贝 → 改项目名称 → 改日期 → 提交

★ 看起来是对的
★ ★ 但"改得不够"的地方，就是错误：
· 项目名称改了，正文里的项目名没改（或者反了）
· 甲方乙方改了一个没改另一个
· ★ 实施内容还是上个项目的
· ★ 忘了改的，比忘了写的更多

★ 为什么人几乎查不出来？
★ 因为 ★ 模板残留"看起来很像对的" —— 它通顺、格式正确、逻辑完整
★ ★ 而人眼对"读起来顺"的东西极度宽容

> **★ 这就是 AI 在这里真正不可替代的地方：它不是更认真，是 ★ 它不认识那些内容。**
>
> **★ 人读一份文档时，「这读起来很顺」本身就是一种信任； ★ 而 AI 判断的是「这句话是不是属于这个项目」，它对上一个项目的故事没有任何感觉。**

★ 第二步： AI 实际做的四件事

★ ① 列出 ★ 与本项目无关的内容（★ 最核心的一件）
★ ② 识别模板残留导致的名称/甲乙方/实施内容差异
★ ③ 挑出打字错误、错别字
★ ④ 给出风险提示

★ 而它的能力边界，作者给的说法很硬：

> 「对于 AI 来说它都能识别和检查出来，给本项目无关的内容它会列出来。 包括还有一些打字错误啊等等的，基本上怎么来说呢？ **他们都要审，而且都审都会有问题。可以说是在初审 AI 可以 100% 发现问题， 发现这些低级问题啊。**「

★ 第三步：★★ 而真正精彩的是它被放进了流程的哪个位置

> 「一般来说，他们都会审完改、改了审，**一直审到 AI 是没有报错了，就是说没有异常错误了，包括没有一些风险提示了，他们才会提交到 PMO 这边来审。**」

★★ 请把这句话读两遍——它描述的不是「AI 是个帮手」，而是「AI 是一个闸门」：

```
PM 写文档
   ↓
┌──────────────────────────────┐
│  ★ AI 初审                   │
│  有告警 → 改 → 再审 → 再改    │ ← ★ 循环，直到零告警
└──────────────────────────────┘
   ↓  ★ 零告警，才允许提交
PMO 复审（人）
   ↓
评审会
```

★ 这个机制的价值有三层，一层比一层实际：

	传统做法	★ 现在的做法
谁来找低级错误	PM 自己，或 PMO 兜底	★ AI，且 ★ 找到 100%
PMO 收到的东西	★ 带着错误	★★ 已知干净（零告警才放行）
PM 的工作	写 → 提交 → 被打回 → 再改	★ 写 → ★ 与 AI 对话到干净 → 提交
PMO 的负担	★ 要替 PM 抓低级错误	★★ 只看该看的
★ 评审会	——	★★ 有底气改成线上（30%~40%）

★★ ★ 最后一行是这个机制真正的价值所在：

★ 线下评审会能转线上，前提是「提交上来的东西已经干净了」。 ★ 而「已经干净」不是靠 PM 更认真实现的，是靠一个 ★ 零告警才能提交的闸门实现的。

★ 这就是「 AI 改变了流程」最朴素的一种样子： ★ 它没有出现在评审会的议程上，它出现在 ★ 谁有资格把材料提交上去。

★ 第四步：★★ 两级分工——AI 审什么，人审什么

「然后 PMO 这边呢，等会也有相应的规则规范检查的要点啊。 比如说立项的，我们会更多的去检查项目组、项目的执行计划呀、它的可行性呀、 以及相关的项目成员的职责划分等等，包括人员匹配是否到位啊，更多的是这些东西啊。 这个项目会检查项目的一些风险是否有相应的一个实施策略，这些作为重点。 **所以他检查的侧重点不一样。**「

★★ 这是 9.7 那张「人机分工表」的**真实落地版本**:

	★ 初审：项目管理小助手（AI）	★ 复审：PMO（人）
审什么	★ 与本项目无关的内容 ★ 模板残留（项目名称/甲方乙方/实施内容差异） ★ 打字错误、错别字 ★ 风险提示	★ 项目组是否成立 ★ 执行计划 ★ 可行性 ★ 成员职责划分 ★ ★ 人员匹配是否到位 ★ ★ 风险有无实施策略
性质	★ 确定性规则	★ ★ 判断与担责
判定标准	★ 零告警才放行	★ 通过 / 不通过 + 签字
★ 人还要不要	★ ★ 要，但只看该看的	★ ★ 责任一点没减

★★ 这就是「AI 拿走了任务，留下了判断」最教科书的一次落地：

★ 确定性规则（模板残留、错别字）交给 AI，它能 100% 覆盖；★ 判断性规则（人员匹不匹配、风险策略到不到位）留给人，因为这些本来就需要担责。

★ 而这两者的分界线，其实非常朴素：★★「这件事有没有唯一正确答案」★·有 → 交给 AI，它比人可靠 ★·没有 → 留给人，它本来就该由人负责

★ 第五步：★★ 一个数字员工，两处岗位，两套要点

「那么 PMO 这边呢，对于这些侧重点呢，他就是给 PM——项目管理小助手，针对 PMO 这方面的一些测试检测。」

★ 注意这句话的结构：**项目管理小助手在两处被使用——**

```
PM 侧   → 用它做文档初审（找低级错误）
PMO 侧  → ★ 用它按 PMO 自己的规则规范要点做针对性检测

★ 同一个数字员工
★ 两处使用
★ ★ 但两处喂进去的检查要点不一样
```

★★ 这就把 9.0 那句「数字员工不是系统，是岗位」往前又推了一步：

★ 它不是某一个岗位的专属工具，★ 而是 ★ 这个职能域里多个岗位共用同一个数字员工，各自按自己的要点配置。

★ 这也正是 9.5.1 讲的「部门级共享智能体」的初级形态：★★ 一套底座 + 多套要点 = 一个职能域的公共能力。

★ 第六步：★★ 关于「提效多少」，作者给了一个诚实的回答

> 「这个我觉得没有具体数量吧。」

★ 我原本准备把「减少了多少低级错误」写成量化指标，看完这句就改了主意——**不写数字，也不标【待核实】**，因为它拿不到，写了就是编。

★★ 但「拿不到」这件事本身，★ 反而值得单独写一段：

```
★ 成功的收益，天然难量化
    · 低级错误被拦在提交之前 → ★ 没有汇总口径，因为它是"每一次提交"的一部分
    · 评审会变短 → 节约的时间散在每个人的日历里
    ★ → 所以作者只能说"没有具体数量"

★ 而失败的结局，天然好量化
    · ★ "零使用量" —— 一个不需要争论的数字
    · ★ "上线 3 个月后无人使用"
```

> **★★ 这给评估 AI 项目提供了一个非常实用的换位思考：**
>
> **★ 别问「它提效了多少」——这个问题在成功的项目上通常答不出来。 ★ 要问「★ 有哪些能用行为数据直接观测的指标」： ★ 上线后第 30 天有多少人主动打开过它？零告警闸门拦下了多少份没提交的稿？ ★ 评审会平均从几小时变成几十分钟？**
>
> **★ 凡是必须靠「大家主观评价」才能回答的指标，大概率永远在扯皮。 ★ 凡是能从系统里直接读出来的数字，才是可以拿去做决策的。**

★ 而这条，恰好能和 8.5 那个失败案例对上——**失败的那个项目，恰恰有一个毫无争议的硬数字：零使用量。**

	程序员	项目经理 / PMO	★ 一线业务人员
AI 以什么形态进入	工具链	既有审批链	另开的入口
需要多少组织变革	几乎不需要	★ 需要（评审会改线上）	★ 需要（大家改用习惯）
收益第一个显现在哪	写码速度	★ 错别字/数据错误/时间	无
为什么是这个顺序	规则最明确	★ 文档密集 + 规则可判定	★ 规则依赖经验、口径不清
本书对应案例	—	**★ 9.8 （本章）**	**★ 8.5 营销助手（失败）**

★ 这张表把全书的两条线接在了一起：

> **营销 AI 助手为什么失败？** 因为它选择了「另开一个入口」的那一列——★ 规则最不明确、变革成本最高、收益最晚显现的那一列。
>
> **而 PMO 案例为什么成功？** 因为它选择了「接进既有审批链」——★ 变革发生在原本就要做的动作上，不需要用户改变习惯，只需要让原来的会变短。

★ 换句话说：

> **★ AI 进流程的正确姿势，是让原本就要做的动作变得更省，而不是让用户养成一个新习惯。**

怎么识别自己在坑里

★ 这一节留给「想在自己公司里找到类似机会」的人。五个自查问题：

□ Q1：我能说出"我的岗位里，哪一部分工作是逐字逐行看文档"吗？
→ 说不出，说明还没找到切入点（★ 这是 PMO 类机会的入口特征）

□ Q2：我能说出"我的岗位里，哪一部分工作规则明确到可以预审"吗？
→ 规则不明确的部分，AI 只能"辅助"，不能"预审"
→ ★ 分不清这两者，是最常见的期望落空来源

□ Q3：这个 AI 是"接进我原本就要做的动作"，还是"另开一个入口"？
→ ★ 另开入口的，用起来一定会掉到"上线即冷"

□ Q4：★ 如果 AI 的预审足够可靠，我组织里的哪个会、哪条流程可以因此变短或改线上？
→ ★ 答不上来，说明你只是在加工具，没有进流程
→ ★ 这个问题答得上来，才叫"AI 改变了岗位"

□ Q5：★ 这个岗位被改变后，"判断"的部分变多了还是变少了？
→ 如果只是把活儿全交出去，那是自动化
→ ★ 如果"要盯的东西"变多了，人才真的被 AI 改变了

★ Q4 是这五个里最狠的一个。**它把「要不要上 AI」这个问题，换成了「上完之后哪个环节会变短」**。★ 前一个问题会引出无穷的立项论证，后一个问题会直接引出流程图。

正确做法

★ 如果 Q4 有答案，做法是固定的四步：

```
★ 第一步：找出那个"逐字逐行看文档"的动作
    → 这是 AI 最容易、也最被低估的切入点

★ 第二步：把它定义成"预审"，而不是"助手"
    → ★ 预审有明确的输出物：问题清单
    → 助手没有输出物，所以永远说不清用得好不好
    → ★ 这一步决定了它能不能接进既有流程

★ 第三步：★ 预审之后必须有人工复核，并写明"低风险可简化、高风险必须上会"
    → 作者原话是"如果没有什么大的问题和风险的情况下……可以调整到线上评审会"
    → ★ 关键在"如果没有大的问题和风险"——这就是分级
    → 没有分级，AI 预审只会变成新的会议负担

★ 第四步：★ 把"预算/进度/超支"这类系统事实也接进来
    → 从"检查文档"升级到"理解项目"
    → ★ 这一步是"助手"和"数字员工"的分水岭（见 9.0）
```

★ 第三步的「分级」需要特别强调，因为它对应第一章讲过的**六环节分级**：**★ 能做成「线上评审会」的，是规则明确、可复核、低风险的环节；★ 必须留在线下开会的，是口径依赖经验、一旦出错代价高、涉及跨部门利益的环节**。

★ 这也顺手回答了第八章 8.3 那个观察：

> **★ 8.3 发现「做成的 23 个里几乎全是助手」。 PMO 案例是个例外，而例外最值得研究—— ★ 它成功的关键不是「做成了助手」，而是「★ 它把 AI 接进了原本就在跑的审批链， ★ 并让评审会从线下搬到了线上」。**
>
> **所以 8.3 的结论要修正为：绝大多数项目做成的是助手； ★ 但真正做出「流程级改变」的，恰恰是那些不另开入口、直接改既有流程的项目。 ★ 而这类项目，占失败项目的大多数——因为它要求的组织变革更大。**

★ 一个必须同时讲的反面：改变人的项目，失败代价也最大

★ 本节讲的是成功案例。但同一天，同一位负责人，还讲了另一个项目的结局：

```
★ 营销 AI 助手
  · 建设周期：约 3 个月
  · 投入：★ 常态 5 人（IT 侧 2.5 / 业务侧 2.5）
            ★ 测试期营销扩到 5 人 → 参与峰值 8 人
  · 上线后：约 3 个月基本无人使用
  · ★ 半年周期，正式宣告失败
  · ★ 现状：系统仍挂在线上，至今 ★ 零使用量
```

★ 把这一条和 9.8 的 PMO 案例并排看，能看到一组刺眼的对照：

	★ PMO / 项目管理小助手	★ 营销 AI 助手
AI 进入的路径	★ 接进既有审批链	★ 另开入口
变革成本	★ 改一次评审方式	★ 要求所有人改习惯
人得到什么	★ 错别字/数据错误/时间不用自己抓	★ 要先自己登录、再自己问、再自己判断
半年后的结果	★ PMO 每天在用	★ ★ 零使用量
★ 超预算何时发现	不适用	★ ★ 立项/变更/支付环节，当场（★ 提示非拦截）
★ 参与人数峰值	不适用（本来就在流程里）	★ ★ 8 人（测试期）
★ 代价	★ 省下来的时间	★ ★ 常态 5 人 × 半年周期，参与峰值 8 人

★ 结论刺眼但很清楚：

> **★ 同样是「要改变人」，一个把成本降下来了，一个把成本花光了。**
>
> **★ 差别不在 AI 能力，在「你有没有让 AI 替人省掉一个原本就存在的动作」。**

★ 再补一句最难听的：

> ★ 再补一个更要命的细节：★ 这个项目参与人数的峰值出现在测试阶段（营销侧从 2 人扩到 5 人），**★ 而它最终的日常使用量是 0 。参与人数最多的时候，恰恰离「会被真正用起来」最远。★ 所以「参与人数」绝不能当成项目健康度的指标——该盯的是上线后第 30 天、第 60 天，★ 有多少人主动打开过它。**
>
> **★ 营销 AI 助手至今仍挂在线上，零使用量，也没有人再管它。★ 一个「正式宣告失败、但系统还挂着」的项目，会一直躺在资源清单上，★ 被当成「还在推进的资产」——它连失败都算不上，所以永远不会被清算。**

★ 这就是第六章「算账」在人的层面的样子：**改变人的项目，成功时收益最大，失败时账单也最大，而且账单最容易被分散到看不见的地方**。

五、★ 三个岗位的对照：谁被改变了，为什么

六、★ 30%~40%：这个数字比「已经转成线上」更值得写

★ 作者给的实测数据是：**线下评审会转线上的比例约 30%~40%**。

★ 第一反应可能是「才三成，是不是没做成功」。

★ 但换个问法，它其实是全书里最诚实的一个数字：

```
★ 为什么不是 100%?
   → 因为剩下的 60%~70% 评审会，本身就该在线下开

   ★ 有些评审会要处理的是:
      · 跨部门利益冲突 —— 需要面对面谈
      · 口径不明确的分歧 —— 文字说不清
      · 高风险决策 —— 需要现场拍板的责任感
   ★ 而这些，★ 恰恰是第一章讲过的"AI 靠不住的地方"

★ 所以 30%~40% 不是"AI 覆盖率低"
★ 而是 "★ AI 可靠地覆盖了它能覆盖的那一段"
```

> **★ 这才是成熟度分级的正确用法：★ 不是「AI 能覆盖 100% 的流程」，而是「★ AI 覆盖了其中规则明确、可复核的那一部分，★ 并且明确知道剩下的哪一部分不该交给它」。**

★ 而 30%~40% 这个数字真正的价值在于——**它让「转线上」从一句口号变成了一个可以跟踪的指标**。

```
★ 一句话就能问出来：
   "上个月的项目评审会，有多少比例是在线上开的？"

★ 如果回答是"基本上都线上了"——大概率是假话
★ 如果回答是"30% 多，而且我们知道剩下那 70% 为什么还在线下"——★ 这是真的
```

★ 顺带一提：**这 30%~40% 也不是从上线第一天就有的**。★ 一个数字员工的价值不是「上线即达标」，而是「★ 每一轮复盘都能让这个比例往上挪一点」。

七、★★ 全书最干净的一次对照实验：同一个底座，一成一败

★ 现在把两个案例并排放，会出现全书最值得记的一张表：

	★ 项目管理小助手	★ 营销 AI 助手
时间	★ 2026 年 3 月上线	★ 同期建设，约 3 个月完成上线
底座/架构	★ 千问的 Agent 架构（智能体架构）	★ AI 开发 + 移动端适配自建
★ 进入方式	★ 接进既有审批链（原本就在跑的流程）	★ 另开入口（要求用户改习惯）
用户要做什么	★ 什么都不用做——流程把人带过来	★ 主动登录、主动提问、主动判断
半年后	★ 仍在用，30%~40% 的评审会转到了线上	★ 宣告失败，至今零使用量
到今天(2026-11)	★ ★ 约 8 个月，仍在用	★ ★ 零使用量
★ 结论	★ AI 进入了流程	★ AI 停在了工具层

★ 两个人事、同一套技术路线、同一批工程师、同一段时期——

> **★ 差别只有一个：★ 项目管理小助手接进了一条本来就在跑的流程；★ 营销 AI 助手要求所有人养成一个新习惯。**
>
> **★★ 一个 8 个月后仍在用且改变了 30%~40% 的会议方式；★ 一个 8 个月后零使用量、至今挂着没人管。**

★ 所以本书那个反复出现的判断，在这里得到了最硬的一次证明：

> **★ 不是 AI 不行，是「★ AI 从哪里进入」决定了它是资产还是负债。**
>
> **★ 技术底座可以一样、投入可以一样、团队可以一样，★ 差别在于：★ 你是把 AI 塞进用户原本就要做的事里，★ 还是要求用户为了 AI 多做一件事。**

★★ 这一条应该写进本书的封面页。 它是全书所有方法论的压缩版：

```
★ 你的 AI 项目需要改变谁的行为？
  · 如果答案是"用户"    → ★ 营销助手的路，代价最高
  · 如果答案是"流程"    → ★ 项目管理小助手的路，代价最低
```

> **金句**
>
> **「AI 改变的第一批人，往往不是最复杂的那个岗位，是最重复的那个岗位。」**
>
> **「项目经理的 AI 价值，第一个不是替他想，是替他挡住错别字、数据错误和时间写错。」**
>
> **「当预审足够可靠，线下评审会就能变成线上评审会——这是判断 AI 有没有真进流程的标志。」**

9.9 本章小结

1. **数字员工不是系统，是岗位**。传统系统按业务闭环垂直切、自己带走一整个环节、有独立入口；数字员工按岗位横向切、穿插在多个系统的节点之间、**没有自己的入口**。搞混了，就会给每个数字员工建一套系统，然后被成本拖死。
2. **先写岗位说明书，再谈技术**。七个字段：岗位名称、原岗位工作、承接什么、不承接什么、在哪些节点、人机分工、岗位价值。**说不清岗位的数字员工，没资格进管理体系**。
3. **IP 与 VI 的本质是识别与授权**。名字是身份证（要集中、要台账、要版本），形象是权限标识（要跟权限绑定），边界是安全底线（网络 / 数据 / 操作三层）。
4. **场景要三段走，1.0 只做三个**。试点跑通 → 结合 IT 系统覆盖 → 生态扩展；2.0 的门槛是数据治理和多模态，不是模型。
5. **制度给户口本，宣传给仪式，两者缺一不可**。制度要有「违规怎么办」，宣传要有「高层定调」和「赛后部署承接」。
6. **最终目标是让员工从「干活的人」变成「会带数字员工的人」**。这才是数字化的协同工作模式，也是效率红利能兑现的落点。
7. **★ 数字员工最终改变的是人，而改变人的方式是「让原本就存在的动作变得更省」**。PMO 案例（★ 项目管理小助手，2026 年 3 月上线，千问 Agent 架构）：AI 预审 → 人工复核 → ★ 线下评审会变线上评审会 → ★ 打通投资管理系统与报销系统（预算+实际支付），★ 超预算在立项/变更/支付环节当场被发现；PM 的第一层收益是错别字、数据错误、时间写错这类低级错误被挡掉。判断标准不是「上线了什么」，而是「★ 哪个环节因此变短了」。而 8.5 营销 AI 助手（★ 常态投入 5 人 = IT 侧 2.5 + 业务侧 2.5，测试期参与峰值 8 人，半年周期宣告失败，至今零使用量）说明：★★ 另开入口的项目，代价是失败时最大的一张账单。
8. **★ 数字员工从工具变成资产的那一步，是「一套知识库服务整个部门」**。部门级共享智能体（一套智能体+一套知识库+一套技能库，全员共用）的三层结构——用户接入层 / 能力引擎层 / ★ 共享资源底座层。底座难在维护不在搭建：没有人对「答案是对的」负责，它活不过三个月。（详见 8.2.6）

> **本章整体金句**
>
> **「数字员工不是把 AI 做得更像人，是把人的工作重新定义一遍。」**

> **用途**：这一组模板是全书的「工具箱」。前三章到第八章讲的是判断，附录给的是**可以直接抄走的表格**。 **纪律声明**：模板中的示例数字均为示意，不指向任何企业；素材未覆盖处标 `【待核实】`。

附录 A　场景评估表（业务价值 × 可行性 打分模板）

来源：作者的实践做法是——把所有候选场景列成清单，逐条按「值不值得做」打分，而不是凭印象挑几个。素材中记录的规模是**七十余条候选需求逐条打分**【待核实：具体条数以作者最终确认为准】。

A.1 评分表（可复制）

#	场景	业务价值 (1-5)	可行性 (1-5)	优先级 =价值×可行性	结论
1					做/缓
2					做/缓
…					

A.2 价值怎么打分（5 分定义）

分	判据（业务价值）
5	直接影响收入、成本或合规风险，且**有明确的责任人**（做错了谁赔）
4	明显减少重复劳动，替代人力明确
3	效率提升，但受益人分散、价值难量化
2	锦上添花，做不做都能活
1	说不清收益

A.3 可行性怎么打分（5 分定义）

分	判据（可行性）
5	数据现成、口径唯一、流程已有负责人
4	数据现成，但口径需对齐一次
3	数据在两个系统内可取，需开发
2	数据需跨系统整合或需外部采购
1	数据不存在或口径混乱

A.4 收敛规则（★ 这一步才是重点）

```
优先级 = 价值 × 可行性

≥ 15   → 进入候选池（第一批做 1–2 个）
9–14   → 备选（第一批跑通后接着做）
< 9    → 暂不做（写进清单，半年后重评）

★ 但最终只选 1–2 个进入试点，其余全部留在清单里
```

「**选完不删表**」**很重要**：被搁置的场景留在清单上，下次复盘时重评，而不是每次重新想一遍。

A.5 一条使用提醒

> **这张表最重要的一列不是分数，是「结论」列里的「为什么不做」。**
>
> 如果 20 个场景里有 18 个写的是「暂不做」却没写原因，那么三个月后你会重新讨论这 18 个场景。

附录 B　AI 项目 ROI 计算表

与第六章 6.5 同一张表，此处做成可直接打印的版本。

项目名称：_____
对应场景：_____
周期：_____ 个月　　负责人：_____

【一、一次性投入】
数据治理前置投入　　人日 _____ × 单价 _____ = _____
流程改造 / 审批环节调整　　= _____
接入与集成开发　　= _____
知识库建设（若适用）　　= _____
外部咨询（若适用）　　= _____
一次性合计 = _____

【二、持续投入（按月）】
算力 / 模型调用　　= _____
运维与内容更新　　= _____
兜底人工复核　　人时 _____ × 单价　　= _____
培训与变革沟通　　= _____
月度合计　　= _____

【三、收益（按月）】
提效类：人时节省 _____ 人时 × 单价　　= _____
决策类：_____（需单独论证）　　= _____
质量类：_____（需单独论证）　　= _____

★ 收益兑现去向（必填其一）：
□ 释放人力做更高价值的事，具体是：_____
□ 减少编制 / 外包支出，具体是：_____
□ 支撑业务量增长，目标：_____

【四、结论】
总投入 = _____ ；月度净收益 = _____
回本周期 = _____ 个月
★ 若"收益兑现去向"未填，本表收益部分不成立

附录 C　AI 模型与工具台账模板

说明：本附录采用通用工程模板，不依赖任何内部制度原件。✅ **2026-11-02 作者确认：就用本通用模板代替，不再追原件。** 以下模板按通用工程实践编写，可直接使用。

C.1 台账字段

#	名称/用途	类型	版本	责任人	上线日	状态
1						在用/退役

C.2 每个条目要补的六项

项	内容	为什么必须有
数据边界	哪些数据会进它、哪些不出厂	出事时第一句话就是问这个
权限与可见范围	谁能用、谁能看它处理过什么	权限继承的落点
日志留存	输入/输出/调用人/时间存多久、谁能查	审计与追溯
降级方案	它挂了怎么办，谁接手	兜底不是可选项
复核机制	谁定期看它的输出、抽查多少	防止它悄悄变差
退役条件	什么情况下下线、下线后数据怎么办	防止僵尸工具累积

C.3 退役条件（建议写法）

出现下列任一情况 → 启动复核 → 决定整改或退役：

① 连续 N 个月价值类指标低于初始目标的 50%
② 人工介入次数持续上升（说明模型在退化或场景在漂移）
③ 依赖的上游系统已下线
④ 出现合规问题且无法通过整改闭环
⑤ 场景的业务流程已不存在

> 【待核实】退役条件中的 N 值与「人工介入次数上升」的判定口径，需按作者所在企业的管理制度确定。

附录 D　数据准备三张清单

与第三章 3.2 同一套，此处为可直接抄的空白表。

D.1 数据可用性清单

数据名称	口径（唯一算法）	来源	时效/延迟	完整性	权限

★ 口径一栏若填不出唯一算法 → 这个数据现在不能用

D.2 文档治理清单

文件名称	版本	生效期	所有者	密级

★ 没有所有者的文件 → 没有人能修它，也没有人能为它的内容负责

D.3 反馈闭环清单

事项	具体约定	责任人
错误上报	入口 / 上报内容	
纠错判断	谁判断、多久给结论	
★ 回流	改源头（知识/规则/口径）的路径	

★ 没有"回流"这一行，这份清单就只是一个抱怨箱

附录 E　风险自查清单

自检方式：每一条打「是/否/部分」。**任何一个「否」且该风险不可接受，就先别上线**。

【数据与合规】
- □ 出厂的数据已逐项分级（可进 / 需脱敏 / 绝对不进）
- □ "绝对不进"的数据，确实没有以任何形式（含摘要）出厂
- □ 个人信息已脱敏或聚合化处理
- □ 检索权限过滤发生在检索之前，而不是生成之后
- □ 输出必须带出处，且无出处时明确拒答

【责任与流程】
- □ 每个结论有明确签字人（AI 辅助，不是 AI 决策）
- □ 错误上报 → 纠错 → 回流的路径已跑通至少一次
- □ 复核能力已复制到推广后的岗位（不依赖能人）

【技术运行】
- □ 模型/工具台账已建立，含数据边界、责任人、退役条件
- □ 降级方案已演练过（工具不可用时怎么办）
- □ 日志留存策略已定，且与供应商条款一致
- □ 模型换版本后做过回归抽查（行为漂移）

【组织】
- □ 推广期有"长期负责人"的名字，不是岗位
- □ 收益兑现去向已写明
- □ 涉及个人行为感知的场景，已写明"不做个人持续监控"的边界并公开

附录 F　常用技术名词速查（给不写代码的读者）

名词	一句话解释	读者最该关心的一点
大模型 / LLM	从海量文本里学出来的「下一个词预测器」，靠它组出像人话的答案	它不保证正确，只保证像人话
Token （令牌）	模型处理文本的最小计量单位，中文大约 1 个字≈1–2 个 token	计费与上下文长度都按它算
上下文窗口	模型一次能「看到」的文字总量	超出窗口的内容它完全不知道 → 这就是 RAG 存在的理由
Embedding （嵌入/向量化）	把一段文字变成一串数字，意思相近的文字数字也相近	靠它做「语义检索」
向量数据库	专门存这些数字并做相似度检索的数据库	知识库的底座；**它不保证权限正确**
RAG （检索增强生成）	先从知识库捞出相关片段，再连同问题一起给模型，让它照着答	**它把「模型不知道」变成「模型查得到」，但不负责查得对**
切片（Chunking）	把长文档切成小段，分别建索引	**切得不好，后面全错**；切片里必须带生效期
混合检索	关键词检索 + 语义检索一起用	制度类文件里，关键词比语义更可靠
微调（Fine-tuning）	用你自己的数据再训练模型，让它更懂你的风格与格式	**它改不了模型的知识时效**，知识要变就用 RAG
Agent （智能体）	能自己拆任务、调用工具、多步完成一件事的模型应用	它让 AI 从「回答」变成「办事」，也把风险从「答错」升级成「做错」
MCP	一种让 AI 以标准方式连接企业工具与数据源的协议	**标准化的意义是权限与审计可以统一做**
私有化部署 / API	模型跑在自己机房，或调用云端服务	决定「数据出不出厂」；**它是数据边界问题，不是性能问题**
幻觉	模型一本正经地说错话或编造事实	只能靠**引用溯源 + 拒答 + 人工确认点**控制，不能靠提示词消除

名词	一句话解释	读者最该关心的一点
置信度	模型对本次答案的把握程度（估算值，不是保证）	**低置信度时不给结论、只列分歧点**，是高管场景的关键设计
Token 成本 / 调用量	按输入输出量计费或占用算力	**它决定了「能不能规模化」，也决定了哪些场景值得私有化**

F.1 一张选型速查表

问自己四个问题：

```
知识经常变吗？          → 变 → RAG（别去微调知识）
要的是风格和格式吗？    → 是 → 少量微调
要多步自主完成吗？      → 是 → Agent（并严格限定工具权限）
数据能出厂吗？          → 不能 → 私有化 / 分片外发 / 路由
```

附录 G　引用与延伸阅读

G.2 延伸阅读（公开资料）

> 【待核实】以下条目来自素材中记录的外部报告与论文清单，**正式定稿前需逐条核实原始出处、发布时间与可引用性**，并按出版社要求标注。本书不引用未经核实的二手转述。

- □ 行业公开报告：AI 在企业落地中的投入产出与失败模式
- □ 公开研究：自建 AI 与采购 AI 的失败率对比（口径见 6.3.3）
- □ 公开研究：企业级 AI 助手的组织接受度与透明化前提
- □ 公开标准/协议文档：MCP（模型上下文协议）规范
- □ 公开标准：GMP / GxP 数据完整性要求（用于第七章 CSV 场景）

G.3 本书与《数据治理实战》的关系

```
《数据治理实战》  →  治责任：谁对数据负责、怎么管住人
《AI 未成之路》→  治决策：做不做、先做哪个、钱怎么花、什么时候停
        ↓
   两本共用一套世界观，但读者不同、处方不同。
```

附录 H　场景筛选表（一页纸自测：这事该不该做）

这张表和附录 A 的区别：

```
附录 A（场景评估表）   →  ★ 已经决定要做了，★ 那怎么排序、怎么收敛
附录 H（场景筛选表）   →  ★ 还没决定要不要做，★ ★★ 先用这一页筛掉不该做的
        ↓
★★ ★★ 顺序很重要：★★ 先筛，再排
★★ ★★ 拿一张 46 个场景的表去做优先级排序，★ ★★ 是本末倒置——
★★ ★★ 因为那张表里，★ ★★ 一半的场景本来就不该做
```

用法：拿一个候选场景，对着下面 12 个问题打「是/否」。**不用写任何方案**，5 分钟能答完。

H.1 十二问筛选表（可复制）

#	问题	是 / 否
1	**这件事现在有人做吗？**（做不了 AI，也要有人在做）	
2	**如果 AI 做好了，收益能被说成「省了多少时间」或「多赚了多少」吗？**	
3	**出错一次，代价有人承担吗？**（能追到具体的人/岗位）	
4	**做这件事需要的文档/数据，今天就拿得到吗？**	
5	**这些数据的口径，是唯一的吗？**（没有第二套算法在打架）	
6	**★ 现在查这件事，老路是什么？**	
7	**★ 老路哪一项比新方案强？**	
8	**使用者今天愿意多打开一个入口吗？**	
9	**这个场景做完，**哪个环节**因此变短了？**（不是「覆盖了几个流程」）	
10	**有没有一个具体的人，会在第一个月里每天用它？**	
11	**试点失败了，我们能说清「为什么」吗？**	
12	**预算从哪出？**（说不出归口部门，就先别立项）	

H.2 怎么读这张表

★★ ★★ 第 6、7 两问是这张表的核心，★★ 也是最容易被跳过的一问

第 6 问：现在这件事，大家是怎么做的？
↓
★★ 如果答案是"有邮件 / 有 Excel / 有个群"——★★ 那就先问第 7 问
★★ 因为老路一定在某一项上强，★★ 不然它早该被换掉了
↓
★★ 第 7 问：老路哪一项比新方案强？
↓
★★ ★★ 常见答案（★ ★★ 这三条是"先别做"的信号）：
· ★★ 自己能挑要哪部分（AI 替用户做了选择）
· ★★ 能转给同事、抄送主管（AI 默认单线程）
· ★★ 零学习成本（AI 要求先学会怎么问）
↓
★★ ★★ 而如果新方案只赢"移动端能查"这一项，★★ ★★ 通常不值得单独立项
★★ ★★ 除非——★ 它是那个不可替代的一项（比如一线在外场，没电脑）

H.3 三条「直接不做」的信号

★★ ★★ 出现下面任何一条，★ ★★ 不用再往下评估了：

❶ ★★ 答不出第 2 问（★ 说不出收益形态）
→ ★★ 这不是技术问题，★ 是这个场景的价值本身没被想清楚

❷ ★★ 答不出第 5 问（★ 口径不唯一）
→ ★★ 这是第三章讲的"翻译"没做，★ ★★ 做 AI 之前先把口径对齐

❸ ★★ 第 7 问的答案是"老路在三项上强，我们只赢一项"
→ ★★ ★★ 老路不是落后，★ ★★ 它是当前的最优解
→ ★★ 这种情况应该做的是：★ ★★ 先去做 5.2 的数据准备与 5.3 的流程再造，
★ ★★ 而不是先做一个 AI 入口

H.4 筛完之后

★★ 过了筛 → ★★ 进附录 A 的评分表，★ ★★ 排队、收敛、选 1–2 个进试点
★★ 没过筛 → ★★ ★★ 写进清单，★ ★★ 半年后重评（★ 别删，★ ★★ 下次复盘要用）

★ 金句

「先筛，再排。拿一张 46 个场景的表去做优先级排序，是本末倒置——因为那张表里，一半的场景本来就不该做。」

「老路不是落后，它是当前的最优解。」

★ 一条使用提醒

这张表最有价值的一行是第 7 问。 如果一个场景永远答不出「老路哪一项比新方案强」，那多半是你还没把老路弄清楚，而不是新方案特别强。

阅读指南：本书怎么读

> **副标题**：为什么大多数企业的 AI 都没做成，以及怎么认出自己正在哪个坑里 **形态**：电子书 → 短视频拆条 → 出版/售卖 **版本**：串联稿 v1（2026-11-01，D7）｜正文约 10.4 万中文字 **作者**：某大型制造企业 智能与数字化中心 项目负责人

全书目录

章	标题	一句话	字数
0	引言：三年里， AI 大多没做成（含三年三阶段）	这本书为什么存在	8,630
1	企业 AI 落地的 5 个坑	输的都不是模型不够强	16,487
2	场景选择：先选事，再选技术	场景选错的代价远大于技术选错	9,654
3	数据准备： AI 的燃料	数据没准备好就想上 AI	8,586
4	技术方案：三棵决策树	「哪个模型更强」是伪问题	11,705
5	企业 AI 落地五步法	从「方案通过」到「真的有人每天用」	8,911
6	算账：老板视角的三问	AI 项目不赚钱就等于没做	11,662
7	合规与风险：哪条线不能碰	AI 的位置必须按出错代价来定	9,414
8	真实案例复盘	三个成的，一个没成的，一个半失败的	33,499
9	数字员工建设	数字员工不是系统，是岗位	20,281
附录	工具箱 A–H	能直接抄走的表	3,846
	合计		**142,675**

各章导语

> ★ 这一节是**读者进入每一章前读的那一段话**。 每段导语都做三件事：① 说清上一章留下了什么问题 ② 说清这一章要解决什么 ③ 预告本章最硬的那个结论。

引言｜三年里， AI 大多没做成

> **三段：技术探索 → 创新专项 → 持续探索。第二年是证据年。**

> **导语**
>
> 前三章之前，先说清一件事：**这本书是站在甲方这边写的。**
>
> 市面上讲 AI 落地的书，绝大多数是乙方视角——卖方案、卖工具、吹成功。而这本书的作者是甲方： 一家制造企业的 AI 创新专项负责人，拿着自己的预算、自己的考核、自己团队的 KPI ，走完了三年——第一年在试，第二年在推，第三年在换一种做法。
>
> 所以这本书里有些东西，乙方视角的书不会写：**哪些场景其实是「看着能做」其实做不了、 哪些钱花了换不回任何东西**、以及**一个项目组在自己的考核压力下怎么做出错误判断**。
>
> 这本书写给三类人：**甲方技术团队**（怎么建起来）、**甲方管理者**（该不该投、什么时候叫停）、 **乙方**（★ 一份反向提醒：客户真正需要什么，以及什么样的推销是有害的）。
>
> 然后是两组数字：三年里投了多少人、做了多少事；以及这些事里，真正跑起来的有几个。 看完你就明白，为什么这本书要叫「未成之路」。

第一章｜企业 AI 落地的 5 个坑

导语

引言里有一句话：技术上没有一家企业是输在模型不够强的。

那「别的地方」具体是哪几个？

第一章把这五个坑一次摆出来：**场景选错、数据没准备好、技术选型凭感觉、员工抵触、以及效率提升之后红利无处兑现**。

这五个坑有一个共同点：**它们都不是技术问题，但每一个都能让项目停下来**。

而这一章最硬的结论是——**第五个坑最贵**：前四个坑会杀死项目，第五个坑会让项目「成功」却毫无意义。

第二章｜场景选择：先选事，再选技术

导语

第一个坑是「选错场景」。这一章把它拆开讲。

先看一份真实的台账：**上百项需求收敛下来，46 个「计划做」、23 个「已实现」，两组零重叠**。 然后是三个可以当场用的筛选问题：**做错了谁赔？现在人工怎么做？做完之后谁的工作方式会变？**

这一章还给了两张能贴在墙上的表：**选场景三问、制造业六环节速查表**。

而最硬的结论是这句：**上 AI 别从「最痛的地方」开始，从「规则最明确的地方」开始**。 因为最痛的地方往往同时是：口径说不清、入口改不动、效果验不了——恰好是 AI 最不擅长的。

第三章｜数据准备：AI 的燃料

导语

场景选完了，接下来是那件最不性感的事：**备料**。

第一章的坑三说了一句话：数据没准备好就想上 AI 。这一章讲为什么这句话是对的，而且比看起来更严重——

不是「数据质量影响效果」，而是「数据治理的成熟度直接决定 AI 能承担多大的责任」。

这一章会讲清楚三件事： AI 的燃料到底长什么样（ RAG 为什么成了企业首选）、 数据分级怎么切（哪些数据出厂、哪些不出厂）、 以及一个最容易被忽略的机制——**反馈闭环没有「回流」，就只是一个抱怨箱**。

第四章｜技术方案：三棵决策树

导语

料备好了，该选技术了。这一章一上来就否定一个伪问题：**「哪个模型更强」**。

因为在企业里，这个问题从来不是决定性的。决定性的是另外三个——

```
决策树一：私有化 vs API      （先看数据能不能出厂）
决策树二：RAG vs 微调 vs Agent   （先看知识会不会变）
决策树三：什么规模的企业该走哪条
```

三棵树的结论形态都不是「二选一」，而是组合： **数据分层外发 + 路由、 RAG 打底 + Agent 编排 + 少量微调**。

而这一章最硬的一句是：**当你的团队只有一个人做 AI ，先别问选什么模型，先问这件事该不该由你做**。

第五章｜企业 AI 落地五步法

导语

第二、三、四章解决了**选事、备料、选型**。但这三件事做完，项目通常还没开始被使用。

真正卡住项目的，是从「方案通过」到「真的有人每天用」之间的这段路。

这一章讲五步：定义试点 → 准备环境 → 试点运行 → 评估迭代 → 推广。 每一步都给出**可验收的产出物**，而不是「加强沟通」「持续关注」这类正确但无用的话。

而最硬的部分是 **5.4 止损条件**——因为大部分项目不是死于没做起来，是死于**不肯承认自己没做起来**。

第六章｜算账：老板视角的三问

导语

老板问的从来不是「AI 好不好」，而是三个问题：**值不值？钱哪来？谁来干？**

这一章讲成本，也讲收益，更讲一件更要命的事——**收益要分三类算，只有第一类好算，而老板想听的是第二类。**

```
成本五项：外部咨询 / 培训 / 平台开发 / 应用开发 / ★ 兜底人工
          （而最贵的第五项，★ 从不出现在预算表里）
收益四类：提效类（好算）/ 决策类（最值钱最难算）/ 质量类（最容易归零）/ ★ 外部支出类（★ 减少
```

而这一章最硬的一句在 6.3：**单点型能力算工时，平台型能力算时间。用错算法，好项目也会被砍掉。**

第七章｜合规与风险：哪条线不能碰

> **导语**
>
> 这一章不讲「合规很重要」——这句话讲一百遍也没用。
>
> 这一章讲的是**具体的三件事**：**AI 的输出在企业里是什么法律地位**（辅助 vs 决策，责任完全不同）、**哪些数据不能出厂**、以及**审计留痕要留什么**。
>
> 因为在制造业，一条线踩错的代价不是罚款，是**整批产品要返工**。
>
> 而这一章最硬的一句是：**AI 的位置必须按出错的代价来定——代价高的地方，AI 只出草稿，人签字**。

第八章｜真实案例复盘

导语

前面七章讲的是道理。道理容易讲，难的是那个没问出口的问题：**「你说的这些，在真实企业里到底长什么样？」**

这一章是全书的**证据章**，用同一家企业三年的真实材料（第一年技术探索、第二年创新专项、第三年持续探索）：

```
成的三个：合同智审（全公司覆盖）／印章识别（进合同审阅部门）／部门级 Agent 平台底座
没成的：营销 AI 助手（★ 全书唯一一个完整失败案例）
半失败的：智能问数（★ 投入少、损失低、结论清楚）
没推成的：46 个还躺在计划里的场景
```

而这一章最硬的结论是：**评价一个 AI 项目该不该继续，不能只看「成没成」，要看「失败的代价是多少」。同样是没成，一种是「没有结论」，一种是「有结论的结论」。**

★ **而这一章还有两节是 D9 补上的方法论：8.0.2「当下的大模型应用很少真正实现闭环管理」**——嵌入和闭环是两种完全不同的生意；**8.5.3「AI 的失败是速生的」**——所以「用几个月没效果才放弃」在 AI 领域并不常见，**要么没撑过第一个月，要么已经被用出粘性。8.7.1 则把「没撑过第一个月」拆成了四条可排查的路径。**

第九章｜数字员工建设

> **导语**
>
> 最后一章讲一件听起来最虚的事：**怎么让 AI 从「一个工具」变成「一群同事」**。
>
> 这一章先做一件反直觉的事：**把「数字员工」和传统信息化系统摆在一起对比**—— 系统是按业务闭环垂直切的烟囱，数字员工是按岗位横向切、穿插在多个系统节点之间的。 **搞混了，就会给每个数字员工建一套系统，然后被成本拖死**。
>
> 然后讲九件事： IP （它是谁）、 VI （识别成本）、匿名与权限边界、范围与场景、 管理体系与制度、适用范围、人机协同、以及怎么让它变得人性化。
>
> 而这一章最硬的结论是：**数字员工不是把 AI 做得更像人，是把人的工作重新定义一遍**。

附录｜工具箱 A–H

> **导语**
>
> 这一节是**能直接抄走的部分**：场景筛选表、场景评估表、 ROI 计算表、模型台账、 三张清单（数据分级 / 风险自查 / 规范执行）、以及名词速查。
>
> ★ **而这八张表不是散的，它们串成一条链：**
>
> ```
> 附录 H 场景筛选表  →  这事该不该做？（12 问，5 分钟）
>       ↓
> 附录 A 场景评估表  →  如果该做，先做哪个？（收敛到 1–2 个）
>       ↓
> 附录 B ROI 计算表  →  这一两个，账算得过来吗？
>       ↓
> 附录 C 模型台账    →  跑起来之后，怎么管住它？
> ```
>
> ★ **这四张表另出了一份单文件：`草稿/附录_模板清单.md`，方便打印与转发。**
>
> 如果你只想拿一样东西走，就拿**场景筛选表**—— **它对应的是第二章那条主线，而且是全书唯一一个「5 分钟能填完」的东西。**

全书金句墙（★ 可截图传播 / 可拆短视频）

> **用法**：每一条都是独立成篇的完整句子，直接可做一条图文或一条短视频。 ★ 标了 ◆ 的是「最可能被转发」的几条。

关于选场景

> **★ 先筛，再排。拿一张 46 个场景的表去做优先级排序，是本末倒置——因为那张表里，一半的场景本来就不该做。**

> **★ 老路不是落后，它是当前的最优解。**

关于人

> **★ 员工抗拒的不是 AI，是位置。**

> **★ 抵触强度与岗位的 AI 替代性成正比，与该岗位的业务重要性无关。**

> **★ 试点挑得太好推，推广期才爆炸。**

★ AI 在传统企业能替代的大概只有 20%~30%，而且替代的是「岗位里的一些点上」，不是整个岗位。

★ 人均效率提升 30% 和公司利润增长 0%，往往不是两件事，是同一件事。

关于入口

★ 当一个入口策略看起来自相矛盾，往往是你把两个问题塞进了同一个入口——解法是分层，不是打通。

★ 形态是最后一步，边界是第一步。

★ 在哪里投入，就在哪里用大厂的能力——而「投」与「不投」，由那个场景的核心瓶颈决定。

★ 嵌入式是无感的，客户端是有感的——同一套管理动作，用错了地方就是错的。

★ 在高保密环境里，入口首先不是「选哪个」，是「哪些不能选」。

★ 无感的项目不能用使用率验收，有感的项目必须正视使用率。

★ 入口策略的方向，比入口的形态重要得多。 ★ 收敛有机会，叠加几乎必败——因为用户永远有选择权。

★ 另开一个入口的风险，来自「选择权」，而不来自「新入口」本身。

★ 方向正确的方案，也会死在时间差上。

★ 不是 AI 不行，是入口策略的方向反了。

关于效率

★ 效率提升的红利，默认归员工，不归公司——除非你设计了一条把它收回去的路。

★ 省下来的时间不会自动变成收入，它会变成员工的自由度。

★ 人均效率提升 30% 和公司利润增长 0%，往往不是两件事，是同一件事。

★ Agent 数量的多少，和这件事最后成不成，几乎没有因果关系。

关于怎么死

★ 如果一个 AI 项目第一个月就死了，先查推广，再查产品——顺序反了，你收集到的所有反馈都是噪音。

★ 当大家开始从 AI 的输出里「抄一段」用，那就是不好用了。

> ★ AI 的失败是速生的，而传统系统的失败是慢性的——所以对 AI 项目，越早停，省下的越多。

关于失败

> ★ 嵌入和闭环，是两种完全不同的生意。嵌入改造的是一个环节；闭环才谈得上改变一门业务的运行方式。

> ★ 不可能仅因某个岗位的工作，就改变整个公司的管理体系。

> ★ AI 的失败是速生的，而传统系统的失败是慢性的——所以对 AI 项目，越早停，省下的越多。

> ★ 用户开始对 AI 提新需求，才是它活下来的真正证据。

> ★ 「老路被用户抛弃」和「老路其实一直在赢」，是两种完全不同的诊断。

> ★ 一个新工具真正的对手，从来不是另一个新工具，而是那个「不需要用户做任何新动作」的老办法。

> ★ 老工具身上那些看起来「不先进」的优点，恰恰是它活到今天的原因。

> ★ 助手型产品有一个结构性的软肋：它替用户做了选择。

★ 需求成立，不等于产品成立。

◆ 同样是没成，一种是「没有结论」，一种是「有结论的结论」。

◆ 评价一个 AI 项目该不该继续，不能只看「成没成」，要看「失败的代价是多少」。

一个「事实已结束、账面未结束」的项目，会一直躺在资源清单上，被当成「还在推进的资产」。

止损线本身早就该触发，真正缺的不是阈值，是「触发之后谁来拍板关掉它」。

AI 项目的失败，大部分不是败在模型，是败在项目组根本不在决策链上。

最贵的失败不是做错了，是拖了一年才承认自己没做对。

关于场景选择

◆ 上 AI 别从「最痛的地方」开始，从「规则最明确的地方」开始。

◆ 一个部门里同时住着适合 AI 的活和不适合 AI 的活。判断的单位是活，不是部门。

> 痛的地方不一定适合，适合的地方不一定痛。选场景要的是后者。

> 按部门粗分场景必然失败，按规则密度找场景不会。

> 场景选错的代价远大于技术选错——技术选错损失几个月，场景选错损失一句「AI 没用」。

> 一张排了 46 个场景却没有排优先级的表，不是路线图，是愿望清单。

> 「这个场景能不能上」的第一问不是「能不能做」，是「做错了谁赔」。

关于数据与口径

> ◆ 模型的精度不来自模型，来自口径。

> AI 助手的价值曲线，不随模型能力上升，只随数据治理程度上升。

> 反馈闭环没有「回流」，就只是一个抱怨箱。

> 数据没准备好就想上 AI，等于在没测过水温的地方下饺子。

关于技术与选型

> 「哪个模型更强」是个伪问题。真正的问题是：你的数据能不能出厂，你的知识会不会变。

> 当你的团队只有一个人做 AI，先别问选什么模型，先问这件事该不该由你做。

> 企业里不存在「最好的技术方案」，只存在「这个季度能落地的方案」。

> 接得上不等于敢让它接——权限代理和审计留痕，是 MCP 落地的两条硬要求。

关于落地与组织

> ◆ 技术部门保留授权权，交出运维权——两难就这样被切开了。

> 当业务能自己动手，IT 的角色不是变少了，是从「排队接单」变成「设闸放行」。

> 能让企业持续变好的 AI，不是替你干活的那个，是帮你把方法建起来的那个。

> 数字员工不是把 AI 做得更像人，是把人的工作重新定义一遍。

> 把 AI 当功能做，一年就见分晓；把 AI 当基础设施做，五年才见分晓。

> 业务部门自己会用 AI 写工具之后，企业的问题从「怎么让业务别乱做」变成了「怎么让业务做的东西能被管理」。

关于账

> ★ 在传统制造企业里， AI 最实在的一笔账，不是替代谁，而是减少对外咨询的费用。

> ★ AI 更像当年的财务软件——它不是一夜之间取代一切，是逐步、不断周期性地淘汰迭代，然后不断深入。

> ◆ 单点型能力算工时，平台型能力算时间。用错算法，好项目也会被砍掉。

> 自建不是省钱，是把钱换成了自己人的时间——而时间是最贵的那一项。

> 专职 AI 人力不足两人，是很多企业最贵、也最不愿写进预算表的一项。

> 不要用 数百人次培训去换组织的信任，要用两个真数字去换。

> 推的产出是声量，扎的产出是数字。

关于合规

> AI 的位置必须按出错的代价来定——代价高的地方， AI 只出草稿，人签字。

> 在制造业，合规不是罚款问题，是整批产品要返工的问题。

> AI 辅助和 AI 决策的区别，不在技术，在签字的人是谁。

关于底座与知识

> ◆ 一个共享知识库要活下来，只需要一件事：有人对「答案是对的」负责。

> 知识库活不过三个月，通常不是因为它不好用，是因为没人敢信它的答案。

> 判断一个平台是不是底座，看第二个部门接进来时，改动大不大。

> 一套知识库服务整个部门，才是数字员工从工具变成资产的那一步。

> 前两个成功证明「AI 能做成事」，第三个证明「AI 能做成组织能力」。

关于立场（★ 给乙方读者，也给甲方读者）

◆ 这本书站在甲方这边——因为钱是甲方出的，责任也是甲方担的。

◆ 不要为了自己的业绩，去推动甲方投入。

劝甲方投一个不合适的场景，代价从来不在乙方那边，在甲方那边。

一份诚实的方案里，一定包含「我们建议你先别做的那部分」。

甲方拿这份东西去立项，责任才清得干净。

甲方自己也做过乙方。内部汇报里的每一个「有前景」，都是一次不经意的推销。

一家企业的 AI 信任是脆弱资产，乙方每做一次过度承诺就在消耗它。

关于第一年与第二年

◆ 第一年学会「试」，第二年才有「扎」。

两年用同一把尺子，汇报就会一直报喜不报忧。

> 数百人次培训落地数个场景；而「扎」出来的场景拿出了 千余小时/年和可核算的年度效益。

版权与免责声明

- 本书案例均来自作者亲历项目与企业内部实践，已对真实企业名、人名、产品名、金额与内部精确数字做泛化处理，仅保留可公开的量级、比例与效益指标。
- 书中标注【待核实】的内容为作者尚未最终确认的部分，正式出版前将逐条校订。
- 书中涉及合规、数据分级与 AI 输出的责任划分，仅为工程与管理层面的经验总结，**不构成法律或合规意见**；具体项目请与贵公司的法务、质量、信息安全部门共同确认。
- 「某大型制造企业」「某制造业集团」「某头部医药集团」等均为泛化称呼，与任何具体企业无对应关系。
- 文中出现的开源系统与商业平台名称，用于说明技术路线形态，不构成对任何产品的推荐或评价。

《AI 未成之路》定稿候选稿 v1.13 ｜ 2026-11-03